

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Giáo trình

THIẾT KẾ HỆ THỐNG NHÚNG

**Tác giả: Đoàn Duy
Lê Hoài Nghĩa**

TP. Hồ Chí Minh, 2020

LỜI MỞ ĐẦU

Ngày nay, máy tính đã không còn xa lạ với cuộc sống của con người. Các hệ thống máy tính được ứng dụng rộng khắp trong các lĩnh vực khoa học, y tế, giáo dục, giao thông, ứng dụng đời sống, sản xuất công nghiệp cho đến các hệ thống có mức độ phức tạp cao nhưng hàng không, vũ trụ và quân sự quốc phòng. Cuộc sống hiện đại gắn liền với các hệ thống máy tính, các hệ thống tự động và hơn hết là các hệ thống thông minh giúp con người xử lý hầu hết các nhu cầu của cuộc sống. Từ những nhu cầu đơn giản như đánh thức mỗi sáng thức dậy bằng chế độ định giờ trên điện thoại; xem tin tức qua truyền hình; chuẩn bị thức ăn với các thiết bị hiện đại như tủ lạnh, lò vi sóng, điện thoại, xe hơi... Tất cả các hệ thống tin toán nói trên đều là các hệ thống nhúng hoặc một phần là hệ thống nhúng.

Trong những năm gần đây, hệ thống nhúng phải giải quyết nhiều thử thách hơn để đáp ứng sự thay đổi của nhu cầu sử dụng, từ việc tương tác với thế giới vật lý nhiều hơn, tiếp cận công nghệ mới, sự bùng nổ của internet và những nhu cầu ứng dụng mới. Thuật ngữ về “Vạn vật kết nối” hay thiết bị thông minh dần trở nên phổ biến. Là nền tảng cơ sở để phát triển các thiết bị thông minh, thiết bị sử dụng cho vạn vật kết nối, hệ thống nhúng được tích hợp các cơ chế tính toán tinh mạnh mẽ hơn và cung cấp nhiều cơ chế điều khiển các quá trình vật lý với những cơ chế phản hồi, tinh chỉnh một cách tinh vi. Các hệ thống nhúng cũng được hỗ trợ công nghệ mạng kết

nói, kỹ thuật tổ chức dữ liệu, phần mềm và những câu cấu vật lý phần cứng phức tạp.

Với vai trò quan trọng đó, giáo trình này được xây dựng để mang đến cho người đọc những kiến thức và kỹ thuật cơ bản để xây dựng các hệ thống nhúng. Giáo trình giúp người đọc tiếp cận với thiết kế hệ thống nhúng từ góc nhìn từ hệ thống đến chi tiết các thành phần. Chúng ta sẽ tìm hiểu qua các khái niệm liên quan, sự kết nối của hệ thống nhúng với các xu hướng công nghệ và ứng dụng. Giáo trình được tổ chức thành tám chương với những ví dụ cụ thể và bài tập đi kèm trong từng chương giúp người đọc dễ tiếp cận với các khái niệm và thiết kế.

Chương 1 là phần giới thiệu tổng quan về thiết kế hệ thống nhúng. Trong chương này, chúng ta sẽ thấy được mối liên hệ của hệ thống nhúng với các khái niệm công nghệ và xu thế phát triển ứng dụng. Sau đó, giáo trình sẽ cung cấp góc nhìn hệ thống để người đọc nắm bắt được các thành phần chính và nguyên lý hoạt động cơ bản của một hệ thống nhúng. Trong các chương tiếp theo, chúng ta sẽ tìm hiểu về từng thành phần cụ thể và cách thiết kế.

Chương 2 giới thiệu tổng quan các vi xử lý nhúng và vi điều khiển được sử dụng trong phát triển các hệ thống nhúng. Bộ vi xử lý nhúng được xem là bộ não của cả hệ thống, là nơi thực thi các tính toán và xử lý dữ liệu. Chúng ta sẽ lần lượt tìm hiểu về kiến trúc

các dòng vi xử lý nhúng từ cổ điển đến hiện đại với các tính năng quan trọng được cập nhật.

Chương 3 sẽ mô tả các kiến trúc bộ nhớ, bao gồm bản khái quát mô hình bộ nhớ trong lập trình, thuộc tính vật lý (công nghệ chế tạo, kích thước...) và thuộc tính kiến trúc của bộ nhớ (kiến trúc phân tần theo cache, bộ nhớ chính...) và nhấn mạnh vào hiểu biết về ảnh hưởng của kiến trúc bộ nhớ lên tính động của hệ thống. Bộ nhớ là một thành phần quan trọng không thể thiếu trong mọi hệ thống nhúng, bên cạnh bộ vi xử lý.

Chương 4 sẽ trình bày về các giao tiếp giữa thành phần phần mềm và thành phần phần cứng trong hệ thống. Chương này sẽ thảo luận các cơ chế nhập xuất trong kiến trúc phần mềm và kiến trúc máy tính và giao tiếp giữa thế giới tín hiệu số và thế giới tín hiệu tương tự. Các thành phần nhập xuất là cơ sở để bộ vi xử lý có thể tương tác được với thế giới bên ngoài.

Chương 5 giới thiệu về cảm biến và các cơ cấu chấp hành được sử dụng trong thiết kế hệ thống nhúng. Không đi sâu trình bày một cách chi tiết yếu tố kỹ thuật, chúng ta xem xét ở khía cạnh mô hình hoạt động của các thành phần này để hướng đến một phương thức thiết kế tổng thể cho hệ thống.

Chương 6 và Chương 7 sẽ tập trung vào kiến trúc thượng tầng của hệ thống nhúng để giới thiệu về hệ điều hành và các cơ chế liên quan. Trong đó, giáo trình sẽ giới thiệu các khái niệm bên liên quan

đến hệ thống đa nhiệm và các cơ chế xử lý song song. Chương này nhấn mạnh và các khó khăn khi sử dụng các cơ cấu mức thấp như luồng (thread) để hy vọng có thể giúp người đọc hiểu được giá trị thực sự của việc sử dụng mô hình. Các kỹ thuật mô hình hóa giúp người kỹ sư tự tin trong thiết kế hệ thống. Giáo trình cũng giới thiệu về cơ chế định thời gian thực và các giải thuật định thời cơ bản được sử dụng.

Bên cạnh giới thiệu các thành phần cơ bản của hệ thống nhúng phục vụ cho quá trình thiết kế, giáo trình này cũng giới thiệu một hướng tiếp cận mới trong quá trình thiết kế: thiết kế hướng mô hình hóa. Mô hình hóa giúp xây dựng hệ thống tổng quan, phân tích đánh giá và phát triển hệ thống một cách toàn diện và thuận tiện. Chương 8 mô tả các khái niệm liên quan đến mô hình hệ thống, máy trạng thái và các thức mô tả hệ thống dựa trên mô hình.

Để có thể sử dụng hiệu quả giáo trình này, người đọc cần có (hoặc bổ sung) kiến thức cơ bản về kiến trúc máy tính và hệ điều hành máy tính. Trong quá trình nghiên cứu tài liệu, người đọc cần kết hợp kiến thức lý thuyết với vận dụng giải các bài tập, kết hợp việc hiểu các nội dung với quan sát đối chiếu với các vấn đề tồn tại ngoài thực tế. Nhóm biên soạn giáo trình cũng mong muốn nhận được phản hồi của người đọc để có những đánh giá và cập nhật cần thiết cho giáo trình tốt hơn.

MỤC LỤC

Chương 1 GIỚI THIỆU	17
1.1 Ứng dụng hệ thống nhúng.....	18
1.2 Hệ thống nhúng và IoT	19
1.3 Các thành phần của hệ thống nhúng	21
1.4 Chuyển đổi giữa tín hiệu tương tự và tín hiệu số.....	26
1.4.1 Chuyển đổi từ tín hiệu tương tự sang tín hiệu số	26
1.4.2 Chuyển đổi từ tín hiệu số sang tín hiệu tương tự	28
1.4.3 Mạch công suất và mạch cách ly.....	29
1.5 Quy trình thiết kế	31
1.6 Tổng kết	33
Chương 2 VI XỬ LÝ NHÚNG	34
2.1 Các loại bộ vi xử lý nhúng	35
2.1.1 Bộ vi xử lý tín hiệu số	36
2.1.2 Bộ vi điều khiển	42
2.1.3 Bộ điều khiển khả lập trình	44
2.1.4 Kiến trúc x86.....	46
2.1.5 Bộ vi xử lý DSP.....	47
2.1.6 Bộ vi xử lý đồ họa	48
2.2 Xử lý song song	49
2.2.1 Xử lý song song và xử lý đồng thời	49
2.2.2 Kiến trúc đường ống.....	53
2.2.3 Song song mức lệnh	60
2.2.4 Kiến trúc đa lõi	66
2.3 Tóm tắt	68
2.4 Bài tập	69
Chương 3 KIẾN TRÚC BỘ NHỚ.....	70
3.1 Công nghệ bộ nhớ	71
3.1.1 RAM.....	71
3.1.2 Bộ nhớ bất biến	73
3.2 Phân cấp bộ nhớ	75
3.2.1 Ánh xạ bộ nhớ	76
3.2.2 Thanh ghi.....	79
3.2.3 Vùng nhớ đệm Scratchpad và bộ nhớ Cache	80
3.3 Mô hình bộ nhớ.....	86
3.3.1 Địa chỉ bộ nhớ	86

3.3.2	Ngăn xếp.....	88
3.3.3	Các đơn vị bảo vệ bộ nhớ.....	89
3.3.4	Cấp phát bộ nhớ động	90
3.3.5	Mô hình bộ nhớ trong ngôn ngữ C.....	92
3.4	Tóm tắt	94
3.5	Bài tập	95
Chương 4 THÀNH PHẦN NHẬP XUẤT		99
4.1	Phân cứng nhập xuất	99
4.1.1	Điều chế độ rộng xung	101
4.1.2	Cổng nhập xuất tín hiệu số đa dụng	102
4.2	Giao tiếp nối tiếp.....	107
4.3	Giao tiếp song song.....	110
4.4	Đường truyền dữ liệu	111
4.5	Phần mềm tuần tự trong một thế giới đồng thời	114
4.5.1	Ngắt và ngoại lệ.....	114
4.5.2	Tính nguyên tử	116
4.5.3	Bộ điều khiển ngắt.....	118
4.5.4	Mô hình hóa ngắt.....	120
4.6	Tóm tắt	124
4.7	Bài tập	124
Chương 5 CẢM BIẾN VÀ CƠ CẤU CHẤP HÀNH.....		129
5.1	Giới thiệu về cảm biến	130
5.1.1	Đo góc nghiêng và gia tốc.....	130
5.1.2	Đo vị trí và vận tốc	133
5.1.3	Đo vòng quay	135
5.1.4	Đo âm thanh	136
5.1.5	Các cảm biến khác.....	137
5.2	Cơ cấu chấp hành	137
5.2.1	Điốt phát sáng.....	137
5.2.2	Điều khiển động cơ	139
5.3	Tóm tắt	143
5.4	Bài tập	143
Chương 6 HỆ THỐNG ĐA NHIỆM		146
6.1	Lập trình lệnh	150
6.2	Luồng xử lý	154
6.2.1	Tạo luồng.....	154

6.2.2	Thực thi các luồng.....	157
6.2.3	Loại trừ tương hỗ.....	160
6.2.4	Tắc nghẽn	164
6.2.5	Đồng nhất bộ nhớ	167
6.2.6	Các vấn đề liên quan đến luồng	168
6.3	Tiến trình và truyền thông điệp.....	172
6.4	Tóm tắt	180
6.5	Bài tập	181
Chương 7	ĐỊNH THỜI TÁC VỤ	184
7.1	Khái niệm cơ bản về định thời tác vụ	185
7.1.1	Quyết định định thời.....	185
7.1.2	Mô hình tác vụ.....	188
7.1.3	So sánh các bộ định thời.....	193
7.1.4	Hiện thực bộ định thời.....	194
7.2	Bộ định thời Rate Monotonic.....	196
7.3	Bộ định thời EDF	203
7.3.1	EDF với các tiền lệ	205
7.4	Định thời và loại trừ tương hỗ.....	207
7.4.1	Đảo ngược độ ưu tiên	207
7.4.2	Giao thức kế thừa độ ưu tiên	209
7.4.3	Giao thức trần ưu tiên.....	211
7.5	Định thời đa xử lý	213
7.5.1	Bất thường trong định thời	214
7.6	Tóm tắt	218
7.7	Bài tập	218
Chương 8	THIẾT KẾ HỆ THỐNG DỰA TRÊN MÔ HÌNH	223
8.1	Mô hình hệ thống thay đổi liên tục	224
8.1.1	Ví dụ mở đầu	224
8.1.2	Mô hình tác nhân – Actor model.....	226
8.1.3	Thuộc tính của hệ thống.....	230
8.1.4	Hệ thống kiểm soát có phản hồi	234
8.2	Mô hình hệ thống thay đổi rời rạc.....	235
8.2.1	Các hệ thống thay đổi rời rạc	235
8.2.2	Khái niệm về trạng thái	239
8.2.3	Máy trạng thái hữu hạn	240
8.2.4	Thuyết không xác định.....	251

8.3	Hệ thống lai	255
8.3.1	Mô hình phương thức	256
8.3.2	Tình chỉnh trạng thái	259
8.4	Tổ hợp máy trạng thái	262
8.4.1	Tổ hợp đồng thời	263
8.4.2	Tổ hợp xếp tầng.....	277
8.4.3	Tổ hợp tổng quát	281
8.4.4	Máy trạng thái phân cấp	282
8.5	Tóm tắt	288
8.6	Bài tập	288
TÀI LIỆU THAM KHẢO		298

DANH MỤC HÌNH ẢNH

Hình 1.1: Hệ sinh thái ứng dụng của hệ thống nhúng	18
Hình 1.2: Mô hình các tầng của hệ thống IoT	19
Hình 1.3: Ứng dụng IoT cho mô hình nhà thông minh	21
Hình 1.4: Sơ đồ các thành phần cơ bản của Hệ thống nhúng	22
Hình 1.5: Sơ đồ khối thiết bị xử lý trong hệ thống nhúng	24
Hình 1.6: Cấu trúc hệ thống nhúng có hệ điều hành và ứng dụng	25
Hình 1.7: Minh họa lấy mẫu chuyển đổi tín hiệu tương tự sang tín hiệu số	27
Hình 1.8: Chuyển đổi tín hiệu số sang tín hiệu tương tự	29
Hình 1.9: Mạch điều khiển động cơ sử dụng opto cách ly	30
Hình 1.10: Các thành phần cơ bản của quá trình thiết kế hệ thống nhúng CPS	31
Hình 2.1: Cấu trúc một hiện thực của bộ lọc FIR	38
Hình 2.2: Sơ đồ logic bậc thang	45
Hình 2.3: Cấu trúc đường ống đơn giản.	54
Hình 2.4: Bảng đặt trước cho đường dẫn được thể hiện trong Hình 2.2	56
Hình 2.5: Khóa liên động giải quyết <i>pipeline hazard</i>	58
Hình 3.1: Tổ chức bộ nhớ của kiến trúc ARM Cortex-M3	77
Hình 3.2: Tổ chức bộ nhớ cache	82
Hình 4.1: Bo mạch thực hành Stellaris R LM3S8962	100
Hình 4.2: Mạch thu và mạch góp hở	106
Hình 4.3: Các đầu nối cho giao tiếp nối tiếp và song song	107
Hình 4.4: Các mô hình máy trạng thái và chương trình tương ứng	120
Hình 4.5: Mô hình máy trạng thái cho sự tương tác giữa ISR và chương trình chính	122
Hình 4.6: Mô hình máy trạng thái phân cấp cho sự tương tác giữa ISR và chương trình chính	124
Hình 4.7: Ví dụ chương trình C với thủ tục gọi hàm	125
Hình 5.1: Mô hình gia tốc kế	131

Hình 5.2: Gia tốc kế silicon	132
Hình 5.3: Điều khiển động cơ DC bằng PWM.....	140
Hình 6.1: Các khái niệm trừu tượng về chương trình xử lý đồng thời	147
Hình 6.2: Minh họa đoạn chương trình C cấp cao	149
Hình 6.3: Mô hình cập nhật trạng thái biến trong chương trình Hình 6.2	152
Hình 6.4: Chương trình C đa luồng đơn giản sử dụng Pthreads	155
Hình 6.5: Ví dụ về một ứng dụng truyền thông điệp đơn giản...	176
Hình 6.6: Các thủ tục truyền thông điệp.....	178
Hình 7.1: Tóm tắt thời gian liên quan đến việc thực thi tác vụ ..	190
Hình 7.2: Hai tác vụ chu kỳ p_1 và p_2	197
Hình 7.3: Hai tác vụ chu kỳ với định thời độ ưu tiên	197
Hình 7.4: Thời gian đáp ứng trường hợp xấu nhất	199
Hình 7.5: Định thời Non-RM ưu tiên cao hơn cho τ_2	199
Hình 7.6: Định thời RM ưu tiên cao hơn cho τ_1	200
Hình 7.7: Ví dụ định thời đa tác vụ với các độ ưu tiên khác nhau	205
Hình 7.8: Máy dò đường sao Hỏa và hình ảnh bề mặt sao Hỏa từ camera của tàu đổ bộ	209
Hình 7.9: Minh họa nghịch đảo độ ưu tiên.....	210
Hình 7.10: Minh họa giao thức kế thừa ưu tiên.....	210
Hình 7.11: Hình minh họa tắc nghẽn.....	211
Hình 7.12: Định thời song song hai bộ vi xử lý cho các tác vụ.	215
Hình 7.13: Biểu đồ mức độ ưu tiên với chín tác vụ	215
Hình 7.14: Các chế độ định thời với ràng buộc thời gian và độ ưu tiên	216
Hình 7.15: Sự bất thường do các khóa loại trừ tương hỗ	218
Hình 8.1: Thiết kế mạch điều khiển đèn diot	225
Hình 8.2: Mô hình hệ thống thay đổi liên tục một ngõ vào/ra ...	226
Hình 8.3: Mô hình tác nhân của điện trở	227
Hình 8.4: Hệ thống nối tiếp mô hình tác nhân.....	228
Hình 8.5: Mô hình tác nhân hai tín hiệu đầu vào	228

Hình 8.6: Biểu tượng của mô hình bộ cộng.....	229
Hình 8.7: Biểu tượng của mô hình bộ trừ.....	229
Hình 8.8: Hệ thống điều khiển có phản hồi.....	235
Hình 8.9: Mô hình của một hệ thống theo dõi số lượng ô tô trong một nhà để xe.....	236
Hình 8.10: Các ký hiệu liên quan máy trạng thái	241
Hình 8.11: Mô hình FSM cho ga ra ô tô của Hình 8.9	242
Hình 8.12: Mô hình máy điều nhiệt có độ trễ.....	244
Hình 8.13: Một quá trình chuyển trạng thái mặc định	247
Hình 8.14: Mô hình máy trạng thái không xác định cho bộ điều khiển đèn cho người đi bộ	252
Hình 8.15: FSM không xác định cho tín hiệu đèn giao thông....	255
Hình 8.16: FSM được biểu thị như một tác nhân.....	258
Hình 8.17: Bộ điều nhiệt được mô hình hóa như một FSM với tín hiệu đầu vào liên tục	259
Hình 8.18: Một bộ điều nhiệt có đầu ra thời gian liên tục.....	260
Hình 8.19: Ký hiệu cho hệ thống lai.....	261
Hình 8.20: Tóm tắt ký hiệu các máy trạng thái được sử dụng trong chương này.....	264
Hình 8.21: Tổ hợp song song của hai tác nhân.....	267
Hình 8.22: Sơ đồ máy trạng thái của tổ hợp hai tác nhân	268
Hình 8.23: Máy trạng thái đơn đưa ra ngữ nghĩa của thành phần đồng bộ song song	270
Hình 8.24: Máy trạng thái đưa ra ngữ nghĩa của thành phần không đồng bộ song song	271
Hình 8.25: Mô hình của hai máy chủ với hàng đợi tác vụ được chia sẻ	274
Hình 8.26: Tổ hợp xếp chồng hai tác nhân.....	278
Hình 8.27: Tổ hợp xếp chồng hai FSM	278
Hình 8.28: Ngữ nghĩa của tổ hợp xếp chồng hai FSM.....	278
Hình 8.29: Mô hình đèn đi bộ trong tổ hợp xếp chồng đồng bộ	280
Hình 8.30: Sơ đồ ngữ nghĩa của tổ hợp xếp tầng đồng bộ của đèn đi bộ	281

Hình 8.31: Sự kết hợp của tổ hợp song song và xếp tầng trong kết nối của máy trạng thái.....	282
Hình 8.32: Tinh chỉnh trạng thái của máy trạng thái phân cấp. .	283
Hình 8.33: Sơ đồ ngữ nghĩa của HFSM	283
Hình 8.34: Máy trạng thái phân cấp với chuyển trạng thái có độ ưu tiên.	284
Hình 8.35: Sơ đồ ngữ nghĩa HFSM với chuyển trạng thái có độ ưu tiên.	285
Hình 8.36: Dạng khác của HFSM với lịch sử chuyển tiếp.....	285
Hình 8.37: Sơ đồ ngữ nghĩa của HFSM với lịch sử chuyển trạng thái.	287
Hình 8.38: Mô hình âm thoa.....	289
Hình 8.39: Mô hình bộ đếm sự kiện.....	291

DANH MỤC BẢNG BIỂU

Bảng 3.1: Tổng hợp các tham số của bộ nhớ cache	82
Bảng 8.1: Mô tả các thành phần của quá trình chuyển trạng thái	242
Bảng 8.2: Các giá trị guard hợp lệ	243
Bảng 8.3: Các giá trị Action hợp hệ	245

DANH MỤC TỪ VIẾT TẮT

LTI	Linear and Time Invariant
FSM	Finite State Machine
BIBO	Bounded Input Bounded Output
ODE	Ordinary Differential Equation
HVAC	Heating, Ventilation, Air Conditioning
PWM	Pulse Width Modulation
ISA	Instruction Set Architecture
μC	Microcontroller
FIR	Finite Impulse Response
ATSC	Advanced Television System Committee
GPU	Graphic Processing Unit
PC	Program Counter
ALU	Arithmetic and Logic Unit
IPL	Instruction Level Parallelism
CISC	Complex Instruction Set Computer
RISC	Reduced Instruction Set Computer
PDA	Personal Digital Assistant
RAM	Random Access Memory
SRAM	Static RAM
DRAM	Dynamic RAM
ROM	Read-only Memory
TLB	Translation Lookaside Buffer

ADC	Analog to Digital Converter
MMU	Memory Management Unit
LRU	Least Recently Used
FIFO	First In, First Out
LIFO	Last In, First Out
IDE	Integrated Development Environment
GPIO	General Purpose Input/Output
SWD	Serial Wire Debug
PCI	Peripheral Component Interface
ATA	Advanced Technology Attachment
MAC	Media Access Control
CSMA	Carrier Sense Multiple Access
DMA	Direct Memory Access
ISR	Interrupt Service Routine
WCET	Worst Case Execution Time
EDD	Earliest Due Date
EDF	Earliest Deadline First
LDF	Lasted Deadline First

Chương 1 GIỚI THIỆU

Trong những năm gần đây, khái niệm **Hệ thống nhúng** (Embedded system) ngày càng trở nên quen thuộc với các kỹ sư trong lĩnh vực điện tử, công nghệ thông tin, viễn thông và ngay cả đối với người dùng các thiết bị điện tử. Một định nghĩa đơn giản, hệ thống nhúng là các hệ thống có sự kết hợp của các thành phần phần cứng, phần mềm và những thành phần khác như các bộ phận cơ khí, truyền động nhằm thực hiện một chức năng chuyên biệt nào đó. Căn cứ vào định nghĩa trên, chúng ta có thể dễ dàng tìm thấy sự hiện diện của hệ thống nhúng thông qua các thiết bị quen thuộc trong cuộc sống hằng ngày như tivi, máy giặt, máy lạnh, điện thoại cho đến các thiết bị văn phòng như máy in, máy photocopy...

Cùng với sự phát triển nhanh chóng của lĩnh vực điện-điện tử và công nghệ thông tin và truyền thông với các khái niệm mới như Tự động hóa (Automotives), Hệ thống thông minh (Smart system) và Vạn vật kết nối (Internet of Things), hệ thống nhúng đã ngày càng trở nên phức tạp hơn về mặt thiết kế, khả năng tính toán và tích hợp nhiều tính năng thông minh.

Trong chương này, chúng ta sẽ cũng tìm hiểu qua một số khái niệm cơ bản về các hệ thống nhúng khác nhau, các kiến trúc và thành phần cơ bản trong hệ thống nhúng. Bên cạnh đó, một số mô hình thiết kế hệ thống nhúng cũng được giới thiệu để người đọc có

được cái nhìn tổng quan về phương pháp tiếp cận thiết kế hệ thống nhúng.

1.1 Ứng dụng hệ thống nhúng

Như đã đề cập ở trên, hệ thống nhúng đang trở nên vô cùng thông dụng trong cuộc sống hiện tại của chúng ta. Bên cạnh những ứng dụng thông thường, chúng ta cũng có thể tìm thấy ứng dụng của hệ thống nhúng ở những quy mô khác nhau và những mức độ tin vi, phức tạp khác nhau. Cụ thể, chúng ta có thể tìm thấy các dây chuyền lắp ráp thiết bị công nghiệp tự động hóa và được điều khiển bằng máy tính ở quy mô hàng trăm thiết bị. Các thiết bị trong ngành viễn thông như trạm thu phát sóng 3G/4G, thiết bị kết nối mạng vô tuyến, điện thoại di động, bộ chuyển mạch...



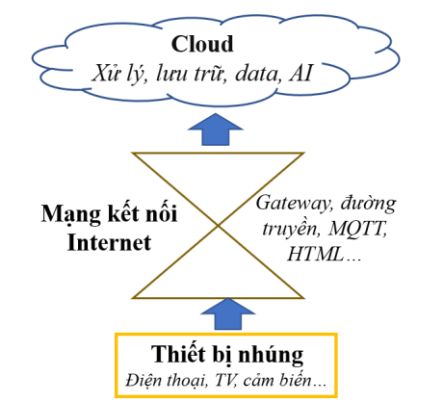
Hình 1.1: Hệ sinh thái ứng dụng của hệ thống nhúng

Ngoài ra, chúng ta cũng có thể tìm thấy các ứng dụng của hệ thống nhúng trên các thiết bị tinh vi hơn như máy bay, trạm vệ tinh

ngiên cứu vũ trụ hay các thiết bị có độ bảo mật cao thuộc về an ninh quốc phòng. Trong tất cả các hệ thống nói trên đều có sự hiện diện của hệ thống nhúng dưới dạng đơn lẻ hoặc kết hợp của hàng chục, hàng trăm đến hàng ngàn hệ thống nhúng với nhau.

1.2 Hệ thống nhúng và IoT

Tầm quan trọng của hệ thống nhúng rõ thể hiện trong xu thế phát triển của khái niệm vạn vật kết nối (IoT). Các thiết bị ứng dụng của hệ thống nhúng được kết nối với mạng Internet để trao đổi thông tin, truyền tải tín hiệu điều khiển và dần được tích hợp những tính năng xử lý phức tạp, thông minh và nhanh chóng. Thiết kế hệ thống nhúng trở thành nền tảng cho sự phát triển của IoT và các ứng dụng liên quan. Hình 1.2 mô tả ba tầng cơ bản của một hệ thống IoT gồm: Tầng 1 – Thiết bị đầu cuối, Tầng 2 – Hạ tầng kết nối mạng Internet, và Tầng 3 – Cloud.



Hình 1.2: Mô hình các tầng của hệ thống IoT

Tầng đầu tiên trong hệ thống là các thiết bị đầu cuối (hay còn gọi là thiết bị người dùng) là các thiết bị dùng để tương tác trực tiếp

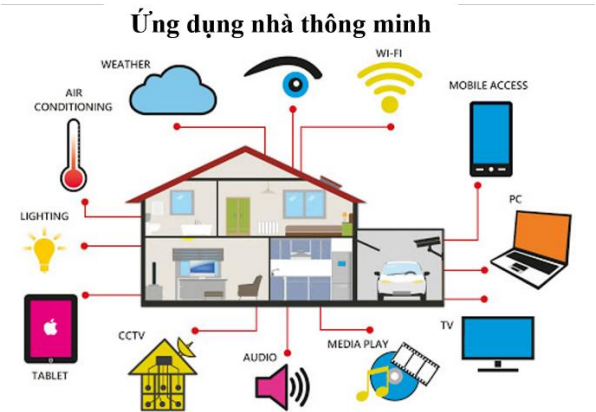
với người dùng hoặc tương tác với môi trường. Các thiết bị này có thể là điện thoại thông minh, TV thông minh, xe hơi thông minh, các cảm biến quan trắc thông số môi trường, các hệ thống chiếu sáng... Về cơ bản, các thiết bị ở tầng này đều là các hệ thống nhúng, được thiết kế để tương tác với người dùng và thu thập dữ liệu cần thiết cho các xử lý trên tầng cao hơn - Cloud.

Tầng thứ hai là hệ thống kết nối mạng Internet. Tầng này cung cấp kết nối giữa các thiết bị đầu cuối (Tầng 1) và hệ thống cloud. Tầng này gồm hai thành phần cơ bản là thiết bị phần cứng phục vụ kết nối như router, access point, cáp truyền... và các giao thức kết nối như HTML, MQTT, Zigbee... Công nghệ không dây dần chiếm ưu thế trong hạ tầng kết nối của hệ thống IoT so với công nghệ kết nối dây truyền thống.

Tầng thứ ba là điện toán đám mây – Cloud – cung cấp nền tảng xử lý và lưu trữ dữ liệu. Cloud là nơi tập hợp dữ liệu được gửi lên từ thiết bị đầu cuối và hỗ trợ các cơ chế xử lý dữ liệu lớn, cho phép ứng dụng các công nghệ thông minh như trí tuệ nhân tạo, máy học...

Thiết kế hệ thống nhúng giữ vai trò quan trọng trong việc phát triển hệ thống IoT từ phát triển các thiết bị đầu cuối ở Tầng 1 đến các thiết bị phần cứng phục vụ cho Tầng 2. Ngoài ra, việc xây dựng các hệ thống nhúng còn ảnh hưởng đến chất lượng cơ sở dữ liệu phục vụ cho các giải pháp được thực hiện trên điện toán đám mây.

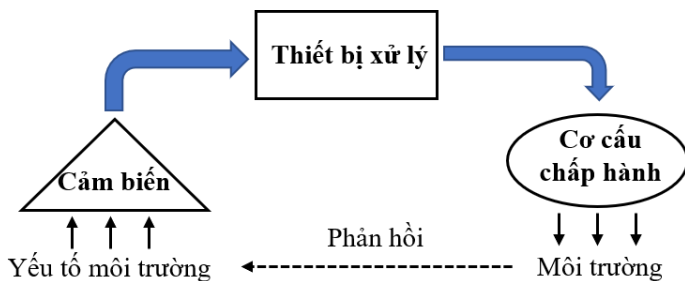
Hình 1.3 mô tả một ứng dụng nhà thông minh trong xu thế phát triển của IoT. Trong ứng dụng này, hầu hết các thiết bị đầu cuối tham gia trong hệ thống đều là các đại diện cho hệ thống nhúng.



Hình 1.3: Ứng dụng IoT cho mô hình nhà thông minh

1.3 Các thành phần của hệ thống nhúng

Một hệ thống nhúng cơ bản là sự kết hợp của hai thành phần phần cứng và phần mềm. Tùy thuộc vào mức độ phức tạp và nhu cầu ứng dụng và các hệ thống nhúng có thể được thiết kế từ đơn giản đến phức tạp. Trong đó, thành phần phần cứng nói nôm na là các thành phần vật lý (có thể nhìn thấy, sờ thấy) của hệ thống. Thành phần phần mềm chính là chương trình điều khiển hoạt động của các cơ cấu phần cứng vật lý để giúp hệ thống tương tác với môi trường và giải quyết bài toán của ứng dụng. Từ “nhúng” ở đây có thể hiểu là chương trình điều khiển được đưa vào một cơ cấu phần cứng để điều khiển cơ cấu này đáp ứng một mục đích ứng dụng cụ thể.



Hình 1.4: Sơ đồ các thành phần cơ bản của Hệ thống nhúng

Hình 1.4 trình bày sơ đồ một hệ thống nhúng với các thành phần chức năng cơ bản. Các hệ thống nhúng được thiết kế để giúp người dùng có thể tương tác với môi trường và tạo ra các thay đổi cần thiết đến môi trường tùy thuộc vào nhiều mục đích khác nhau. Một quy trình xử lý cơ bản của một hệ thống nhúng bao gồm thu thập hay ghi nhận các thông tin về môi trường (thông tin đầu vào của hệ thống), xử lý thông tin theo yêu cầu thiết kế và tạo ra tín hiệu hay một tác động ngược lại môi trường (thông tin đầu ra). Thông thường, thông tin đầu ra sẽ được tiếp tục kiểm tra thông qua **cơ chế phản hồi** (*feedback*) để giúp cho hệ thống hoạt động liên tục.

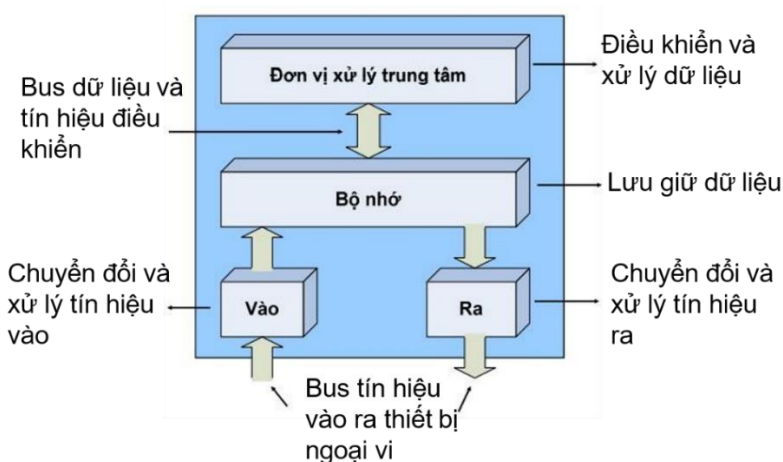
Các thông tin đầu môi trường đầu vào có thể là nhiệt độ, độ ẩm, ánh sáng, chuyển động hay một thao tác điều khiển của người dùng. Các tín hiệu này được ghi nhận thông qua một bộ phận gọi là **cảm biến** (*sensor*). Tín hiệu từ cảm biến được truyền đến **Thiết bị xử lý** để thực thi các tính toán và đưa ra các tín hiệu điều khiển tương ứng ở đầu ra. Tín hiệu điều khiển đầu ra của thiết bị xử lý sẽ điều khiển một **cơ cấu chấp hành** (*actuator*) để tạo ra những thay đổi cần thiết đến môi trường nhằm đáp ứng yêu cầu thiết kế của hệ

thống. Cơ cấu chấp hành có thể là động cơ, đèn, thiết bị phát nhiệt, loa phát để tạo ra thay đổi ở môi trường hay màn hình hiển thị để tạo tương tác với người dùng.

Để dễ hình dung, chúng ta xem xét ví dụ máy lạnh trong một căn phòng là một hệ thống nhúng. Trong máy lạnh có các cảm biến để theo dõi nhiệt độ từ môi trường cùng với cảm biến để ghi nhận tín hiệu điều khiển từ người dùng để thiết lập mức nhiệt độ yêu cầu. Giả sử máy lạnh được thiết lập ở 21°C và nhiệt độ phòng hiện tại là 30°C . Từ thông tin nhiệt độ phòng và mức nhiệt độ được thiết lập, một bo mạch điều khiển (thiết bị xử lý) được tích hợp bên trong sẽ tính toán để quyết định rằng nên mở động cơ bơm hơi lạnh vào phòng. Động cơ bơm hơi lạnh là một cơ cấu chấp hành trong trường hợp này. Nhiệt độ trong phòng tiếp tục được ghi nhận bằng cảm biến (cơ chế phản hồi) và khi nhiệt độ xuống dưới 21°C thì động cơ được tắt.

Hình 1.5 mô tả thiết kế của một thiết bị xử lý trong hệ thống nhúng đơn giản với bốn thành phần chính gồm **đơn vị xử lý trung tâm** (bộ vi xử lý, processor), **bộ nhớ** và các thành phần **đầu vào và đầu ra** (hay còn gọi là thành phần nhập xuất). Trong đó, bộ nhớ là nơi lưu trữ chương trình điều khiển đã được thiết kế từ trước. Bộ vi xử lý sẽ thực thi chương trình điều khiển để có thể xử lý tín hiệu ở đầu vào để tạo ra kết quả ở đầu ra tương ứng. Kết nối giữa bộ vi xử lý, bộ nhớ và các thành phần nhập xuất là hệ thống các bus kết nối.

Thông thường có ba loại bus cơ bản: bus điều khiển, bus dữ liệu và bus địa chỉ. Lưu ý thêm rằng kích thước bus dữ liệu sẽ quyết định đến năng lực tính toán của vi xử lý và khi nhắc đến vi xử lý 8-bit, 16-bit hay 32-bit thì các con số ở đây là biểu thị cho kích thước của bus dữ liệu. Trong một hệ thống nhúng tổng thể, đầu vào của thiết bị xử lý được kết nối với đầu ra của thiết bị cảm biến và đầu ra của sẽ được kết nối đến cơ cấu chấp hành.

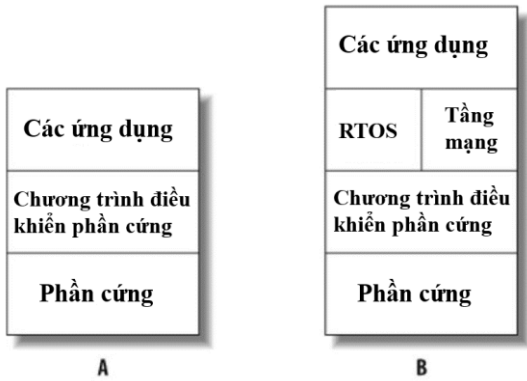


Hình 1.5: Sơ đồ khối thiết bị xử lý trong hệ thống nhúng

Bên cạnh thiết kế phần cứng, chương trình điều khiển là phần quan trọng khác trong hệ thống nhúng. Trong các hệ thống đơn giản, thông thường chỉ có một chương trình điều khiển đã được thiết kế sẵn, được cài đặt trong thiết bị và khi triển khai ứng dụng thì không thể thay đổi chương trình điều khiển. Trong khi đó, để đáp ứng nhu cầu ứng dụng đa dạng và phức tạp, các hệ thống nhúng cần cho phép cài đặt nhiều loại ứng dụng, thay đổi các cài đặt và cập nhật

chương trình điều khiển... Các tính năng này thông thường cần sự hỗ trợ của một chương trình phức tạp hơn gọi là **hệ điều hành** (Operating system).

Hình 1.6 mô tả kiến trúc của các hệ thống nhúng phức tạp hơn, cho phép cài đặt các ứng dụng trên hệ thống. Hình 1.6(A) là hệ thống cho phép cài đặt một hay nhiều ứng dụng trên một nền tảng phần cứng. Để các ứng dụng có thể sử dụng (điều khiển) cơ cấu phần cứng một cách nhịp nhàng, hệ thống được cài đặt các chương trình điều khiển phần cứng (device driver) cho phép ứng dụng tương tác với phần cứng một cách thuận tiện. Các chương trình điều khiển phần cứng thường tuân theo các chuẩn giao tiếp để tạo sự thuận lợi cho các kỹ sư thiết kế.



Hình 1.6: Cấu trúc hệ thống nhúng có hệ điều hành và ứng dụng

Tuy nhiên, khi độ phức tạp của hệ thống tăng lên, các hệ thống nhúng được cài đặt một hệ điều hành cung cấp các dịch vụ quản lý ứng dụng và kiểm soát phần cứng hiệu quả để giúp cho người dùng sử dụng các ứng dụng một cách thuận lợi, như thể hiện trong hình

Hình 1.6(B). Có nhiều loại hệ điều hành khác nhau cung cấp đa dạng các dịch vụ, trong đó, **hệ điều hành thời gian thực** (real-time operating system - RTOS) là thông dụng nhất trong các hệ thống nhúng ngày nay. Hệ điều hành thời gian thực cung cấp thời gian đáp ứng hiệu quả cho ứng dụng và tương tác hiệu quả với người dùng. Bên cạnh hệ điều hành, các hệ thống nhúng ngày nay còn được tích hợp các giao thức tầng mạng, giao thức kết nối, cho phép tạo các kết nối liên thiết bị, kết nối với mạng Internet và hình thành các mạng lưới thiết bị trao đổi dữ liệu liên tục và thuận tiện.

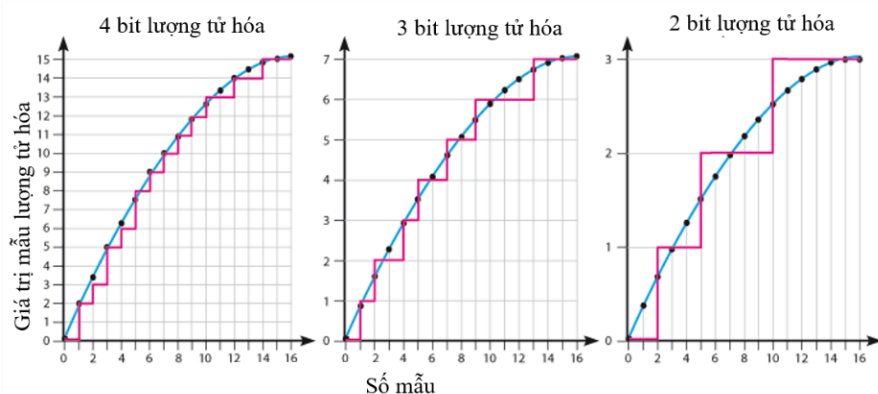
1.4 Chuyển đổi giữa tín hiệu tương tự và tín hiệu số

1.4.1 Chuyển đổi từ tín hiệu tương tự sang tín hiệu số

Như đã đề cập ở phần trên, tín hiệu đầu vào cho các thiết bị điều khiển là tín hiệu đầu ra từ các cảm biến. Trên thực tế, đầu ra của các cảm biến là các **tín hiệu tương tự** (*analog signal*) dưới dạng điện áp hay dòng điện. Trong khi đó, thiết bị xử lý thực hiện các tính toán trên **tín hiệu số** (*digital signal*). Do đó, để thiết bị xử lý có thể thu nhận và xử lý thông tin từ cảm biến, một thành phần chuyển đổi cần thiết được sử dụng gọi là bộ chuyển đổi tín hiệu tương tự sang tín hiệu số (Analog-to-Digital Converter - ADC).

ADC giúp tạo ra tín hiệu số có giá trị thay đổi tương ứng với thay đổi giá trị ở đầu vào. Lý thuyết về ADC thì khá là phức tạp tùy thuộc vào chức năng và độ chính xác của nó. Giáo trình này không đi sâu giới thiệu về lý thuyết ADC và người đọc có thể tham khảo

thêm lý thuyết về ADC trong các tài liệu đính kèm [1-3]. Một số thông số quan trọng khi sử dụng ADC gồm có: tốc độ lấy mẫu (*sampling rate*), độ phân giải (*resolution*), thời gian chuyển đổi (*conversion rate*) và lỗi chuyển đổi (*conversion error*). Hình 1.7 trình bày minh họa sự chuyển đổi tín hiệu tương tự (đường màu liền nét) sang tín hiệu số (đường bậc thang). Với số bit lượng tử khác nhau, một tín hiệu tương tự có thể được chuyển đổi sang tín hiệu số với số lượng mẫu và giá trị lấy mẫu khác nhau. Tập hợp các mẫu phản ánh độ chính xác của tín hiệu số so với tín hiệu tương tự.



Hình 1.7: Minh họa lấy mẫu chuyển đổi tín hiệu tương tự sang tín hiệu số

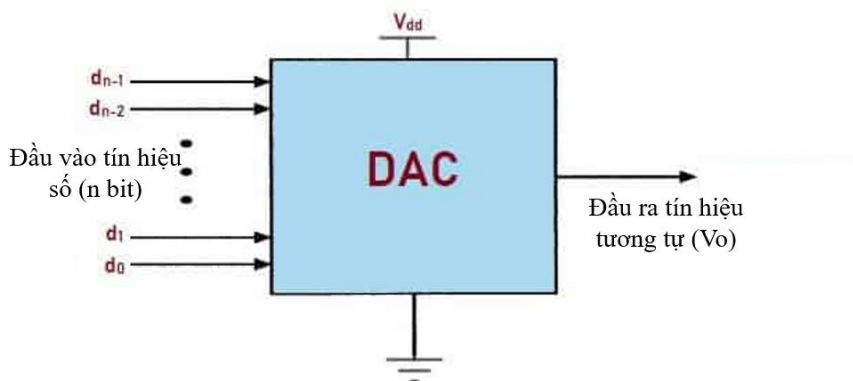
Trong quá trình thiết kế, người thiết kế cần tìm hiểu kỹ yêu cầu của thiết kế và các thông số kỹ thuật quan trọng của ADC để từ đó có thể chọn lựa một ADC phù hợp. Có nhiều loại ADC trên thị trường với các độ phân giải khác nhau, từ các loại có độ phân giải thấp 4-bit, 8-bit cho đến các ADC có độ phân giải cao 12-bit, 24-bit. Các ADC có độ phân giải bit càng lớn thì độ chính xác càng

cao và khả năng phản ánh tín hiệu tương tự thực tế tốt hơn. Tuy nhiên, các ADC có độ phân giải cao cũng tương ứng với giá thành cao, nhạy với nhiễu và đồng thời tạo ra dữ liệu lớn gây áp lực tính toán lên thiết bị xử lý và bộ nhớ.

Thông thường, các cảm biến được tích hợp với ADC để tạo thành các mô-dun cảm biến thuận tiện cho người thiết kế. Trong các trường hợp khác, người thiết kế có thể sử dụng chip ADC rời hoặc ADC được tích hợp trong các vi điều khiển. Một số loại chip ADC thông dụng như PCF8591 8-bit, MCP3208 12-bit, AD7327 12-bit, ADS1216Y 24-bit.

1.4.2 Chuyển đổi từ tín hiệu số sang tín hiệu tương tự

Tương tự với việc chuyển đổi tín hiệu từ tương tự sang tín hiệu số ở đầu vào của thiết bị xử lý, một hệ thống nhúng thường cũng cần chuyển đổi từ tín hiệu số sang tín hiệu tương tự ở đầu ra của thiết bị xử lý. Cụ thể, các cơ cấu chấp hành thông thường hoạt động dựa trên tín hiệu điện áp và dòng điện. Do đó, chúng ta cần sử dụng bộ chuyển đổi từ tín hiệu số (tín hiệu điều khiển ở đầu ra của thiết bị xử lý) sang tín hiệu điện áp hay dòng điện. Hình 1.8 mô tả chức năng chuyển đổi từ tín hiệu số (n bit) sang tín hiệu tương tự dạng điện áp.



Hình 1.8: Chuyển đổi tín hiệu số sang tín hiệu tương tự

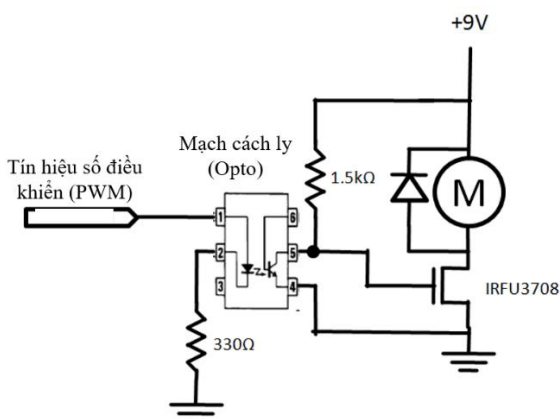
1.4.3 Mạch công suất và mạch cách ly

Chúng ta thấy rằng trong một hệ thống nhúng thường có hai trường tín hiệu khác nhau: tín hiệu số và tín hiệu tương tự. Tín hiệu số được sử dụng chủ yếu trong thiết bị xử lý, có điện áp và dòng thấp. Trong khi đó, tín hiệu tương tự xuất hiện trên thiết bị cảm biến và cơ cấu chấp hành. Đặc biệt ở cơ cấu chấp hành, điện áp và dòng có thể lớn hơn rất nhiều lần so với mức điện áp và dòng được sử dụng cho thiết bị xử lý.

Cơ cấu chấp hành đóng vai trò như một tải trong hệ thống. Ví dụ như một hệ thống điều khiển một mạng các đèn LED trong bảng hiệu quảng cáo. Mạng các LED cần tiêu thụ một dòng điện lớn hơn nhiều so với vi điều khiển được sử dụng. Hay trong mạch điều khiển động cơ, bên cạnh việc yêu cầu một mức điện áp và dòng điện lớn, một số trường hợp động cơ có thể xoay và tạo ra dòng điện phản

hồi ngược về vi điều khiển nếu chúng ta nối trực tiếp động cơ vào chân vi điều khiển.

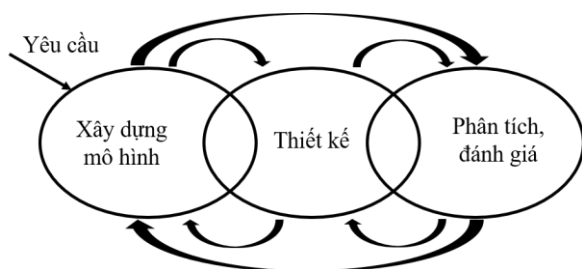
Các vấn đề về điện áp và dòng lớn có thể gây hư hỏng trên thiết bị xử lý. Trong thiết kế hệ thống nhúng, để tránh các tác động có hại trên, thông thường các mạch công suất được thiết kế riêng để cấp nguồn cho các cơ cấu chấp hành và một thiết bị cách ly (*isolator*) được sử dụng để cách ly mạch công suất này và thiết bị xử lý. Hình 1.9 trình bày một ví dụ về mạch điều khiển động cơ DC 9V (động cơ M) được điều khiển bởi tín hiệu số được điều chế xung (PWM). Tín hiệu điều khiển được cấp từ thiết bị xử lý và được cách ly với mạch điều khiển động cơ thông qua một thiết bị gọi là opto. Opto giúp bảo vệ vi điều khiển và các mạch xử lý số khỏi những tác hại từ mạch công suất lớn hơn ở ngõ ra.



Hình 1.9: Mạch điều khiển động cơ sử dụng opto cách ly

1.5 Quy trình thiết kế

Quy trình thiết kế hệ thống nhúng tiêu chuẩn gồm ba giai đoạn chính: xây dựng mô hình, thiết kế và phân tích/đánh giá, như được thể hiện Hình 1.10. **Xây dựng mô hình** (modeling) là quá trình làm việc để đạt được những hiểu biết sâu hơn về hệ thống thông qua việc mô phỏng, hay giả lập. Các mô hình giả lập hoạt động của hệ thống và phản ánh các đặc tính của hệ thống đó. Quá trình này giúp kiểm tra được tính năng của hệ thống và sự đáp ứng đối với yêu cầu đặt ra của đề bài. Quá trình xây dựng mô hình tạo ra một sơ đồ kết nối các thành phần cơ bản của hệ thống gọi là mạch nguyên lý (*schematic*). Matlab, Proteus, Altium là một số các phần mềm phổ biến được dùng để thiết kế mạch nguyên lý và mô phỏng hoạt động của các hệ thống nhúng với thư viện hỗ trợ đa dạng các loại vi điều khiển và linh kiện. Sơ đồ mạch nguyên lý tập trung vào mô tả danh sách các thành phần trong mạch và kết nối giữa các thành phần mà không đề cập đến vị trí bố trí, kích thước cụ thể của các thành phần trong thực tế.



Hình 1.10: Các thành phần cơ bản của quá trình thiết kế hệ thống nhúng CPS

Thiết kế (designing) là quá trình tạo ra cấu trúc của hệ thống thực tế. Quá trình này đặc tả cách thức cụ thể mà hệ thống sẽ hoạt động để đảm bảo các đặc tính như được mô tả trong phần xây dựng mô hình. Quá trình này liên quan đến việc thiết kế mạch in (Printed Circuit Board - PCB), bố trí linh kiện, thiết kế mạch nguồn, kết nối thiết bị và lập trình điều khiển hệ thống. Quá trình thiết kế mạch in từ mạch nguyên lý (được thiết kế trong giai đoạn xây dựng mô hình) được hỗ trợ bởi nhiều phần mềm như Altium, Kicad. Quá trình thiết kế quan tâm đến vị trí bố trí, kích thước các linh kiện, chi tiết kết nối các linh kiện và hoạt động thực tế của sản phẩm. Khác với sơ đồ mạch nguyên lý được mô phỏng dựa vào dữ liệu giả lập, quá trình thiết kế sử dụng thông tin đầu vào là tín hiệu thực tế từ môi trường. Kết quả hoạt động thực tế có thể khác nhiều kết quả mô phỏng và cần nhiều tinh chỉnh để hệ thống hoạt động đúng yêu cầu.

Cuối cùng, phân tích và đánh giá là bước quan trọng cần được thực hiện để kiểm tra được mức độ chính xác của thiết kế. Phân tích là quá trình làm việc để hiểu sâu hơn hệ thống thông qua việc phân tách các thành phần của hệ thống và đánh giá hoạt động của từng thành phần. Quá trình này quyết định tính đúng đắn, hoặc lỗi của hệ thống để thay đổi hay cải tiến mô hình và thiết kế.

Cả ba quá trình xây dựng mô hình, thiết kế và phân tích/đánh giá có mối liên kết chặt chẽ và tác động qua lại. Trong quá trình phát triển một hệ thống nhúng, các bước trên có mức độ đầu tư về

thời gian và công sức khác nhau tùy thuộc vào đối tượng xử lý, điều kiện môi trường và mức độ yêu cầu chính xác của hệ thống.

1.6 Tổng kết

Hệ thống nhúng đang ngày càng phát triển nhanh chóng cả về số lượng, sự đa dạng và độ phức tạp. Một hệ thống nhúng về cơ bản gồm 3 thành phần chính: thành phần cảm biến, thiết bị xử lý và cơ cấu chấp hành. Thành phần cảm biến giúp đo lường các thông số môi trường phản ánh vào tín hiệu đầu vào cho thiết bị xử lý. Thiết bị xử lý là nơi thực thi các tính toán, áp dụng các giải thuật cần thiết để đưa ra những quyết định hay tín hiệu điều khiển cần thiết phản ánh yêu cầu thiết kế của hệ thống. Cuối cùng, cơ cấu chấp hành giúp tạo ra thay đổi cần thiết đến môi trường hoặc tương tác với người dùng.

Đặc biệt trong xu thế phát triển của IoT, các thiết bị nhúng càng trở nên phong phú và đóng vai trò quan trọng trong cuộc sống của chúng ta. Hệ thống nhúng có vai trò quan trọng và là cơ sở phát triển các thiết bị được kết nối trong hệ thống IoT. Để đáp ứng được yêu cầu của IoT, các hệ thống nhúng dần được thiết kế với năng lực tính toán cao hơn, tích hợp nhiều tính năng phức tạp và thông minh hơn.

Chương 2 VI XỬ LÝ NHÚNG

Như được giới thiệu trong Chương 1, thiết bị xử lý là thành phần quan trọng nhất để thực hiện các tính toán và xử lý dữ liệu trong một hệ thống nhúng. Để thực hiện được chức năng này, thiết bị xử lý được thiết kế sử dụng vi xử lý nhúng hay vi điều khiển được kết nối với các thành phần linh kiện khác nhau. Vi xử lý hay vi điều khiển có thể xem là bộ não của một hệ thống nhúng.

Trong các hệ thống tính toán đa dụng ngày nay, sự đa dạng của các kiến trúc tập lệnh bị hạn chế, trong đó kiến trúc x86 của Intel chiếm ưu thế hoàn toàn. Trái lại, trong thế giới của hệ thống nhúng, không hề có một sự thống trị nào như vậy và các kiến trúc tập lệnh vô cùng đa dạng. Trong chương này, chúng ta sẽ tìm hiểu cơ bản về các thuộc tính của bộ vi xử lý nhúng. Chúng ta các cơ chế cung cấp sự đồng thời và kiểm soát thời gian bởi vì những vấn đề này xuất hiện rất nhiều trong việc thiết kế các hệ thống nhúng, đặc biệt là đối với hệ thống IoT.

Khi được triển khai trong một sản phẩm, các bộ vi xử lý nhúng thường có một chức năng chuyên dụng. Chúng có thể điều khiển động cơ ô tô hoặc đo độ dày băng ở Bắc Cực. Chúng không được yêu cầu thực hiện các chức năng tùy ý bằng phần mềm do người dùng định nghĩa. Do đó, các bộ vi xử lý nhúng có thể chuyên biệt hơn giúp tiết kiệm năng lượng hay có thể được tích hợp với các phần cứng chuyên dụng để tăng tốc độ xử lý.

Khi đánh giá bộ vi xử lý nhúng, điều quan trọng là phải hiểu sự khác biệt giữa kiến trúc tập lệnh (Instruction Set Architecture - ISA) và khả năng nhận biết bộ vi xử lý nhúng hoặc chip. Khả năng nhận biết bộ vi xử lý nhúng phụ thuộc bên ngoài của chip được thiết kế và chế tạo bởi nhà cung cấp chất bán dẫn. Còn ISA là định nghĩa về các lệnh mà bộ vi xử lý có thể thực thi và các ràng buộc cấu trúc nhất định cho việc thực thi. x86 hay ARM-7 là những ví dụ điển hình về ISA.

Các ISA thường được chia sẻ trong một nhóm các bộ vi xử lý. Lợi thế của việc chia sẻ ISA này là các công cụ phần mềm có thể được chia sẻ và (đôi khi) các chương trình giống nhau có thể chạy chính xác trên nhiều phiên bản vi xử lý. Tuy nhiên, các ISA không có ràng buộc về thời gian, kiến trúc thực thi của các bộ vi xử lý là khác nhau và môi trường ứng dụng của hệ thống nhúng là khác nhau. Do đó, mặc dù một chương trình có thể thực thi giống nhau về mặt luận lý nhưng hiệu suất có thể là khác nhau khi thực thi trên các hệ thống nhúng khác nhau.

2.1 Các loại bộ vi xử lý nhúng

Do sự đa dạng của các ứng dụng nhúng, có rất nhiều bộ vi xử lý nhúng khác nhau được sử dụng. Vi xử lý nhúng có thể là loại rất nhỏ, chậm, rẻ tiền, công suất thấp, cho đến các loại được thiết kế với mục đích đặc biệt, tốc độ nhanh và hiệu suất cao. Trong phần

này, chúng ta sẽ tìm hiểu tổng quan về một số bộ vi xử lý nhúng phổ biến.

2.1.1 Bộ vi xử lý tín hiệu số

Nhiều ứng dụng nhúng thực hiện khá nhiều xử lý tín hiệu. Tín hiệu là một tập hợp các giá trị lấy mẫu của thế giới vật lý. Ví dụ, một ứng dụng điều khiển chuyển động có thể đọc vị trí hoặc thông tin vị trí từ các cảm biến với tốc độ lấy mẫu từ vài Hertz (Hz hoặc mẫu mỗi giây) đến vài trăm Hertz. Tín hiệu âm thanh được lấy mẫu ở các tốc độ từ 8.000 Hz (hoặc 8 KHz) đến 44,1 KHz. Các ứng dụng siêu âm (chẳng hạn như hình ảnh y tế) và các ứng dụng âm nhạc hiệu suất cao có thể lấy mẫu tín hiệu âm thanh ở tốc độ cao hơn nhiều. Video thường sử dụng tốc độ lấy mẫu từ 25 Hz hoặc 30 Hz cho các thiết bị tiêu dùng đến tốc độ cao hơn nhiều cho các ứng dụng đo lường đặc biệt. Tất nhiên, mỗi mẫu chứa toàn bộ một hình ảnh (gọi là khung), bản thân nó có nhiều mẫu (gọi là pixel) được phân bố theo không gian chứ không phải theo thời gian. Các ứng dụng vô tuyến được định nghĩa bằng phần mềm có tốc độ mẫu có thể nằm trong khoảng từ hàng trăm đến hàng ngàn KHz. Các ứng dụng nhúng khác sử dụng nhiều xử lý tín hiệu bao gồm các trò chơi tương tác; hệ thống hình ảnh radar, sonar, và LIDAR (phát hiện ánh sáng và khoảng cách); phân tích video; hệ thống hỗ trợ lái xe ô tô; điện tử y tế; và thiết bị đo đạc khoa học.

Các ứng dụng xử lý tín hiệu số đều có những đặc điểm chung nhất định. Thứ nhất, chúng xử lý một lượng lớn dữ liệu. Dữ liệu có thể đại diện cho các mẫu theo thời gian của bộ vi xử lý vật lý, mẫu trong không gian hoặc cả. Thứ hai, các ứng dụng dạng này thường thực hiện các phép toán phức tạp trên dữ liệu, bao gồm lọc, nhận dạng hệ thống, phân tích tần số, học máy và trích xuất tính năng. Các phép toán này chuyên sâu về mặt toán học.

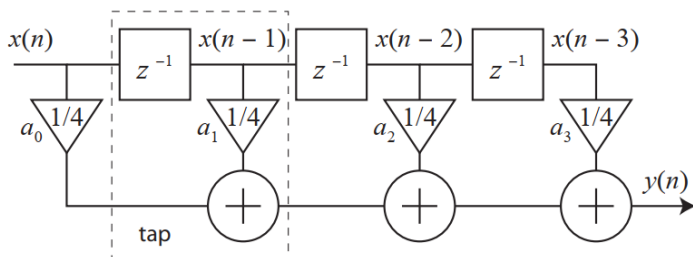
Các bộ vi xử lý được thiết kế đặc biệt để hỗ trợ các ứng dụng xử lý tín hiệu số chuyên sâu được gọi là bộ vi xử lý tín hiệu số (Digital Signal Processing -DSP). Để có cái nhìn sâu sắc về cấu trúc của các bộ vi xử lý như vậy và ý nghĩa đối với kỹ sư thiết kế phần mềm nhúng, chúng ta cần tìm hiểu cấu trúc của các thuật toán xử lý tín hiệu điển hình.

Một thuật toán xử lý tín hiệu phổ biến là lọc đáp ứng xung hữu hạn (Finite Impulse Response - FIR). Dạng thuật toán này đơn giản nhưng có ý nghĩa đối với phần cứng. Ở dạng đơn giản nhất này, tín hiệu đầu vào x bao gồm một chuỗi các giá trị số rất dài gần như là vô hạn. Một đầu vào như vậy có thể được mô hình hóa như một hàm $x: \mathbb{N} \rightarrow D$, trong đó D là tập hợp các giá trị trong một số kiểu dữ liệu. Ví dụ: D có thể là tập hợp tất cả các số nguyên 16 bit. Khi đó, $x(0)$ là giá trị đầu vào đầu tiên (số nguyên 16 bit), $x(1)$ là giá trị đầu vào thứ hai, v.v. Để thuận tiện cho toán học, chúng ta có thể tăng giá trị này thành $x: \mathbb{Z} \rightarrow D$ bằng cách xác định $x(n) = 0$ với

mọi $n < 0$. Đối với mỗi giá trị đầu vào $x(n)$, bộ lọc FIR phải tính giá trị đầu ra $y(n)$ theo công thức sau:

$$y(n) = \sum_{i=0}^{N-1} a_i x(n-i) \quad (2.1)$$

trong đó N là độ dài của bộ lọc FIR và các hệ số a_i được gọi là các giá trị **tap** của nó. Từ công thức này, bạn có thể thấy lý do tại sao nó hữu ích khi tăng miền của hàm x , vì ví dụ, phép tính $y(0)$ liên quan đến các giá trị $x(-1)$, $x(-2)$.



Hình 2.1: Cấu trúc một hiện thực của bộ lọc FIR

Tỷ lệ mà các giá trị đầu vào $x(n)$ được cung cấp và phải được xử lý được gọi là tỷ lệ mẫu (sample rate). Nếu bạn biết tỷ lệ mẫu và N , bạn có thể xác định số phép tính số học phải tính trong mỗi giây.

Một hình ảnh có thể được mô hình hóa tương tự như một hàm $x: H \times V \rightarrow D$, trong đó $H \subset N$ đại diện cho chỉ số ngang, $V \subset N$ đại diện cho chỉ số dọc và D là tập hợp tất cả các giá trị pixel có thể có. Pixel (hoặc phần tử ảnh) là một mẫu đại diện cho màu sắc và cường độ của một điểm trong hình ảnh. Có nhiều cách để thực hiện việc này, nhưng tất cả đều sử dụng một hoặc nhiều giá trị số cho mỗi pixel. Tập hợp H và V phụ thuộc vào độ phân giải của hình ảnh.

Ví dụ 2.1: Giả sử $N = 4$ và $a_0 = a_1 = a_2 = a_3 = 1/4$. Khi đó $\forall n \in N$, ta có :

$$y(n) = (x(n) + x(n-1) + x(n-2) + x(n-3))/4$$

Mỗi mẫu đầu ra là giá trị trung bình của bốn mẫu đầu vào gần nhất. Cấu trúc của phép tính này được thể hiện trong Hình 2.2. Trong hình đó, các giá trị đầu vào đến từ bên trái và truyền xuống đường trễ, được khai thác sau mỗi phần tử trễ. Cấu trúc này được gọi là đường trễ nhẹ.

Ví dụ 2.2: Giả sử rằng một bộ lọc FIR được cung cấp với các mẫu ở tốc độ 1 MHz (một triệu mẫu mỗi giây) và $N = 32$. Sau đó, các đầu ra phải được tính toán ở tốc độ 1 MHz và mỗi đầu ra yêu cầu 32 phép nhân và 31 phép cộng. Bộ vi xử lý phải có khả năng duy trì tốc độ tính toán là 63 triệu phép tính số học mỗi giây để triển khai ứng dụng này. Tất nhiên, để duy trì tốc độ tính toán, không chỉ cần phần cứng số học đủ nhanh mà còn cả các cơ chế lấy dữ liệu vào và ra khỏi bộ nhớ và bật và tắt chip phải đủ nhanh.

Đối với hình ảnh thang độ xám, thao tác lọc thông thường sẽ xây dựng hình ảnh mới y từ hình ảnh ban đầu x theo công thức sau:

$$\forall i \in H, j \in V, y(i, j) = \sum_{n=-N}^N \sum_{m=-M}^M a_{n,m} x(i-n, j-m) \quad (2.2)$$

Ví dụ 2.3: Truyền hình tương tự đang dần được thay thế bởi các định dạng kỹ thuật số như ATSC, một bộ tiêu chuẩn do Ủy ban Hệ thống Truyền hình Tiên tiến phát triển. Tại Hoa Kỳ, phần lớn các đường truyền NTSC “qua không khí” (Ủy ban Hệ thống Truyền hình Quốc gia) đã được thay thế bằng ATSC vào ngày 12 tháng 6 năm 2009. ATSC hỗ trợ một số tốc độ khung hình từ chỉ dưới 24 Hz đến 60 Hz và một số các giải pháp. Ví dụ, video độ nét cao theo tiêu chuẩn ATSC (Advanced Television System Committee) hỗ trợ độ phân giải 1080 x 1920 pixel ở tốc độ khung hình 30 Hz. Do đó, $H = \{0, \dots, 1919\}$ và $V = \{0, \dots, 1079\}$. Độ phân giải này được gọi là 1080p trong ngành. Thiết bị video chuyên nghiệp ngày nay tăng gấp bốn lần độ phân giải này (4320 x 7680). Tốc độ khung hình cũng có thể cao hơn nhiều so với 30 Hz. Tốc độ khung hình rất cao rất hữu ích để chụp các hiện tượng cực nhanh trong chuyển động chậm.

trong đó $a_{n,m}$ là các hệ số lọc. Đây là bộ lọc FIR hai chiều. Một phép tính như vậy yêu cầu xác định x bên ngoài vùng $H \times V$. Có một nghệ thuật đối với điều này (để tránh hiệu ứng cạnh), nhưng đối với mục đích của chúng ta ở đây, đủ để hiểu cấu trúc của tính toán mà không cần quan tâm đến chi tiết này.

Một hình ảnh màu sẽ có nhiều kênh màu. Chúng có thể đại diện cho độ chói (độ sáng của pixel) và sắc độ (màu của pixel là gì) hoặc chúng có thể đại diện cho các màu có thể được cấu tạo để có được màu tùy ý. Trong trường hợp thứ hai, một lựa chọn phổ biến là định dạng RGBA, có bốn kênh đại diện cho màu đỏ, xanh lục, xanh lam và kênh alpha, thể hiện độ trong suốt. Ví dụ: giá trị 0 cho R, G và B đại diện cho màu đen. Giá trị 0 của A thể hiện hoàn toàn trong suốt (vô hình). Mỗi kênh cũng có một giá trị tối đa, chẳng hạn như 1.0. Nếu cả bốn kênh đều ở mức tối đa, màu thu được là màu trắng đục hoàn toàn.

Tải tính toán của bộ lọc trong công thức (2.2 phụ thuộc vào số kênh, hệ số lọc (giá trị của N và M), độ phân giải (kích thước của bộ H và V), và tốc độ khung hình.

Ngoài một số lượng lớn các phép toán số học, bộ vi xử lý còn phải xử lý sự di chuyển của dữ liệu xuống đường trễ như trong Hình 2.1. Bằng cách cung cấp hỗ trợ cho các đường trễ và lệnh nhân tích lũy, bộ vi xử lý DSP có thể nhận ra một *tap* của bộ lọc FIR trong một chu kỳ. Trong chu kỳ đó, hai số đầu vào được nhân với nhau, cộng kết quả vào bộ tích lũy và tăng hoặc giảm hai con trỏ bằng cách sử dụng số học mô-đun.

Ví dụ 2.4: Giả sử rằng hoạt động lọc như công thức (với $N = 1$ và $M = 1$ (giá trị nhỏ nhất cho bộ lọc hữu dụng) được thực hiện trên tín hiệu video độ nét cao như trong Ví dụ 2.3. Sau đó, mỗi pixel của hình ảnh đầu ra y yêu cầu thực hiện 9 phép nhân và 8 phép cộng. Giả sử chúng ta có một hình ảnh màu với ba kênh (giả sử, RGB, không có độ trong suốt), thì điều này sẽ cần thực hiện 3 lần cho mỗi pixel. Do đó, mỗi khung ảnh thu được sẽ yêu cầu $1080 \times 1920 \times 3 \times 9 = 55,987,200$ phép nhân và một số phép cộng tương tự. Ở tốc độ 30 khung hình/giây, điều này chuyển thành 1,679,616,000 phép nhân mỗi giây và một số phép cộng tương tự. Vì đây là hoạt động đơn giản nhất mà người ta có thể thực hiện trên tín hiệu video độ nét cao, chúng ta có thể thấy rằng kiến trúc bộ vi xử lý xử lý tín hiệu video như vậy thực sự phải khá nhanh.

2.1.2 Bộ vi điều khiển

Vi điều khiển (Microcontroller - μC) là một máy tính nhỏ trên một mạch tích hợp duy nhất bao gồm bộ vi xử lý trung tâm tương đối đơn giản (CPU) kết hợp với các thiết bị ngoại vi như bộ nhớ, thiết bị nhập xuất và bộ định thời. Các khảo sát cho thấy hơn một nửa tổng số CPU được bán trên toàn thế giới là vi điều khiển. Các bộ vi điều khiển đơn giản nhất là các vi điều khiển 8-bit và thích hợp cho các ứng dụng có kích thước bộ nhớ nhỏ và các yêu cầu tính toán đơn. Chúng tiêu thụ năng lượng cực kỳ nhỏ và thường hỗ trợ chế độ ngủ (sleep mode) để giảm tiêu thụ điện năng. Các thành phần

nhúng như mạng cảm biến và thiết bị giám sát đã được chứng minh có thể hoạt động bằng pin nhỏ trong vài năm.

Bộ vi điều khiển có thể khá phức tạp về mặt kiểu dáng, chủng loại, kiến trúc tập lệnh, số lượng ngoại vi được tích hợp và tính năng hoạt động. Việc phân biệt chúng với các bộ vi xử lý đa dụng khá là khó khăn. Ví dụ, Intel Atom là một họ CPU x86 được sử dụng chủ yếu trong netbook và các máy tính di động nhỏ khác. Các bộ vi xử lý này được thiết kế để sử dụng tương đối ít năng lượng mà không làm giảm quá nhiều hiệu suất so với các bộ vi xử lý được sử dụng trong các máy tính cao cấp hơn. Các chip Intel Atom được sử dụng khá phổ biến trong các ứng dụng nhúng và trong các máy chủ. AMD's Geode là một ví dụ khác về bộ vi xử lý nhúng gần với bộ vi xử lý đa dụng.

Hầu hết các nhà cung cấp chất bán dẫn bao gồm một hoặc nhiều họ vi điều khiển (VĐK) trong dòng sản phẩm của họ. Một số kiến trúc khá cũ như Motorola 6800 và Intel 8080 8-bit được giới thiệu từ năm 1974. Các dòng VĐK của những kiến trúc này vẫn tồn tại cho đến ngày nay.

Một kiến trúc rất phổ biến và bền bỉ khác là Intel 8051, một VĐK 8 bit do Intel phát triển vào năm 1980. 8051 ISA ngày nay được hỗ trợ bởi nhiều nhà cung cấp, bao gồm Atmel, Infineon Technologies, Dallas Semiconductor, NXP, ST Microelectronics, Texas Instruments và Cypress Semiconductor. Bộ vi điều khiển

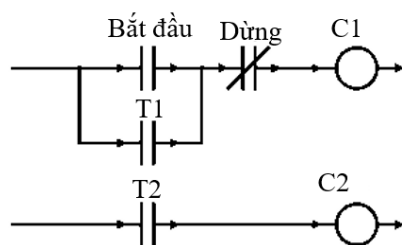
Atmel AVR 8-bit, được phát triển bởi Atmel vào năm 1996, là một trong những bộ vi điều khiển đầu tiên sử dụng bộ nhớ flash trên chip để lưu trữ chương trình.

Nhiều bộ VDK 32 bit dựa trên tập lệnh ARM được phát triển bởi ARM Limited. ARM ban đầu là viết tắt của Advanced RISC Machine, và trước đó là Acorn RISC Machine, nhưng ngày nay nó chỉ đơn giản là ARM. Các bộ vi xử lý kiến trúc ARM được sử dụng rộng rãi trong điện thoại di động và nhiều hệ thống nhúng khác. Các nhà cung cấp chất bán dẫn được cấp bản quyền tập lệnh từ ARM Limited và sản xuất chip của riêng họ. Các bộ vi xử lý ARM hiện được sản xuất bởi Alcatel, Atmel, Broadcom, Cirrus Logic, Freescale, LG, Marvell Technology Group, NEC, NVIDIA, NXP, Samsung, Sharp, ST Microelectronics, Texas Instruments, VLSI Technology và nhiều đơn vị khác.

Các kiến trúc VDK nhúng đáng chú ý khác bao gồm Motorola ColdFire, Hitachi H8 và SuperH, MIPS, PIC, và PowerPC.

2.1.3 Bộ điều khiển khả lập trình

Bộ điều khiển luận lý khả lập trình (Programmable Logic Controller-PLC) là một dạng VDK chuyên dụng cho tự động hóa công nghiệp. PLC có nguồn gốc là sự thay thế cho các mạch điều khiển sử dụng rơ le điện để điều khiển máy móc. Chúng thường được thiết kế để hoạt động liên tục trong các môi trường khắc nghiệt (nhiệt độ cao, độ ẩm, bụi, v.v.).



Hình 2.2: Sơ đồ logic bậc thang

PLC thường được lập trình bằng cách sử dụng logic bậc thang, một khái niệm ban đầu được sử dụng để chỉ định logic được xây dựng bằng rơ le và công tắc. Rơ le là một công tắc mà tiếp điểm được điều khiển bằng cuộn dây. Khi một điện áp được đặt vào cuộn dây, tiếp điểm sẽ đóng lại, tạo điều kiện cho dòng điện chạy qua rơ le. Bằng cách kết nối các tiếp điểm và cuộn dây, rơ le có thể được sử dụng để xây dựng các bộ điều khiển kỹ thuật số tuân theo các mẫu được chỉ định.

Thông thường, ký hiệu hai thanh dọc biểu thị một tiếp điểm và vòng tròn biểu thị cho cuộn dây. Sơ đồ trên có hai bậc. Cuộn dây C2 ở bậc dưới sẽ bật hoặc tắt động cơ. Hai tiếp điểm *Bắt đầu* và *Dừng* đại diện cho các công tắc nút bấm. Khi người vận hành nhấn nút *Bắt đầu*, tiếp điểm sẽ đóng và dòng điện có thể chạy từ bên trái (đường ray điện) sang bên phải (mặt đất). *Bắt đầu* là một tiếp điểm thường mở. Tiếp điểm *Dừng* thường đóng, được biểu thị bằng dấu gạch chéo, nghĩa là nó sẽ mở khi người vận hành ấn ngắt điện. Nguyên lý hoạt động ở bậc trên rất thú vị. Khi người vận hành nhấn *Bắt đầu*, dòng điện chạy đến cuộn dây C1, làm cho cả hai tiếp điểm

T1 và *T2* đóng lại. Động cơ sẽ chạy, ngay cả khi đã thả nút *Bắt đầu*. Khi người vận hành nhấn *Dừng*, dòng điện bị ngắt và cả hai tiếp điểm *T1* và *T2* đều mở, khiến động cơ dừng lại. Các tiếp điểm có dây song song thực hiện một hàm luận lý OR và các tiếp điểm có dây nối tiếp thực hiện một luận lý AND. Bậc trên có phản hồi, ý nghĩa của bậc thang là một nghiệm điểm cố định đối với phương trình luận lý mà biểu đồ biểu diễn.

Ngày nay, PLC chỉ là bộ vi điều khiển ở dạng đóng gói thô với giao diện nhập xuất thích hợp cho điều khiển công nghiệp và logic bậc thang là một ký hiệu lập trình đồ họa cho các chương trình. Những sơ đồ này có thể khá phức tạp, với hàng nghìn bậc thang. Chi tiết thêm, người đọc có thể tìm hiểu trong tác phẩm của Kamen (1999) [4].

2.1.4 Kiến trúc x86

Cho đến nay, ISA x86 vẫn là kiến trúc tập lệnh phổ biến nhất đối với các máy tính đa dụng. Thuật ngữ này bắt nguồn từ Intel 8086, một chip vi xử lý 16 bit được Intel giới thiệu vào năm 1978. Một biến thể của 8086, được gọi là 8088, đã được sử dụng trong IBM PC ban đầu và dòng vi xử lý này đã thống trị thị trường PC kể từ đó. Các bộ vi xử lý tiếp theo trong họ này được đặt tên kết thúc bằng “86” và thường được duy trì khả năng tương thích với các dòng vi xử lý trước đó. Intel 80386 là phiên bản 32 bit đầu tiên của tập lệnh này, được giới thiệu vào năm 1985. Ngày nay, thuật ngữ

“x86” thường dùng để chỉ phiên bản 32 bit, với các phiên bản 64 bit được ký hiệu là “x86-64”. Intel Atom, được giới thiệu vào năm 2008, là một bộ vi xử lý x86 với mức tiêu thụ năng lượng giảm đáng kể. Mặc dù chủ yếu nhắm đến netbook và các máy tính di động nhỏ, Intel Atom cũng là một lựa chọn hấp dẫn cho một số ứng dụng nhúng. Kiến trúc x86 cũng đã được triển khai trong các bộ vi xử lý của AMD, Cyrix và một số nhà sản xuất khác.

2.1.5 Bộ vi xử lý DSP

Các kiến trúc máy tính chuyên dụng để xử lý tín hiệu đã có từ khá lâu. Chip DSP đơn xuất hiện đầu tiên vào đầu những năm 1980 với Western Electric DSP1 của Bell Labs, S28211 của AMI, TMS32010 của Texas Instruments, uPD7720 của NEC và một số loại khác. Các ứng dụng ban đầu của các DSP này bao gồm thiết bị truyền âm thanh, tổng hợp giọng nói, đồ họa và bộ điều khiển ổ đĩa.

Đặc điểm chủ yếu của DSP bao gồm một khối tích lũy-nhân phần cứng (hardware multiply-accumulate unit); một số biến thể của kiến trúc Harvard (để hỗ trợ đa dữ liệu đồng thời và nạp chương trình); và các chế độ định địa chỉ hỗ trợ tăng tự động, bộ đệm tròn và định địa chỉ đảo ngược bit (hỗ trợ tính toán FFT). Hầu hết hỗ trợ dữ liệu cố định 16-24 bit, bộ tích lũy (accumulator) thường rộng hơn nhiều (40-56 bit) giúp thực hiện ác lệnh tích lũy-nhân liên tiếp mà không bị tràn. Một số DSP đang trong quá trình nghiên cứu với phần cứng có độ dài bit thay đổi.

Các DSP rất khó lập trình so với các kiến trúc RISC, chủ yếu là do các lệnh chuyên biệt phức tạp, một liên kết mở ra với các kỹ sư phần mềm và các kiến trúc bộ nhớ không đối xứng. Cho đến cuối những năm 1990, các thiết bị này hầu như luôn được lập trình bằng hợp ngữ. Ngay cả ngày nay, các chương trình C vẫn sử dụng rộng rãi các thư viện được mã hóa thủ công bằng hợp ngữ để tận dụng các tính năng bí truyền nhất của kiến trúc.

2.1.6 Bộ vi xử lý đồ họa

Bộ vi xử lý đồ họa (Graphic Processing Unit - GPU) là một bộ vi xử lý chuyên dụng được thiết kế đặc biệt để thực hiện các phép tính cần thiết trong đồ họa máy tính. Những bộ vi xử lý như vậy có từ những năm 1970, khi chúng được sử dụng để xử lý văn bản và đồ họa hình học. GPU hiện đại hỗ trợ đồ họa 3D, đổ bóng và video kỹ thuật số. Các nhà cung cấp GPU phổ biến nhất ngày nay là Intel, NVIDIA và AMD.

Một số ứng dụng nhúng, đặc biệt là trò chơi, rất phù hợp với GPU. Hơn nữa, GPU đã phát triển theo hướng các mô hình lập trình tổng quát hơn, và do đó đã bắt đầu xuất hiện trong các ứng dụng máy tính chuyên sâu khác, chẳng hạn như thiết bị đo đạc. GPU thường khá tốn điện và do đó ngày nay không phù hợp với các ứng dụng nhúng hạn chế năng lượng.

2.2 Xử lý song song

Hầu hết các bộ vi xử lý ngày nay cung cấp nhiều dạng xử lý song song khác nhau. Các cơ chế này ảnh hưởng mạnh mẽ đến thời gian thực hiện một chương trình, vì vậy các kỹ sư thiết kế hệ thống nhúng phải hiểu chúng. Phần này cung cấp tổng quan về một số hình thức xử lý song song cho các kỹ sư thiết kế hệ thống.

2.2.1 Xử lý song song và xử lý đồng thời

Xử lý đồng thời là đặt tính cốt lõi của các hệ thống nhúng. Về mặt khái niệm, một chương trình máy tính được cho là *đồng thời* (concurrency) nếu các phần khác nhau của chương trình thực thi đồng thời. Một chương trình được cho là *song song* (parallel) nếu các phần khác nhau của chương trình thực thi vật lý đồng thời trên các phần cứng riêng biệt (trên các hệ thống đa lõi, đa vi xử lý).

Các chương trình không đồng thời chỉ định một chuỗi các lệnh để thực thi. Một ngôn ngữ lập trình thực hiện thao tác tính toán theo chuỗi các hoạt động được gọi là ngôn ngữ mệnh lệnh. C là một ngôn ngữ mệnh lệnh. Khi sử dụng C để viết các chương trình đồng thời, chúng ta phải bước ra ngoài chính ngôn ngữ đó, điển hình là sử dụng thư viện luồng. Thư viện luồng sử dụng các tiện ích không phải do C cung cấp mà do hệ điều hành và/hoặc phần cứng cung cấp. Java là một ngôn ngữ chủ yếu bắt buộc được mở rộng với các cấu trúc hỗ trợ trực tiếp các luồng. Do đó, người ta có thể viết các

chương trình đồng thời bằng Java mà không cần bước ra ngoài ngôn ngữ này.

Ví dụ 2.5: Xét các câu C sau:

```
double pi, piSquared, piCubed;  
pi = 3,14159;  
piSquared = pi * pi;  
piCubed = pi * pi * pi;
```

Hai câu lệnh gán cuối cùng là độc lập và do đó có thể được thực hiện song song hoặc theo thứ tự ngược lại mà không làm thay đổi hành vi của chương trình. Tuy nhiên, nếu chúng tôi viết chúng như sau, chúng sẽ không còn độc lập nữa:

```
double pi, piSquared, piCubed;  
pi = 3,14159;  
piSquared = pi * pi;  
piCubed = piSquared * pi;
```

Trong trường hợp này, câu lệnh cuối cùng phụ thuộc vào câu lệnh thứ ba theo nghĩa là câu lệnh thứ ba phải hoàn thành việc thực thi trước khi câu lệnh cuối cùng bắt đầu.

Mọi thực thi (đúng) của một chương trình bằng ngôn ngữ mệnh lệnh phải hoạt động như thể các lệnh được thực thi chính xác theo trình tự đã chỉ định. Tuy nhiên, thường có thể thực hiện các lệnh song song hoặc theo một thứ tự khác với lệnh được chỉ định bởi chương trình và vẫn nhận được kết quả đúng tương tự.

Một trình biên dịch có thể phân tích sự phụ thuộc giữa các lệnh trong một chương trình và tạo ra mã thực thi song song, nếu máy

đích hỗ trợ nó. Phân tích này được gọi là phân tích luồng dữ liệu. Nhiều bộ vi xử lý ngày nay hỗ trợ thực thi song song, đa luồng hay kiến trúc lệnh dài (Very Large Instruction Word - VLIW). Các bộ vi xử lý đa luồng có thể thực hiện đồng thời các lệnh độc lập. Phần cứng phân tích nhanh sự phụ thuộc của các lệnh và xem xét thực thi nhiều hơn một lệnh cùng một lúc. Ngày nay, các máy VLIW hỗ trợ các lệnh tổ hợp chỉ định nhiều hoạt động được thực hiện cùng nhau. Trong trường hợp này, trình biên dịch thường được yêu cầu tạo ra các lệnh song song thích hợp. Phân tích sự phụ thuộc được thực hiện ở cấp độ hợp ngữ hoặc ở cấp độ các hoạt động riêng lẻ, không phải ở cấp độ dòng lệnh C. Một dòng lệnh C có thể chỉ định nhiều hoạt động, hoặc thậm chí các hoạt động phức tạp như lệnh gọi thủ tục. Trong cả hai trường hợp của đa luồng và VLIW, một chương trình mệnh lệnh được phân tích đồng thời để cho phép thực hiện song song. Mục đích chung là tăng tốc độ thực thi chương trình. Mục tiêu là cải thiện hiệu suất, trong đó giả định là hoàn thành tác vụ sớm hơn luôn tốt hơn.

Tuy nhiên, trong bối cảnh của các hệ thống nhúng, tính đồng thời đóng một vai trò quan trọng hơn nhiều so với việc chỉ đơn thuần là cải thiện hiệu suất. Các chương trình nhúng tương tác với các quá trình vật lý và trong thế giới vật lý, nhiều hoạt động tiến triển cùng một lúc. Một chương trình nhúng thường cần theo dõi và phản ứng với nhiều nguồn kích thích đồng thời và cũng điều khiển nhiều thiết bị đầu ra ảnh hưởng đến thế giới vật lý. Các chương

trình nhúng hầu như luôn là các chương trình đồng thời. Xử lý đồng thời không chỉ là một cách để cải thiện hiệu suất. Thật vậy, hoàn thành một tác vụ sớm hơn chưa chắc đã tốt hơn hoàn thành nó muộn hơn. Tất nhiên là vấn đề đảm bảo các ràng buộc thời gian; các hành động được thực hiện trong thế giới vật lý thường cần được thực hiện vào đúng thời điểm (không sớm cũng không muộn). Hình ảnh ví dụ một bộ điều khiển động cơ cho động cơ xăng. Kích hoạt bugi sớm hơn chắc chắn không tốt hơn kích hoạt sau đó. Tín hiệu điều khiển phải được tạo ra vào đúng thời điểm.

Cũng giống như các chương trình mệnh lệnh, các chương trình đồng thời cũng có thể được thực hiện tuần tự hoặc song song. Việc thực thi tuần tự một chương trình đồng thời thường được thực hiện ngày nay bởi một hệ điều hành đa nhiệm. Hệ điều hành này đan xen việc thực thi nhiều tác vụ trong một dòng lệnh tuần tự duy nhất. Tất nhiên, phần cứng có thể thực hiện song song quá trình thực thi đó nếu bộ vi xử lý có kiến trúc đa luồng hoặc VLIW. Do đó, một chương trình đồng thời có thể được chuyển đổi thành một luồng tuần tự bởi một hệ điều hành và quay trở lại chương trình đồng thời bởi phần cứng, nơi bản dịch thứ hai được thực hiện để cải thiện hiệu suất. Những bản dịch phức tạp này càng làm rối vấn đề về việc đảm bảo rằng mọi thứ xảy ra vào đúng thời điểm. Chúng ta sẽ tìm hiểu sâu hơn về vấn đề này trong Chương 7: Định thời tác vụ.

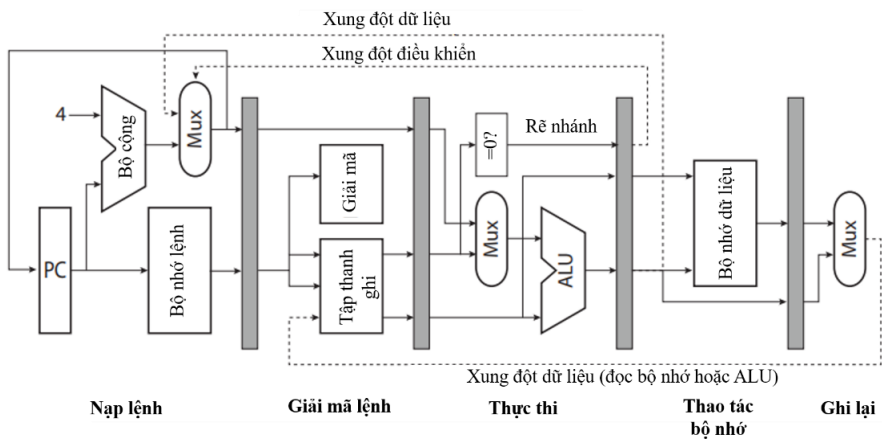
Xử lý song song trong phần cứng mục đích để cải thiện hiệu suất cho các ứng dụng tính toán chuyên sâu. Từ quan điểm của kỹ sư phần mềm, tính đồng thời phát sinh do phần cứng được thiết kế để cải thiện hiệu suất, không phải do vấn đề ứng dụng được giải quyết. Nói cách khác, ứng dụng không (nhất thiết) yêu cầu nhiều hoạt động tiến hành đồng thời, nó chỉ yêu cầu mọi thứ được thực hiện rất nhanh chóng. Tất nhiên, nhiều ứng dụng thú vị sẽ kết hợp cả hai dạng đồng thời, phát sinh từ tính song song và từ yêu cầu ứng dụng.

Các loại giải thuật được tìm thấy trong các chương trình nhúng chuyên sâu về máy tính có ảnh hưởng sâu sắc đến thiết kế của phần cứng. Trong phần này, chúng tôi tập trung vào các phương pháp tiếp cận phần cứng cung cấp song song, cụ thể là pipelining, song song ở mức lệnh và kiến trúc đa lõi.

2.2.2 Kiến trúc đường ống

Hầu hết các bộ vi xử lý hiện đại đều sử dụng kiến trúc đường ống (pipeline). *Pipeline* năm tầng đơn giản cho máy 32 bit được thể hiện trong Hình 2.3. Trong đó, các hình chữ nhật được tô đậm là các mạch chốt. Các mạch chốt này sẽ chốt dữ liệu theo tốc độ xung nhịp của bộ vi xử lý. Tại mỗi cạnh của xung clock, giá trị ở đầu vào được lưu trữ trong thanh ghi chốt. Đầu ra sau đó được giữ không đổi cho đến cạnh tiếp theo của xung clock, cho phép các mạch giữa

các chốt ổn định. Sơ đồ này có thể được xem như là một mô hình hoạt động đồng bộ về thực thi của bộ vi xử lý.



Hình 2.3: Cấu trúc đường ống đơn giản.

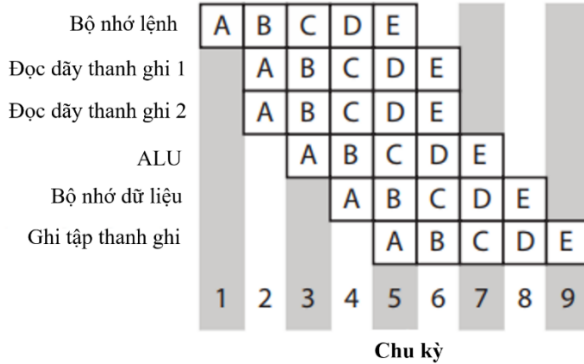
Trong giai đoạn **Nạp lệnh** của đường dẫn, bộ đếm chương trình (Program Counter - PC) cung cấp địa chỉ cho bộ nhớ lệnh. Bộ nhớ lệnh cung cấp các lệnh được mã hóa, trong hình được giả định là có chiều rộng 32 bit. Trong giai đoạn **Nạp lệnh**, PC được tăng thêm 4 (byte) để xác định địa chỉ của lệnh tiếp theo, trừ khi một lệnh con có điều kiện cung cấp một địa chỉ hoàn toàn mới cho PC. Giai đoạn **Giải mã lệnh**, địa chỉ thanh ghi được trích xuất từ lệnh 32-bit và dữ liệu trong các thanh ghi được tìm nạp từ tập thanh ghi. Giai đoạn **Thực thi** thực hiện thao tác trên dữ liệu được tìm nạp từ các thanh ghi hoặc trên PC bằng cách sử dụng **bộ luận lý và số học** (Arithmetic and Logic Unit - ALU), thực hiện các phép toán số học và luận lý. Giai đoạn **Thao tác bộ nhớ** sẽ đọc hoặc ghi vào một vị

trí bộ nhớ được cung cấp bởi một thanh ghi. Giai đoạn cuối cùng là **Ghi lại** lưu trữ kết quả trong thanh ghi được chỉ định.

Bộ vi xử lý DSP thường thêm một hoặc hai giai đoạn bổ sung để thực hiện phép nhân, cung cấp các ALU riêng biệt để tính toán địa chỉ và cung cấp bộ nhớ dữ liệu kép để truy cập đồng thời vào hai toán hạng (thiết kế sau này được gọi là kiến trúc Harvard). Tuy nhiên, phiên bản đơn giản không có ALU riêng biệt đủ để minh họa các vấn đề mà một kỹ sư thiết kế hệ thống nhúng phải đối mặt.

Các phần giữa các chốt hoạt động song song. Do đó, chúng ta có thể thấy ngay rằng có năm lệnh được thực thi đồng thời, mỗi lệnh ở một giai đoạn thực thi khác nhau. Điều này có thể dễ dàng hình dung với kết quả thực thi như trong Hình 2.4. Hình hiển thị các tài nguyên phần cứng có thể được sử dụng đồng thời ở bên trái. Trong trường hợp này, tập thanh ghi xuất hiện ba lần vì *pipeline* trên Hình 2.3 giả định rằng hai lần đọc và ghi tệp của thanh ghi có thể xảy ra trong mỗi chu kỳ.

Tài nguyên phần cứng:



Hình 2.4: Bảng đặt trước cho đường dẫn được thể hiện trong Hình 2.2

2.2.2.1 Trở ngại dữ liệu trong kiến trúc đường ống

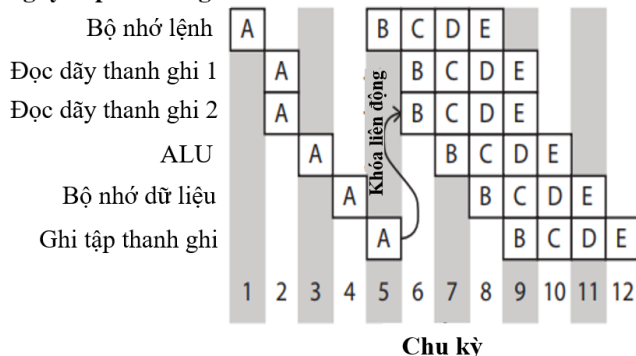
Kết quả thực thi trong Hình 2.4 cho thấy một chuỗi A, B, C, D, E của các lệnh trong một chương trình. Trong chu kỳ 5, E đang được tìm nạp trong khi D đang đọc từ tập thanh ghi, C đang sử dụng ALU, B đang đọc từ hoặc ghi vào bộ nhớ dữ liệu, và A đang ghi kết quả vào tập thanh ghi. Việc ghi dữ liệu bởi A xảy ra ở chu kỳ 5, nhưng đọc bởi B xảy ra ở chu kỳ 3. Như vậy, giá trị mà B đọc sẽ không phải là giá trị mà A ghi.

Hiện tượng này được gọi là trở ngại dữ liệu (data hazard), một dạng của trở ngại đường dẫn (pipeline hazard). Các *pipeline hazard* xảy ra trên các đường dữ liệu đứt nét trong Hình 2.3. Các kỹ sư phần mềm thường mong đợi rằng nếu lệnh A trước lệnh B, thì bất kỳ kết quả nào do A tính toán sẽ có sẵn cho B. Vì vậy hành vi này có thể không được chấp nhận.

Các kiến trúc sư máy tính đã giải quyết vấn đề *pipeline hazard* theo nhiều cách khác nhau. Kỹ thuật đơn giản nhất được gọi là một đường dẫn tường minh (explicit pipeline). Trong kỹ thuật này, *pipeline hazard* được biên tập lại đoạn lệnh một cách đơn giản và người lập trình (hoặc trình biên dịch) phải giải quyết nó. Đối với ví dụ khi B đọc một thanh ghi được viết bởi A, trình biên dịch có thể chèn ba lệnh no-op (lệnh không có thực thi) giữa A và B để đảm bảo rằng việc ghi xảy ra trước khi đọc. Các lệnh no-op này tạo thành bong bóng (pipeline bubble) lan truyền xuống đường dẫn.

Một kỹ thuật phức tạp hơn là cung cấp các khóa liên động (interlock). Trong kỹ thuật này, phần cứng giải mã lệnh, khi gặp lệnh B đọc một thanh ghi do A viết, sẽ phát hiện ra *pipeline hazard* và trì hoãn việc thực thi lệnh B cho đến khi A hoàn thành giai đoạn ghi lại. Đối với *pipeline* này, B phải được trì hoãn ba chu kỳ clock để cho phép A hoàn thành, như thể hiện trong Hình 2.5. Điều này có thể giảm xuống còn hai chu kỳ nếu cung cấp luận lý chuyển tiếp phức tạp hơn một chút, phát hiện ra rằng A đang ghi cùng vị trí mà B đang đọc và trực tiếp cung cấp dữ liệu thay vì yêu cầu quá trình ghi xảy ra trước khi đọc. Các khóa liên động do đó cung cấp phần cứng tự động chèn *pipeline bubble*.

Tài nguyên phần cứng:



Hình 2.5: Khóa liên động giải quyết *pipeline hazard*.

Một kỹ thuật vẫn phức tạp hơn là thực thi không theo thứ tự, trong đó phần cứng được cung cấp để phát hiện trở ngại, nhưng thay vì chỉ trì hoãn việc thực thi B, nó tiến hành tìm nạp C và nếu C không đọc các thanh ghi được viết bởi A hoặc B, và không ghi các thanh ghi được đọc bởi B, sau đó tiến hành thực thi C trước B. Điều này càng làm giảm số lượng *pipeline bubble*.

2.2.2.2 Trở ngại điều khiển trong kiến trúc đường ống

Một dạng khác của *pipeline hazard* được minh họa trong Hình 2.3 là trở ngại điều khiển (control hazard). Trong hình, một lệnh rẽ nhánh có điều kiện thay đổi giá trị của PC nếu một thanh ghi được chỉ định có giá trị bằng không. Giá trị mới của PC được cung cấp (tùy chọn) do kết quả của hoạt động ALU. Trong trường hợp này, nếu A là một lệnh rẽ nhánh có điều kiện, thì A phải đạt đến giai đoạn ghi nhớ trước khi PC có thể được cập nhật. Các lệnh theo sau A trong bộ nhớ sẽ được tìm nạp và sẽ ở giai đoạn giải mã và thực thi trước thời điểm nó được xác định rằng các lệnh đó trên thực tế

không được thực thi. Giống như các trở ngại về dữ liệu, có nhiều kỹ thuật để đối phó với các *control hazard*. Một nhánh bị trì hoãn chỉ đơn giản ghi lại thực tế rằng nhánh sẽ được thực hiện một số chu kỳ sau khi nó chạm trán và giao nó cho kỹ sư phần mềm (hoặc trình biên dịch) để đảm bảo rằng các lệnh tuân theo lệnh rẽ nhánh có điều kiện là vô hại (giống như no-ops) hoặc làm công việc hữu ích mà không phụ thuộc vào việc nhánh được thực hiện. *Interlock* cung cấp phần cứng để chen *bubble* khi cần thiết, giống như các trở ngại về dữ liệu. Trong kỹ thuật phức tạp nhất, thực thi suy đoán, phần cứng ước tính xem nhánh có khả năng được thực hiện hay không và bắt đầu thực hiện các lệnh mà nó mong đợi sẽ thực thi. Nếu kỳ vọng của nó không được đáp ứng, thì nó sẽ hoàn tác bất kỳ phản ứng phụ nào (chẳng hạn như ghi thanh ghi) mà các lệnh được thực thi theo suy đoán gây ra.

Ngoại trừ các đường dẫn rõ ràng và các nhánh bị trễ, tất cả các kỹ thuật này đều tạo ra sự thay đổi trong thời gian thực hiện chuỗi lệnh. Việc phân tích thời gian của một chương trình có thể trở nên cực kỳ khó khăn khi có một *pipeline* sâu với quá trình chuyển tiếp và suy đoán phức tạp. Các *explicit pipeline* tương đối phổ biến trong bộ vi xử lý DSP, thường được áp dụng trong các bối cảnh mà thời gian chính xác là điều cần thiết. Thực thi không theo thứ tự (out-of-order) và suy đoán (speculative execution) là phổ biến trong các bộ vi xử lý đa dụng. Một kỹ sư thiết kế hệ thống nhúng cần phải hiểu

các yêu cầu của ứng dụng và tránh các bộ vi xử lý mà mức độ cần thiết của việc ước đoán thời gian là không thể đạt được.

2.2.3 Song song mức lệnh

Để đạt được hiệu suất cao đòi hỏi xử lý song song trong phần cứng. Xử lý song song có hai dạng chính gồm kiến trúc đa lõi và song song mức lệnh (Instruction Level Parallelism - ILP), là chủ đề của phần này. Bộ vi xử lý hỗ trợ ILP có thể thực hiện nhiều hoạt động độc lập trong mỗi chu kỳ lệnh. Chúng ta thảo luận về bốn hình thức chính của ILP: lệnh CISC, *subword*, *superscalar* và VLIW.

2.2.3.1 Lệnh CISC

Một bộ vi xử lý với các lệnh phức tạp (và thường là khá chuyên biệt) được gọi là máy CISC (Complex Instruction Set Computer). Triết lý đằng sau các bộ vi xử lý như vậy khác biệt rõ ràng với các máy có tập lệnh đơn giản RISC (Reduced Instruction Set Computer). DSP thường là máy CISC và bao gồm các lệnh hỗ trợ đặc biệt cho việc lọc FIR. Trên thực tế, để đủ điều kiện là DSP, bộ vi xử lý phải có khả năng thực hiện lọc FIR trong một chu kỳ lệnh cho mỗi lần nhân.

Ví dụ 2.6: Dòng bộ vi xử lý DSP Texas Instruments TMS320c54x được thiết kế để sử dụng trong các ứng dụng nhúng hạn chế về nguồn điện đòi hỏi hiệu suất xử lý tín hiệu cao, chẳng hạn như hệ thống truyền thông không dây và máy cá nhân (Personal

Digital Assistant - PDA). Vòng lặp bên trong của phép tính FIR (công thức 2.1) là:

1. $RPT\ numberOfTaps - 1$
2. $MAC * AR2 +, * AR3 +, A$

Lệnh đầu tiên minh họa các vòng lặp zero-overhead thường thấy trong các DSP. Lệnh xuất hiện sau nó sẽ thực thi một số lần bằng một cộng với đối số của lệnh RPT. Lệnh MAC là một lệnh đa tích lũy, cũng phổ biến trong kiến trúc DSP. Nó có ba đối số chỉ định phép tính sau:

$$a := a + x * y \quad (2.3)$$

trong đó a là nội dung của thanh ghi bộ tích lũy có tên A, và x và y là các giá trị được tìm thấy trong bộ nhớ. Địa chỉ của các giá trị này được chứa bởi các thanh ghi phụ AR2 và AR3. Các thanh ghi này được tăng tự động sau khi truy cập. Hơn nữa, các thanh ghi này có thể được thiết lập để triển khai các bộ đệm tròn. Bộ vi xử lý C54x bao gồm một phần bộ nhớ trên chip hỗ trợ hai truy cập trong một chu kỳ duy nhất và miễn là các địa chỉ tham chiếu phần này của bộ nhớ, lệnh MAC sẽ thực hiện trong một chu kỳ duy nhất. Do đó, mỗi chu kỳ, bộ vi xử lý thực hiện hai lần tìm nạp bộ nhớ, một phép nhân, một phép cộng thông thường và hai lần tăng địa chỉ (có thể theo mô-đun). Tất cả các DSP đều có khả năng tương tự.

Ví dụ 2.7: Các hệ số của bộ lọc FIR trong công thức

(2.1 thường đối xứng, nghĩa là N chẵn và

$$a_i = a_{N-i-1} \quad (2.4)$$

Lý do cho điều này là các bộ lọc như vậy có pha tuyến tính. Trong trường hợp này, chúng ta có thể giảm số lượng phép nhân bằng cách viết lại công thức

(2.1 như sau:

$$y(n) = \sum_{i=0}^{\left(\frac{N}{2}\right)-1} a_i(x(n-i) + x(n-N+i+1)) \quad (2.5)$$

Tập lệnh Texas Instruments TMS320c54x bao gồm lệnh FIRS có chức năng tương tự như MAC trong Ví dụ 2.6, nhưng sử dụng phép tính này chứ không phải của công thức

(2.1. Điều này tận dụng lợi thế của thực tế là c54x có hai ALU và do đó có thể thực hiện các phép cộng gấp đôi so với các phép nhân. Giờ đây, thời gian để thực thi bộ lọc FIR giảm xuống còn 1/2 chu kỳ mỗi lần nhân.

Các tập lệnh CISC có nhược điểm của chúng. Đầu tiên, việc trình biên dịch sử dụng một cách tối ưu tập lệnh như vậy là vô cùng thách thức. Do đó, bộ vi xử lý DSP thường được sử dụng với các thư viện mã được viết và tối ưu hóa bằng hợp ngữ.

Ngoài ra, các tập lệnh CISC có thể có các vấn đề phức tạp về thời gian có thể cản trở việc lên lịch thời gian thực khó khăn. Trong

các ví dụ trên, cách bố trí dữ liệu trong bộ nhớ ảnh hưởng mạnh đến thời gian thực thi. Thậm chí tinh tế hơn nữa, việc sử dụng các vòng lặp zero-overhead (lệnh RPT ở trên) có thể gây ra một số vấn đề phức tạp. Trên TI C54x, ngắt bị vô hiệu hóa trong quá trình thực hiện lặp lại lệnh sau lệnh RPT. Điều này có thể dẫn đến độ trễ dài ngoài dự kiến trong việc phản hồi các ngắt.

2.2.3.2 *Subword*

Nhiều ứng dụng nhúng hoạt động trên các kiểu dữ liệu nhỏ hơn đáng kể so với kích thước từ của bộ vi xử lý.

Ví dụ 2.8: Trong Ví dụ 2.3 và 2.4, kiểu dữ liệu thường là số nguyên 8 bit, mỗi số đại diện cho một cường độ màu. Màu của pixel có thể được biểu thị bằng ba byte ở định dạng RGB. Mỗi byte RGB có giá trị nằm trong khoảng từ 0 đến 255 thể hiện cường độ của màu tương ứng. Chẳng hạn, sẽ lãng phí tài nguyên nếu sử dụng ALU 64 bit để xử lý một số 8 bit.

Để hỗ trợ các kiểu dữ liệu như vậy, một số bộ vi xử lý hỗ trợ thực thi song song từ lệnh con (subword), trong đó ALU có độ dài lệnh lớn được chia thành các mảng hẹp hơn cho phép đồng thời các thao tác số học hoặc luận lý trên các từ nhỏ hơn.

Bộ vi xử lý vector là bộ vi xử lý trong đó tập lệnh bao gồm các hoạt động trên nhiều phần tử dữ liệu đồng thời. Phép toán song song từ khóa con là một dạng xử lý vector cụ thể.

Ví dụ 2.9: Intel đưa khái niệm song song từ khóa con vào bộ vi xử lý Pentium đa năng được sử dụng rộng rãi và được gọi là công nghệ MMX [5]. Lệnh MMX chia đường dữ liệu 64-bit thành các mảng nhỏ tới 8 bit, hỗ trợ các hoạt động giống hệt nhau đồng thời trên nhiều byte dữ liệu pixel hình ảnh. Công nghệ này đã được sử dụng để nâng cao hiệu suất của các ứng dụng thao tác hình ảnh cũng như các ứng dụng hỗ trợ phát trực tuyến video. Các kỹ thuật tương tự đã được Sun Microsystems giới thiệu cho bộ vi xử lý SparcTM [6] và Hewlett Packard cho bộ vi xử lý PA RISC [7]. Nhiều kiến trúc bộ vi xử lý được thiết kế cho các ứng dụng nhúng, bao gồm nhiều bộ vi xử lý DSP, cũng hỗ trợ song song từ khóa con này.

2.2.3.3 Superscalar

Bộ vi xử lý superscalar sử dụng các tập lệnh tuần tự khá thông thường, nhưng phần cứng có thể gửi đồng thời nhiều lệnh đến các đơn vị phần cứng riêng biệt khi nó phát hiện ra rằng việc gửi đồng thời đó sẽ không thay đổi hoạt động của chương trình. Có nghĩa là, việc thực thi chương trình giống hệt với những gì nó sẽ xảy ra nếu nó được thực thi theo trình tự. Các bộ vi xử lý như vậy thậm chí còn hỗ trợ thực thi không theo thứ tự, nơi các lệnh sau trong luồng được thực thi trước các lệnh trước đó. Bộ vi xử lý superscalar có một nhược điểm đáng kể đối với các hệ thống nhúng, đó là thời gian thực thi có thể cực kỳ khó dự đoán và trong bối cảnh đa nhiệm (ngắt và đe dọa), thậm chí có thể không lặp lại. Thời gian thực thi có thể rất nhạy cảm với thời gian ngắt chính xác, trong đó những thay đổi

nhỏ về thời gian như vậy có thể ảnh hưởng lớn đến thời gian thực thi của chương trình.

2.2.3.4 VLIW

Các bộ vi xử lý dành cho các ứng dụng nhúng thường sử dụng kiến trúc VLIW thay vì superscalar để có được thời gian có thể lặp lại và dễ đoán hơn. Bộ vi xử lý VLIW (rất lớn trong từ struction) bao gồm nhiều đơn vị chức năng, giống như bộ vi xử lý superscalar, nhưng thay vì xác định các lệnh nào có thể được thực thi đồng thời, mỗi lệnh chỉ định những gì mỗi đơn vị chức năng phải làm trong một chu kỳ cụ thể. Nghĩa là, một tập lệnh VLIW kết hợp nhiều hoạt động độc lập thành một lệnh duy nhất. Giống như các kiến trúc superscalar, nhiều hoạt động này được thực hiện đồng thời trên các phần cứng riêng biệt. Tuy nhiên, không giống như superscalar, thứ tự và tính đồng thời của việc thực thi được cố định trong chương trình chứ không phải được quyết định ngay lập tức. Đó là tùy thuộc vào kỹ sư phần mềm (làm việc ở cấp hợp ngữ) hoặc trình biên dịch để đảm bảo rằng các hoạt động đồng thời thực sự độc lập. Để đối lấy sự phức tạp bổ sung này trong lập trình, thời gian thực thi trở nên có thể lặp lại và (thường) có thể dự đoán được.

Ví dụ 2.10: Trong Ví dụ 2.7, chúng ta đã thấy lệnh FIRS chuyên biệt của kiến trúc c54x chỉ định các hoạt động cho hai ALU và một hệ số nhân. Đây có thể được coi là một dạng nguyên thủy của VLIW, nhưng các thế hệ bộ vi xử lý tiếp theo rõ ràng hơn nhiều về bản chất

VLIW của chúng. Texas Instruments TMS320c55x, thế hệ tiếp theo của c54x, bao gồm hai đơn vị tích lũy nhân và có thể hỗ trợ các hướng dẫn giống như sau:

1. $MAC * AR2 +, * CDP +, AC0$
2. $MAC * AR3 +, * CDP +, AC1$

Ở đây, AC0 và AC1 là hai thanh ghi tích lũy và CDP là một thanh ghi chuyên dùng để trỏ đến các hệ số lọc. Ký hiệu :: có nghĩa là hai lệnh này phải được phát hành và thực hiện trong cùng một chu kỳ. Người lập trình hoặc trình biên dịch quyết định xem trên thực tế có thể thực hiện đồng thời các lệnh này hay không. Giả sử các địa chỉ bộ nhớ có thể xảy ra đồng thời, hai lệnh MAC này thực thi trong một chu kỳ, chia đôi một cách hiệu quả thời gian cần thiết để thực thi bộ lọc FIR.

Đối với các ứng dụng vẫn đòi hỏi hiệu suất cao hơn, kiến trúc VLIW có thể khá phức tạp.

Ví dụ 2.11: Dòng bộ vi xử lý Texas Instruments c6000 có tập lệnh VLIW. Bao gồm trong họ này là ba phân họ bộ vi xử lý, bộ vi xử lý điểm cố định c62x và c64x và bộ vi xử lý điểm nổi c67x. Các bộ vi xử lý này được thiết kế để sử dụng trong cơ sở hạ tầng không dây (chẳng hạn như trạm gốc di động và ăng-ten thích ứng), cơ sở hạ tầng viễn thông (chẳng hạn như thoại qua IP và hội nghị truyền hình) và các ứng dụng hình ảnh (chẳng hạn như hình ảnh y tế, giám sát, tầm nhìn máy hoặc giám sát, và ra đa).

2.2.4 Kiến trúc đa lõi

Máy đa lõi là sự kết hợp của nhiều bộ vi xử lý trên một chip duy nhất. Mặc dù máy đa lõi đã tồn tại từ đầu những năm 1990, nhưng gần đây chúng mới thâm nhập vào lĩnh vực điện toán đa năng. Sự thâm nhập này chiếm phần lớn sự quan tâm của công chúng ngày nay. Các máy đa lõi không đồng nhất kết hợp nhiều loại bộ vi xử lý trên một chip duy nhất, so với nhiều phiên bản của cùng một loại bộ vi xử lý.

Ví dụ 2.12: Các kiến trúc OMAP (nền tảng ứng dụng đa phương tiện mở) của Texas Instruments được sử dụng rộng rãi trong điện thoại di động, thường kết hợp một hoặc nhiều bộ vi xử lý DSP với một hoặc nhiều bộ vi xử lý có kiểu dáng gần với bộ vi xử lý đa năng hơn. Bộ vi xử lý DSP xử lý radio, giọng nói và xử lý phương tiện (âm thanh, hình ảnh và video). Các bộ vi xử lý khác xử lý giao diện người dùng, chức năng cơ sở dữ liệu, mạng và các ứng dụng có thể tải xuống. Cụ thể, OMAP4440 bao gồm bộ vi xử lý ARM Cortex lõi kép 1 GHz, c64x DSP, GPU và bộ vi xử lý tín hiệu hình ảnh.

Đối với các ứng dụng nhúng, kiến trúc đa lõi có lợi thế tiềm năng đáng kể so với kiến trúc lõi đơn vì các tác vụ quan trọng trong thời gian thực và an toàn có thể có một bộ vi xử lý chuyên dụng. Đây là lý do cho các kiến trúc không đồng nhất được sử dụng cho điện thoại di động, vì các chức năng xử lý giọng nói và radio là các chức năng thời gian thực khó với tải tính toán đáng kể. Trong kiến

trúc như vậy, các ứng dụng người dùng không thể can thiệp vào các chức năng thời gian thực.

Sự thiếu can thiệp này là khó khăn nhiều hơn trong các kiến trúc đa lõi đa năng. Ví dụ, phổ biến là sử dụng bộ đệm đa cấp, trong đó bộ đệm cấp hai hoặc cao hơn được chia sẻ giữa các lõi. Thật không may, việc chia sẻ như vậy làm cho rất khó để tách hành vi thời gian thực của các chương trình trên các lõi riêng biệt, vì mỗi chương trình có thể gây ra lỗi bộ nhớ cache trong lõi khác. Bộ nhớ đệm nhiều cấp như vậy không thích hợp cho các ứng dụng thời gian thực.

Một kiểu kiến trúc đa lõi rất khác mà đôi khi được sử dụng trong các ứng dụng nhúng sử dụng một hoặc nhiều lõi mềm cùng với phần cứng tùy chỉnh trên một mảng cổng lập trình trường (FPGA). FPGA là các chip có chức năng phần cứng có thể lập trình được bằng các công cụ thiết kế phần cứng. Lõi mềm là bộ vi xử lý được thực hiện trên FPGA. Ưu điểm của lõi mềm là chúng có thể được kết hợp chặt chẽ với phần cứng tùy chỉnh dễ dàng hơn so với các bộ vi xử lý bán sẵn.

2.3 Tóm tắt

Việc lựa chọn kiến trúc bộ vi xử lý cho một hệ thống nhúng có hệ quả quan trọng đối với người lập trình. Các kỹ sư phần mềm có thể cần sử dụng hợp ngữ để tận dụng các tính năng của kiến trúc bí truyền. Đối với các ứng dụng yêu cầu thời gian chính xác, có thể khó kiểm soát thời gian của một chương trình do các kỹ thuật trong

phần cứng để đối phó với các trở ngại đường dẫn và tài nguyên song song.

2.4 Bài tập

Bài tập 1. Xem xét bảng đặt chỗ trong Hình 2.5. Giả sử rằng bộ vi xử lý bao gồm luận lý chuyển tiếp có thể thông báo rằng lệnh A đang ghi vào cùng một thanh ghi mà lệnh B đang đọc từ đó và do đó kết quả do A ghi có thể được chuyển tiếp trực tiếp đến ALU trước khi quá trình ghi được thực hiện. Giả sử bản thân luận lý chuyển tiếp không mất thời gian. Hãy sửa đổi bảng đặt chỗ trên cho việc thực thi lệnh. Có bao nhiêu chu kỳ bị mất cho các *pipeline bubble*?

Bài tập 2. Hãy xem xét lệnh sau, được thảo luận trong Ví dụ 2.6:

$MAC * AR2 +, * AR3 +, A$

Giả sử bộ vi xử lý có ba ALU, một cho mỗi phép toán số học trên các địa chỉ chứa trong thanh ghi AR2 và AR3 và một để thực hiện phép cộng trong lệnh nhân tích lũy MAC. Giả sử mỗi ALU này yêu cầu một chu kỳ đồng hồ để thực thi. Giả sử rằng một hệ số nhân cũng yêu cầu một chu kỳ đồng hồ để thực thi. Giả sử thêm rằng ngân hàng thanh ghi hỗ trợ hai lần đọc và hai lần ghi trên mỗi chu kỳ, và thanh ghi tích lũy A có thể được ghi riêng biệt và không mất thời gian để ghi. Đưa ra một bảng đặt chỗ cho thấy việc thực hiện một chuỗi các hướng dẫn như vậy.

Chương 3 KIẾN TRÚC BỘ NHỚ

Các kỹ sư thiết kế vi xử lý cho rằng hệ thống bộ nhớ có ảnh hưởng lớn đến hiệu suất tổng thể của hệ thống hơn là kiến trúc truyền dữ liệu. Đối với nhiều ứng dụng, điều này là đúng. Bộ nhớ trong hệ thống nhúng là một hệ thống phức tạp thể hiện qua ba yếu tố. Đầu tiên, thông thường cần phải kết hợp nhiều loại công nghệ bộ nhớ trong cùng một hệ thống nhúng. Nhiều loại bộ nhớ là “khả biến” hay còn được gọi là “động”, có nghĩa là nội dung của bộ nhớ sẽ bị mất nếu ngưng cung cấp điện. Hầu hết hệ thống nhúng yêu cầu kết hợp giữa bộ nhớ khả biến và bộ nhớ bất biến (không mất dữ liệu khi ngưng cung cấp điện). Việc lựa chọn bộ nhớ ảnh hưởng nhiều đến người thiết kế hệ thống. Thứ hai, phân cấp bộ nhớ thường cần thiết vì các bộ nhớ có dung lượng lớn hơn và/hoặc mức tiêu thụ điện năng thấp hơn sẽ chậm hơn. Để đạt được hiệu suất hợp lý với chi phí hợp lý, bộ nhớ nhanh hơn thường được phối hợp với bộ nhớ chậm hơn trong một hệ thống. Thứ ba, không gian địa chỉ của kiến trúc bộ vi xử lý được phân chia để cung cấp quyền truy cập vào các loại bộ nhớ khác nhau, để hỗ trợ cho các mô hình lập trình phổ biến và ánh xạ địa chỉ để tương tác với các thiết bị không phải là bộ nhớ, chẳng hạn như thiết bị I/O. Trong chương này, chúng ta sẽ lần lượt thảo luận qua vấn đề này.

3.1 Công nghệ bộ nhớ

Trong các hệ thống nhúng, vấn đề về bộ nhớ là rất quan trọng. Ví dụ, một kỹ sư phần mềm có thể cần phải lo lắng về việc liệu dữ liệu có tiếp tục tồn tại khi ngắt nguồn điện hay không hay là hệ thống có hỗ trợ chế độ chờ tiết kiệm năng lượng hay không. Bộ nhớ có nội dung bị mất khi ngắt nguồn được gọi là **bộ nhớ khả biến** (volatile memory) và ngược lại có bộ nhớ không bị mất nội dung khi ngắt nguồn điện gọi là **bộ nhớ bất biến** (non-volatile memory). Trong phần này, chúng tôi thảo luận về một số công nghệ hiện có và sự cân bằng của chúng.

3.1.1 RAM

Ngoài tập thanh ghi, một vi máy tính thường bao gồm bộ nhớ RAM (bộ nhớ truy cập ngẫu nhiên - Random Access Memory) với một kích thước nhất định. Bộ nhớ RAM là nơi lưu trữ dữ liệu (dưới dạng byte hoặc từ nhớ (word)) có thể được ghi và đọc một cách tương đối nhanh chóng. Có hai loại RAM phổ biến là SRAM (static RAM – RAM tĩnh) và DRAM (dynamic RAM - RAM động). SRAM có tốc độ truy cập nhanh hơn DRAM nhưng nó chiếm diện tích lớn hơn. DRAM chỉ lưu giữ dữ liệu trong một thời gian ngắn và cần một chế độ làm mới (refreshing) mỗi vị trí bộ nhớ một cách định kỳ. SRAM lưu giữ dữ liệu miễn liên tục khi được duy trì nguồn điện. Cả hai đều là bộ nhớ khả biến; trong đó DRAM được cho là

có tính khả biến hơn SRAM vì nó mất nội dung ngay cả khi nguồn điện được duy trì.

Hầu hết các hệ thống máy tính nhúng đều có bộ nhớ SRAM. Nhiều hệ thống cũng bao gồm DRAM vì việc cung cấp đủ bộ nhớ chỉ với công nghệ SRAM có thể không thực tế. Một kỹ sư phần mềm quan tâm đến thời gian chương trình thực thi phải biết liệu các địa chỉ bộ nhớ đang được truy cập có được kết nối tới SRAM hay DRAM hay không. Hơn nữa, chu kỳ làm mới của DRAM có thể gây ra sự thay đổi về thời gian truy cập. Ngoài ra, lịch sử truy cập có thể ảnh hưởng đến thời gian truy cập. Thời gian cần thiết để truy cập một địa chỉ bộ nhớ có thể phụ thuộc vào địa chỉ bộ nhớ được truy cập lần cuối.

DRAM cần được làm mới mỗi vị trí bộ nhớ một cách chu kỳ. Chẳng hạn như, cứ sau 64 ms và một số vị trí (một “hàng”) được làm mới cùng nhau. Hành động đọc bộ nhớ cũng sẽ làm mới các vị trí được đọc. Tuy nhiên, vì các ứng dụng có thể không truy cập tất cả các hàng trong khoảng thời gian được chỉ định, DRAM cần có bộ điều khiển đảm bảo rằng tất cả các vị trí nhớ được làm mới thường xuyên. Bộ điều khiển sẽ ngừng cho phép truy cập nếu bộ nhớ bận làm mới. Điều này ảnh hưởng đến thời gian thực thi của chương trình.

3.1.2 Bộ nhớ bất biến

Các hệ thống nhúng luôn cần lưu trữ dữ liệu ngay cả khi nguồn điện bị tắt. Có tùy chọn cho việc này; pin dự phòng để điện năng không bao giờ bị mất cũng là một lựa chọn. Tuy nhiên pin bị suy hao năng lượng theo thời gian và có kích thước lớn nên không phải lúc nào cũng tích hợp vào hệ thống được. Trong trường hợp đó, bộ nhớ bất biến được sử dụng. Dạng bộ nhớ bất biến đầu tiên là bộ nhớ lõi từ (**magnetic core memory**), trong đó vòng điện từ đã được từ hóa để lưu trữ dữ liệu.

Bộ nhớ không thay đổi phổ biến nhất hiện nay là **ROM** (Read-only Memory - bộ nhớ chỉ đọc) hoặc **mask ROM** (bộ nhớ có dữ liệu tạo cố định tại nhà máy sản xuất). Điều này có thể hữu ích đối với các sản phẩm được sản xuất hàng loạt chỉ cần có một chương trình và dữ liệu liên tục được lưu trữ không thay đổi. Các chương trình như vậy được gọi là phần **firmware**. Công nghệ phát triển tạo ra nhiều loại bộ nhớ ROM có thể được lập trình và đủ tốt để thay thế cho *mask ROM*. **EEPROM** là bộ nhớ ROM cho phép lập trình và thay đổi nội dung bằng điện. Thời gian ghi thường lâu hơn nhiều so với thời gian đọc và số lần ghi bị giới hạn trong suốt thời gian hoạt động của thiết bị. Một dạng EEPROM đặc biệt hữu ích là bộ nhớ *flash*. *Flash* thường được sử dụng để lưu trữ chương trình cơ sở và dữ liệu người dùng cần duy trì khi tắt nguồn.

Bộ nhớ flash, được phát minh bởi Tiến sĩ Fujio Masuoka tại Toshiba vào khoảng năm 1980, là một dạng bộ nhớ không thay đổi đặc biệt tiện lợi, nhưng nó cũng mang đến một số thách thức thú vị đối với kỹ sư thiết kế hệ thống nhúng. Thông thường, bộ nhớ flash có thời gian đọc khá nhanh, dù không bằng SRAM và DRAM, vì vậy dữ liệu được truy cập thường xuyên thường phải được chuyển từ flash sang RAM trước khi được chương trình sử dụng. Thời gian ghi lâu hơn nhiều so với thời gian đọc và tổng số lần ghi có hạn, vì vậy những bộ nhớ này không thể thay thế cho bộ nhớ chính như RAM.

Có hai loại bộ nhớ flash, được gọi là NOR flash và NAND flash. NOR flash có thời gian xóa và ghi lâu hơn, nhưng nó có thể được truy cập như RAM. NAND flash rẻ hơn và có thời gian xóa và ghi nhanh hơn, nhưng dữ liệu phải được đọc một khối tại một thời điểm, trong đó một khối là hàng trăm đến hàng nghìn bit. Điều này có nghĩa là từ góc độ hệ thống, nó hoạt động giống như một thiết bị lưu trữ thứ cấp. Cả hai loại flash chỉ có thể được xóa và ghi lại một số lần nhất định, thường là dưới 1.000.000 đối với NOR Flash và dưới 10.000.000 đối với NAND Flash.

Thời gian truy cập dài hơn, số lần ghi hạn chế và truy cập theo khối (đối với NAND) cũng là những thử thách cho các kỹ sư thiết kế hệ thống nhúng. Những đặc tính này không chỉ được tính đến khi thiết kế phần cứng, mà cả phần mềm.

Đĩa nhớ (disk) cũng là một dạng khác của bộ nhớ không thay đổi. Chúng có thể lưu trữ một lượng dữ liệu rất lớn, nhưng thời gian truy cập có thể trở nên nhiều lớn. Đặc biệt, cơ chế quay của đĩa và di chuyển đầu đọc-ghi định vị trước khi dữ liệu tại vị trí đó có thể được đọc gây nhiều hạn chế cho ứng dụng và thiết kế. Thời gian diễn ra các thao tác này rất khác nhau tùy vào loại đĩa. Đĩa cũng dễ bị rung và kích thước lớn nên khó sử dụng trong nhiều ứng dụng nhúng.

3.2 Phân cấp bộ nhớ

Nhiều ứng dụng yêu cầu lượng bộ nhớ đáng kể, nhiều hơn những gì có sẵn trên chip vi điều khiển. Nhiều bộ vi xử lý sử dụng hệ thống phân cấp bộ nhớ, kết hợp các công nghệ bộ nhớ khác nhau để tăng dung lượng bộ nhớ tổng thể nhằm tối ưu hóa chi phí, độ trễ và tiêu thụ năng lượng. Thông thường, một SRAM có kích thước tương đối nhỏ trên chip sẽ được sử dụng kết hợp với một lượng DRAM lớn hơn nằm ngoài chip. Hệ thống cũng có thể kết hợp thêm với mức thứ ba, chẳng hạn như ổ đĩa, có dung lượng rất lớn nhưng tốc độ truy cập chậm hơn.

Người lập trình ứng dụng có thể không biết rằng bộ nhớ bị phân mảnh trên các công nghệ này. Công nghệ ánh xạ bộ nhớ ảo (virtual memory) ra đời giúp cho việc quản lý bộ nhớ trở nên dễ dàng hơn với kỹ sư phần mềm; đối với họ, bộ nhớ như một không gian địa chỉ liên tiếp. Bên cạnh, hệ điều hành và/hoặc phần cứng cung cấp

tính năng dịch địa chỉ, giúp chuyển đổi các địa chỉ luận lý trong không gian địa chỉ sang các vị trí vật lý. Quá trình dịch địa chỉ này thường được hỗ trợ bởi một phần cứng chuyên dụng được gọi là bộ đệm chuyển đổi **Translation lookaside buffer** (TLB). Đối với một ký sư thiết kế hệ thống nhúng, các kỹ thuật này có thể tạo ra các vấn đề thử thách khi ước lượng chính xác thời gian truy cập bộ nhớ. Do đó, các ký sư thiết kế hệ thống nhúng thường cần hiểu sâu hơn về hệ thống bộ nhớ so với các kỹ sư phần mềm phát triển ứng dụng.

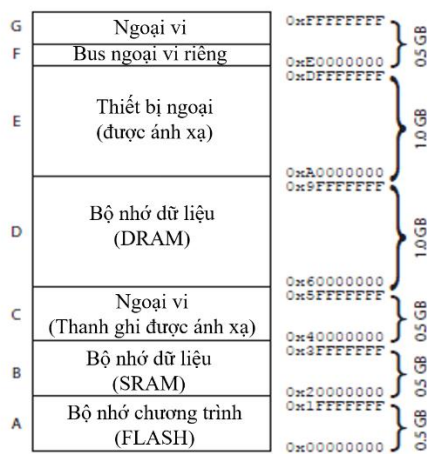
3.2.1 Ánh xạ bộ nhớ

Ánh xạ bộ nhớ cho bộ vi xử lý xác định cách địa chỉ được kế nối tới phần cứng. Tổng kích thước của không gian địa chỉ bị giới hạn bởi độ rộng địa chỉ của bộ vi xử lý. Ví dụ, một bộ vi xử lý 32 bit có thể giải quyết 2^{32} vị trí hoặc 4 GB, giả sử mỗi địa chỉ tham chiếu đến một byte. Độ rộng địa chỉ thường khớp với độ rộng từ nhớ; ngoại trừ bộ vi xử lý 8 bit, trong đó độ rộng địa chỉ thường cao hơn (thường là 16 bit). Ví dụ, một kiến trúc ARM Cortex™ - M3, có ánh xạ bộ nhớ được hiển thị trong Hình 3.1. Các kiến trúc khác sẽ có bố cục khác, nhưng mô hình hình chung là tương tự.

Kiến trúc tổ chức bộ nhớ này phân tách các địa chỉ được sử dụng cho bộ nhớ chương trình (có nhãn A) với các địa chỉ được sử dụng cho bộ nhớ dữ liệu (B và D). Mô hình cơ bản này cho phép việc truy cập bộ nhớ thông qua các *bus* riêng biệt và do đó có thể tìm nạp đồng thời các lệnh và dữ liệu. Điều này giúp tăng gấp đôi *băng*

thông bộ nhớ. Sự tách biệt bộ nhớ chương trình khỏi bộ nhớ dữ liệu như vậy được gọi là kiến trúc Harvard. Ngoài ra, một dạng tổ chức bộ nhớ khác là kiến trúc Von Neumann cổ điển với chương trình và dữ liệu được lưu trữ chung trong cùng một vùng nhớ.

Kiến trúc ánh xạ bộ nhớ ảnh hưởng trực tiếp đến kiến trúc thiết kế và chế tạo của chip nhớ và cả bộ vi xử lý. Ví dụ, bộ điều khiển Luminary Micro LM3S8962, bao gồm lõi ARM Cortex™ - M3, có 256 KB bộ nhớ flash trên chip, nhỏ hơn 0.5 GB tối đa mà kiến trúc cho phép. Bộ nhớ này được cấp địa chỉ từ 0x00000000 đến 0x0003FFFF. Các địa chỉ còn lại từ 0x00040000 đến 0x1FFFFFFF là “địa chỉ dành riêng” cho trình biên dịch cho từng thiết bị cụ thể.



Hình 3.1: Tổ chức bộ nhớ của kiến trúc ARM Cortex-M3

Ngoài ra, LM3S8962 còn có 64 KB SRAM có địa chỉ từ 0x20000000 đến 0x2000FFFF. Nó cũng bao gồm một số thiết bị ngoại vi trên chip có địa chỉ từ 0x40000000 đến 0x5FFFFFFF (vùng C trong hình). Các thiết bị ngoại vi bao gồm bộ hẹn giờ

(timer), bộ chuyển đổi tín hiệu tương tự sang tín hiệu số (ADC), cổng nhập xuất đa dụng (GPIO), giao thức nối (UART) và các thiết bị I/O khác. Mỗi ngoại vi được cấp phát một vài địa chỉ tương ứng với các thanh ghi (trong tập thanh ghi) để cấu hình và/hoặc điều khiển

Đọc thêm: Kiến trúc Harvard

Thuật ngữ “kiến trúc Harvard” xuất phát từ máy tính Mark I, sử dụng các bộ nhớ riêng biệt cho chương trình và dữ liệu. Mark I được IBM chế tạo bằng rô-le điện cơ và được chuyển đến Harvard vào năm 1944. Máy lưu trữ các chỉ lệnh trên băng đục lỗ và dữ liệu trong các bộ đếm cơ điện. Nó được IBM gọi là Máy tính kiểm soát bắn Mark 1 (ASCC) và được Howard H. Aiken phát minh ra để giải các phương trình vi phân bằng số. Phó đề đốc Grace Murray Hopper của Hải quân Hoa Kỳ và sự tài trợ từ IBM là công cụ để biến cỗ máy thành hiện thực.

thiết bị ngoại vi, hoặc để cung cấp dữ liệu được tạo trên một đầu ra. Một số thanh ghi có thể được đọc để truy xuất dữ liệu đầu vào do thiết bị ngoại vi thu được. Một số địa chỉ trong vùng *bus* ngoại vi được sử dụng để truy cập bộ điều khiển ngắt (interrupt controller).

Việc cấp phát các địa chỉ nhớ cho ngoại vi khiến nhiều địa chỉ bộ nhớ không được sử dụng trong thực tế. ARM đã giới thiệu một cách thông minh để tận dụng các địa chỉ không sử dụng này được gọi là dây bit (*bit banding*). *Bit banding* cho phép truy cập các bit riêng lẻ thay vì toàn bộ byte hoặc từ trong bộ nhớ và thiết bị ngoại vi. Điều này làm cho một số thao tác trở nên hiệu quả hơn vì không

cần thiết phải thêm các lệnh để che các bit mong muốn trở nên không cần thiết.

3.2.2 Thanh ghi

Bộ nhớ tích hợp gần nhất trong bộ vi xử lý là tập thanh ghi. Mỗi thanh ghi trong tệp lưu trữ một từ (word). Kích thước của một từ là thuộc tính chính của kiến trúc bộ vi xử lý. *Word* là một byte trên kiến trúc 8 bit, bốn byte trên kiến trúc 32 bit và tám byte trên kiến trúc 64 bit. Tập thanh ghi có thể được triển khai trực tiếp bằng cách sử dụng *flip-flops* trong mạch vi xử lý hoặc các thanh ghi có thể được phân thành các nhóm riêng gọi là *register bank*.

Số lượng thanh ghi trong một bộ vi xử lý thường nhỏ. Lý do chính cho điều này là chi phí của các bit trong một từ lệnh. Các ISA thường cung cấp các lệnh có thể truy cập một, hai hoặc ba thanh ghi. Để lưu trữ hiệu quả các chương trình trong bộ nhớ, các lệnh này không thể yêu cầu quá nhiều bit để mã hóa chúng, và do đó chúng không thể dành quá nhiều bit để xác định các thanh ghi. Nếu tập thanh ghi có 16 thanh ghi, thì mỗi tham chiếu đến một thanh ghi yêu cầu 4 bit. Nếu một lệnh có thể tham chiếu đến 3 thanh ghi, thì lệnh đó yêu cầu tổng cộng 12 bit. Ví dụ, nếu một từ lệnh là 16 bit, thì từ này chỉ để lại 4 bit cho các thông tin khác trong lệnh, chẳng hạn như mã lệnh (op-code).

3.2.3 Vùng nhớ đệm Scratchpad và bộ nhớ Cache

Do các hệ thống nhúng kết hợp nhiều loại bộ nhớ với kích thước và thời gian truy cập khác nhau, hệ thống phải được phân cấp thứ tự truy cập. Ví dụ, bộ nhớ SRAM thường được sử dụng để lưu trữ dữ liệu làm việc tạm thời trong lúc chương trình hoạt động trên đó. Bộ nhớ này được gọi là *bộ nhớ gần* của vi xử lý. Nếu bộ nhớ gần có một bộ địa chỉ riêng biệt và một chương trình để chuyển dữ liệu vào hoặc ra khỏi bộ nhớ đó thì nó được gọi là **scratchpad**. Nếu bộ nhớ gần sao chép dữ liệu trong bộ nhớ xa với phần cứng tự động xử lý việc sao chép thì nó được gọi là **cache**. Đối với các ứng dụng nhúng với các ràng buộc chặt chẽ về thời gian thực, bộ nhớ đệm gây ra một số trở ngại lớn vì hành vi thời gian của chúng có thể thay đổi đáng kể theo cách khó dự đoán. Mặt khác, quản lý thủ công dữ liệu trong vùng nhớ đệm có thể khá tế nhị đối với một kỹ sư phần mềm và các phương pháp trình biên dịch tự động là vẫn còn sơ khai.

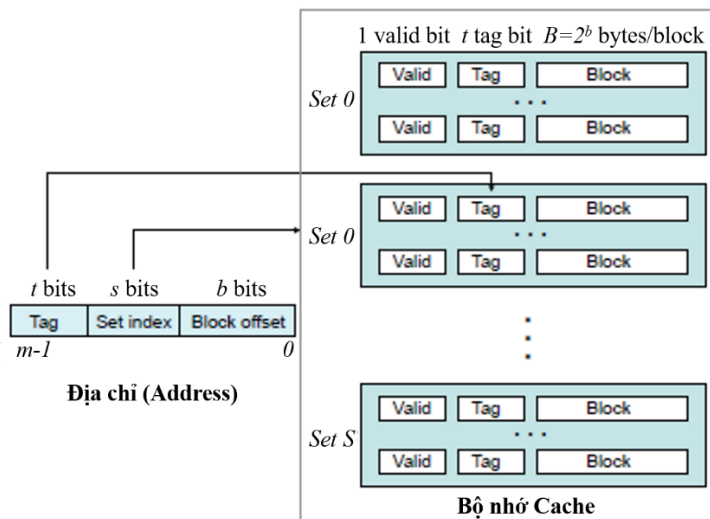
Như đã giải thích trong Phần 3.2.1, một kiến trúc thường sẽ hỗ trợ một không gian địa chỉ lớn hơn nhiều so với những gì thực sự có thể được lưu trữ trong bộ nhớ vật lý của bộ vi xử lý thông qua cơ chế bộ nhớ ảo. Nếu bộ vi xử lý được trang bị **bộ quản lý bộ nhớ** (Memory Management Unit - MMU) thì các chương trình sẽ tham chiếu đến các địa chỉ luận lý và MMU chuyển các địa chỉ này thành địa chỉ vật lý. MMU có thể hiện thực một bộ nhớ cache sử dụng địa chỉ nhớ vật lý có trong vùng B. Khi chương trình cung cấp địa chỉ

bộ nhớ, MMU sẽ xác định xem vị trí đó có được lưu trong bộ nhớ cache hay không, và nếu có, sẽ dịch địa chỉ và hoàn tất quá trình tìm nạp. Nếu không, một lỗi bộ nhớ cache được phát sinh và MMU xử lý việc tìm nạp dữ liệu từ bộ nhớ phụ (trong vùng D) vào bộ nhớ cache (vùng B). Nếu vị trí cũng không có trong vùng D, thì MMU sẽ gây ra lỗi trang, điều này có thể dẫn đến việc phần mềm xử lý chuyển dữ liệu từ đĩa vào bộ nhớ. Do đó, chương trình được tạo ra ảo giác về một lượng lớn bộ nhớ với số lần truy cập bộ nhớ trở nên khá khó dự đoán.

Trong phần tiếp theo, chúng ta sẽ tìm hiểu cơ bản về tổ chức và hoạt động của bộ nhớ cache.

3.2.3.1 Tổ chức bộ cache cơ bản

Giả sử rằng mỗi địa chỉ trong hệ thống bộ nhớ bao gồm m bit, với tối đa $M = 2^m$ địa chỉ duy nhất. Bộ nhớ đệm được tổ chức dưới dạng một mảng bộ nhớ *cachec* có kích thước $S = 2^s$. Mỗi bộ đệm lần lượt bao gồm E dòng. Một dòng bộ nhớ cache lưu trữ một khối dữ liệu $B = 2^b$ byte, cùng với các *valid bits* và *tag bits*. *Valid bits* cho biết liệu dòng trong bộ đệm có lưu trữ thông tin có ý nghĩa hay không, trong khi tag bits (bao gồm các bit $t = m - s - b$) là định danh của khối được lưu trong dòng. Hình 3.2 mô tả tổ chức cache cơ bản và định dạng địa chỉ.



Hình 3.2: Tổ chức bộ nhớ cache

Do đó, một bộ nhớ cache có thể được đặc trưng bởi bộ giá trị (m, S, E, B) . Các thông số này được tóm tắt trong Bảng 3.1. Kích thước C của bộ nhớ cache là $C = S \times E \times B$ byte.

Bảng 3.1: Tổng hợp các tham số của bộ nhớ cache

Tham số	Mô tả
m	Số bit trong địa chỉ vật lý
$S = 2^s$	Số tập hợp trong bộ nhớ cache
E	Số lượng hàng trong tập hợp
$B = 2^b$	Kích thước một khối (byte)
$t = m - s - b$	Số lượng bit tag
$\underline{C}$	Kích thước bộ nhớ cache

Giả sử một chương trình đọc giá trị được lưu trữ tại địa chỉ a . Chúng ta hãy giả định trong phần còn lại của phần này rằng giá trị này là một từ duy nhất w . Đầu tiên CPU gửi địa chỉ a đến bộ nhớ đệm để xác định xem nó có ở đó hay không. Địa chỉ a có thể được

xem như được chia thành ba đoạn bit: t bit trên cùng mã hóa *the tag*, s bit tiếp theo mã hóa *set index*, và b bit cuối cùng mã hóa vị trí của từ trong *block*. Nếu w có trong bộ nhớ đệm cache, truy cập bộ nhớ là *cache hit*; nếu không có thì nó là *cache miss*.

Các bộ nhớ đệm được phân loại thành các lớp dựa trên giá trị của E . Tiếp theo, chúng ta sẽ xem xét các loại bộ nhớ đệm này và mô tả ngắn gọn cách chúng hoạt động.

3.2.3.2 Bộ nhớ đệm ánh xạ trực tiếp

Bộ nhớ cache có chính xác một dòng trên mỗi tập hợp ($E = 1$) được gọi là **ánh xạ trực tiếp** (*Direct-Mapped Caches*). Đối với một bộ nhớ cache như vậy, với một từ w được yêu cầu từ bộ nhớ, trong đó w được lưu trữ tại địa chỉ a , có ba bước để xác định xem w là *cache hit* hay *cache miss*:

1. Lựa chọn tập cache (*Set Selection*): Các bit s mã hóa tập hợp được trích xuất từ địa chỉ a và được sử dụng như một chỉ mục để chọn tập cache tương ứng.
2. Khớp dòng (*Line Matching*): Bước tiếp theo là kiểm tra xem bản sao của w có trong dòng cache duy nhất cho tập hợp này hay không. Điều này được thực hiện bằng cách kiểm tra các *valid bit* và *tag bit* cho dòng bộ nhớ cache đó. Nếu *valid bit* được đặt và các *tag bit* của dòng khớp với các bit của địa chỉ a , thì w có trong dòng và chúng ta có lần truy cập bộ nhớ

cache hit. Nếu không, chúng ta sẽ có lệch bộ nhớ đệm *cache miss*.

3. Lựa chọn từ (*Word Selection*): Khi *word* được biết là có trong khối bộ đệm, chúng ta sử dụng *b* của địa chỉ *a* mã hóa vị trí của *word* trong khối để đọc *word* dữ liệu.

Trong trường hợp *cache miss*, từ *w* phải được yêu cầu từ cấp tiếp theo trong hệ thống phân cấp bộ nhớ. Khi khối này đã được tìm nạp, nó sẽ thay thế khối hiện đang chiếm dòng bộ nhớ *cache* cho *w*.

Mặc dù *cache* được ánh xạ trực tiếp đơn giản để hiểu và triển khai, nhưng nó có thể gặp phải các lỗi xung đột. Lỗi xung đột xảy ra khi các từ trong hai hoặc nhiều khối ánh xạ tới cùng một dòng bộ nhớ *cache* được truy cập liên tục. Khi đó, các khối nhớ sẽ bị thay thế liên tục dẫn đến tỷ lệ *cache miss* tăng lên. Bộ nhớ đệm liên kết có thể giúp giải quyết vấn đề này.

3.2.3.3 Bộ nhớ đệm liên kết

Bộ nhớ đệm liên kết (*Set-Associative Caches*) có thể lưu trữ nhiều hơn một dòng bộ nhớ đệm trên mỗi bộ. Nếu mỗi tập hợp trong bộ đệm có thể lưu trữ *E* dòng, trong đó $1 < E < C/B$ thì bộ đệm đó được gọi là bộ nhớ đệm liên kết *E-way*. Từ “liên kết” xuất phát từ “bộ nhớ liên kết”, là bộ nhớ được thể hiện bằng chính nội dung của nó. Có nghĩa là, mỗi từ trong bộ nhớ được lưu trữ cùng với một khóa duy nhất (*unique key*) và được truy xuất bằng khóa (*key*) chứ không phải địa chỉ vật lý cho biết nơi nó được lưu trữ. Bộ nhớ đệm

liên kết còn được gọi là bộ nhớ có địa chỉ nội dung (*content-addressable memory*).

Đối với bộ nhớ đệm liên kết, việc truy cập một từ w tại địa chỉ a bao gồm các bước sau:

1. *Set selection*: Bước này giống với bộ đệm được ánh xạ trực tiếp.
2. *Line matching*: Bước này phức tạp hơn so với bộ đệm được ánh xạ trực tiếp vì có thể có nhiều dòng nằm trong đó; tức là, các *tag bit* của a có thể khớp với các *tag bit* của bất kỳ dòng nào trong cache của nó. Về mặt hoạt động, mỗi tập hợp trong bộ nhớ đệm liên hợp có thể được xem như một bộ nhớ liên kết, trong đó *key* là sự ghép nối của *tag bit*, các *valid bit* và các giá trị dữ liệu là nội dung của khối tương ứng.
3. *Word selection*: Khi dòng bộ đệm được khớp, việc chọn từ được thực hiện giống như đối với bộ đệm được ánh xạ trực tiếp.

Trong trường hợp *catch miss*, việc thay thế dòng bộ đệm có thể liên quan nhiều hơn so với bộ đệm được ánh xạ trực tiếp. Đối với bộ đệm ánh xạ trực tiếp, không có lựa chọn thay thế vì khối mới sẽ thay thế khối hiện có trên dòng. Tuy nhiên, trong trường hợp bộ đệm liên kết, chúng ta có một tùy chọn để chọn dòng bộ đệm để loại bỏ khỏi từ đó.

Chiến thuật thay thế cache phổ biến *Least-recently used* (LRU), dòng ít được truy cập gần nhất của cache line sẽ bị loại bỏ. Một phương pháp phổ biến khác là *first-in, first-out* (FIFO), trong đó

cache line bị loại bỏ là dòng nằm trong bộ đệm lâu nhất. Các phương pháp thay thế bộ nhớ cache tốt là điều cần thiết để có hiệu suất truy cập tốt.

Bộ nhớ đệm liên kết hoàn toàn (fully associative cache) là bộ đệm có $E = C/B$, tức là chỉ có một bộ duy nhất. Đối với bộ nhớ đệm như vậy, *line matching* có thể khá tốn kém đối với kích thước bộ đệm lớn. Do đó, các bộ nhớ đệm liên kết hoàn toàn thường chỉ được sử dụng cho các bộ đệm nhỏ, chẳng hạn như vùng lưu trữ dữ liệu tạm thời TLB đã đề cập trước đó.

3.3 Mô hình bộ nhớ

Mô hình bộ nhớ xác định cách bộ nhớ được sử dụng bởi các chương trình. Phần cứng, hệ điều hành, ngôn ngữ lập trình và trình biên dịch đều đóng góp vào mô hình bộ nhớ. Một số vấn đề phổ biến phát sinh với các mô hình bộ nhớ sẽ được thảo luận trong phần này.

3.3.1 Địa chỉ bộ nhớ

Ở mức tối thiểu, một mô hình bộ nhớ xác định một loạt các địa chỉ bộ nhớ mà chương trình có thể truy cập được. Trong lập trình C, các địa chỉ này được lưu trữ trong các con trỏ *Pointer*. Trong kiến trúc 32 bit, địa chỉ bộ nhớ là số nguyên không dấu 32 bit, có khả năng biểu diễn các địa chỉ từ 0 đến $2^{32} - 1$. Mỗi địa chỉ tham chiếu đến một byte (tám bit) trong bộ nhớ.

Kiểu dữ liệu *char* C tham chiếu đến một byte. Kiểu dữ liệu *int* C tham chiếu đến một chuỗi có ít nhất hai byte. Trong kiến trúc 32 bit, nó thường sẽ tham chiếu đến 4 byte, có thể biểu diễn các số nguyên từ -2^{31} đến $2^{31} - 1$. *Double data* trong C đề cập đến một chuỗi tám byte được mã hóa theo tiêu chuẩn IEEE (IEEE 754).

Vì địa chỉ bộ nhớ dùng để chỉ một byte nên khi viết một chương trình thao tác trực tiếp các địa chỉ bộ nhớ, có hai mối quan tâm quan trọng về tính tương thích. Đầu tiên là sự liên kết của dữ liệu. Một *int* thường sẽ chiếm bốn byte liên tiếp bắt đầu từ một địa chỉ là bội số của bốn. Trong hệ thập lục phân, những địa chỉ này luôn kết thúc bằng 0, 4, 8 hoặc c.

Mối quan tâm thứ hai là thứ tự byte. Byte đầu tiên (ở địa chỉ kết thúc bằng 0, 4, 8 hoặc c) có thể đại diện cho tám bit bậc thấp của *int* (gọi là *little endian*), hoặc nó có thể đại diện cho tám bit bậc cao của *int* (gọi là *big endian*).

Trong khi biểu diễn dữ liệu đã trở thành tiêu chuẩn chung (chẳng hạn như thứ tự bit trong một byte), thứ tự byte thì còn nhiều tranh cãi. Theo mặc định, kiến trúc Intel's x86 và bộ vi xử lý ARM sử dụng *little-endian*, trong khi IBM's PowerPC sử dụng *big endian*. Một số bộ vi xử lý hỗ trợ cả hai. Thứ tự byte cũng quan trọng trong các giao thức mạng, thường sử dụng *big endian*.

3.3.2 Ngăn xếp

Ngăn xếp (*stack*) là một vùng bộ nhớ được chỉ định cho chương trình theo kiểu *last-in, first-out* (LIFO). *Stack pointer* (thường là một thanh ghi) chứa địa chỉ bộ nhớ của đỉnh ngăn xếp. Khi một mục được lưu vào ngăn xếp, *stack pointer* sẽ tăng lên và mục đó được lưu trữ tại vị trí mới được tham chiếu bởi *stack pointer*. Khi một mục được lấy ra khỏi ngăn xếp, vị trí bộ nhớ được tham chiếu bởi *stack pointer* được sao chép ở một nơi khác (ví dụ: vào một thanh ghi) và *stack pointer* bị giảm.

Các ngăn xếp thường được sử dụng để thực hiện các thủ tục gọi hàm (*procedure calls*). Ví dụ, với một lệnh gọi thủ tục trong C, trình biên dịch tạo ra mã lưu vào ngăn xếp vị trí của lệnh trả về từ thủ tục, giá trị hiện tại của một số hoặc tất cả các thanh ghi máy và các đối số của thủ tục, và sau đó thiết lập thanh ghi PC bằng vị trí của đoạn chương trình cần thực thi. Dữ liệu cho một thủ tục được lưu vào ngăn xếp được gọi là khung ngăn xếp của thủ tục (*stack frame*). Khi một thủ tục trả về, trình biên dịch sẽ đọc khung ngăn xếp của nó, cuối cùng truy xuất vị trí chương trình để tiếp tục thực thi.

Đối với phần mềm nhúng, có thể là thảm họa nếu *stack pointer* được tăng lên vượt quá bộ nhớ được cấp cho ngăn xếp. Việc tràn ngăn xếp như vậy có thể dẫn đến ghi đè bộ nhớ đang được sử dụng cho các mục đích khác, dẫn đến kết quả không thể đoán trước. Do đó, hạn chế ngăn xếp là một mục tiêu quan trọng. Điều này trở nên

đặc biệt khó khăn với các chương trình đệ quy (*recursive programs*), trong đó một thủ tục gọi chính nó. Các kỹ sư thiết kế phần mềm nhúng thường tránh sử dụng đệ quy để khắc phục khó khăn này.

Các lỗi nhỏ có thể phát sinh do sử dụng sai hoặc hiểu sai về ngăn xếp. Hãy xem xét chương trình C sau:

```
int* foo(int a){
    int b;
    b = a * 10;
    return &b;
}
int main(void){
    int* c;
    c = foo(10);
    ...
}
```

Biến *b* là một biến cục bộ, với bộ nhớ của nó trên ngăn xếp. Khi thủ tục trả về, biến *c* sẽ chứa con trỏ đến một vị trí bộ nhớ phía trên *stack pointer*.

Nội dung của vị trí bộ nhớ đó sẽ được ghi đè khi các mục tiếp theo được lưu vào ngăn xếp. Do đó, thủ tục *foo* trả về con trỏ tới *b* là không chính xác. Vào thời điểm con trỏ đó được hủy tham chiếu, vị trí bộ nhớ có thể chứa điều hoàn toàn khác với những gì đã được chỉ định trong *foo*. Thật không may, C không hỗ trợ cơ chế bảo vệ để tránh được những lỗi như vậy

3.3.3 Các đơn vị bảo vệ bộ nhớ

Một vấn đề quan trọng trong các hệ thống hỗ trợ nhiều tác vụ đồng thời là ngăn một tác vụ làm ngắt việc thực thi của tác vụ khác.

Điều này đặc biệt quan trọng trong các ứng dụng nhúng cho phép tải và cài đặt phần mềm của bên thứ ba. Hệ thống cũng có thể cung cấp các biện pháp bảo vệ quan trọng chống lại các lỗi phần mềm trong các ứng dụng có rủi ro về an toàn.

Nhiều bộ vi xử lý cung cấp khả năng bảo vệ bộ nhớ trong phần cứng. Các tác vụ được chỉ định *không gian địa chỉ riêng* của chúng và nếu một tác vụ cố gắng truy cập bộ nhớ bên ngoài không gian địa chỉ của chính nó, thì sẽ dẫn đến lỗi phân đoạn (*segmentation fault*) hoặc các kết quả ngoại lệ khác. Điều này thường sẽ dẫn đến việc dừng ứng dụng vi phạm.

3.3.4 Cấp phát bộ nhớ động

Các ứng dụng phần mềm đa mục đích thường có các yêu cầu không xác định đối với bộ nhớ, tùy thuộc vào các tham số và/hoặc thông tin đầu vào của người dùng. Để hỗ trợ các ứng dụng như vậy, các kỹ sư thiết kế đã phát triển các sơ đồ cấp phát bộ nhớ động, trong đó một chương trình có thể yêu cầu hệ điều hành cấp phát thêm bộ nhớ bất kỳ lúc nào. Bộ nhớ được cấp phát theo một cấu trúc dữ liệu gọi là *heap*, điều này tạo điều kiện thuận lợi cho việc theo dõi các phần bộ nhớ đang được ứng dụng nào sử dụng. Cấp phát bộ nhớ xảy ra thông qua một lời gọi hệ thống (chẳng hạn như *malloc* in C). Khi chương trình không còn cần truy cập vào vùng nhớ đã được cấp phát, nó sẽ giải phóng vùng nhớ (bằng cách gọi *free* in C).

Hỗ trợ cho việc cấp phát vùng nhớ còn có một cơ chế gọi là bộ thu thập “rác” (garbage collector). Ví dụ, *garbage collector* là được tích hợp mặc định trong ngôn ngữ lập trình Java. *Garbage collector* là một tác vụ chạy định kỳ hoặc khi bộ nhớ bị thiếu hụt, phân tích cấu trúc dữ liệu mà chương trình đã cấp phát và tự động giải phóng bất kỳ phần nào của bộ nhớ không còn được tham chiếu trong chương trình. Về nguyên tắc, khi sử dụng *garbage collector*, một kỹ sư phần mềm không cần phải lo lắng về việc giải phóng bộ nhớ một cách rõ ràng.

Dù có hay không có *garbage collector*, chương trình có thể vô tình tích lũy bộ nhớ không bao giờ được giải phóng. Đây được gọi là rò rỉ bộ nhớ và đối với các ứng dụng nhúng với việc thực thi trong một thời gian dài, nó có thể là một thảm họa. Cuối cùng chương trình sẽ lỗi hoặc dừng khi bộ nhớ vật lý bị cạn kiệt.

Một vấn đề khác nảy sinh với các lược đồ cấp phát bộ nhớ là phân mảnh bộ nhớ. Điều này xảy ra khi một chương trình phân bổ và giải phóng bộ nhớ một cách hỗn loạn ở các kích thước khác nhau. Bộ nhớ bị phân mảnh khi các phần bộ nhớ được cấp phát và vùng nhớ trống xen kẽ, và thường phần bộ nhớ trống trở nên quá nhỏ để sử dụng. Trong trường hợp này, cần phải có cơ chế chống phân mảnh.

Chống phân mảnh và thu gom rác đều rất khó khăn đối với các hệ thống thời gian thực. Việc triển khai các tác vụ này yêu cầu tất

cả các tác vụ đang thực thi khác phải dừng (thường gọi nôm na là “*dừng thế giới (stop the world)*”) lại trong lúc thực hiện chống phân mảnh hoặc thu gom rác. Việc hiện thực các kỹ thuật dạng “*stop the world*” như vậy có thể có thời gian tạm dừng đáng kể, đôi khi đến hàng mili giây. Các tác vụ khác không thể thực thi trong thời gian này vì các tham chiếu đến dữ liệu trong cấu trúc dữ liệu (pointers) không nhất quán trong tác vụ. Một kỹ thuật có thể giảm thời gian tạm dừng là thu gom rác gia tăng (incremental garbage collection), điều này làm cách ly các phần bộ nhớ và rác, để thu thập chúng riêng biệt. Tuy nhiên, các kỹ thuật như vậy mới là thử nghiệm và chưa được triển khai rộng rãi.

3.3.5 Mô hình bộ nhớ trong ngôn ngữ C

Các chương trình C lưu trữ dữ liệu trên ngăn xếp, trên *heap* và trong các vị trí bộ nhớ được cố định bởi trình biên dịch. Hãy xem xét chương trình C sau:

```
int a = 2;
void foo(int b, int* c){
    ...
}
int main(void){
    int d;
    int* e;
    d=...;           // Gán một giá trị cho d
    e = malloc(sizeof(int)); // Cấp phát vùng nhớ cho e
    *e = ...;        // Gán một giá trị cho e
    foo(d, e);
    ...
}
```

Trong chương trình này, biến a là một biến toàn cục vì nó được khai báo bên ngoài bất kỳ định nghĩa của thủ tục nào. Trình biên dịch sẽ gán cho nó một vị trí bộ nhớ cố định. Các biến b và c là các tham số, được phân bổ vị trí trên ngăn xếp khi *foo* được gọi (trình biên dịch cũng có thể đặt chúng trong các thanh ghi chứ không phải trên ngăn xếp). Các biến d và e là biến cục bộ. Chúng được khai báo trong phần chính của một thủ tục (trong trường hợp này là *main*). Trình biên dịch sẽ phân bổ không gian trên ngăn xếp cho chúng.

Khi thủ tục *foo* được gọi trong hàm *main*, vị trí ngăn xếp cho b sẽ nhận được một bản sao giá trị của biến d được gán ở dòng 8. Đây là một ví dụ về truyền giá trị (*pass by value*), trong đó giá trị của tham số được sao chép vào ngăn xếp để sử dụng bằng thủ tục được gọi. Ngoài ra, dữ liệu còn được tham chiếu bởi con trỏ e , được lưu trữ trong bộ nhớ được cấp phát trên *heap*, và sau đó nó được chuyển qua tham chiếu giá trị. Địa chỉ được lưu trong vị trí ngăn xếp đối với c . Nếu *foo* bao gồm một phép gán cho $*c$, thì sau khi *foo* trả về, giá trị đó có thể được đọc bằng cách tham chiếu e .

Biến toàn cục a được gán giá trị khởi tạo ở đây chương trình. Tuy nhiên, có một bẫy nhỏ ở đây. Vị trí bộ nhớ lưu trữ a sẽ được khởi tạo với giá trị 2 khi chương trình được tải. Điều này có nghĩa là nếu chương trình được thực thi lần thứ hai mà không cần tải lại, thì giá trị ban đầu của a sẽ không nhất thiết là 2! Giá trị của nó sẽ

là bất kỳ giá trị nào khi lệnh gọi đầu tiên của chương trình kết thúc. Trong hầu hết các hệ điều hành máy tính, chương trình được tải lại mỗi lần thực thi, vì vậy sự cố này không xuất hiện. Nhưng trong nhiều hệ thống nhúng, chương trình không nhất thiết phải tải lại cho mỗi lần chạy. Ví dụ, chương trình có thể được chạy từ đầu mỗi khi hệ thống được thiết lập lại. Để đề phòng vấn đề này, sẽ an toàn hơn khi khởi tạo các biến toàn cục trong phần chính của main, hơn là cách đã làm ở trên.

3.4 Tóm tắt

Một kỹ sư thiết kế hệ thống nhúng cần hiểu kiến trúc bộ nhớ của máy tính đích và mô hình bộ nhớ của ngôn ngữ lập trình. Việc sử dụng bộ nhớ không đúng cách có thể dẫn đến các lỗi cực kỳ nhỏ, một số lỗi sẽ không xuất hiện trong quá trình kiểm tra. Các lỗi chỉ hiển thị trong một sản phẩm có thể gây ra kết quả không mong muốn cho cả người dùng hệ thống và nhà cung cấp công nghệ.

Cụ thể, một kỹ sư thiết kế cần hiểu phần nào của không gian địa chỉ đề cập đến bộ nhớ khả biến và bộ nhớ không dễ thay đổi. Đối với các ứng dụng nhạy cảm với thời gian (là hầu hết các hệ thống nhúng), người thiết kế cũng cần phải biết công nghệ bộ nhớ và kiến trúc bộ nhớ đệm (nếu có) để hiểu thời gian thực thi của chương trình. Ngoài ra, kỹ sư phát triển ứng dụng cần hiểu mô hình bộ nhớ của ngôn ngữ lập trình để tránh đọc dữ liệu có thể không hợp lệ. Ngoài ra, người phát triển ứng dụng cần phải rất cẩn thận với việc

cấp phát bộ nhớ động, đặc biệt đối với các hệ thống nhúng dự kiến sẽ chạy trong một thời gian rất dài. Việc cạn kiệt bộ nhớ khả dụng có thể gây ra sự cố hệ thống hoặc các hành vi không mong muốn khác.

3.5 Bài tập

Bài tập 1: Hãy xem xét hàm compute variance trong đoạn chương trình bên dưới, trong đó giá trị phương sai của các số nguyên được lưu trữ trong biến *data*. Giả sử chương trình này đang thực thi trên bộ vi xử lý 32-bit với bộ đệm được ánh xạ trực tiếp với các tham số (m; S; E; B) = (32; 8; 1; 8). Chúng tôi đưa ra các giả định bổ sung sau:

- Kiểu *int* có kích thước 4 byte
- Các biến *s1*, *s2*, *result* và *i* được lưu trong các thanh ghi
- Biến *data* được lưu tại địa chỉ 0x0.

```
int data[N];
int var_cal(){
    int s1 = 0, s2 = 0, result;
    int i;
    for(i=0; i < N; i++){
        s1 += data[i];
    }
    s2 /= N;

    result = (s2 - s1*s1);
    return result;
}
```

Trả lời các câu hỏi sau:

- (a) Hãy xem xét trường hợp $N = 16$. Sẽ có bao nhiêu *cache miss*?
- (b) Bây giờ giả sử rằng $N = 32$. Tính lại số *cache miss*
- (c) Bây giờ hãy xem xét việc thực thi $N = 16$ trên bộ nhớ đệm liên kết 2 chiều với tham số $(m, S, E, B) = (32, 8, 2, 4)$. Nói cách khác, kích thước khối là giảm một nửa, trong khi có hai *cache lines* trên mỗi bộ. Còn bao nhiêu *cache miss*?

Bài tập 2. Nhớ lại phần 3.2.3 rằng bộ nhớ đệm sử dụng dải giữa của bit địa chỉ làm *set index* và bit bậc cao là *tag*. Tại sao điều này được thực hiện? Hiệu suất bộ nhớ cache có thể bị ảnh hưởng như thế nào nếu các bit ở giữa được sử dụng làm *tag* và bit bậc cao được sử dụng làm *set index*?

Bài tập 3. Hãy xem xét chương trình C và bảng ánh xạ địa chỉ bộ nhớ đơn giản cho vi điều khiển 16 bit được hiển thị bên dưới. Giả sử rằng ngăn xếp bắt đầu từ trên cùng (vùng D) và chương trình và các biến tĩnh được lưu trữ ở dưới cùng (vùng C) của vùng nhớ dữ liệu và chương trình. Ngoài ra, giả sử rằng toàn bộ không gian địa chỉ có bộ nhớ vật lý được liên kết với nó.

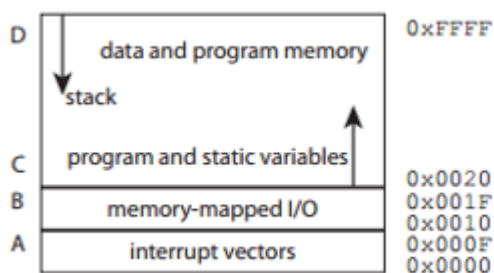
Giả định rằng trong hệ thống này, *int* là một số 16-bit, rằng không có hệ điều hành và không có cơ chế bảo vệ bộ nhớ, và chương trình đã được biên dịch và tải vào vùng C của bộ nhớ.

- (a) Đối với mỗi biến n , m và a , cho biết vị trí trong bộ nhớ (vùng A, B, C hoặc D) biến đó sẽ được lưu trữ.

(b) Xác định chương trình sẽ làm gì nếu nội dung tại địa chỉ 0x0010 là 0 khi nhập.

(c) Xác định chương trình sẽ làm gì nếu nội dung của vị trí bộ nhớ 0x0010 là 1 khi nhập.

```
#include <stdio.h>
#define FOO 0x0010
int n;
int* m;
void foo(int a){
    if(a>0){
        n = n + 1;
        foo(n);
    }
}
int main(){
    n = 0;
    m = (int*)FOO;
    foo(*m);
    printf("n = %d\n",
n)
}
```



Bài tập 4. Hãy xem xét chương trình sau:

```
1  int a = 2;
2  void foo (int b){
3      printf ("%d,b");
4  }
5  int main (void){
6      foo(a);
7      a    = 1;
8  }
```

Đúng hay sai khi giá trị của *a* được chuyển đến *foo* sẽ luôn là 2? Giải thích. Giả sử rằng đây là toàn bộ chương trình, chương trình

này được lưu trữ trong bộ nhớ liên tục và chương trình được thực thi trên một bộ vi điều khiển *bare-iron* mỗi khi nhấn nút đặt lại.

Chương 4 THÀNH PHẦN NHẬP XUẤT

Các hệ thống nhúng hường bao gồm các thành phần tính toán kết hợp với các cơ cấu thay thu nhận và tác động đến môi trường vật lý. Do đó, bộ vi xử lý cần được hỗ trợ các cơ chế để tương tác với thế giới bên ngoài. Đây là một trong những yêu cầu quan trọng và bắt buộc mà các kỹ sư thiết kế hệ thống phải giải quyết. Trong chương này, chúng ta sẽ tìm hiểu về các đặc tính cơ điện và cách thức thiết kế các cơ chế giao tiếp này. Việc sử dụng không đúng các bộ phận giao tiếp có thể khiến hệ thống hoạt động sai hoặc làm giảm tuổi thọ của hệ thống.

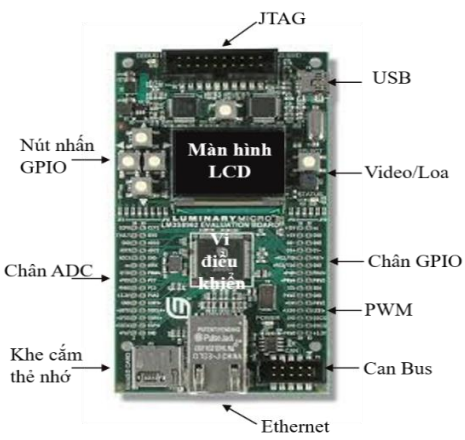
Trong thế giới vật lý, nhiều thứ diễn ra đồng thời. Ngược lại, trong thế giới phần mềm hầu hết mọi thứ diễn ra tuần tự. Việc kết hợp hai đặc tính khác nhau này là một thách thức lớn và thường là yếu tố rủi ro lớn nhất trong việc thiết kế các hệ thống nhúng. Tương tác không chính xác giữa phần mềm và thế giới vật lý có thể gây ra lỗi hệ thống nghiêm trọng. Trong chương này, chúng ta sẽ xem xét các vấn đề trên.

4.1 Phần cứng nhập xuất

Bộ vi xử lý nhúng, có thể là bộ vi điều khiển, bộ vi xử lý DSP hoặc bộ vi xử lý đa năng, thường bao gồm một số cơ chế đầu vào và đầu ra (Input/Output - I/O) gắn liền trên chip hay còn được gọi là các chân của chip (*pin*). Trong phần này, chúng ta tìm hiểu một

số giao tiếp phổ biến với phần minh họa các thuộc tính trong ví dụ sau.

Ví dụ 4.1: Hình 4.1 trình bày bo mạch thực hành (evaluation board) cho vi điều khiển Luminary Micro Stellaris R với lõi xử lý ARM Cortex™-M3 32-bit [8]. Bên dưới màn hình LCD là bộ vi điều khiển ở trung tâm của bảng mạch. Nhiều chân của bộ vi điều khiển có sẵn ở các cạnh bên của bộ vi điều khiển và ở cạnh trên và dưới của bảng mạch. Bo mạch thực hành thường được sử dụng để tạo mẫu thử một ứng dụng nhúng và trong sản phẩm cuối cùng, nó sẽ được thay thế bằng một bảng mạch tùy chỉnh chỉ bao gồm phần cứng mà ứng dụng yêu cầu. Một kỹ sư sẽ phát triển phần mềm cho bo mạch bằng cách sử dụng môi trường phát triển ứng dụng tích hợp (Integrated Development Environment - IDE) và tải phần mềm vào bộ nhớ flash. Ngoài ra, phần mềm có thể được tải lên bảng mạch thông qua giao tiếp USB ở trên cùng từ máy tính phát triển.



Hình 4.1: Bo mạch thực hành Stellaris R LM3S8962

4.1.1 Điều chế độ rộng xung

Điều chế độ rộng xung (Pulse Width Modulation - PWM) là một kỹ thuật để cung cấp một lượng điện năng hiệu quả cho các thiết bị phần cứng bên ngoài. Nó có thể được sử dụng để điều khiển tốc độ của động cơ điện, độ sáng của đèn LED và nhiệt độ của bộ phận sưởi ấm. Nói chung, PWM có thể cung cấp các lượng điện năng khác nhau cho các thiết bị chịu được sự thay đổi nhanh chóng và đột ngột của điện áp và dòng điện.

Phần cứng PWM chỉ sử dụng các mạch kỹ thuật số và do đó dễ dàng tích hợp trên cùng chip với vi điều khiển. Các mạch kỹ thuật số được thiết kế chỉ tạo ra hai mức điện áp, cao và thấp. Tín hiệu PWM nhanh chóng chuyển đổi giữa cao và thấp ở một số tần số cố định, thay đổi khoảng thời gian mà nó giữ tín hiệu ở mức cao. Chu kỳ làm việc là sự cân đối thời gian mà ở đó điện áp cao. Nếu chu kỳ làm việc là 100%, thì điện áp luôn ở mức cao. Nếu chu kỳ làm việc là 0%, thì điện áp luôn ở mức thấp.

Nhiều bộ vi điều khiển cung cấp các thiết bị ngoại vi PWM (như trong Hình 4.1). Để sử dụng chúng, một kỹ sư phần mềm thường ghi một giá trị vào thanh ghi được ánh xạ cho PWM để thiết lập chu kỳ làm việc (tần số cũng có thể được cài đặt). Sau đó, thiết bị sẽ cung cấp năng lượng cho phần cứng bên ngoài tương ứng với chu kỳ làm việc được chỉ định.

PWM là một cách hiệu quả để cung cấp các lượng điện năng khác nhau, nhưng chỉ cho một số thiết bị nhất định. Ví dụ, một phần tử gia nhiệt là một điện trở có nhiệt độ tăng khi có nhiều dòng điện chạy qua nó. Nhiệt độ thay đổi chậm, so với tần số của tín hiệu PWM, vì vậy điện áp thay đổi nhanh chóng của tín hiệu được tính trung bình bởi điện trở và nhiệt độ sẽ rất gần với một hằng số không đổi trong một chu kỳ làm việc cố định. Tương tự như vậy, các động cơ đưa ra các biến thể nhanh chóng của điện áp đầu vào. Đèn sợi đốt và đèn LED cũng vậy. Bất kỳ thiết bị nào có phản ứng với các thay đổi về dòng điện hoặc điện áp chậm so với tần số của tín hiệu PWM là đối tượng để được điều khiển bởi PWM.

4.1.2 Cổng nhập xuất tín hiệu số đa dụng

Các kỹ sư thiết kế hệ thống nhúng thường xuyên phải kết nối các phần cứng xử lý số với các bộ vi xử lý nhúng. Các bộ vi xử lý nhúng có một số chân nhập xuất tín hiệu số đa dụng (General Purpose Input/Output - GPIO), cho phép phần mềm đọc hoặc ghi các mức điện áp biểu thị một giá trị 1 hoặc 0. Nếu điện áp cung cấp của bộ vi xử lý là V_{DD} thì ở dạng tích cực mức cao, V_{DD} đại diện cho giá trị luận lý 1, ngược lại điện áp gần 0 biểu thị giá trị luận lý 0. Nếu hệ thống thiết kế ở dạng tích cực mức thấp thì giá trị điện áp và luận lý tương ứng đảo ngược lại.

Trong nhiều thiết kế, chân GPIO có thể được định cấu hình để trở thành đầu ra. Điều này cho phép phần mềm sau đó ghi vào thanh

ghi được ánh xạ tương ứng để đặt điện áp đầu ra là cao hoặc thấp. Bằng cơ chế này, phần mềm có thể điều khiển trực tiếp các thiết bị vật lý bên ngoài. Tuy nhiên, cần thận trọng. Khi giao tiếp phần cứng với các chân GPIO, kỹ sư thiết kế cần hiểu các thông số kỹ thuật của thiết bị. Đặc biệt, mức điện áp và dòng điện khác nhau tùy theo thiết bị. Nếu chân GPIO tạo ra điện áp đầu ra là V_{DD} khi được cung cấp một giá trị luận lý, thì kỹ sư thiết kế cần biết các giới hạn hiện tại trước khi kết nối thiết bị với nó. Ví dụ: nếu một thiết bị có điện trở R ohms được kết nối với vi điều khiển, thì định luật Ohm cho chúng ta biết rằng dòng điện đầu ra sẽ là:

$$I = V_{DD}/R \quad (4.1)$$

Điều cần thiết là giữ cho dòng điện này trong các dung sai cụ thể (một sai số chấp nhận được). Việc vượt ra ngoài những dung sai này có thể khiến thiết bị quá nóng và hỏng hóc. Có thể cần một bộ khuếch đại công suất để cung cấp đủ dòng điện. Cũng có thể cần một bộ khuếch đại để thay đổi mức điện áp.

Ngoài ra, điều quan trọng là phải duy trì sự cách ly về điện giữa bộ vi xử lý và các thiết bị bên ngoài. Các thiết bị bên ngoài có thể có các đặc

Ví dụ 4.2: Các chân GPIO của bộ vi điều khiển Luminary Micro Stellaris R được hiển thị trong Hình 4.1 có thể được cấu hình thành source hoặc sink biến đổi lượng dòng điện lên đến 18 mA.

Ví dụ 4.3: Các chân GPIO của bộ vi điều khiển được hiển thị trong Hình 4.1, khi được cấu hình làm đầu vào, được kích hoạt Schmitt.

tính điện lộn xộn (nhiều) sẽ làm cho bộ vi xử lý không đáng tin cậy nếu sự nhiễu tràn vào đường dây điện hoặc đường nối đất của bộ vi xử lý. Hoặc thiết bị bên ngoài có thể hoạt động ở chế độ điện áp hoặc công suất rất khác so với bộ vi xử lý. Một chiến lược hữu ích là chia một mạch điện thành các vùng điện, có thể với các nguồn điện riêng biệt, có ảnh hưởng tương đối ít đến nhau. Các thiết bị cách ly như bộ cách ly quang (op-to) và máy biến áp có thể được sử dụng để cho phép giao tiếp giữa các miền điện áp.

Ví dụ 4.4: Hãy xem xét một sàn nhà máy nơi một số bộ vi điều khiển độc lập đều có thể tắt một bộ phận của máy móc bằng cách khẳng định một logic 0 trên một dòng GPIO đầu ra. Thiết kế như vậy có thể gia tăng thêm sự an toàn vì các bộ vi điều khiển có thể dư thừa, do đó lỗi của một bộ vi điều khiển không ngăn cản việc tắt nguồn vì lí do an toàn diễn ra. Nếu tất cả các đường GPIO này được nối với nhau đến một đầu vào điều khiển duy nhất của phần máy móc, thì chúng ta phải thực hiện các biện pháp phòng ngừa để đảm bảo rằng các bộ vi điều khiển không bị chập lẫn nhau. Điều này sẽ xảy ra nếu một bộ vi điều khiển cố gắng điều khiển đường dây dùng chung đến điện áp cao trong khi một bộ vi điều khiển khác cố gắng điều khiển cùng một đường dây đến điện áp thấp.

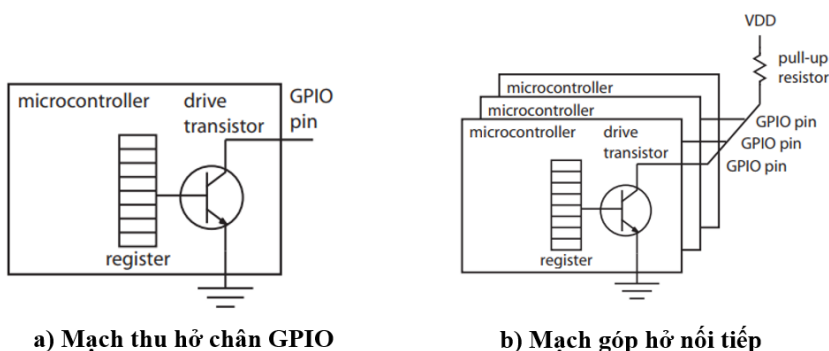
Các chân GPIO cũng có thể được cấu hình làm đầu vào. Trong trường hợp đó, phần mềm sẽ có thể phản ứng với các mức điện áp được cung cấp bên ngoài.

Trong nhiều ứng dụng, một số thiết bị có thể dùng chung một mối nối điện. Người thiết kế phải cẩn thận để đảm bảo rằng các thiết bị này không

đồng thời điều khiển điện áp của mỗi nối điện đơn lẻ này đến các giá trị khác nhau, dẫn đến đoạn mạch có thể gây quá nhiệt và hỏng thiết bị.

Các đầu ra GPIO có thể sử dụng mạch thu hở (open collector), như trong Hình 4.2(a). Trong một mạch như vậy, việc ghi một logic vào thanh ghi (ánh xạ bộ nhớ) sẽ bật bóng bán dẫn, kéo điện áp trên chân đầu ra xuống (gần) bằng không. Việc ghi logic 0 vào thanh ghi sẽ tắt bóng bán dẫn, khiến chân đầu ra không được kết nối, hoặc "hở".

Một số giao tiếp bộ thu hở có thể được kết nối như trong Hình 4.2(b). Đường dây dùng chung được kết nối với một điện trở kéo lên, đưa điện áp của đường dây lên đến V_{DD} khi tất cả các bóng bán dẫn được tắt. Nếu bất kỳ một bóng bán dẫn nào được bật, thì nó sẽ đưa điện áp của toàn bộ đường dây xuống 0 mà không tạo ra ngắn mạch với các chân GPIO khác. Về mặt luận lý, tất cả các thanh ghi phải có số 0 trong chúng để kết quả đầu ra cao. Nếu bất kỳ một trong các thanh ghi có 1 trong đó, thì kết quả đầu ra sẽ thấp. Giả sử logic hoạt động cao, hàm logic đang được thực hiện là NOR, thì mạch như vậy được gọi là NOR có dây. Bằng cách thay đổi cấu hình, người ta có thể tạo OR có dây hoặc AND có dây một cách tương tự.



Hình 4.2: Mạch thu và mạch góp hở

Thuật ngữ "open collector" xuất phát từ tên của thiết bị đầu cuối của bóng bán dẫn lưỡng cực. Trong công nghệ CMOS, loại giao tiếp này thường được gọi là giao tiếp cực máng mở (open drain). Về cơ bản, nó hoạt động theo cùng một cách.

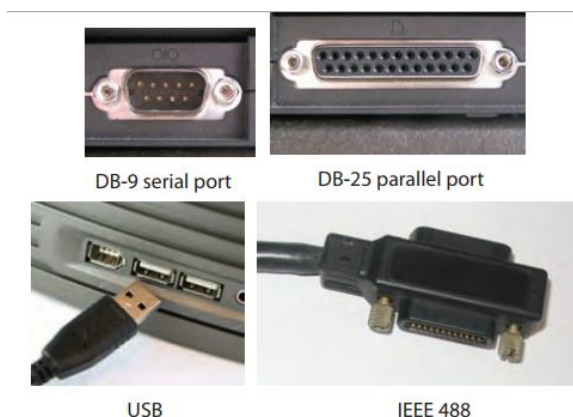
Ví dụ 4.5: Các chân GPIO của bộ vi điều khiển thể hiện trong Hình 10.1, khi được cấu hình làm đầu ra, có thể được chỉ định là các mạch thoát hở. Họ cũng có thể tùy chọn cung cấp điện trở kéo lên, giúp giảm số lượng linh kiện rời bên ngoài cần thiết trên bảng mạch in một cách thuận tiện.

Các đầu ra GPIO cũng có thể được hiện thực với cổng ba trạng thái (tristate logic); có nghĩa là ngoài hai giá trị điện áp đầu ra cao hoặc thấp, cổng còn

một trạng thái ngắt kết nối (turn off). Giống như open collector, tristate logic có thể tạo điều kiện cho việc chia sẻ các mạch bên ngoài giống nhau giữa nhiều thiết bị. Không giống như giao diện bộ thu hở, thiết kế ba ngôi có thể xác định cả điện áp cao và thấp, thay vì chỉ một trong hai.

4.2 Giao tiếp nối tiếp

Một trong những giới hạn mà các kỹ sư thiết kế bộ vi xử lý nhúng phải đối mặt là yêu cầu đóng gói chip vật lý nhỏ và tiêu thụ điện năng thấp. Hệ quả là số lượng chân cắm trên mạch tích hợp bộ vi xử lý bị hạn chế. Vì vậy, mỗi chân phải được sử dụng hiệu quả. Ngoài ra, khi nối các hệ thống thành phần với nhau, số lượng dây cần được giới hạn để giữ cho khối lượng tổng thể và giá thành của sản phẩm luôn trong tầm kiểm soát. Do đó, dây cũng phải được sử dụng hiệu quả. Một cách để sử dụng chân và dây một cách hiệu quả là gửi thông tin qua chúng một cách tuần tự dưới dạng chuỗi các bit. Giao tiếp như vậy được gọi là giao tiếp nối tiếp (serial interface). Một số tiêu chuẩn đã phát triển cho các giao tiếp nối tiếp để các thiết bị từ các nhà sản xuất khác nhau có thể được kết nối.



Hình 4.3: Các đầu nối cho giao tiếp nối tiếp và song song

Một tiêu chuẩn cũ nhưng ổn định là RS-232, lần đầu tiên được giới thiệu vào năm 1962 để kết nối teletypes với modem. Tiêu

chuẩn này xác định tín hiệu điện và kiểu đầu nối cho các thiết bị. Nó vẫn tồn tại vì tính đơn giản và sự phổ biến của các thiết bị công nghiệp đang sử dụng giao tiếp này. RS-232 xác định cách một thiết bị có thể truyền một byte sang thiết bị khác một cách không đồng bộ (không chia sẻ tín hiệu clock). Trên các PC cũ hơn, kết nối RS-232 có thể được cung cấp qua đầu nối DB-9, như thể hiện trong Hình 4.3. Một bộ vi điều khiển thường sẽ sử dụng bộ thu/phát không đồng bộ đa năng (UART) để chuyển đổi nội dung của thanh ghi 8 bit thành một chuỗi các bit để truyền qua liên kết nối tiếp RS-232.

Đối với một kỹ sư thiết kế hệ thống nhúng, vấn đề chính cần xem xét là giao diện RS-232 có thể khá chậm và có thể làm chậm phần mềm ứng dụng, nếu người lập trình không cẩn thận.

Cơ chế RS-232 rất đơn giản. Người gửi và người nhận trước tiên phải đồng ý về tốc độ truyền (tốc độ này là chậm theo tiêu chuẩn hiện đại). Người gửi bắt đầu truyền một byte với một bit bắt đầu, bit này thông báo cho người nhận rằng một byte đang đến. Sau đó, người gửi định giờ chuỗi các bit theo tỷ lệ đã thỏa thuận, theo sau chúng bằng một hoặc hai bit dừng. Clock của người nhận được thiết lập lại khi nhận được bit bắt đầu và sẽ theo dõi clock của người gửi để có thể lấy mẫu tín hiệu đến một cách tuần tự và khôi phục chuỗi bit. RS-232 có nhiều dạng kế thừa hỗ trợ giao tiếp tốc độ cao hơn, chẳng hạn như RS-422, RS-423, v.v.

Các thiết bị mới hơn được thiết kế để kết nối với máy tính cá nhân thường sử dụng giao diện bus nối tiếp chung (USB), được tiêu chuẩn hóa bởi một nhóm các nhà cung cấp. USB 1.0 xuất hiện vào năm 1996 và hỗ trợ tốc độ dữ liệu 12 Mbits/giây. USB 2.0 xuất hiện vào năm 2000 và hỗ trợ tốc độ dữ liệu lên đến 480 Mbits/giây. USB 3.0 xuất hiện vào năm 2008 và hỗ trợ tốc độ dữ liệu lên đến 4,8 Gbits/giây.

USB đơn giản hơn về mặt điện so với RS-232 và sử dụng các đầu nối đơn giản hơn, mạnh mẽ hơn, như trong Hình 4.3. Nhưng tiêu chuẩn USB xác định nhiều thứ hơn là vận chuyển điện của các byte, và cần có luận lý điều khiển phức tạp hơn để hỗ trợ nó. Vì các thiết bị ngoại vi hiện đại như máy in, ổ đĩa, thiết bị âm thanh và video đều bao gồm bộ vi điều khiển, nên việc hỗ trợ các giao thức USB phức tạp hơn là phù hợp cho các thiết bị này.

Một giao tiếp nối tiếp khác được triển khai rộng rãi trong các bộ vi xử lý nhúng được gọi là JTAG, tiêu chuẩn IEEE 1149.1 và kiến trúc quét ranh giới. Giao diện này xuất hiện vào giữa những năm 1980 để giải quyết vấn đề mà các gói mạch tích hợp và công nghệ bảng mạch in đã phát triển đến mức việc kiểm tra các mạch sử dụng đầu dò điện trở nên khó khăn hoặc không thể. Các điểm trong mạch cần được truy cập trở nên không thể tiếp cận được đối với các đầu dò. Khái niệm về quét biên cho phép trạng thái của một ranh giới logic của một mạch (những gì theo truyền thống là các chân có thể

truy cập được đối với các đầu dò) được đọc hoặc ghi tuần tự thông qua các chân có thể truy cập được. Ngày nay, các cổng JTAG được sử dụng rộng rãi để cung cấp giao tiếp gỡ lỗi cho bộ vi xử lý nhúng, cho phép môi trường gỡ lỗi được lưu trữ trên máy tính để kiểm tra và kiểm soát trạng thái của bộ vi xử lý nhúng. Ví dụ, cổng JTAG được sử dụng để đọc trạng thái của các thanh ghi bộ vi xử lý, để đặt các điểm ngắt trong một chương trình và từng bước trong một chương trình. Một biến thể mới hơn là gỡ lỗi dây nối tiếp (Serial Wire Debug - SWD), cung cấp chức năng tương tự với ít chân hơn.

Có một số giao tiếp nối tiếp khác đang được sử dụng ngày nay, bao gồm I2C, SPI, PCI Express, FireWire, MIDI và các phiên bản nối tiếp của SCSI. Mỗi thứ đều có công dụng của nó. Ngoài ra, các giao tiếp mạng thường là nối tiếp.

4.3 Giao tiếp song song

Một giao tiếp nối tiếp gửi hoặc nhận một chuỗi các bit tuần tự trên một đường truyền (dây). Một giao song song sử dụng nhiều dây để gửi đồng thời các bit. Tất nhiên, mỗi dây của giao tiếp song song cũng là một giao tiếp nối tiếp, nhưng việc nhóm hợp lý và phối hợp hoạt động của các dây này là những gì làm cho một giao tiếp trở thành song song.

Trong lịch sử, một trong những giao tiếp song song được sử dụng rộng rãi nhất là cổng máy in IEEE-1284, cổng này trên máy tính IBM đã sử dụng đầu nối DB-25, như trong Hình 4.3. Giao tiếp

này ra đời từ năm 1970 với máy in Centronics model 101, và do đó đôi khi được gọi là công máy in Centronics. Ngày nay, máy in thường được kết nối bằng USB hoặc mạng không dây.

Với việc lập trình một cách cẩn thận, một nhóm các chân GPIO có thể được sử dụng cùng nhau để tạo ra một giao tiếp song song. Trên thực tế, các kỹ sư thiết kế hệ thống nhúng đôi khi sử dụng các chân GPIO để tạo ra một giao diện mà phần cứng của vi điều khiển không hỗ trợ trực tiếp.

Có vẻ trực quan rằng các giao tiếp song song sẽ mang lại hiệu suất cao hơn các giao tiếp nối tiếp, vì nhiều dây hơn được sử dụng cho kết nối. Tuy nhiên, đó là việc không cần thiết. Một thách thức đáng kể với các giao tiếp song song là duy trì tính đồng bộ trên nhiều dây. Điều này trở nên khó khăn hơn khi độ dài vật lý của kết nối tăng lên. Thực tế này, kết hợp với yêu cầu về sự công kênh và nhiều chân I/O hơn đã dẫn đến nhiều giao tiếp song song truyền thống bị thay thế bằng giao tiếp nối tiếp.

4.4 Đường truyền dữ liệu

Bus là một giao tiếp được chia sẻ giữa nhiều thiết bị, trái ngược với kết nối *point-to-point* liên kết chính xác hai thiết bị. Các cổng có thể là giao tiếp nối tiếp (chẳng hạn như USB) hoặc giao tiếp song song. Một bus song song phổ biến là SCSI (phát âm là scuzzy, dành cho giao diện hệ thống máy tính nhỏ), thường được sử dụng để kết nối ổ cứng và băng từ với máy tính. Tuy nhiên, các biến thể gần

đây của giao diện SCSI bắt đầu từ giao tiếp song song truyền thống để trở thành giao tiếp nối tiếp. SCSI là một ví dụ về kiến trúc bus ngoại vi, được sử dụng để kết nối máy tính với thiết bị ngoại vi như card âm thanh và ổ đĩa.

Các tiêu chuẩn bus ngoại vi được sử dụng rộng rãi khác bao gồm bus ISA (kiến trúc tiêu chuẩn công nghiệp, được sử dụng trong kiến trúc IBM PC phổ biến), PCI (Peripheral Component Interface) và Parallel ATA (Advanced Technology Attachment). Một loại chuẩn bus ngoại vi hơi khác là IEEE-488, ban đầu được phát triển cách đây hơn 30 năm để kết nối thiết bị kiểm tra tự động với máy tính điều khiển. Giao tiếp này được thiết kế tại Hewlett Packard và còn được biết đến rộng rãi với

Ví dụ 4.6: Một bus ngoại vi cung cấp một cơ chế để các thiết bị bên ngoài giao tiếp với CPU. Nếu một thiết bị bên ngoài cần chuyển một lượng lớn dữ liệu vào bộ nhớ chính, thì việc yêu cầu CPU thực hiện mỗi lần truyền có thể không hiệu quả và/hoặc gây ngất. Một giải pháp thay thế là truy cập bộ nhớ trực tiếp (Direct Memory Access - DMA). Trong lược đồ DMA được sử dụng trên bus ISA, việc truyền được thực hiện bởi một thiết bị riêng biệt được gọi là bộ điều khiển DMA, thiết bị này sẽ điều khiển bus và truyền dữ liệu. Trong một số thiết kế gần đây hơn, chẳng hạn như PCI, thiết bị bên ngoài trực tiếp kiểm soát bus và thực hiện chuyển giao mà không cần sự trợ giúp của bộ điều khiển DMA chuyên dụng. Trong cả hai trường hợp, CPU có thể tự do thực thi phần mềm trong khi quá trình truyền đang diễn ra, nhưng nếu mã

tên gọi HP-IB (bus giao diện Hewlett Packard) và GPIB (bus giao diện mục đích chung). Nhiều mạng cũng sử dụng kiến trúc bus.

Bởi vì một bus được chia sẻ giữa nhiều thiết bị, bất kỳ kiến trúc bus nào cũng phải bao gồm giao thức kiểm soát truy cập (Media Access Control - MAC) để phân xử các truy cập đang tranh nhau. Một giao thức MAC đơn giản có một *bus master* để truy vấn các *bus slaver*. USB sử dụng một cơ chế như vậy. Một giải pháp thay thế là bus được kích hoạt theo thời gian, trong đó các thiết bị được chỉ định các vị trí thời gian trong đó chúng có thể truyền (hoặc không, nếu chúng không có gì để gửi). Giải pháp thay thế thứ ba là *token ring*, trong đó các thiết bị trên bus phải nhận được mã thông báo trước khi chúng có thể sử dụng phương tiện được chia sẻ và mã thông báo được chuyển xung quanh các thiết bị theo một số mẫu. Một giải pháp thay thế thứ tư là sử dụng bộ điều khiển bus, đây là một mạch xử lý các yêu cầu cho bus theo một số mức độ ưu tiên. Một giải pháp thay thế thứ năm là đa truy cập cảm biến sóng mang (Carrier Sense Multiple Access - CSMA), trong đó các thiết bị cảm nhận sóng mang để xác định xem đường truyền có đang được sử dụng hay không trước khi bắt đầu sử dụng, phát hiện xung đột có thể xảy ra khi nó bắt đầu sử dụng và thử lại sau đó khi có xung đột xảy ra.

Trong mọi trường hợp, việc chia sẻ đường truyền vật lý có tác động đến thời gian của các ứng dụng.

4.5 Phần mềm tuần tự trong một thế giới đồng thời

Như chúng ta đã thấy trong Ví dụ 4.5, khi phần mềm tương tác với thế giới bên ngoài, thời gian thực thi của phần mềm có thể bị ảnh hưởng mạnh mẽ. Về bản chất, phần mềm là xử lý tuần tự, thường thực thi càng nhanh càng tốt. Tuy nhiên, thế giới vật lý là đồng thời, với nhiều thứ xảy ra cùng một lúc và với tốc độ chúng xảy ra được xác định bởi các đặc tính vật lý. Hệ thống kết hợp những yếu tố này là một trong những thách thức lớn mà kỹ sư thiết kế hệ thống nhúng phải đối mặt. Trong phần này, chúng ta thảo luận về một số cơ chế chính để đạt được điều này.

4.5.1 Ngắt và ngoại lệ

Ngắt (interrupt) là một cơ chế để tạm dừng thực thi công việc hiện tại và chuyển sang thực hiện một đoạn chương trình phục vụ một sự kiện ngắt. Đoạn chương trình này gọi là **chương trình phục vụ ngắt** (Interrupt Service Routine - ISR). Ba loại sự kiện có thể kích hoạt ngắt. Một là ngắt phần cứng; một số phần cứng bên ngoài thay đổi mức điện áp trên đường tín hiệu yêu cầu ngắt. Thứ hai là ngắt phần mềm; chương trình đang thực thi sẽ kích hoạt ngắt bằng cách thực hiện một lệnh đặc biệt hoặc bằng cách ghi vào thanh ghi ánh xạ bộ nhớ dành cho ngắt. Trường hợp thứ ba được gọi là ngoại lệ (exception); ngắt được kích hoạt bởi phần cứng bên trong phát hiện lỗi, chẳng hạn như lỗi phân đoạn bộ nhớ.

Đối với hai biến thể đầu tiên, sau khi ISR hoàn tất, chương trình bị ngắt sẽ tiếp tục lại từ vị trí nó bị dừng. Trong trường hợp ngoại lệ, khi ISR đã hoàn thành, chương trình đã kích hoạt ngoại lệ thường không được tiếp tục. Thay vào đó, bộ đếm chương trình được đặt ở một số vị trí cố định, ví dụ như hệ điều hành có thể chấm dứt chương trình vi phạm.

Khi xảy ra kích hoạt ngắt, phần cứng trước tiên phải quyết định xem có phản hồi hay không. Nếu ngắt bị tắt, nó sẽ không phản hồi. Cơ chế bật hoặc tắt ngắt khác nhau tùy theo bộ vi xử lý. Hơn nữa, có thể một số ngắt được kích hoạt và những ngắt khác thì không.

Ngắt và ngoại lệ thường có mức độ ưu tiên, và một ngắt sẽ chỉ được phục vụ nếu bộ vi xử lý không bận phục vụ ngắt có mức ưu tiên cao hơn. Thông thường, các trường hợp ngoại lệ có mức ưu tiên cao nhất và luôn được phục vụ.

Ví dụ 4.7: ARM Cortex™ - M3 là bộ vi điều khiển 32 bit được sử dụng trong tự động hóa công nghiệp và các ứng dụng khác. Nó bao gồm một bộ đếm thời gian hệ thống được gọi là SysTick. Bộ đếm thời gian này có thể được sử dụng để kích hoạt ISR thực thi cứ sau 1 ms. Ví dụ, giả sử rằng cứ sau 1 ms chúng ta muốn đếm ngược từ một số lần đếm ban đầu cho đến khi số lượng đó đạt đến 0, và sau đó dừng đếm ngược. Mã lệnh trong C sau đây xác định ISR thực hiện điều này:

```
1 volatile uint timerCount = 0;
2 void countdown(void) {
3     if (timerCount != 0) {
4         timerCount --;
5     }
6 }
```

Ở đây, biến *timerCount* là một biến toàn cục và nó sẽ giảm đi mỗi khi *countDown()* được gọi, cho đến khi nó đạt đến 0.

Khi phần cứng quyết định phục vụ ngắt, trước tiên nó sẽ vô hiệu hóa các ngắt khác, sao lưu thanh ghi PC hiện tại và các thanh ghi trạng thái stack và nhảy đến địa chỉ được chỉ định của ISR. ISR phải sao lưu lên stack các giá trị hiện có trong bất kỳ thanh ghi nào mà nó sẽ sử dụng và khôi phục các giá trị của chúng trước khi kết thúc ISR để chương trình bị ngắt có thể được phục hồi. ISR hoặc phần cứng phải kích hoạt lại ngắt thoát khỏi ISR.

4.5.2 Tính nguyên tử

Chương trình phục vụ ngắt có thể được gọi giữa hai lệnh bất kỳ của chương trình chính (hoặc giữa hai lệnh bất kỳ của ISR có mức độ ưu tiên thấp hơn). Một trong những thách thức lớn đối với các kỹ sư thiết kế phần mềm nhúng là lý luận về sự xen kẽ các lệnh có thể trở nên cực kỳ khó khăn. Trong ví dụ trước, chương trình phục vụ ngắt và chương trình chính đang tương tác thông qua một biến được chia sẻ, cụ thể là *timerCount*. Giá trị của biến đó có thể thay đổi giữa bất kỳ hai phép toán nguyên tử (atomic operation) nào của chương trình chính. Thật không may, có thể khá khó khăn để biết những hoạt động nào là nguyên tử. Thuật ngữ "nguyên tử" xuất phát từ tiếng Hy Lạp có nghĩa là "không thể phân chia", và một kỹ sư phần mềm không phân biệt rõ được là hoạt động nào là không thể phân chia. Nếu kỹ sư phần

Ví dụ 4.8: Tập lệnh ARM bao gồm lệnh LDM, lệnh này tải nhiều thanh ghi từ các vị trí bộ nhớ liên tiếp. Nó có thể bị ngắt một phần khi tải.

mềm đang viết mã hợp ngữ, thì có thể an toàn khi giả định rằng mỗi lệnh hợp ngữ là nguyên tử, nhưng nhiều ISA bao gồm các lệnh cấp hợp ngữ không phải là nguyên tử.

Ở cấp độ của một chương trình C, có thể còn khó khăn hơn để biết các phép toán nào là nguyên tử. Hãy xem xét câu lệnh đơn giản sau:

```
timerCount = 2000;
```

Trên vi điều khiển 8 bit, câu lệnh này có thể mất nhiều hơn một chu kỳ lệnh để thực thi (một từ 8 bit không thể lưu trữ cả lệnh và hằng số 2000; trên thực tế, một mình hằng số không phù hợp với một từ 8 bit). Một ngắt có thể xảy ra một phần thông qua việc thực hiện các chu trình đó. Giả sử rằng ISR cũng ghi vào biến *timerCount*. Trong trường hợp này, giá trị cuối cùng của biến *timerCount* có thể bao gồm 8 bit được đặt trong ISR và các bit còn lại được đặt bởi dòng C ở trên. Giá trị cuối cùng có thể rất khác so với 2000 và cũng khác với giá trị được chỉ định trong quy trình dịch vụ ngắt. Lỗi này có xảy ra trên vi điều khiển 32 bit không? Cách duy nhất để biết chắc chắn là hiểu đầy đủ về ISA và trình biên dịch. Trong những trường hợp như vậy, không có lợi ích gì khi viết mã bằng C thay vì hợp ngữ.

Các lỗi như thế này trong một chương trình rất khó xác định và sửa chữa. Tệ hơn nữa, sự đan xen khó giải quyết như vậy thì khá khó xảy ra và do đó có thể không xuất hiện trong kiểm tra. Đối với

các hệ thống quan trọng về an toàn, các kỹ sư phần mềm phải cố gắng hết sức để tránh những lỗi như vậy. Một cách để làm điều này là xây dựng các chương trình sử dụng các mô hình tính toán đồng thời cấp cao hơn. Tất nhiên, việc triển khai các mô hình tính toán đó cần phải chính xác. Việc triển khai các mô hình đó được các chuyên gia trong lĩnh vực xây dựng đồng thời, chứ không phải bởi các kỹ sư phần mềm.

Khi làm việc ở cấp độ C và ISR, một kỹ sư phần mềm phải suy luận cẩn thận về thứ tự hoạt động. Mặc dù có thể thực hiện nhiều phép xen kẽ, nhưng các hoạt động được đưa ra dưới dạng một chuỗi các câu lệnh C phải thực hiện theo thứ tự (chính xác hơn, chúng phải hoạt động như thể chúng đã được thực thi theo thứ tự, ngay cả khi thực thi không theo thứ tự được sử dụng).

4.5.3 Bộ điều khiển ngắt

Bộ điều khiển ngắt là logic trong bộ vi xử lý xử lý các ngắt. Nó hỗ trợ một số ngắt và một số mức độ ưu tiên. Mỗi ngắt có một vector ngắt, là địa chỉ của ISR hoặc một chỉ số của một mảng được gọi là bảng vector ngắt có chứa địa chỉ của tất cả các ISR.

Khi ngắt được xác nhận bằng cách thay đổi điện áp trên một chân, phản ứng có thể được kích hoạt mức (level triggered) hoặc kích hoạt cạnh (edge triggered).

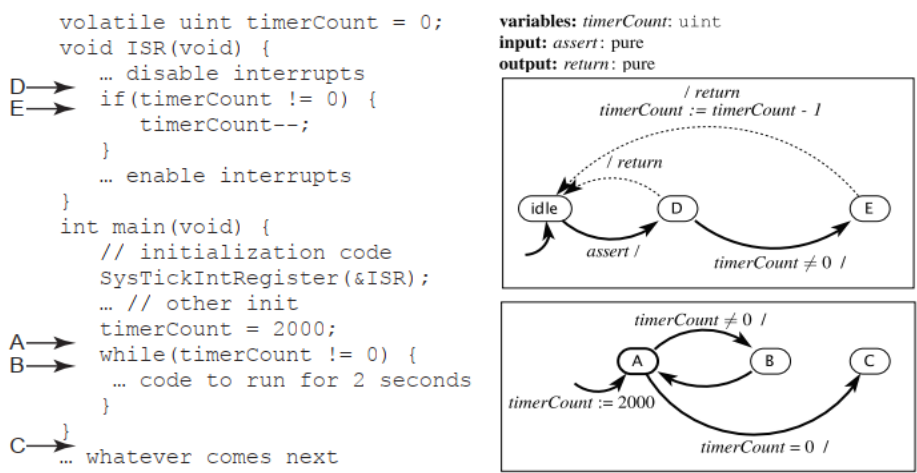
Ví dụ 4.9: Bộ điều khiển Luminary Micro LM3S8962, được hiển thị trong Hình 4.1, bao gồm bộ vi điều khiển lõi ARM Cortex™ - M3 hỗ trợ 36 ngắt với tám mức ưu tiên. Nếu hai ngắt được gán cùng một số ưu tiên, thì ngắt có vector thấp hơn sẽ được ưu tiên hơn ngắt có vector cao hơn.

Đối với các ngắt được kích hoạt mức, phần cứng xác nhận ngắt thường sẽ giữ điện áp trên đường dây cho đến khi nó nhận được thông báo xác nhận, điều này cho biết rằng ngắt đang được xử lý. Đối với ngắt được kích hoạt cạnh, phần cứng xác nhận ngắt sẽ thay đổi điện áp chỉ trong một thời gian ngắn. Trong cả hai trường hợp, các đường dây của open collector có thể được sử dụng sao cho cùng một đường truyền vật lý có thể được chia sẻ giữa một số thiết bị (tất nhiên, ISR sẽ yêu cầu một số cơ chế để xác định thiết bị nào đã xác nhận ngắt, ví dụ bằng cách đọc một thanh ghi được ánh xạ bộ nhớ trong mỗi thiết bị có thể đã xác nhận ngắt).

Việc chia sẻ ngắt giữa các thiết bị có thể phức tạp và phải xem xét cẩn thận để tránh các ngắt có mức độ ưu tiên thấp chặn các ngắt có mức độ ưu tiên cao. Việc xác nhận ngắt bằng cách ghi vào một địa chỉ được chỉ định trên một bus có ưu điểm là cùng một phần cứng có thể hỗ trợ nhiều ngắt khác nhau, nhưng nhược điểm là các thiết bị ngoại vi trở nên phức tạp hơn. Các thiết bị ngoại vi phải bao gồm một giao tiếp với bus bộ nhớ.

4.5.4 Mô hình hóa ngắt

Hành vi ngắt có thể khá khó để hiểu đầy đủ và nhiều lỗi hệ thống nặng nề là do các hành vi không mong muốn. Thật không may, luận lý của bộ điều khiển ngắt thường được mô tả trong nhiều tài liệu về bộ vi xử lý rất không chính xác, để lại nhiều hành vi có thể xảy ra mà không xác định. Một cách để làm cho luận lý này chính xác hơn là mô hình hóa nó như một FSM.



Hình 4.4: Các mô hình máy trạng thái và chương trình tương ứng

Ví dụ 4.10: Chương trình của Ví dụ 4.7 thể hiện trong Hình 4.4 cùng với hai máy trạng thái cho ISR và chương trình chính. Các trạng thái của FSM tương ứng với các vị trí trong quá trình thực thi có nhãn từ A đến E. Các vị trí này nằm giữa các câu lệnh C, vì vậy ở đây chúng ta giả định rằng các câu lệnh này là các phép toán nguyên tử.

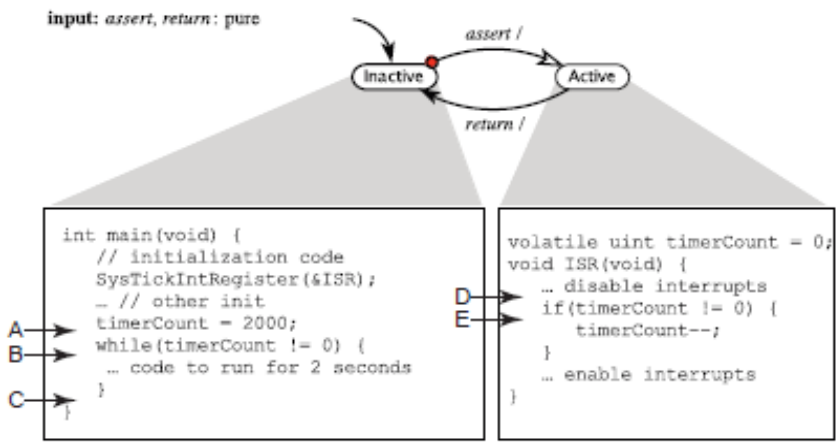
Mục đích của chúng ta là xác định xem chương trình có được đảm bảo luôn đạt đến vị trí C. Nói cách khác, chúng ta có thể tự tin khẳng định rằng chương trình cuối cùng sẽ vượt qua bất kỳ phép tính nào mà nó đã thực hiện trong hai giây không? Một mô hình máy trạng thái sẽ giúp chúng ta trả lời câu hỏi đó.

Câu hỏi quan trọng bây giờ là làm thế nào để cấu hình các máy trạng thái này để mô hình hóa chính xác sự tương tác giữa hai đoạn mã tuần tự trong các thủ tục ISR và hàm *main*. Dễ dàng nhận thấy bố cục không đồng bộ không phải là lựa chọn phù hợp vì sự đan xen không thể tùy tiện. Đặc biệt, hàm *main* có thể bị ngắt bởi ISR, nhưng ISR không thể bị ngắt bởi *main*. Thành phần không đồng bộ sẽ không thể kiểm soát hết được sự bất đối xứng này.

Giả sử rằng ngắt luôn được phục vụ ngay lập tức khi được yêu cầu, chúng tôi muốn có một mô hình giống như trong Hình 4.5. Trong đó, một FSM hai trạng thái mô hình hóa xem ngắt có đang phục vụ hay không. Quá trình chuyển đổi trạng thái từ không hoạt động (Inactive) sang hoạt động (Active) được kích hoạt bởi một đầu vào thuần túy *assert* yêu cầu dịch vụ ngắt. Khi ISR hoàn thành việc thực thi, một phản hồi đầu vào thuần túy khác sẽ kích hoạt quay trở lại trạng thái *Inactive*. Lưu ý ở đây rằng quá trình chuyển hóa từ *Inactive* sang *Active* là một quá trình chuyển hóa ưu tiên, được biểu thị bằng vòng tròn nhỏ khi bắt đầu quá trình chuyển hóa. Điều này có nghĩa là nó nên được thực hiện ngay lập tức khi *assert*

xảy ra và đó là một chuyển hóa tái thiết lập, cho thấy rằng việc tinh chỉnh trạng thái *Active* sẽ bắt đầu ở trạng thái ban đầu của chính nó khi vào.

Nếu kết hợp Hình 4.4 và Hình 4.5, chúng ta nhận được một FSM phân cấp trong Hình 4.6(a). Lưu ý rằng tín hiệu trả về bây giờ vừa là đầu vào vừa là đầu ra. Đây là đầu ra được tạo ra bởi quá trình tinh chỉnh trạng thái của *Active* và nó là đầu vào cho FSM cấp cao nhất, nơi nó kích hoạt chuyển đổi sang *Inactive*. Việc có một đầu ra cũng là một đầu vào cung cấp một cơ chế tinh chỉnh trạng thái để kích hoạt quá trình chuyển trạng thái.



Hình 4.5: Mô hình máy trạng thái cho sự tương tác giữa ISR và chương trình chính

Để xác định xem chương trình có đạt đến trạng thái C hay không, chúng ta có thể nghiên cứu máy trạng thái trong Hình 4.6(b). Chúng ta thấy rằng trên thực tế không có gì đảm bảo sẽ đạt trạng thái C!

Ví dụ, nếu *assert* xuất hiện trong mọi phản ứng, thì C không bao giờ đạt được.

Điều này có thể xảy ra trong thực tế? Với chương trình này, điều đó là không thể xảy ra, nhưng không phải là không thể. Nó có thể xảy ra nếu bản thân ISR mất nhiều thời gian để thực thi hơn thời gian giữa các lần ngắt. Có đảm bảo rằng điều này sẽ không xảy ra? Thật không may, đảm bảo duy nhất của chúng ta là một khái niệm mơ hồ rằng bộ vi xử lý có thể nhanh hơn thế!

Trong ví dụ trên, mô hình hóa sự tương tác giữa một chương trình chính và một chương trình phục vụ ngắt cho thấy một lỗ hổng tiềm ẩn trong chương trình. Mặc dù lỗ hổng có thể khó xảy ra trong ví dụ này, nhưng thực tế, lỗ hổng tồn tại là điều đáng lo ngại. Trong mọi trường hợp, tốt hơn là nhận thức được lỗ hổng tồn tại và quyết định rằng rủi ro có thể chấp nhận được, còn hơn là không biết nó đang tồn tại.

Cơ chế ngắt có thể khá phức tạp. Phần mềm sử dụng các cơ chế này để cung cấp I/O cho thiết bị bên ngoài được gọi là trình điều khiển thiết bị. Viết trình điều khiển thiết bị chính xác và mạnh mẽ là một tác vụ kỹ thuật đầy thách thức, đòi hỏi sự hiểu biết sâu sắc về kiến trúc và kỹ năng lý luận đáng kể về tính đồng thời. Nhiều lỗi trong hệ thống máy tính là do tương tác không mong muốn giữa trình điều khiển thiết bị và các chương trình khác.

đếm với giá trị 3 và sau đó gọi *foo()* cho đến khi bộ đếm bằng không.

```
#include <avr/interrupt.h>
volatile uint16_t timer_count = 0;

//Chương trình phục vụ ngắt
SIGNAL(SIG_OUTPUT_COMPARELA){
    if(timer_count > 0) {
        timer_count--;
    }
}

//Chương trình main()
int main(void){
    //Cấu hình ngắt 1s
    ...
    // Tạo timer 3s
    timer_count = 3;
    // Chương trình lặp lại trong 3s
    while(timer_count > 0){
        foo();
    }
}
```

Hình 4.7: Ví dụ chương trình C với thủ tục gọi hàm

- (a) Chúng ta muốn giả định rằng các đoạn mã trong các hộp màu xám, có nhãn A, B và C, là nguyên tử. Nêu các điều kiện làm cho giả định này có giá trị.
- (b) Xây dựng một mô hình máy trạng thái cho chương trình này, giả sử như trong phần (a) rằng A, B và C là nguyên tử. Các chuyển trạng thái trong máy trạng thái này nên được gắn nhãn "*guard/action*", trong đó *action* có thể là bất kỳ A, B, C hoặc không có gì. Các hành động A, B hoặc C phải tương ứng với các phần mã trong các hộp màu xám có nhãn tương ứng. Bạn có thể giả sử những hành động này là nguyên tử.

(c) Máy trạng thái trong câu (b) có xác định không? Nó cho bạn biết điều gì về số lần *foo()* có thể được gọi? Tất cả các hành vi có thể có của mô hình có tương ứng với những gì mà kỹ sư phần mềm có thể dự định không?

Lưu ý rằng có rất nhiều câu trả lời có thể. Các mô hình đơn giản được ưu tiên hơn những mô hình phức tạp và những mô hình hoàn chỉnh (nơi mọi thứ được xác định) hơn những mô hình không hoàn chỉnh. Hãy đưa ra nhiều hơn một mô hình.

Bài tập 2. Xem xét màn hình bảng điều khiển hiển thị “*normal*” khi phanh trong xe hoạt động bình thường và “*emergency*” khi có sự cố. Hành vi dự kiến là một khi “*emergency*” đã được hiển thị, “*normal*” sẽ không được hiển thị nữa. Tức là, “*emergency*” vẫn hiển thị trên màn hình cho đến khi hệ thống được thiết lập lại.

Trong đoạn mã sau, giả sử rằng biến *display* xác định những gì được hiển thị. Dù giá trị của nó là gì, đó là những gì xuất hiện trên bảng điều khiển.

Giả sử rằng ISRA là một ISR ngắt quãng được gọi khi người lái xe đạp phanh. Giả sử rằng ISRB được kích hoạt nếu một cảm biến chỉ ra rằng phanh đang được dùng cùng lúc với bàn đạp ga được nhấn.

Giả sử rằng cả ISR đều không thể tự ngắt, nhưng ISRB có mức ưu tiên cao hơn ISRA và do đó ISRB có thể ngắt ISRA, nhưng

ISRA không thể ngắt ISRB. Giả định thêm (không thực tế) rằng mỗi dòng mã là nguyên tử.

```
volatile static uint8_t alt;
volatile static char* dis;
void ISRA(){
    if(alt == 0){
        dis = "normal";
    }
}
void ISRB(){
    dis = "emergency";
    alt = 1;
}
void main(){
    alt = 0;
    //---Khởi tạo ngắt---//
    //---Kích hoạt ngắt---//
    //-Chương trình khác---//
```

(a) Chương trình này có luôn thể hiện hành vi dự kiến không? Giải thích. Trong các phần còn lại của bài tập này, bạn sẽ xây dựng các mô hình khác nhau để chứng minh rằng hành vi là đúng hoặc sẽ minh họa cách nó có thể không chính xác.

(b) Xây dựng ISRA mô hình máy trạng thái mở rộng xác định. Giả sử:

- *alerted* là một biến kiểu $\{0, 1\} \subset \text{uint8_t}$
- Có một đầu vào tuần tự *A* mà khi xuất hiện chỉ ra một yêu cầu ngắt cho ISRA, và
- *display* là đầu ra có kiểu char *.

(c) Xác định kích thước của không gian trạng thái cho lời giải trên.

- (d) Giải thích các giả định của bạn về thời điểm máy trạng thái ở (b) phản ứng. Đây có phải là kích hoạt thời gian hay kích hoạt sự kiện hay cả hai đều không phải?
- (e) Xây dựng máy trạng thái mở rộng xác định mô hình hóa ISRB. Nó có một đầu vào thuần túy B mà khi xuất hiện chỉ ra một yêu cầu ngắt cho ISRB.
- (f) Xây dựng một máy trạng thái mở rộng xác định (không phân cấp) mô tả hoạt động chung của hai ISR . Sử dụng mô hình này để lập luận về tính đúng đắn của câu trả lời của bạn cho phần (a).
- (g) Cho một máy trạng thái phân cấp tương đương. Sử dụng mô hình này để lập luận về tính đúng đắn của câu trả lời của bạn cho phần (a).

Chương 5 CẢM BIẾN VÀ CƠ CẤU CHẤP HÀNH

Cảm biến (sensor) là thiết bị để đo lường các đại lượng vật lý như ánh sáng, âm thanh, nhiệt độ... **Cơ cấu chấp hành** (actuator) là một thiết bị làm thay đổi một đại lượng vật lý. Trong các hệ thống điện tử, kết quả đầu ra của các cảm biến thường là giá trị điện áp tỷ lệ với đại lượng vật lý mà cảm biến đó đang đo. Để đưa kết quả điện áp này vào máy tính xử lý, điện áp có thể được chuyển đổi thành một dạng tín hiệu số thông qua bộ chuyển đổi tín hiệu tương tự sang tín hiệu số (Analog to Digital Converter - ADC). Bộ cảm biến được tích hợp ADC được gọi là cảm biến kỹ thuật số (digital sensor), trong khi cảm biến không có ADC được gọi là cảm biến tương tự (analog sensor). Một cảm biến kỹ thuật số sẽ có độ chính xác giới hạn và được xác định bởi số lượng bit được sử dụng để biểu diễn một giá trị (con số). Trong khi đó, một bộ truyền động thường được điều khiển bởi một điện áp. Điện áp điều khiển này có thể được tạo ra từ giá trị số thông qua bộ chuyển đổi từ tín hiệu số sang tín hiệu tương tự (Digital to Analog Converter - DAC). Một bộ truyền động được tích hợp DAC được gọi là bộ truyền động kỹ thuật số (digital actuator).

Ngày nay, cảm biến và thiết bị truyền động thường được kết hợp cùng với bộ vi xử lý và kết nối mạng. Do đó, các dịch vụ từ hệ thống có thể được cung cấp và sử dụng thông qua mạng internet.

Các hệ thống CPS do đó được hình thành và cung cấp các dịch vụ rộng khắp. Các hệ thống như thế có thể xuất hiện dưới dạng các khái niệm về vạn vật kết nối (Internet of Things - IoT), cách mạng Công nghiệp 4.0, Machine-to-Machine (M2M), Internet of Everything, smart planet, TSensors (Nghìn tỷ cảm biến) hay các hệ thống điện toán đám mây.

Cảm biến và các thiết bị truyền động dần trở nên đa dạng và phổ biến cùng với sự phát triển của hệ thống CPS. Trong chương này, chúng ta sẽ cùng tìm hiểu về cảm biến và thiết bị truyền động một cách cơ bản. Chúng ta sẽ tập trung tìm hiểu các thuộc tính của cảm biến và thiết bị truyền động.

5.1 Giới thiệu về cảm biến

Trong phần này, chúng ta tìm hiểu một số cảm biến cơ bản và chỉ ra các mô hình phù hợp để sử dụng cảm biến này.

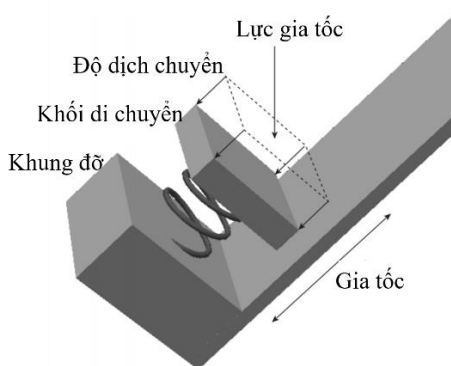
5.1.1 Đo góc nghiêng và gia tốc

Gia tốc kế (accelerometer) là một cảm biến đo chỉ xác gia tốc. Chúng ta cũng cần lưu ý rằng rất khó để phân biệt giữa lực hấp dẫn và gia tốc. Do đó, một gia tốc kế thường sẽ đo lường cả gia tốc và lực hấp dẫn. Kết quả này là một tiền đề đối với Thuyết tương đối rộng của Albert Einstein.

Sơ đồ của một gia tốc kế được thể hiện trong Hình 5.1. Một khối chuyển động được gắn qua lò xo vào một khung cố định. Giả sử

rằng mạch cảm biến có thể đo vị trí của khối chuyển động này so với khung cố định (ví dụ như thông qua đo điện dung). Khi khung tăng tốc theo hướng mũi tên kép trong hình, gia tốc khiến cho khối di chuyển, và do đó gia tốc này có thể đo lường được.

Vị trí trung hòa của khối là vị trí khi lò xo không bị biến dạng. Nó sẽ chiếm vị trí trung hòa này nếu toàn bộ cụm lắp ráp rơi tự do, hoặc nếu cụm nằm ngang



Hình 5.1: Mô hình gia tốc kế

Thay vào đó, nếu bộ phận được sắp xếp thẳng hàng theo phương thẳng đứng, thì lực hấp dẫn sẽ nén lò xo và làm dịch chuyển khối. Đối với một người quan sát khi rơi tự do, điều này trông giống như thể khối lắp ráp đang tăng tốc hướng lên **gia tốc của trọng lực**, xấp xỉ $g = 9.8 \text{ m/s}^2$.

Do đó, một gia tốc kế có thể đo độ nghiêng (so với trọng lực) của khung cố định. Bất kỳ gia tốc nào đi qua khung sẽ cộng hoặc trừ từ phép đo này. Việc tách biệt tác động của trọng lực và gia tốc thật sự là thử thách. Sự kết hợp của cả hai là cái mà chúng ta gọi là

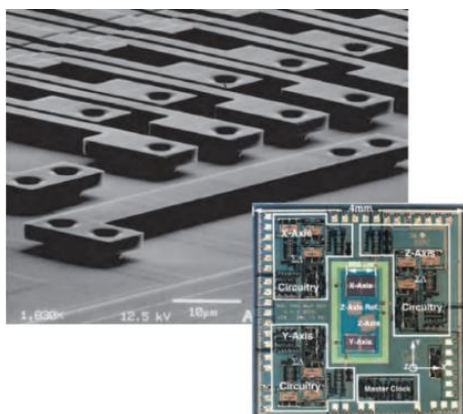
gia tốc riêng (proper acceleration), hay là gia tốc của một vật so với trạng thái nghỉ của nó.

Giả sử x là gia tốc riêng của khung của một gia tốc kế tại một thời điểm cụ thể. Một gia tốc kế kỹ thuật số sẽ tạo ra phép đo $f(x)$, trong đó:

$$f: (L; H) \rightarrow \{0, \dots, 2^b - 1\}$$

với $L \in R$ là gia tốc riêng nhỏ nhất có thể đo được và $H \in R$ là cực đại, và $b \in N$ là số bit của ADC.

Ngày nay, gia tốc kế thường được thực hiện bằng silicon (Hình 5.2) với biến dạng dưới lực hấp dẫn hoặc gia tốc. Mạch đo độ biến dạng và cung cấp giá trị ngõ ra dạng tín hiệu số. Thông thường, ba gia tốc kế được sử dụng kết hợp cùng nhau để tạo ra một gia tốc kế ba trục. Gia tốc kế ba trục có thể được sử dụng để đo hướng của một vật thể so với trọng lực, cộng với gia tốc theo bất kỳ hướng nào trong không gian ba chiều.



Hình 5.2: Gia tốc kế silicon

5.1.2 Đo vị trí và vận tốc

Về lý thuyết, với một số đo x của gia tốc theo thời gian, có thể xác định được vận tốc và vị trí của vật. Chúng ta hãy xem xét một vật chuyển động trong không gian một chiều. Gọi vị trí của vật theo thời gian là $p: R_+ \rightarrow R$, với vị trí ban đầu $p(0)$. Chúng ta giả sử gọi $v: R_+ \rightarrow R$ là vận tốc của vật với vận tốc ban đầu là $v(0)$ và gia tốc là $x: R_+ \rightarrow R$. Khi đó ta có các công thức sau:

$$p(t) = p(0) + \int_0^t v(\tau) d\tau \quad (5.1)$$

$$v(t) = v(0) + \int_0^t x(\tau) d\tau \quad (5.2)$$

Chúng ta cần lưu ý rằng nếu có sai số khác 0 trong phép đo gia tốc, thì $p(t)$ sẽ có sai số tăng tỷ lệ với t^2 . Sai số này được gọi là **độ lệch** (drift) và nó làm cho việc sử dụng gia tốc kế để xác định vị trí không hữu ích lắm. Tuy nhiên, nếu vị trí có thể được đặt lại một giá trị tốt theo định kỳ, ví dụ như sử dụng GPS, khi đó gia tốc kế trở nên hữu ích để ước tính vị trí giữa các cài đặt như vậy.

Trong một số trường hợp, chúng ta có thể đo vận tốc của một vật chuyển động trong một môi trường. Ví dụ, một máy đo gió (đo lưu lượng không khí) có thể ước tính vận tốc của một máy bay so với không khí xung quanh. Nhưng sử dụng phép đo này để ước tính vị trí thì một lần nữa có thể bị lệch, đặc biệt vì chuyển động của không khí xung quanh đảm bảo sai lệch.

Việc đo trực tiếp vị trí là rất khó. Hệ thống định vị toàn cầu (GPS) là một hệ thống định vị dựa trên vệ tinh tinh vi sử dụng phương pháp tam giác. Máy thu GPS thu nhận tín hiệu từ bốn hoặc nhiều vệ tinh GPS mang đồng hồ cực kỳ chính xác. Các vệ tinh truyền tín hiệu bao gồm thời gian truyền và vị trí của vệ tinh tại thời điểm truyền. Nếu máy thu có đồng hồ chính xác như nhau, khi nhận được tín hiệu như vậy từ vệ tinh, hệ thống sẽ có thể tính toán khoảng cách của nó với vệ tinh bằng tốc độ ánh sáng. Với ba khoảng cách được đưa ra như vậy, hệ thống sẽ có thể tính toán vị trí của riêng nó. Tuy nhiên, đồng hồ chính xác là cực kỳ đắt. Do đó, máy thu sử dụng phép đo khoảng cách thứ tư như vậy để có được một hệ thống của bốn phương trình với bốn ẩn số, vị trí ba chiều của nó và sai số trong đồng hồ của thiết bị.

Tín hiệu từ vệ tinh GPS tương đối yếu và dễ bị chặn bởi các tòa nhà và những trở ngại khác. Do đó, phải sử dụng các cơ chế khác để xác định vị trí trong nhà. Một cơ chế đó là định vị WiFi, trong đó thiết bị sử dụng vị trí đã biết của điểm phát WiFi, cường độ tín hiệu từ các điểm truy cập đó và thông tin cục bộ khác. Một công nghệ khác hữu ích cho việc định vị trong nhà là Bluetooth. Tín hiệu Bluetooth có thể được sử dụng làm báo hiệu và tín hiệu cường độ có thể cung cấp một chỉ báo sơ bộ về khoảng cách đến đèn hiệu.

Sử dụng cường độ của tín hiệu vô tuyến là thước đo khoảng cách thì độ chính xác thấp vì tín hiệu vô tuyến dễ bị nhiễu và phản xạ

cục bộ. Môi trường trong nhà, tín hiệu vô tuyến có tính chất **đa đường**, nơi nó truyền dọc theo nhiều đường tới một mục tiêu, và tại mục tiêu trải qua sự giao thoa mang tính xây dựng hoặc phá hủy. Sự giao thoa như vậy gây ra sự biến đổi lớn về cường độ tín hiệu có thể dẫn đến sai lệch các phép đo khoảng cách. Theo bài viết này, các cơ chế để định vị trong nhà chính xác không có sẵn, trong khi đó định vị ngoài trời, nơi GPS có sẵn trên toàn thế giới.

5.1.3 Đo vòng quay

Con quay hồi chuyển (gyroscope) là một thiết bị đo những thay đổi về hướng (quay). Không giống như một gia tốc kế, nó không bị ảnh hưởng bởi trường hấp dẫn. Con quay truyền thống là các thiết bị cơ khí quay công kênh trên giá treo bằng đồng. Con quay hồi chuyển hiện đại là thiết bị **MEMS** (hệ thống vi cơ điện tử - microelectromechanical systems) sử dụng cấu trúc cộng hưởng nhỏ, hoặc các thiết bị quang học đo sự khác biệt về khoảng cách di chuyển của chùm tia laze xung quanh đường dẫn khép kín theo các hướng ngược nhau hoặc (đối với độ chính xác cực cao) các thiết bị tận dụng hiệu ứng lượng tử.

Con quay hồi chuyển và máy đo gia tốc có thể được kết hợp để cải thiện độ chính xác của quán tính điều hướng (inertial navigation), trong đó vị trí được ước tính bằng phương pháp dẫn đường dự đoán (dead reckoning). Còn được gọi là tính toán suy luận (đối với tính toán suy diễn), phương pháp dẫn đường dự đoán

bắt đầu từ vị trí và hướng ban đầu đã biết, sau đó sử dụng các phép đo chuyển động để ước tính vị trí và hướng tiếp theo. Một đơn vị đo lường quán tính (IMU) hoặc hệ thống định vị quán tính (INS) sử dụng con quay hồi chuyển để đo những thay đổi về hướng và một gia tốc kế để đo những thay đổi trong vận tốc. Tất nhiên, các đơn vị như vậy có thể bị lệch, vì vậy chúng thường được kết hợp với GPS đơn vị, có thể cung cấp định kỳ thông tin vị trí “tốt đã biết” (mặc dù không định hướng). IMU có thể khá phức tạp và đắt tiền.

5.1.4 Đo âm thanh

Micrô đo những thay đổi áp suất âm thanh. Một số kỹ thuật được sử dụng, bao gồm cả cảm ứng điện từ (trong đó áp suất âm thanh làm cho dây dẫn chuyển động trong từ trường), điện dung (trong đó khoảng cách giữa một tấm bị biến dạng bởi áp suất âm thanh và một tấm cố định khác nhau, gây ra sự thay đổi có thể đo được trong điện dung), hoặc hiệu ứng áp điện (nơi điện tích tích tụ trong tinh thể do áp lực cơ học).

Micrô cho âm thanh của con người được tạo ra có độ biến dạng âm thấp và tiếng ồn thấp trong dải tần số nghe của con người, khoảng 20 Hz đến 20.000 Hz. Nhưng micro cũng được sử dụng bên ngoài phạm vi này. Ví dụ: máy đo khoảng cách siêu âm phát ra âm thanh bên ngoài phạm vi thính giác của con người và lắng nghe tiếng vang. Nó có thể được sử dụng để đo khoảng cách bề mặt phản xạ âm thanh.

5.1.5 Các cảm biến khác

Ngoài các loại cảm biến trên, trong thực tế còn có nhiều loại cảm biến khác cảm biến nhiệt, cảm biến hóa chất, cảm biến ánh sáng, đồng hồ thời gian... Công tắc cũng có thể được xem là một cảm biến đặc biệt đơn giản được thiết kế phù hợp để cảm nhận được áp lực, độ nghiêng hoặc chuyển động và có thể được kết nối trực tiếp với các chân GPIO của vi điều khiển.

5.2 Cơ cấu chấp hành

Cơ cấu chấp hành rất là đa dạng tùy thuộc vào các mục đích ứng dụng. Việc trình bày đầy đủ các loại bộ truyền động là khá khó khăn. Trong phạm vi giáo trình này, chúng ta chỉ tìm hiểu về hai loại bộ truyền động phổ biến nhất là LED và động cơ điều khiển.

5.2.1 Điốt phát sáng

Rất ít thiết bị truyền động có thể được điều khiển trực tiếp bằng tín hiệu kỹ thuật số từ các chân nhập xuất của vi điều khiển (GPIO pins) vì các bộ truyền động có thể tạo ra các dòng hoặc gây sụt dòng dẫn đến nguy cơ tổn hại vi điều khiển. Tuy nhiên, một ngoại lệ của thiết bị truyền động là điốt phát quang (hay còn gọi là LED). LED có thể được nối trực tiếp với các chân của vi điều khiển. Điều này tạo sự thuận tiện cho một hệ thống nhúng để cung cấp một hình ảnh trực quan cho một số ứng dụng.

Ví dụ 5.1: Chúng ta xem xét một bộ vi điều khiển hoạt động ở điện áp 3 V và thiết lập chân GPIO của nó có thể chịu dòng điện tụt đến 18 mA. Giả sử chúng ta muốn bật và tắt LED trong bằng phần mềm. Đèn LED khi phân cực thuận (chế độ mở) và có hiệu điện thế giảm 2 V. Câu hỏi đặt ra ở đây là điện trở nhỏ nhất để nối tiếp với LED là bao nhiêu để giữ an toàn dòng điện trong giới hạn 18 mA?

Để trả lời cho câu hỏi trên, chúng ta dựa vào Định luật Ohm's như sau:

$$V_R = I * R \quad (5.3)$$

trong đó V_R là điện áp trên điện trở, I là dòng điện và R là điện trở. Khi đi qua điện trở, giá trị điện áp bị rơi là: $V_R = 3 - 2 = 1V$. Do đó, dòng điện đi qua điện trở sẽ là:

$$I = 1/R \quad (5.4)$$

Để giới hạn dòng điện này đến 18 mA, chúng ta cần một điện trở

$$R \geq \frac{1}{0.018} \approx 56 \Omega$$

Nếu chúng ta sử dụng một điện trở 100Ω thì dòng điện chạy qua điện trở và LED là: $I = V_R/100 = 10mA$.

Nếu dung lượng pin là 200 mAh thì việc sử dụng đèn LED trong 20 giờ sẽ làm cạn kiệt pin hoàn toàn, không tính bất kỳ mức tiêu hao năng lượng nào trong bộ vi điều khiển hoặc các mạch khác. Công suất tiêu tán trong điện trở sẽ là $P_R = V_R I = 10mW$. Công

suất tiêu tán trong đèn LED sẽ là $P_L = 2I = 20mW$. Những con số này cho biết nhiệt lượng tạo ra bởi mạch LED.

Các phép tính trong ví dụ trên là điển hình về những gì chúng ta cần làm để kết nối bất kỳ thiết bị nào đến bộ vi điều khiển.

5.2.2 Điều khiển động cơ

Một **động cơ** (motor) tác dụng một mômen (lực góc) lên tải tỷ lệ với dòng điện chạy qua cuộn dây động cơ. Do đó, một ý tưởng hấp dẫn là đặt đặt vào động cơ một điện áp tỷ lệ với mômen mong muốn. Tuy nhiên, đây hiếm khi là một ý tưởng hay. Đầu tiên, nếu điện áp được điều khiển kỹ thuật số thông qua DAC thì chúng ta phải rất cẩn thận để không vượt quá giới hạn hiện tại của DAC. Hầu hết các DAC

Ví dụ 5.2: Bộ khuếch đại âm thanh được thiết kế để điều khiển loa 8Ω thường có thể cung cấp $100W$ cho loa. Kết hợp công thức tính công suất và Định luật Ohm, chúng ta có thể xác định được dòng điện như sau:

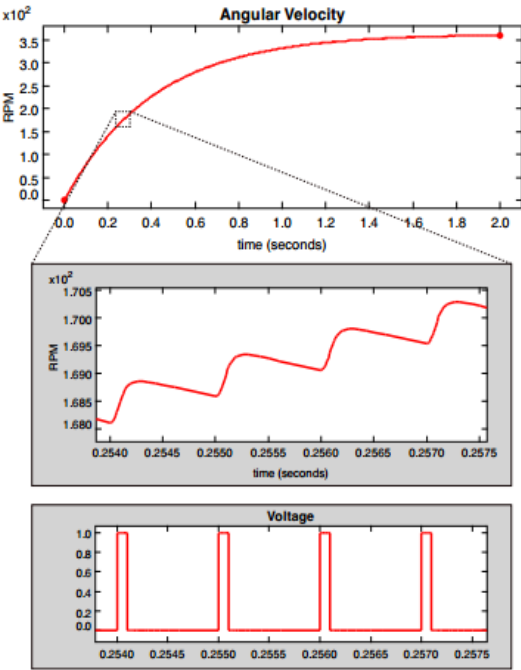
$$I = \sqrt{P/R} = \sqrt{100/8} \approx 3.5A$$

Trong đó, $P = RI^2$, R là điện trở. Do đó, ở 100 watt, dòng điện qua loa 8Ω là một dòng điện thực. Mạch điện trong bộ khuếch đại công suất có thể cung cấp một dòng điện như vậy mà không quá nóng và không gây biến dạng là khá tinh vi.

không thể cung cấp nhiều điện năng và yêu cầu nguồn cấp cho bộ khuếch đại giữa DAC và thiết bị được cấp nguồn. Một giải pháp có thể là kết nối với DAC một bộ khuếch đại công suất do đầu vào cho bộ khuếch đại công suất có trở kháng cao nên nó tiêu tốn rất ít dòng

điện ở một điện áp cố định. Tuy nhiên, kết quả đầu ra có thể liên quan đến dòng điện thực.

Bộ khuếch đại công suất có độ tuyến tính tốt (trong đó điện áp đầu ra và dòng điện tỷ lệ với điện áp đầu vào) có thể đắt tiền, cồng kềnh và kém hiệu quả (bản thân bộ khuếch đại tiêu hao năng lượng đáng kể). May mắn thay, khi điều khiển động cơ, chúng ta thường không cần như một bộ khuếch đại công suất. Chỉ cần sử dụng một công tắc mà chúng ta có thể bật và tắt bằng tín hiệu kỹ thuật số từ vi điều khiển. Làm một công tắc chịu được dòng điện cao là dễ hơn chế tạo một bộ khuếch đại công suất.



Hình 5.3: Điều khiển động cơ DC bằng PWM

Ngoài ra, chúng ta còn có thể sử dụng bộ điều chế độ rộng xung như đề cập ở phần trước để có thể cung cấp hiệu quả lượng điện năng lớn dưới sự điều khiển kỹ thuật số, miễn là thiết bị mà nguồn điện đang được phân phối có thể chịu được việc bật và tắt nguồn điện nhanh chóng. Thiết bị chịu được điều này bao gồm đèn LED, đèn sợi đốt và DC động cơ. Tín hiệu PWM, như được hiển thị ở dưới cùng của Hình 5.3, chuyển đổi giữa mức cao và mức thấp ở một tần số xác định. Nó giữ tín hiệu cao cho một phần nhỏ của chu kỳ. Phần này được gọi là chu trình hoạt động (duty cycle), và trong Hình 5.3, là 0.1 tức 10%.

Động cơ điện một chiều (DC) bao gồm một nam châm điện được tạo ra bằng cách cuộn dây xung quanh một lõi được đặt trong từ trường được tạo ra bằng nam châm vĩnh cửu hoặc nam châm điện. Khi dòng điện chạy qua các dây, lõi quay. Một động cơ như vậy có cả quán tính và điện cảm giúp động cơ hoạt động mượt mà khi dòng điện được bật và tắt đột ngột. Các động cơ như vậy có thể chịu được PWM tính hiệu tốt.

Gọi $\omega: \mathbb{R} \rightarrow \mathbb{R}$ là vận tốc góc của động cơ dưới dạng một hàm của thời gian. Giả định chúng ta áp dụng một điện áp v cho động cơ, cũng là một hàm của thời gian. Sau đó, sử dụng lý thuyết mạch cơ bản, chúng ta xác định điện áp và dòng điện qua động cơ thỏa mãn phương trình sau:

$$v(t) = Ri(t) + L \frac{di(t)}{dt} \quad (5.5)$$

trong đó R là điện trở và L là độ tự cảm của các cuộn dây trong động cơ. Đó là, cuộn dây của động cơ được mô hình như chuỗi kết nối của một điện trở và điện cảm. Điện áp rơi trên điện trở tỷ lệ với dòng điện, và điện áp giảm trên điện cảm tỷ lệ với tốc độ thay đổi của dòng điện.

Tuy nhiên, động cơ có hiện tượng khi cuộn dây quay trong từ trường, nó tạo ra một dòng điện (và điện áp tương ứng). Trên thực tế, một động cơ cũng có thể hoạt động như một máy phát điện. Ngoài ra, động cơ có là một số mô-men xoắn chống lại sự quay, được gọi là lực điện từ ngược, do xu hướng tạo ra dòng điện khi quay. Do đó, phương trình xác định vận tốc động cơ trở thành:

$$v(t) = Ri(t) + L \frac{di(t)}{dt} + k_b \omega(t) \quad (5.6)$$

trong đó k_b là hằng số lực điện từ ngược xác định, thường được biểu thị bằng đơn vị V/rpm (volt trên vòng quay mỗi phút).

Trong cách sử dụng điển hình của bộ điều khiển PWM để điều khiển động cơ, chúng ta sử dụng kỹ thuật điều khiển phản hồi (feedback control) để đặt tốc độ của động cơ thành RPM mong muốn. Để làm điều này, chúng ta cần đo tốc độ của động cơ. Chúng ta có thể sử dụng một cảm biến gọi là máy quay bộ mã hóa hoặc chỉ bộ mã hóa, thông báo vị trí góc hoặc vận tốc (hoặc cả hai) của một trục quay. Có nhiều thiết kế khác nhau cho các bộ mã hóa như vậy.

Một cái rất đơn giản cung cấp một xung điện mỗi khi trục quay một góc nhất định, để đếm xung trên một đơn vị thời gian sẽ cung cấp phép đo vận tốc góc.

5.3 Tóm tắt

Cảm biến và thiết bị truyền động là rất đa dạng cho các kỹ sư thiết kế ứng dụng. Trong chương này, chúng ta vừa tìm hiểu khái quát về các mô hình của các cảm biến và bộ truyền động. Các mô hình như vậy là một phần thiết yếu trong thiết kế hệ thống nhúng. Nếu không có những mô hình như vậy, các kỹ sư sẽ gặp khó khăn trong việc mô phỏng hệ thống, thử nghiệm và đánh giá.

5.4 Bài tập

Bạn là một phi công chiến đấu của Liên minh nổi dậy trốn tránh sự truy đuổi từ Đế chế Ngân hà bằng cách di chuyển con tàu không gian của bạn bên dưới những đám mây của hành tinh Cory. Đặt hướng dương z hướng lên trên và là vị trí của tàu so với mặt đất và v là vận tốc thẳng đứng của bạn. Lực hấp dẫn tác động mạnh lên hành tinh này và tạo ra một gia tốc (trong chân không) với giá trị tuyệt đối g . Lực từ lực cản của không khí là tuyến tính đối với vận tốc và bằng rv , trong đó hệ số cản $r \leq 0$ là một tham số không đổi của mô hình. Con tàu có khối lượng M . Động cơ của bạn cung cấp một lực thẳng đứng.

- (a) Gọi $L(t)$ là đầu vào là lực nâng thẳng đứng được cung cấp từ động cơ của bạn. Xác định công thức động lực học cho

con tàu của bạn ở vị trí $z(t)$ và vận tốc $v(t)$. Bỏ qua giả thuyết khi tàu của bạn gặp sự cố. Vẽ bên phải của công thức chứa $v(t)$ và $L(t)$.

- (b) Đưa ra câu trả lời của bạn cho bài toán trước, hãy viết ra phương án rõ ràng cho $z(t)$ và $v(t)$ khi lực cản của không khí không đáng kể và $r = 0$. Tại thời điểm ban đầu $t = 0$, bạn đang ở độ cao 30m so với mặt đất và có vận tốc đầu là $-10 \frac{m}{s}$. Gợi ý: Viết $v(t)$ trước sau đó viết $z(t)$ dưới dạng $v(t)$.
- (c) Vẽ mô hình tác nhân bằng cách sử dụng bộ tích hợp, bộ cộng, v.v. cho hệ thống tạo ra vị trí thẳng đứng và vận tốc của bạn. Đảm bảo gắn nhãn tất cả các biến trong mô hình tác nhân của bạn.
- (d) Động cơ của bạn bị hư hỏng nhẹ và bạn chỉ có thể kiểm soát nó bằng cách đưa một đầu vào thuận túy, công tắc (SW), khi đó ngay lập tức thể hiện chuyển trạng thái của động cơ từ bật sang tắt và ngược lại. Khi bật, động cơ tạo ra lực nâng dương L và khi tắt $L = 0$. Bảng điều khiển thiết bị của bạn có một gia tốc kế. Giả sử tàu vũ trụ của bạn ngang bằng (tức là góc độ bằng không) và trục z dương của gia tốc kế hướng lên trên. Hãy để trình tự đầu vào của các lệnh chuyển đổi động cơ là:

$$SW(t) = \begin{cases} 1 & \text{nếu } t \in \{.5, 1.5, 2.5, \dots\} \\ 0 & \text{ngược lại} \end{cases}$$

Để giải quyết sự mơ hồ tại thời điểm chuyển mạch $t = .5, 1.5; 2.5; \dots$, tại thời điểm chuyển đổi, lực của động cơ nhận giá trị mới ngay lập tức. Giả sử rằng lực cản của không khí là không đáng kể (tức là $r = 0$), bỏ qua trạng thái va chạm và động cơ đang bật ở $t = 0$. Phác họa thành phần thẳng đứng của số đọc gia tốc kế như một hàm của thời gian $t \in R$. Ghi nhận các giá trị quan trọng trên các trục. Gợi ý: Việc phác thảo biểu đồ lực trước tiên sẽ rất hữu ích.

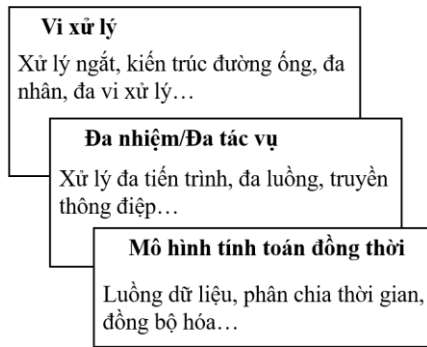
Nếu tàu vũ trụ đang bay ở độ cao không đổi, giá trị được đọc bởi gia tốc kế là bao nhiêu?

Chương 6 HỆ THỐNG ĐA NHIỆM

Trong chương này, chúng ta thảo luận về các cơ chế được sử dụng trong phần mềm để cung cấp việc thực thi đồng thời các mã chương trình tuần tự. Có nhiều lý do để thực thi đồng thời nhiều chương trình tuần tự, nhưng tất cả đều liên quan đến thời gian. Lý do thứ nhất là để cải thiện khả năng đáp ứng bằng cách tránh các trường hợp trong đó các chương trình chạy lâu có thể cản trở chương trình đáp ứng với các kích thích bên ngoài như cảm biến hoặc yêu cầu của người dùng. Khả năng đáp ứng được cải thiện làm giảm độ trễ, thời gian từ khi xuất hiện kích thích đến phản hồi. Một lý do khác là để cải thiện hiệu suất bằng cách thực thi một chương trình đồng thời trên nhiều bộ vi xử lý hoặc lõi. Đây cũng là một vấn đề về thời gian, vì hoàn thành tác vụ sớm hơn sẽ tốt hơn là muộn hơn. Lý do thứ ba là trực tiếp kiểm soát thời gian của các tương tác bên ngoài. Một chương trình có thể cần thực hiện một số thao tác, chẳng hạn như cập nhật một màn hình, vào những thời điểm cụ thể, bất kể những tác vụ khác có thể đang thực hiện tại thời điểm đó.

Chúng ta đã thảo luận về tính đồng thời trong nhiều ngữ cảnh khác nhau. Hình 6.1 mô tả mối quan hệ giữa các cấp độ xử lý đồng thời. Ở tầng thấp nhất là phần cứng cung cấp các cơ chế đồng thời cho kỹ sư thiết kế phần mềm bao gồm các mô hình trừu tượng của xử lý đồng thời, thành phần đồng bộ, luồng dữ liệu và mô hình kích hoạt thời gian. Tầng hai là mức độ xử lý song song trên tiến trình,

tác vụ và các kỹ thuật liên quan cho phép đồng bộ hóa giữa các tiến trình. Tầng thứ 3 là các cơ chế phân cứng hỗ trợ xử lý song song như kiến trúc đường ống, đa nhân, đa vi xử lý. Nội dung chương này sẽ trình bày cơ chế ở tầng hai – đa tác vụ - để kết nối tầng một và tầng ba của mô hình xử lý song song. Chúng ta sẽ xem xét các cơ chế để thực hiện các xử lý ở cấp thấp và cung cấp cơ sở để hiện thực hóa các cơ chế cấp cao. Nói chung, các kỹ thuật này được gọi là đa nhiệm, nghĩa là thực hiện đồng thời nhiều tác vụ.



Hình 6.1: Các khái niệm trừu tượng về chương trình xử lý đồng thời

Các kỹ sư thiết kế hệ thống nhúng thường sử dụng trực tiếp các cơ chế cấp trung này để xây dựng các ứng dụng; tuy nhiên, việc các kỹ sư thiết kế sử dụng các cơ chế cấp cao ngày càng trở nên phổ biến. Người thiết kế xây dựng một mô hình bằng cách sử dụng một công cụ phần mềm hỗ trợ một hoặc một số mô hình tính toán. Sau đó, mô hình được biên dịch tự động hoặc bán tự động thành một chương trình sử dụng các cơ chế cấp trung hoặc cấp thấp. Cách gọi khác của quá trình dịch là tạo mã hoặc tự động mã hóa.

Các cơ chế được mô tả trong chương này thường được cung cấp bởi một hệ điều hành, một vi nhân hoặc một thư viện các thủ tục. Chúng có thể khá phức tạp để thực hiện một cách chính xác, và do đó cần các chuyên gia. Các kỹ sư phần mềm ứng dụng hệ thống nhúng thường thực hiện các cơ chế như vậy dựa trên một bộ vi xử lý không có hệ điều hành (thường gọi là **bare iron**). Để làm được một cách chính xác đòi hỏi sự hiểu biết sâu sắc về các vấn đề xử lý đồng thời.

Chương này bắt đầu với mô tả ngắn gọn về các mô hình cho các chương trình tuần tự, cho phép các mô hình kết hợp đồng thời của các chương trình tuần tự đó. Sau đó, chúng ta tiến hành thảo luận về các luồng, quy trình và truyền thông điệp, là ba kiểu cấu tạo của các chương trình tuần tự.

```

#include <stdlib.h>
#include <stdio.h>
int x; // Biến cập nhật
typedef void notifyProcedure(int); // Kiểu của thủ tục Notify
struct element{
    notifyProcedure* listener; // Con trỏ trỏ tới thủ tục Notify
    struct element* next; // Con trỏ trỏ tới thành phần tiếp theo
};
typedef struct element element_t; // Kiểu của danh sách thành phần
element_t* head = 0; // Con trỏ đầu danh sách
element_t* tail = 0; // Con trỏ cuối danh sách

// Thủ tục tạo một "listener"
void addListener(notifyProcedure* listener){
    if(head == 0){
        head = malloc(sizeof(element_t));
        head -> listener = listener;
        head -> next = 0;
        tail = head;
    } else {
        tail -> next = malloc(sizeof(element_t));
        tail = tail -> next;
        tail -> listener = listener;
        tail -> next = 0;
    }
}

// Thủ tục để cập nhật biến "x"
void update(int newx){
    x = newx;
    //Notify listener
    element_t* element = head;
    while(element != 0){
        (*(element -> listener))(newx);
        element = element -> next;
    }
}

// Ví dụ sử dụng thủ tục
void print(int arg){
    printf("%d", arg);
}

```

Hình 6.2: Minh họa đoạn chương trình C cấp cao

6.1 Lập trình lệnh

Một ngôn ngữ lập trình thể hiện các tính toán như một chuỗi các thao tác được gọi là ngôn ngữ mệnh lệnh. C là một ngôn ngữ mệnh lệnh.

Ví dụ 6.1: Trong chương này, chúng ta minh họa một số điểm chính bằng cách sử dụng chương trình C. Chương trình này thực hiện một mẫu thiết kế thường được sử dụng gọi là mẫu bộ quan sát [12]. Trong mẫu này, một thủ tục cập nhật thay đổi giá trị của một biến x . Bộ giám sát (là các chương trình khác hoặc các phần khác của chương trình) sẽ được thông báo bất cứ khi nào x được thay đổi bằng cách gọi một thủ tục *callback*. Ví dụ: giá trị của x có thể được bộ quan sát hiển thị trên màn hình. Bất cứ khi nào giá trị thay đổi, bộ giám sát cần được thông báo để có thể cập nhật hiển thị trên màn hình. Tiến trình chính sau đây sử dụng các tiến trình được xác định trong Hình 6.2:

```
1  int main(void){
2      addListener (&print);
3      addListener (&print);
4      update (1);
5      addListener (&print);
6      update (2)
7      return 0;
8  }
```

Chương trình kiểm thử này đăng ký thủ tục *print* dưới dạng gọi lại hai lần, sau đó thực hiện cập nhật (thiết lập $x = 1$), sau đó đăng

ký lại thủ tục *print* và cuối cùng thực hiện cập nhật khác (thiết lập $x = 2$). Quy trình *print* chỉ đơn giản là in giá trị hiện tại, vì vậy kết quả đầu ra khi thực hiện chương trình kiểm thử này là 1 1 2 2 2.

Chương trình C chỉ định một chuỗi các bước, trong đó mỗi bước thay đổi trạng thái của bộ nhớ trong máy. Trong C, trạng thái của bộ nhớ trong máy được biểu diễn bằng giá trị của các biến.

Sử dụng máy trạng thái mở rộng, chúng ta có thể mô hình hóa việc thực thi các chương trình

C đơn giản nhất định, giả sử các chương trình có một số biến cố định và có giới hạn. Các biến của chương trình C sẽ là các biến của máy trạng thái. Các trạng thái của máy trạng thái sẽ đại diện cho các vị trí trong chương trình, và các chuyển hóa sẽ đại diện cho việc thực thi chương trình.

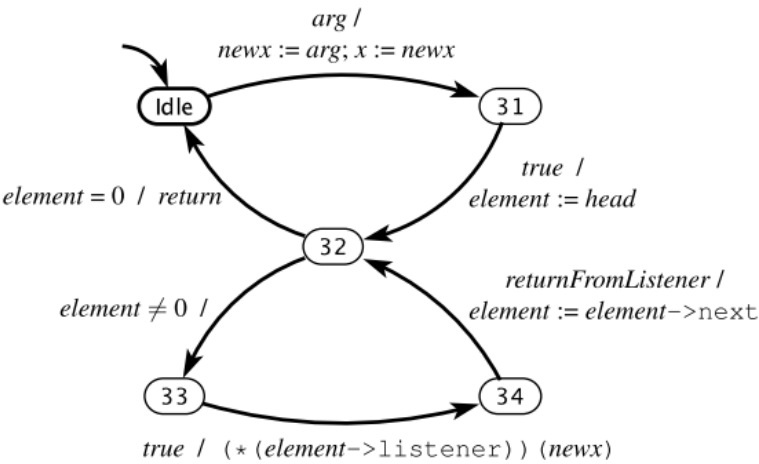
Ví dụ 6.2: Trong chương trình Hình 6.2, trạng thái bộ nhớ của máy bao gồm giá trị của biến x (là một biến toàn cục) và danh sách các phần tử được trỏ tới bởi biến *head* (một biến toàn cục khác). Bản thân danh sách được biểu diễn dưới dạng danh sách liên kết, trong đó mỗi phần tử trong danh sách chứa một con trỏ hàm tham chiếu đến một thủ tục được gọi khi x thay đổi.

Trong quá trình thực thi chương trình C, trạng thái của bộ nhớ của máy sẽ cần bao gồm trạng thái của *stack*, nó bao gồm bất kỳ biến cục bộ nào.

Ví dụ 6.3: Hình 6.3 cho thấy một mô hình của thủ tục cập nhật trong Hình 6.2. Máy chuyển từ trạng thái *Idle* ban đầu khi quy trình cập nhật được gọi. Cuộc gọi được báo hiệu bởi đối số đầu vào có

mặt; giá trị của nó sẽ là đối số int cho thủ tục cập nhật. Khi quá trình chuyển hóa này được thực hiện, newx (trên stack) sẽ được gán giá trị của đối số. Ngoài ra, x (một biến toàn cục) sẽ được cập nhật.

```
inputs: arg: int, returnFromListener: pure
outputs: return: pure
local variables: newx: int, element: element_t*
global variables: x: int, head: element_t*
```



Hình 6.3: Mô hình cập nhật trạng thái biến trong chương trình Hình 6.2

Sau lần chuyển trạng thái đầu tiên này, máy ở trạng thái 31, tương ứng với vị trí bộ đếm chương trình ngay trước khi thực hiện dòng 31 trong Hình 6.2. Sau đó, nó chuyển đổi hóa điều kiện sang trạng thái 32 và đặt giá trị của phần tử. Từ trạng thái 32, có hai khả năng; nếu phần tử = 0, thì máy chuyển trở lại trạng thái Idle và tạo ra đầu ra thuần túy return. Nếu không, nó chuyển sang 33.

Trong quá trình chuyển hóa từ 33 sang 34, action là một lệnh gọi thủ tục tới listener với đối số là biến stack. Quá trình chuyển

đổi từ 34 trở lại 32 xảy ra khi nhận được đầu vào thuần túy `returnFromListener`, cho biết rằng thủ tục trình nghe trả về.

Mô hình trong Hình 6.3 không phải là mô hình duy nhất mà chúng ta có thể xây dựng về thủ tục cập nhật. Khi xây dựng một mô hình như vậy, chúng ta cần quyết định mức độ chi tiết và chúng ta cần quyết định hành động nào có thể được coi là hoạt động nguyên tử một cách an toàn. Hình 6.3 sử dụng các dòng mã ở một mức độ chi tiết, nhưng không có gì đảm bảo rằng một dòng mã C thực thi nguyên tử (thường thì không).

Ngoài ra, các mô hình chính xác của chương trình C thường không phải là hệ thống trạng thái hữu hạn. Chỉ xem xét đoạn mã trong Hình 6.2, mô hình trạng thái hữu hạn không thích hợp vì mã hỗ trợ thêm một số lượng listener tùy ý vào danh sách. Nếu chúng ta kết hợp Hình 6.3 với thủ tục `main` trong Ví dụ 6.1, thì hệ thống là trạng thái hữu hạn vì chỉ có ba *listener* được đưa vào danh sách. Do đó, một mô hình trạng thái hữu hạn chính xác sẽ cần phải bao gồm chương trình hoàn chỉnh, khiến cho việc lập luận theo mô-đun về mã rất khó khăn.

Vấn đề trở nên tồi tệ hơn nhiều khi chúng ta thêm tính đồng thời vào hòa trộn này. Chúng tôi sẽ chỉ ra trong chương này rằng lý luận chính xác về các chương trình C với cơ chế đồng thời cấp trung như các luồng là một điều đáng kinh ngạc và dễ bị lỗi. Chính vì lý do này mà các kỹ sư thiết kế đang hướng tới lớp trên trong Hình 6.1.

6.2 Luồng xử lý

Luồng là các chương trình bắt buộc chạy đồng thời và chia sẻ một không gian bộ nhớ. Chúng có thể truy cập các biến của nhau. Các kỹ sư trong lĩnh vực thiết kế nhúng sử dụng thuật ngữ "luồng" hẹp hơn để chỉ các cách xây dựng chương trình chia sẻ một bộ nhớ cụ thể. Trong giáo trình này, chúng ta sẽ sử dụng thuật ngữ này một cách rộng hơn để chỉ bất kỳ cơ chế nào mà các chương trình mệnh lệnh chạy đồng thời và chia sẻ bộ nhớ. Theo nghĩa rộng này, các luồng tồn tại dưới dạng ngắt trên hầu hết tất cả các bộ vi xử lý, thậm chí không có bất kỳ hệ điều hành nào (bare iron).

6.2.1 Tạo luồng

Hầu hết các hệ điều hành cung cấp cơ chế cấp cao hơn ngắt để nhận ra các chương trình mệnh lệnh chia sẻ bộ nhớ. Cơ chế được cung cấp dưới dạng một tập hợp các thủ tục mà một kỹ sư phần mềm có thể sử dụng. Các thủ tục như vậy thường tuân theo một API chuẩn hóa (giao diện chương trình ứng dụng), giúp các kỹ sư xây dựng chương trình linh hoạt hơn (chương trình sẽ chạy trên nhiều bộ vi xử lý và/hoặc nhiều hệ điều hành). **Pthreads** (hoặc chuỗi POSIX) là một API như vậy; nó được tích hợp vào nhiều hệ điều hành hiện đại. Pthreads định nghĩa một tập hợp các kiểu ngôn ngữ lập trình C, các hàm và các hằng số. Nó được IEEE chuẩn hóa vào năm 1988 để thống nhất các biến thể của Unix. Trong Pthreads,

một luồng được xác định bởi một thủ tục C và được tạo ra bằng cách gọi thủ tục tạo pthread.

Ví dụ 6.4: Một chương trình C đa luồng đơn giản sử dụng Pthreads được thể hiện trong Hình 6.4. Thủ tục printN (dòng 3-9) - thủ tục mà luồng bắt đầu thực thi - được gọi là quy trình bắt đầu; trong trường hợp này, quy trình bắt đầu in đối số được truyền cho nó 10 lần rồi thoát ra, điều này sẽ khiến luồng kết thúc. Thủ tục *main* tạo ra hai luồng, mỗi luồng sẽ thực hiện quy trình bắt đầu. Giá trị đầu tiên, được tạo trên dòng 14, sẽ in giá trị 1. Giá trị thứ hai, được tạo trên dòng 15, sẽ in giá trị 2. Khi bạn chạy chương trình này, các giá trị 1 và 2 sẽ được in theo một số thứ tự xen kẽ phụ thuộc vào bộ định thời luồng. Thông thường, các lần chạy lặp lại sẽ mang lại các thứ tự xen kẽ khác nhau của 1 và 2.

```
1  #include <pthread.h>
2  #include <stdio.h>
3  void* printN(void* arg) {
4      int i;
5      for (i = 0; i < 10; i++) {
6          printf("My ID: %d\n", *(int*)arg);
7      }
8      return NULL;
9  }
10 int main(void) {
11     pthread_t threadID1, threadID2;
12     void* exitStatus;
13     int x1 = 1, x2 = 2;
14     pthread_create(&threadID1, NULL, printN, &x1);
15     pthread_create(&threadID2, NULL, printN, &x2);
16     printf("Started threads.\n");
17     pthread_join(threadID1, &exitStatus);
18     pthread_join(threadID2, &exitStatus);
19     return 0;
20 }
```

Hình 6.4: Chương trình C đa luồng đơn giản sử dụng Pthreads

Thủ tục tạo pthread tạo một luồng và trả về ngay lập tức. Quy trình bắt đầu có thể đã thực sự bắt đầu chạy khi nó quay trở lại. Dòng 17 và 18 sử dụng tham gia pthread để đảm bảo rằng chương trình chính không kết thúc trước khi các luồng kết thúc. Nếu không có hai dòng này, việc chạy chương trình có thể không mang lại bất kỳ đầu ra nào từ các luồng.

Một thủ tục bắt đầu thường lệ có thể trở lại hoặc không. Trong các ứng dụng nhúng, việc xác định các quy trình khởi động không bao giờ quay trở lại là điều khá phổ biến. Ví dụ, quy trình bắt đầu có thể thực thi vĩnh viễn và cập nhật màn hình định kỳ. Nếu quy trình bắt đầu thường lệ không trở lại, thì bất kỳ luồng nào khác gọi *pthread_join* của nó sẽ bị chặn vô thời hạn.

Như trong Hình 6.4, quy trình bắt đầu có thể được cung cấp một đối số và có thể trả về một giá trị. Đối số thứ tư pthread_create là địa chỉ của đối số sẽ được

Ví dụ 6.5: Giả sử chúng ta cố gắng tạo một luồng bên trong một thủ tục như sau:

```
1 pthread_t createThread(int x) {  
2     pthread_t ID;  
3     pthread_create(&ID, NULL, printN, &x)  
4     return ID;  
5 }
```

Đoạn mã này sẽ không chính xác vì đối số cho quy trình bắt đầu thường lệ được cung cấp bởi một con trỏ đến một biến trên stack. Vào thời điểm luồng truy cập địa chỉ bộ nhớ được chỉ định, thủ tục createThread có thể sẽ quay trở lại và địa chỉ bộ nhớ sẽ bị ghi đè bởi bất cứ điều gì xảy ra trên stack tiếp theo.

chuyển đến quy trình bắt đầu thường lệ. Điều quan trọng là phải hiểu mô hình bộ nhớ của C nếu không một số lỗi rất nhỏ có thể xảy ra, như được minh họa trong **Ví dụ 6.5**.

6.2.2 Thực thi các luồng

Cốt lõi của việc triển khai các luồng là một bộ định thời quyết định luồng nào sẽ thực thi tiếp theo khi một bộ vi xử lý sẵn sàng. Quyết định có thể dựa trên sự công bằng, trong đó nguyên tắc là cung cấp cho mọi luồng hoạt động một cơ hội bình đẳng để được thực thi, dựa trên các ràng buộc về thời gian hoặc dựa trên một số thước đo về mức độ quan trọng hoặc độ ưu tiên. Các thuật toán định thời được thảo luận chi tiết trong Chương 7. Trong phần này, chúng tôi chỉ đơn giản mô tả cách một bộ định thời luồng sẽ hoạt động mà không cần quan tâm nhiều về cách nó đưa ra quyết định về luồng nào sẽ thực thi.

Câu hỏi quan trọng đầu tiên là làm thế nào và khi nào bộ định thời được gọi. Một kỹ thuật đơn giản được gọi là **đa nhiệm hợp tác** (*cooperative multitasking*) mà không làm ngắt một luồng trừ khi chính luồng đó gọi một thủ tục nhất định hoặc một trong một tập hợp thủ tục nhất định. Ví dụ, bộ định thời có thể can thiệp bất cứ khi nào và bất kỳ dịch vụ hệ điều hành nào được gọi bởi luồng đang thực thi. Một dịch vụ hệ điều hành được gọi bằng cách gọi một thủ tục trong thư viện. Mỗi luồng có stack riêng của nó và khi lệnh gọi thủ tục được thực hiện, địa chỉ trả về sẽ được đẩy lên stack. Nếu bộ

định thời xác định rằng luồng hiện đang thực thi sẽ tiếp tục thực thi thì dịch vụ được yêu cầu đã hoàn tất và thủ tục trở lại như bình thường. Nếu bộ định thời xác định rằng luồng nên bị tạm dừng và một luồng khác nên được chọn để thực thi thì thay vì trả về, bộ định thời tạo một bản ghi con trỏ stack của luồng hiện đang thực thi, rồi sửa đổi con trỏ stack để trỏ đến stack của luồng đã chọn. Sau đó, nó trở lại như bình thường bằng cách lấy địa chỉ trả về khỏi stack và tiếp tục thực thi, nhưng bây giờ trong một luồng mới.

Nhược điểm chính của đa nhiệm hợp tác là một chương trình có thể thực thi trong một thời gian dài mà không thực hiện bất kỳ lệnh gọi dịch vụ hệ điều hành nào, trong trường hợp đó các luồng khác sẽ bị **đói tài nguyên (starvation)**. Để khắc phục điều này, hầu hết các hệ điều hành có một ISR được thực thi vào những khoảng thời gian cố định. ISR này sẽ duy trì đồng hồ hệ thống, cung cấp cho các kỹ sư phần mềm ứng dụng một cách để có được thời gian hiện tại trong ngày và cho phép gọi định kỳ bộ định thời thông qua ngắt timer. Đối với hệ điều hành có đồng hồ hệ thống, *jiffy* là khoảng thời gian mà đồng hồ hệ thống ISR được gọi. Lưu ý, Các giá trị jiffy trong các phiên bản Linux thường thay đổi trong khoảng từ 1 ms đến 10 ms.

Giá trị của một *jiffy* được xác định bằng cách cân bằng các mối quan tâm về hiệu suất với độ chính xác thời gian cần thiết. Thời gian ngắn hơn có nghĩa là các chức năng định thời được thực hiện thường xuyên hơn, điều này có thể làm giảm hiệu suất tổng thể. Tốc độ nhanh hơn có nghĩa là độ chính xác của đồng hồ hệ thống kém hơn và việc chuyển đổi tác vụ xảy ra ít thường xuyên hơn, điều này có thể khiến các ràng buộc thời gian thực bị vi phạm. Đôi khi, khoảng *jiffy* được quy định bởi ứng dụng.

Ví dụ 6.6: Bảng điều khiển trò chơi thường sẽ sử dụng một giá trị *jiffy* được đồng bộ hóa với tốc độ khung hình của hệ thống truyền hình nhắm đến vì tác vụ quan trọng về thời gian đối với các hệ thống đó là tạo đồ họa ở tốc độ khung hình này. Ví dụ, NTSC là hệ thống truyền hình tương tự được sử dụng trong lịch sử ở hầu hết các nước Châu Mỹ, Nhật Bản, Hàn Quốc, Đài Loan và một số nơi khác. Nó có tốc độ khung hình là 59,94 Hz, do đó, một *jiffy* phù hợp sẽ là $1/59.94$ hoặc khoảng 16,68 ms. Với tiêu chuẩn truyền hình PAL (đường dây xen kẽ pha), được sử dụng ở hầu hết châu Âu và phần lớn phần còn lại của thế giới, tốc độ khung hình là 50 Hz, mang lại *jiffy* nhanh 20 ms.

Truyền hình analog đang dần được thay thế bởi các định dạng kỹ thuật số như ATSC. ATSC hỗ trợ một số tốc độ khung hình từ chỉ dưới 24 Hz đến 60 Hz và một số độ phân giải. Giả sử TV tuân thủ tiêu chuẩn, ký sư thiết kế bảng điều khiển trò chơi có thể chọn tốc độ khung hình và độ phân giải phù hợp với mục tiêu chất lượng và chi phí.

Ngoài các ngắt định kỳ và các lời gọi dịch vụ hệ điều hành, bộ định thời còn có thể được gọi khi luồng chặn vì một số lý do. Chúng ta sẽ thảo luận về một số cơ chế để chặn như vậy tiếp theo đây.

6.2.3 Loại trừ tương hỗ

Một luồng có thể bị tạm dừng giữa hai hoạt động nguyên tử bất kỳ để thực thi một luồng khác và/hoặc ISR. Thực tế này có thể làm cho việc suy luận về tương tác giữa các luồng trở nên cực kỳ khó khăn.

Ví dụ 6.7: Nhắc lại quy trình sau từ Hình 6.2:

```
14 void addListener(notifyProcedure* listener) {
15     if (head == 0) {
16         head = malloc(sizeof(element_t));
17         head->listener = listener;
18         head->next = 0;
19         tail = head;
20     } else {
21         tail->next = malloc(sizeof(element_t));
22         tail = tail->next;
23         tail->listener = listener;
24         tail->next = 0;
25     }
26 }
```

Giả sử rằng *addListener* được gọi từ nhiều hơn một luồng. Vậy thì điều gì có thể xảy ra? Đầu tiên, hai luồng có thể đồng thời sửa đổi cấu trúc dữ liệu danh sách liên kết, điều này có thể dễ dàng dẫn đến cấu trúc dữ liệu bị hỏng. Ví dụ, giả sử rằng một luồng bị treo ngay trước khi thực thi dòng 23. Giả sử rằng trong khi luồng bị treo, một luồng khác gọi *addListener*. Khi luồng đầu tiên tiếp tục thực thi ở dòng 23, giá trị của *tail* đã thay đổi. Nó không còn là giá trị

được đặt ở dòng 22! Phân tích cẩn thận cho thấy rằng điều này có thể dẫn đến một danh sách trong đó phần tử thứ hai đến cuối cùng của danh sách trở đến một địa chỉ ngẫu nhiên cho listener (bất kỳ thứ gì trong bộ nhớ được cấp phát bởi *malloc*) và listener thứ hai được thêm vào danh sách thì không còn trong danh sách nữa. Khi cập nhật được gọi, nó sẽ cố gắng thực thi một thủ tục tại địa chỉ ngẫu nhiên, điều này có thể dẫn đến lỗi phân đoạn hoặc tệ hơn, thực thi các nội dung bộ nhớ ngẫu nhiên như thể chúng là các lệnh!

Vấn đề được minh họa trong ví dụ trước được gọi là **tranh chấp** (*race condition*). Hai đoạn mã đồng thời chạy đua để truy cập vào cùng một tài nguyên và thứ tự chính xác mà việc truy cập của chúng xảy ra ảnh hưởng đến kết quả của chương trình. Không phải tất cả các tranh chấp đều tồi tệ như ví dụ trước, nơi một số kết quả của cuộc đua gây ra thất bại thảm hại. Một cách để ngăn chặn những thảm họa như vậy là sử dụng khóa loại trừ tương hỗ (*mutual exclusion lock* hay *mutex*), như được minh họa trong **Ví dụ 6.8**.

Ví dụ 6.8: Trong *Pthreads*, *mutex* được thực hiện bằng cách tạo thực thể của cấu trúc gọi là *pthread_mutex_t*. Ví dụ: chúng ta có thể sửa đổi thủ tục *addListener* như sau:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void addListener(notifyProcedure* listener) {
    pthread_mutex_lock(&lock);
    if (head == 0) {
        ...
    } else {
        ...
    }
    pthread_mutex_unlock(&lock);
}
```

Dòng đầu tiên tạo và khởi tạo một biến toàn cục được gọi là khóa. Dòng đầu tiên trong thủ tục *addListener* có được khóa. Nguyên tắc là chỉ có một luồng có thể giữ khóa tại một thời điểm. Thủ tục khóa *pthread_mutex* sẽ chặn cho đến khi luồng gọi có thể nhận được khóa.

Trong đoạn mã trên, khi *addListener* được gọi bởi một luồng và bắt đầu thực thi, khóa *pthread_mutex* sẽ không return cho đến khi không có luồng nào khác giữ khóa. Khi nó quay trở lại, chuỗi gọi này sẽ giữ khóa. *Pthread_mutex_unlock* gọi ở cuối sẽ giải phóng khóa. Việc không thể giải phóng khóa là một lỗi nghiêm trọng trong lập trình đa luồng.

Khóa loại trừ tương hỗ ngăn không cho hai luồng bất kỳ truy cập đồng thời hoặc sửa đổi tài nguyên được chia sẻ. Đoạn mã giữa khóa và mở khóa là một phần cực kì quan trọng. Tại bất kỳ thời điểm nào, chỉ một luồng có thể thực thi mã trong phần quan trọng như vậy. Một kỹ sư phần mềm có thể cần phải đảm bảo rằng tất cả các quyền truy cập vào tài nguyên được chia sẻ đều được bảo vệ tương tự bằng các khóa.

Ví dụ 6.9: Quy trình cập nhật trong Hình 6.2 không sửa đổi danh sách *listener*, nhưng nó đọc danh sách. Giả sử rằng luồng A gọi *addListener* và bị treo ngay sau dòng 21, điều này thực thi dòng sau:

21 *tail* → *next* = *malloc(sizeof(element_t));*

Giả sử rằng trong khi A bị treo, một luồng B khác gọi cập nhật, bao gồm đoạn mã sau:

```
element_t* element = head;
while (element != 0){
    (*(element->listener))(newx);
    element = element->next;
}
```

Điều gì sẽ xảy ra trên dòng 33 khi *element* == *tail* → *next*? Tại thời điểm đó, luồng B sẽ coi bất kỳ nội dung ngẫu nhiên nào có trong bộ nhớ được trả về bởi *malloc* ở dòng 21 như một con trỏ hàm và cố gắng thực hiện một thủ tục được trỏ tới bởi con trỏ đó. Một lần nữa, điều này sẽ dẫn đến lỗi phân đoạn hoặc tệ hơn.

Mutex được thêm vào trong Ví dụ 6.8 không đủ để ngăn chặn thảm họa này. *Mutex* không ngăn luồng A bị treo. Do đó, chúng ta cần bảo vệ tất cả các truy cập của cấu trúc dữ liệu bằng *mutex*, điều này chúng ta có thể thực hiện bằng cách sửa đổi cập nhật như sau:

```

void update(int newx){
    x = newx;
    // Gửi thông báo đến listener
    pthread_mutex_lock(&lock);
    element_t* element = head;
    while (element != 0){
        (*(element ->
listener))(newx);
        element = element -> next;
    }
    pthread_mutex_unlock(&lock);
}

```

Điều này sẽ ngăn thủ tục cập nhật đọc cấu trúc dữ liệu danh sách trong khi nó đang được sửa đổi bởi bất kỳ luồng nào khác.

6.2.4 Tắc nghẽn

Khi khóa *mutex* phát triển trong các chương trình, nguy cơ các *thread* trong hệ thống bị khóa sẽ tăng lên. **Tắc nghẽn** (Deadlock) xảy ra khi một số luồng bị chặn vĩnh viễn khi cố gắng lấy khóa. Ví dụ, nếu luồng A giữ *lock1* và sau đó các khối đang cố gắng lấy *lock2*, được giữ bởi luồng B và sau đó các khối luồng B cố gắng lấy *lock1*. Những vòng lặp như vậy không có lối thoát rõ ràng. Do đó, chương trình cần bị hủy bỏ.

Ví dụ 6.10: Giả sử rằng cả *addListener* và *update* trong Hình 6.2 đều được bảo vệ bởi *mutex*, như trong hai ví dụ trước. Quy trình cập nhật bao gồm dòng:

```

33  (* (element → listener))(newx);

```

để gọi thủ tục trở đến danh sách phần tử. Sẽ không vô lý khi thủ tục đó tự nó cần phải có một khóa *mutex*. Ví dụ, giả sử rằng thủ tục *listener* cần cập nhật một màn hình. Màn hình thường là tài nguyên được chia sẻ và do đó có thể sẽ phải được bảo vệ bằng khóa *mutex* của riêng nó. Giả sử rằng luồng A gọi cập nhật, đến dòng 33 và sau đó chặn vì thủ tục *listener* cố gắng lấy một khóa khác được giữ bởi luồng B. Giả sử sau đó luồng B gọi *addListener*. Tắc nghẽn!

Deadlock có thể được ngăn chặn. Trong công trình nghiên cứu của mình, Coffman đưa ra các điều kiện cần thiết để xảy ra tắc nghẽn, và bất kỳ điều kiện nào trong số đó có thể được gỡ bỏ để tránh tắc nghẽn [10]. Một kỹ thuật đơn giản là chỉ sử dụng một khóa trong toàn bộ chương trình đa luồng. Tuy nhiên, kỹ thuật này không dẫn đến lập trình mô-đun. Hơn nữa, nó có thể gây khó khăn cho việc đáp ứng các hạn chế về thời gian thực vì một số tài nguyên được chia sẻ (ví dụ: màn hình) có thể cần được lưu giữ đủ lâu để khiến thời hạn bị bỏ lỡ trong các chuỗi khác.

Trong một *microkernel* rất đơn giản, đôi khi chúng ta có thể sử dụng tính năng bật và tắt ngắt như một *mutex* chung. Giả sử rằng chúng ta có một bộ vi xử lý duy nhất (không phải đa lõi) và các ngắt là cơ chế duy nhất mà một luồng có thể bị tạm ngưng (tức là chúng không bị treo khi gọi các dịch vụ nhân hoặc chặn trên I/O). Với những giả định này, việc vô hiệu hóa ngắt sẽ ngăn một luồng bị treo.

Tuy nhiên, trong hầu hết các hệ điều hành, các luồng có thể bị tạm ngưng vì nhiều lý do, vì vậy kỹ thuật này sẽ không hoạt động.

Kỹ thuật thứ ba là đảm bảo rằng khi có nhiều khóa *mutex*, mọi luồng đều có được các khóa theo cùng một thứ tự. Tuy nhiên, điều này có thể khó đảm bảo vì một số lý do. Đầu tiên, hầu hết các chương trình được viết bởi nhiều người và các khóa có được trong một thủ tục không phải là một phần nổi bật của thủ tục. Vì vậy, kỹ thuật này dựa trên tài liệu và sự hợp tác rất cẩn thận và nhất quán giữa một nhóm phát triển. Và bất kỳ khi nào một khóa được thêm vào, thì tất cả các phần của chương trình có được khóa có thể phải được sửa đổi.

Thứ hai, nó có thể làm cho việc viết mã chính xác trở nên cực kỳ khó khăn. Nếu một kỹ sư phần mềm muốn gọi một thủ tục có được *lock1*, theo quy ước trong chương trình luôn là khóa đầu tiên có được, thì trước tiên nó phải giải phóng bất kỳ khóa nào mà nó giữ. Ngay sau khi nó giải phóng những ổ khóa đó, nó có thể bị treo và tài nguyên mà nó giữ những ổ khóa đó để bảo vệ có thể bị sửa đổi. Khi nó đã có được *lock1*, sau đó nó phải yêu cầu lại các khóa đó, nhưng sau đó nó sẽ cần phải cho rằng nó không còn biết gì về trạng thái của tài nguyên và nó có thể phải thực hiện lại công việc đáng kể.

Có nhiều cách khác để ngăn chặn tắc nghẽn. Tuy nhiên, hầu hết các kỹ thuật có sẵn hoặc áp đặt những ràng buộc chặt chẽ đối với

kỹ sư phần mềm hoặc đòi hỏi sự phức tạp đáng kể để áp dụng, điều này cho thấy rằng vấn đề có thể là với mô hình lập trình đồng thời của các luồng.

6.2.5 Đồng nhất bộ nhớ

Như thế điều kiện thác nước và tắc nghẽn không đủ phức tạp, các luồng cũng gặp phải các vấn đề phức tạp tiềm ẩn với mô hình bộ nhớ của chương trình. Bất kỳ sự triển khai cụ thể nào của các luồng đều cung cấp một số loại mô hình nhất quán của bộ nhớ (*memory consistancy*), mô hình này xác định cách các biến được đọc và ghi bởi các luồng khác nhau xuất hiện với các luồng đó. Theo trực giác, việc đọc một biến sẽ mang lại giá trị cuối cùng được ghi vào biến, nhưng “cuối cùng” có nghĩa là gì? Hãy xem xét một tình huống, ví dụ, trong đó tất cả các biến được khởi tạo với giá trị 0 và luồng A thực hiện hai câu lệnh sau:

$$1 \quad x = 1;$$

$$2 \quad w = y;$$

trong khi luồng B thực thi hai câu lệnh sau:

$$1 \quad y = 1;$$

$$2 \quad z = x;$$

Theo trực giác, sau khi cả hai luồng đã thực hiện các câu lệnh này, chúng ta sẽ mong đợi rằng ít nhất một trong hai biến w và z có giá trị 1. Sự đảm bảo như vậy được gọi là tính nhất quán tuần tự (*sequential consistancy*). Tính nhất quán tuần tự có nghĩa là kết quả của bất kỳ quá trình thực thi nào cũng giống như khi các hoạt động

của tất cả các luồng được thực thi theo một số thứ tự tuần tự và các hoạt động của từng luồng riêng lẻ xuất hiện trong trình tự này theo thứ tự được chỉ định bởi luồng.

Tuy nhiên, tính nhất quán tuần tự không được đảm bảo bởi hầu hết (hoặc có thể là tất cả) việc triển khai *Pthreads*. Trên thực tế, việc cung cấp một bảo đảm như vậy là khá khó khăn trên các bộ vi xử lý hiện đại sử dụng trình biên dịch hiện đại. Ví dụ, một trình biên dịch có thể tự do sắp xếp lại các chỉ lệnh trong mỗi luồng này bởi vì không có sự phụ thuộc nào giữa chúng (điều này được hiển thị cho trình biên dịch). Ngay cả khi trình biên dịch không sắp xếp lại chúng, phần cứng có thể. Một chiến thuật đề phòng tốt đó là bảo vệ rất cẩn thận các truy cập như vậy vào các biến được chia sẻ bằng cách sử dụng các khóa loại trừ lẫn nhau (và hy vọng rằng bản thân các khóa loại trừ lẫn nhau đó được triển khai chính xác).

6.2.6 Các vấn đề liên quan đến luồng

Các chương trình đa luồng có thể rất khó hiểu. Hơn nữa, có thể khó xây dựng độ tin cậy trong các chương trình vì các vấn đề trong mã có thể không hiển thị trong quá trình thử nghiệm. Ví dụ, một chương trình có thể có khả năng bị tắc nghẽn, nhưng vẫn chạy đúng cách trong nhiều năm mà không bao giờ xuất hiện tắc nghẽn. Các kỹ sư phần mềm phải rất thận trọng, nhưng việc suy luận về các chương trình là đủ khó khiến các lỗi lập trình có thể tồn tại dai dẳng.

Trong ví dụ của Hình 6.2, chúng ta có thể tránh được tắc nghẽn tiềm ẩn của **Ví dụ 6.10** bằng cách sử dụng một thủ thuật đơn giản, nhưng thủ thuật này dẫn đến một lỗi khó hiểu hơn (một lỗi có thể không xảy ra trong thử nghiệm và có thể không được chú ý khi nó xảy ra, không giống như một tắc nghẽn, mà hầu như luôn luôn được chú ý khi nó xảy ra).

Ví dụ 6.11: Giả sử chúng ta sửa đổi thủ tục cập nhật như sau:

Việc triển khai này không giữ khóa khi nó gọi thủ tục listener. Thay vào đó, nó giữ khóa trong khi tạo một bản sao của danh sách listener, và sau đó nó giải phóng khóa. Sau khi giải phóng khóa, nó sử dụng bản sao của danh sách *listener* để thông báo cho *listener*.

```

1. void update(int newx){
2.     x= newx;
3.     // Copy danh sách thành phần
4.     pthread_mutex_lock(&lock);
5.     element_t* headc = NULL;
6.     element_t* tailc = NULL;
7.     element_t* element = head;
8.     while (element != 0){
9.         if(headc == NULL){
10.             headc = malloc(sizeof(element_t));
11.             headc -> listener = head -> listener;
12.             headc -> next = 0;
13.             tailc = headc;
14.         } else {
15.             tailc -> next = malloc(sizeof(element_t));
16.             tail = tail -> next;
17.             tail -> listener = element -> listener;
18.             tail -> next = 0;
19.         }
20.         element = element -> next;
21.     }
22.     pthread_mutex_unlock(&lock);
23.
24.     // Thủ tục sử dụng chương trình copy
25.     element = headc;
26.     while (element != 0){
27.         (*(element -> listener))(newx);
28.         element = element -> next;
29.     }
30. }

```

Tuy nhiên, mã này có một vấn đề nghiêm trọng tiềm ẩn mà có thể không được phát hiện trong quá trình thử nghiệm. Cụ thể, giả sử luồng A gọi cập nhật với đối số $newx = 0$, cho biết “tất cả hệ thống đều bình thường”. Giả sử rằng A bị treo ngay sau khi giải phóng khóa, nhưng trước khi thực hiện các thông báo. Giả sử rằng trong khi nó bị treo, luồng B gọi cập nhật với đối số $newx = 1$,

có nghĩa là “khẩn cấp! động cơ đang cháy! ” Giả sử rằng lệnh gọi cập nhật này hoàn tất trước khi luồng A có cơ hội tiếp tục. Khi luồng A hoạt động trở lại, nó sẽ thông báo cho tất cả listener, nhưng nó sẽ thông báo cho họ về giá trị sai! Nếu một trong những listener đang cập nhật màn hình hướng dẫn cho một máy bay, màn hình sẽ cho biết rằng tất cả các hệ thống đều bình thường, trong khi thực tế là động cơ đang cháy.

Nhiều kỹ sư phần mềm đã quen thuộc với các luồng và đánh giá cao sự dễ dàng mà họ khai thác phần cứng song song bên dưới. Có thể, nhưng không dễ dàng, để xây dựng các chương trình đa luồng chính xác và đáng tin cậy. Đến năm 2005, các thư viện Java tiêu chuẩn bao gồm các cấu trúc và cơ chế dữ liệu đồng thời dựa trên các luồng. Các thư viện như OpenMP cũng cung cấp hỗ trợ cho các mẫu đa luồng được sử dụng phổ biến như các cấu trúc vòng lặp song song. Tuy nhiên, các kỹ sư phần mềm hệ thống nhúng hiếm khi sử dụng Java hoặc các gói lớn phức tạp như OpenMP. Và ngay cả khi họ đã làm vậy, những rủi ro tắc nghẽn tương tự và những lỗi ngầm sẽ xảy ra.

Các luồng có một số khó khăn khiến người ta nghi ngờ khi tiếp xúc với các kỹ sư phần mềm như một cách để xây dựng các chương trình đồng thời. Trên thực tế, trước những năm 1990, các luồng không được sử dụng bởi các kỹ sư phần mềm ứng dụng. Chính sự xuất hiện của các thư viện như *Pthreads* và các ngôn ngữ như Java

và C # đã phơi bày các cơ chế này cho các kỹ sư phần mềm ứng dụng.

Các chương trình đa luồng thông thường khó hiểu một cách đáng kinh ngạc và có thể gây ra các lỗi ngầm, điều kiện thác nước và tắc nghẽn. Các vấn đề có thể tiềm ẩn trong các chương trình đa luồng qua nhiều năm thậm chí sử dụng nhiều chương trình. Những mối quan tâm này đặc biệt quan trọng đối với các hệ thống nhúng có ảnh hưởng đến sự an toàn và sinh kế của con người. Vì hầu như mọi hệ thống nhúng đều liên quan đến phần mềm đồng thời, các kỹ sư thiết kế hệ thống nhúng phải đương đầu với những nguy cơ.

6.3 Tiến trình và truyền thông điệp

Các tiến trình là các chương trình bắt buộc với không gian bộ nhớ riêng của chúng. Các chương trình này không thể tham chiếu đến các biến của nhau, và do đó chúng không thể hiện những khó khăn giống như các luồng. Giao tiếp giữa các chương trình phải được thực hiện thông qua các cơ chế được cung cấp bởi hệ điều hành, nhân vi mô hoặc thư viện. Việc thực hiện các quy trình một cách chính xác thường yêu cầu hỗ trợ phần cứng dưới dạng đơn vị quản lý bộ nhớ hoặc MMU. MMU bảo vệ bộ nhớ của một quá trình khỏi việc đọc hoặc ghi ngẫu nhiên bởi một quá trình khác. Nó thường cung cấp dịch địa chỉ, cung cấp cho mỗi quá trình giả về một không gian địa chỉ bộ nhớ cố định giống nhau cho tất cả các quá trình. Khi một tiến trình truy cập một vị trí bộ nhớ trong không

gian địa chỉ đó, MMU sẽ dịch chuyển địa chỉ đó để tham chiếu đến một vị trí trong phân bộ nhớ vật lý được cấp phát cho tiến trình đó.

Để đạt được sự đồng thời, các quy trình cần có khả năng giao tiếp. Các hệ điều hành thường cung cấp nhiều cơ chế khác nhau, thậm chí bao gồm cả khả năng tạo không gian bộ nhớ dùng chung, điều này tất nhiên sẽ mở ra cho kỹ sư phần mềm tất cả những khó khăn tiềm ẩn của lập trình đa luồng.

Một cơ chế như vậy có ít khó khăn hơn là hệ thống tệp. Hệ thống tệp chỉ đơn giản là một cách để tạo ra một phần dữ liệu liên tục theo nghĩa là nó tồn tại lâu hơn quá trình tạo ra nó. Một quá trình có thể tạo dữ liệu và ghi nó vào một tệp, và một quá trình khác có thể đọc dữ liệu từ cùng một tệp. Việc thực hiện hệ thống tệp để đảm bảo rằng quá trình đọc dữ liệu không đọc nó trước khi nó được ghi. Ví dụ, điều này có thể được thực hiện bằng cách cho phép không quá một tiến trình hoạt động trên một tệp cùng một lúc.

Một cơ chế linh hoạt hơn để giao tiếp giữa các tiến trình là truyền thông điệp. Tại đây, một quá trình tạo ra một phần dữ liệu, lưu trữ nó vào một phần được kiểm soát cẩn thận của bộ nhớ được chia sẻ, và sau đó thông báo cho các quá trình khác rằng thông báo đã sẵn sàng. Những quy trình khác có thể chặn việc chờ dữ liệu sẵn sàng. Việc truyền thông điệp yêu cầu một số bộ nhớ được chia sẻ, nhưng nó được thực hiện trong các thư viện có thể được viết bởi

các chuyên gia. Một kỹ sư phần mềm ứng dụng gọi một thủ tục thư viện để gửi một tin nhắn hoặc để nhận một tin nhắn.

Ví dụ 6.12: Một ví dụ đơn giản về chương trình truyền thông báo được hiển thị trong Hình 6.5. Chương trình này sử dụng mẫu producer/consumer, trong đó một luồng tạo ra một chuỗi thông báo (một luồng) và một luồng khác sử dụng các thông báo. Mô hình này có thể được sử dụng để triển khai mô hình quan sát mà không có rủi ro tắc nghẽn và không có lỗi ngầm được thảo luận trong phần trước. Thủ tục cập nhật sẽ luôn thực hiện trong một chuỗi khác với các bộ quan sát và sẽ tạo ra các thông báo được sử dụng bởi các bộ quan sát.

Trong Hình 6.5, mã được thực thi bởi luồng sản xuất được đưa ra bởi thủ tục producer và mã cho luồng tiêu thụ bởi thủ tục consumer. Nhà sản xuất gọi một thủ tục gọi là send (được định nghĩa) trên dòng 4 để gửi một thông báo có giá trị số nguyên. Consumer sử dụng get (cũng được xác định) trên dòng 10 để nhận thông báo. Consumer được đảm bảo rằng get sẽ không return cho đến khi họ thực sự nhận được tin nhắn. Lưu ý rằng trong trường hợp này, consumer không bao giờ return, vì vậy chương trình này sẽ không tự kết thúc.

Việc thực hiện *send* và *get* bằng Pthreads được thể hiện trong Hình 6.6. Việc triển khai này sử dụng một danh sách được liên kết tương tự như trong Hình 6.2, nhưng trọng tải là một *int*. Ở đây, danh

sách được liên kết đang triển khai hàng đợi đầu tiên, xuất trước (FIFO) không bị ràng buộc, trong đó các phần tử mới được chèn ở phần đuôi và phần tử cũ bị xóa khỏi phần đầu.

Trước tiên hãy xem xét việc thực hiện send. Nó sử dụng *mutex* để đảm bảo rằng send và get không đồng thời sửa đổi danh sách liên kết như trước đây. Nhưng ngoài ra, nó sử dụng một biến điều kiện để thông báo với quá trình consumer rằng kích thước của hàng đợi đã thay đổi. Biến điều kiện được gọi là sent được khai báo và khởi tạo trên dòng 7. Trên dòng 23, luồng sản xuất gọi tín hiệu

```
1. void* procedure(void* arg){
2.     int i;
3.     for(i = 0; i < 10; i++)
4.     {
5.         send(i);
6.     }
7.     return NULL;
8. }
9. void* consumer(void* arg){
10.    while(1){
11.        printf("received %d\n", get());
12.    }
13.    return NULL;
14. }
15. int main(void){
16.    pthread_t threadID1, threadID2;
17.    void* exitStatus;
18.    pthread_create(&threadID1, NULL, producer, NULL);
19.    pthread_create(&threadID2, NULL, consumer, NULL);
20.    pthread_join(threadID1, &exitStatus);
21.    pthread_join(threadID2, &exitStatus);
22.    return 0;
23. }
```

pthread cond, tín hiệu này sẽ “đánh thức” một luồng khác bị chặn trên biến điều kiện, nếu có một luồng như vậy.

Hình 6.5: Ví dụ về một ứng dụng truyền thông điệp đơn giản

Để biết ý nghĩa của việc “đánh thức” một chuỗi khác, hãy xem quy trình get. Trên dòng 31, nếu lệnh gọi luồng get đã phát hiện ra rằng kích thước hiện tại của hàng đợi bằng 0, thì nó gọi *pthread_cond_wait*, sẽ chặn luồng cho đến khi một số luồng khác gọi tín hiệu *pthread_cond*.

Lưu ý rằng thủ tục get lấy *mutex* trước khi kiểm tra biến kích thước. Lưu ý thêm trên dòng 31 rằng pthread cond wait lấy *&mutex* làm đối số. Trên thực tế, trong khi luồng bị chặn trong thời gian chờ, nó sẽ tạm thời giải phóng khóa *mutex*. Nếu không làm điều này, thì luồng producer sẽ không thể vào phần quan trọng của nó, và do đó sẽ không thể gửi tin nhắn. Chương trình sẽ tắc nghẽn. Trước khi *pthread_cond_wait* trả về, nó sẽ lấy lại khóa *mutex*.

Các kỹ sư phần mềm phải rất cẩn thận khi gọi *pthread_cond_wait*, vì khóa *mutex* tạm thời được giải phóng trong quá trình gọi. Do đó, giá trị của bất kỳ biến được chia sẻ nào sau lệnh gọi đến *pthread_cond_wait* có thể không giống với giá trị của nó trước khi gọi (xem Bài tập 3). Do đó, lệnh gọi đến pthread cond wait nằm trong vòng lặp while (dòng 30) liên tục kiểm tra biến size. Điều này giải thích khả năng có thể có nhiều luồng bị chặn đồng thời trên dòng 31 (có thể xảy ra do sự giải phóng tạm thời của

mutex). Khi một luồng gọi tín hiệu `pthread_cond`, tất cả các luồng đang chờ sẽ được thông báo. Nhưng chính xác một luồng sẽ lấy lại mutex trước những luồng khác và sử dụng tin nhắn đã gửi, khiến size được đặt lại về 0. Các luồng được thông báo khác, khi cuối cùng chúng nó có được mutex, sẽ thấy size là `== 0` và sẽ tiếp tục chờ đợi.

Biến điều kiện được sử dụng trong ví dụ trước là một dạng tổng quát của một *semaphore*. *Semaphore* được đặt tên theo các tín hiệu cơ học thường được sử dụng trên đường ray để báo hiệu rằng một đoạn đường ray có tàu trên đó. Sử dụng các semaphores như vậy, có thể sử dụng một đoạn đường ray duy nhất cho các đoàn tàu đi theo cả hai hướng (semaphore thực hiện loại trừ lẫn nhau, ngăn không cho hai đoàn tàu đồng thời đi trên cùng một đoạn đường ray).

Vào những năm 1960, Edsger W. Dijkstra, giáo sư Khoa Toán học tại Đại học Công nghệ Eindhoven, Hà Lan, đã mượn ý tưởng này để chỉ ra cách các chương trình có thể chia sẻ tài nguyên một cách an toàn. Semaphore đếm (mà Dijkstra gọi là semaphore PV) là một biến có giá trị là một số nguyên không âm. Giá trị bằng 0 được coi là khác biệt rõ ràng với giá trị lớn hơn 0. Trên thực tế, biến size trong Ví dụ 6.12 hoạt động như một semaphore. Nó được tăng lên bằng cách gửi tin nhắn và giá trị 0 sẽ chặn consumer cho đến khi giá trị khác 0.

```

1.  #include <pthread.h>
2.  struct element {
3.      int payload;
4.      struct element* next;
5.  };
6.  typedef struct element element_t;
7.  element_t *head = 0, *tail = 0;
8.  int size = 0;
9.  pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
10. pthread_cond_t sent = PTHREAD_COND_INITIALIZER;
11.
12. void send(int msg){
13.     pthread_mutex_lock(&mutex);
14.     if (head == 0){
15.         head = malloc(sizeof(element_t));
16.         head -> payload = msg;
17.         head -> next = 0;
18.         tail = head;
19.     } else {
20.         tail -> next = malloc(sizeof(element_t));
21.         tail = tail -> next;
22.         tail -> payload = msg;
23.         tail -> next = 0;
24.     }
25.     size++;
26.     pthread_cond_signal(&sent);
27.     pthread_mutex_unlock(&mutex);
28. }
29. int get(){
30.     element_t* element;
31.     int result;
32.     pthread_mutex_lock(&mutex);
33.     while (size == 0){
34.         pthread_cond_wait(&sent, &mutex);
35.     }
36.     result = head -> payload;
37.     element = head;
38.     head = head -> next;
39.     free(element);
40.     size--;
41.     pthread_mutex_unlock(&mutex);
42.     return result;
43. }

```

Hình 6.6: Các thủ tục truyền thông điệp

Các biến điều kiện khái quát hóa ý tưởng này bằng cách hỗ trợ các điều kiện tùy ý, thay vì chỉ bằng không hoặc khác không, làm tiêu chí kiểm soát để chặn. Hơn nữa, ít nhất trong Pthreads, các biến điều kiện cũng phối hợp với mutex để làm cho các mẫu như trong Ví dụ 6.12 dễ viết hơn. Dijkstra đã nhận được Giải thưởng Turing năm 1972 cho công trình của ông về lập trình đồng thời.

Sử dụng truyền thông điệp trong các ứng dụng có thể dễ dàng hơn so với sử dụng trực tiếp các chuỗi và các biến được chia sẻ. Nhưng ngay cả thông điệp đi qua cũng không phải là không có nguy cơ. Trên thực tế, việc thực hiện mô hình producer/consumer trong Ví dụ 6.12, có một sai sót khá nghiêm trọng. Cụ thể, nó không áp đặt ràng buộc về kích thước của hàng đợi thông điệp. Bất cứ khi nào một lệnh gọi luồng sản xuất gửi đi, bộ nhớ sẽ được cấp phát để lưu tin nhắn và bộ nhớ đó sẽ không được phân bổ cho đến khi tin nhắn được sử dụng. Nếu luồng producer tạo ra các thông điệp nhanh hơn mức mà consumer sử dụng, thì chương trình cuối cùng sẽ cạn kiệt bộ nhớ khả dụng. Điều này có thể được khắc phục bằng cách giới hạn kích thước của bộ đệm, nhưng kích thước nào là phù hợp? Chọn bộ đệm quá nhỏ có thể khiến chương trình bị tắc nghẽn và việc chọn bộ đệm quá lớn sẽ gây lãng phí tài nguyên. Vấn đề này không phải là dễ giải quyết triệt để.

Cũng có những nguy cơ khác. Các kỹ sư phần mềm có thể vô tình tạo ra các chương trình vượt thông báo bị tắc nghẽn, trong đó

một tập hợp các chuỗi đang chờ thông báo từ nhau. Ngoài ra, các kỹ sư phần mềm có thể vô tình xây dựng các chương trình truyền thông điệp không xác định, theo nghĩa là kết quả tính toán phụ thuộc vào thứ tự (tùy ý) mà bộ định thời luồng xuất hiện để định thời cho các luồng.

Giải pháp đơn giản nhất là các nhà lập trình ứng dụng sử dụng các mức trừu tượng cao hơn cho sự đồng thời, lớp trên cùng trong Hình 6.1, như được mô tả trong 8.4. Tất nhiên, họ chỉ có thể sử dụng chiến lược đó nếu họ có sẵn một triển khai đáng tin cậy ở mức cao hơn mô hình tính toán đồng thời.

6.4 Tóm tắt

Chương này đã tập trung vào mức độ trừu tượng cấp trung cho các chương trình đồng thời, trên mức độ ngắt và phần cứng song song, nhưng dưới mức độ của các mô hình tính toán đồng thời. Cụ thể, nó đã giải thích các luồng, là các chương trình tuần tự thực thi đồng thời và chia sẻ các biến. Chúng tôi đã giải thích về loại trừ lẫn nhau và việc sử dụng các *semaphore*. Chúng tôi đã chỉ ra rằng các luồng chứa đầy nguy cơ và việc viết các chương trình đa luồng chính xác là vô cùng khó khăn. Các chương trình truyền thông điệp tránh được một số khó khăn, nhưng không phải tất cả, với chi phí là có phần hạn chế hơn bằng cách ngăn chia sẻ trực tiếp dữ liệu. Các kỹ sư thiết kế sẽ làm tốt hơn khi sử dụng các mức trừu tượng

cao hơn ở dạng thiết kế mô hình trong **Error! Reference source not found.**

6.5 Bài tập

Bài tập 1. Đưa ra một mô hình máy trạng thái mở rộng của thủ tục *addListener* trong Hình 6.2 tương tự như trong Hình 6.3.

Bài tập 2. Giả sử rằng hai biến toàn cục *int a* và *int b* được chia sẻ giữa một số luồng. Giả sử rằng khóa *a* và khóa *b* là hai khóa

```
1.  int a, b;
2.  pthread_mutex_t lock_a = PTHREAD_MUTEX_INITIALIZER;
3.  pthread_mutex_t lock_b = PTHREAD_MUTEX_INITIALIZER;
4.
5.  void procedure1(int arg){
6.      pthread_mutex_lock(&lock_a);
7.      if (a == arg){
8.          procedure2(arg);
9.      }
10.     pthread_mutex_unlock(&lock_a);
11. }
12. void procedure2(int arg){
13.     pthread_mutex_lock(&lock_b);
14.     b = arg;
15.     pthread_mutex_unlock(&lock_b);
16. }
```

mutex bảo vệ quyền truy cập vào *a* và *b*. Giả sử chúng ta không thể cho rằng việc đọc và ghi các biến toàn cục *int* là nguyên tử. Hãy xem xét đoạn mã sau:

Giả sử rằng để đảm bảo rằng tắc nghẽn không xảy ra, nhóm phát triển đã đồng ý rằng khóa *b* phải luôn được lấy trước khóa *a* bởi bất

kỳ luồng nào có được cả hai khóa. Lưu ý rằng mã được liệt kê ở trên không phải là mã duy nhất trong chương trình. Hơn nữa, vì lý do hiệu suất, nhóm nghiên cứu khẳng định rằng không có khóa nào được lấy một cách không cần thiết. Do đó, sẽ không được chấp nhận nếu sửa đổi *procedure1* như sau:

Một thủ tục gọi *procedure1* sẽ nhận được khóa b một cách không cần thiết khi a không bằng arg.² Đưa ra một thiết kế cho *procedure1* để giảm thiểu việc lấy khóa không cần thiết b. Giải pháp của bạn có loại bỏ việc lấy khóa b không cần thiết không? Có giải pháp nào thực hiện điều này không?

Bài tập 3. Việc thực hiện get trong Hình 6.6 cho phép có nhiều hơn một luồng gọi hàm *get*. Tuy nhiên, nếu chúng ta thay đổi mã

```
1.    if (size == 0){
2.        pthread_cond_wait(&sent, &mutex);
3.    }
-----
1.    void procedure1(int arg){
2.        pthread_mutex_lock(&lock_b);
3.        pthread_mutex_lock(&lock_a);
4.        if (a == arg){
5.            procedure2(arg);
6.        }
7.        pthread_mutex_unlock(&lock_a);
8.        pthread_mutex_unlock(&lock_b);
9.    }
```

trên dòng 30-32 thành:

thì mã này sẽ chỉ hoạt động nếu hai điều kiện được thỏa mãn:

- *pthread_cond_wait* chỉ trả về nếu có một lệnh gọi phù hợp với tín hiệu *pthread_cond*, và
- chỉ có một luồng *consumer*.

Giải thích tại sao điều kiện thứ hai là bắt buộc.

Bài tập 4. Việc triển khai mô hình *producer/consumer* trong Ví dụ 6.12 có nhược điểm là kích thước của hàng đợi được sử dụng để đệm các thông điệp là không bị giới hạn. Một chương trình có thể bị lỗi do sử dụng hết bộ nhớ có sẵn (điều này sẽ gây ra lỗi *malloc*). Xây dựng một biến thể của thủ tục *send* và *get* của Hình 6.6 giới hạn kích thước bộ đệm ở 5 thông điệp.

Bài tập 5. Một hình thức truyền thông điệp thay thế được gọi là điểm gặp, tương tự như mô hình *producer/consumer* của Ví dụ 6.12, nhưng nó đồng bộ hóa *producer* và *consumer* chặt chẽ hơn. Đặc biệt, trong Ví dụ 6.12, thủ tục *send* trả về ngay lập tức, bất kể có bất kỳ luồng *consumer* nào sẵn sàng nhận thông điệp hay không. Trong giao tiếp kiểu điểm gặp, thủ tục *send* sẽ không *return* cho đến khi một chuỗi *consumer* đạt được một lệnh gọi tương ứng để nhận. Do đó, không cần đệm các tin nhắn. Xây dựng triển khai *send* và *get* thực hiện một điểm gặp như vậy.

Chương 7 ĐỊNH THỜI TÁC VỤ

Chương 6 đã giải thích về đa nhiệm, trong đó nhiều tác vụ thực thi đồng thời xen kẽ trên một bộ vi xử lý đơn lẻ hoặc thực thi song song trên nhiều bộ vi xử lý. Khi có ít bộ vi xử lý hơn các tác vụ (trường hợp thông thường) hoặc khi các tác vụ phải được thực hiện tại một thời điểm cụ thể, **bộ định thời** (scheduler) phải can thiệp. Bộ định thời đưa ra quyết định về việc cần làm tiếp theo tại một số thời điểm nhất định, chẳng hạn như thời điểm bộ vi xử lý sẵn sàng thực thi tác vụ.

Hệ thống thời gian thực là tập hợp các tác vụ trong đó bên cạnh sự ràng buộc thứ tự được áp đặt bởi các mức độ ưu tiên giữa các tác vụ, còn có các ràng buộc về thời gian. Những ràng buộc này liên quan việc thực thi một tác vụ với thời gian thực, là thời gian vật lý trong môi trường của máy tính thực thi tác vụ. Thông thường, tác vụ có **thời hạn** (deadline) là giá trị của thời gian vật lý mà tác vụ phải được hoàn thành trễ nhất là thời hạn đó. Nói một cách tổng quát hơn, các chương trình thời gian thực có thể có mọi ràng buộc về thời gian, không chỉ là thời hạn hoàn thành. Ví dụ, một tác vụ có thể được yêu cầu thực hiện không sớm hơn một thời điểm cụ thể; hoặc nó có thể được yêu cầu thực hiện không quá một khoảng thời gian nhất định sau khi một tác vụ khác được thực hiện; hoặc nó có thể được yêu cầu thực hiện định kỳ với một số khoảng thời gian xác định. Các tác vụ có thể phụ thuộc vào nhau và có thể cùng nhau tạo

thành một ứng dụng. Hoặc chúng có thể không liên quan ngoại trừ việc chúng chia sẻ tài Nguyên là bộ vi xử lý. Tất cả những tình huống này đều cần một chiến lược định thời.

7.1 Khái niệm cơ bản về định thời tác vụ

Trong phần này, chúng ta thảo luận về các khả năng định thời, các thuộc tính của tác vụ mà bộ định thời sử dụng để hướng dẫn quá trình và việc triển khai bộ định thời trong hệ điều hành hoặc trong *Microkernel*.

7.1.1 Quyết định định thời

Bộ định thời quyết định tác vụ nào sẽ thực thi tiếp theo khi phải lựa chọn thực hiện một chương trình tiếp theo hoặc một tập hợp các chương trình. Nói chung, một bộ định thời có thể có sẵn nhiều bộ vi xử lý (ví dụ trong hệ thống đa nhân). Một bộ định thời đa vi xử lý không chỉ cần quyết định tác vụ nào sẽ thực thi tiếp theo mà còn cả bộ vi xử lý nào sẽ thực thi tác vụ đó. Sự lựa chọn bộ vi xử lý được gọi là phân bổ bộ vi xử lý (*processor assignment*).

Quyết định định thời (scheduling decision) là một quyết định thực thi một tác vụ và gồm ba phần sau:

1. Phân phối vi xử lý: bộ vi xử lý nào sẽ thực thi tác vụ;
2. Sắp xếp thứ tự: mỗi bộ vi xử lý sẽ thực hiện các tác vụ của nó theo thứ tự nào; và
3. Tính thời gian: thời gian mà mỗi tác vụ thực hiện.

Mỗi quyết định trong số ba quyết định này có thể được thực hiện tại thời điểm thiết lập, trước khi chương trình bắt đầu thực thi hoặc tại thời điểm chạy chương trình, trong quá trình thực thi chương trình.

Tùy thuộc vào thời điểm đưa ra quyết định, chúng ta có thể phân biệt một số loại định thời khác nhau. Một bộ định thời tĩnh (fully-static scheduler) đưa ra cả ba quyết định tại thời điểm thiết lập. Kết quả của việc định thời là một thông số kỹ thuật chính xác cho từng bộ vi xử lý làm khi nào. Một bộ định thời tĩnh thường không cần các *semaphore* hoặc *lock*. Thay vào đó, nó sử dụng thời gian để thực thi các ràng buộc loại trừ lẫn nhau và ưu tiên. Tuy nhiên, các bộ định thời tĩnh rất khó thực hiện với hầu hết các bộ vi xử lý hiện đại vì thời gian thực hiện một tác vụ khó dự đoán chính xác và vì các tác vụ thường sẽ có thời gian thực thi phụ thuộc vào dữ liệu.

Bộ định thời thứ tự tĩnh (static order scheduler) thực hiện việc phân bổ vi xử lý và sắp xếp thứ tự thực thi tác vụ tại thời điểm thiết lập, nhưng sẽ xác định thời gian thực thi tác vụ tại thời điểm thực thi (run-time). Ví dụ, quyết định đó có thể bị ảnh hưởng bởi một *mutex*. Trong định thời lệnh tĩnh, mỗi bộ vi xử lý được cung cấp thứ tự chuyển tác vụ của nó trước khi chương trình bắt đầu thực thi và nó chỉ thực hiện thứ tự đó càng nhanh càng tốt. Ví dụ, nó không thay đổi thứ tự của các tác vụ dựa trên trạng thái của một *semaphore* hoặc *lock*. Tuy nhiên, bản thân một tác vụ có thể chặn trên

semaphore hoặc *lock*, trong trường hợp đó, nó chặn toàn bộ chuỗi tác vụ trên bộ vi xử lý đó. Bộ định thời lệnh tĩnh (static order scheduler) thường được gọi là bộ định thời ngoại tuyến.

Bộ định thời phân bổ tĩnh (static assignment scheduler) thực hiện việc phân bổ vi xử lý tại thời điểm thiết lập và các vấn đề khác tại thời gian thực thi. Mỗi bộ vi xử lý được cung cấp một tập hợp các tác vụ để thực thi và một bộ định thời thời gian hoạt động (run-time scheduler) sẽ quyết định trong quá trình thực thi thì tác vụ nào sẽ thực thi tiếp theo.

Một bộ định thời động (fully-dynamic scheduler) thực hiện tất cả các quyết định tại thời điểm thực thi. Khi một bộ vi xử lý sẵn sàng (ví dụ: nó kết thúc thực thi một tác vụ hoặc một khối tác vụ có được mutex), bộ định thời sẽ đưa ra quyết định tại thời điểm đó về tác vụ sẽ thực thi tiếp theo trên bộ vi xử lý đó. Cả bộ định thời phân bổ tĩnh và bộ định thời động thường được gọi là bộ định thời trực tuyến (on-line schedulers).

Tất nhiên, có những khả năng định thời khác. Ví dụ, phân bổ tác vụ có thể được thực hiện một lần cho một tác vụ, tại thời điểm hoạt động ngay trước khi thực hiện tác vụ đầu tiên. Đối với các lần thực hiện tiếp theo của cùng một tác vụ, sử dụng cùng một phân bổ. Một số kết hợp không có nhiều ý nghĩa. Ví dụ, không có ý nghĩa gì khi xác định thời gian thực hiện một tác vụ tại thời điểm thiết lập và thứ tự tại thời điểm thực thi.

Bộ định thời trung dụng (*preemptive scheduler*) có thể đưa ra quyết định định thời trong quá trình thực hiện một tác vụ, phân bổ một tác vụ mới cho cùng một bộ vi xử lý. Nghĩa là, một tác vụ có thể đang trong quá trình thực thi khi bộ định thời quyết định dừng việc thực thi đó và bắt đầu thực hiện một tác vụ khác. Việc ngưng thực thi tác vụ đầu tiên được gọi là **trung dụng** (*preemption*). Bộ định thời luôn cho phép các tác vụ chạy đến khi hoàn thành trước khi gán một tác vụ khác để thực thi trên cùng một bộ vi xử lý được gọi là bộ định thời không trung dụng (*non-preemptive scheduler*).

Trong định thời trung dụng, một tác vụ có thể được ưu tiên nếu *mutex* và *lock* không khả dụng. Khi điều này xảy ra, tác vụ được cho là bị chặn trên *lock*. Khi một tác vụ khác giải phóng *lock*, tác vụ bị chặn có thể tiếp tục. Hơn nữa, một tác vụ có thể được thực hiện ưu tiên khi nó giải phóng *lock*. Điều này có thể xảy ra chẳng hạn nếu có một tác vụ ưu tiên cao hơn bị chặn trên khóa. Chúng tôi sẽ giả định trong chương này các chương trình có cấu trúc tốt, trong đó bất kỳ tác vụ nào có được *lock* thì cuối cùng sẽ giải phóng nó.

7.1.2 Mô hình tác vụ

Để một bộ định thời đưa ra quyết định, nó cần một số thông tin về cấu trúc của chương trình. Một giả định điển hình là bộ định thời được cung cấp một tập T hữu hạn các tác vụ. Mỗi tác vụ có thể được cho là hữu hạn (nó kết thúc trong thời gian hữu hạn), hoặc không. Một bộ định thời cho hệ điều hành điển hình không cho rằng các

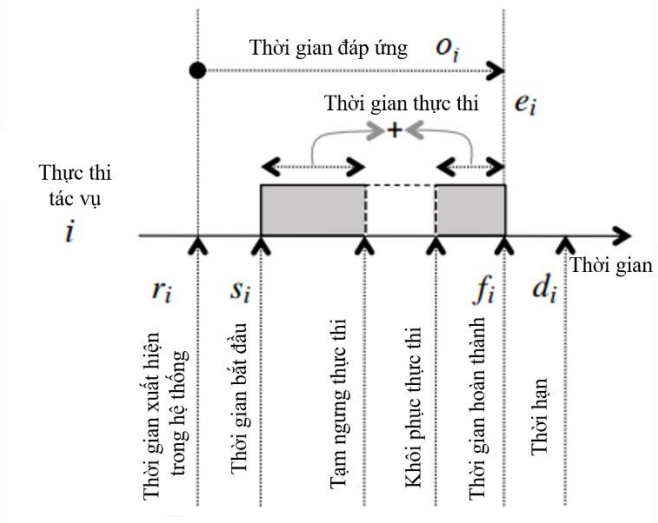
tác vụ kết thúc, nhưng bộ định thời thời gian thực thường làm như vậy.

Bộ định thời có thể đưa ra nhiều giả định khác về các tác vụ, một vài trong số đó chúng ta sẽ thảo luận trong phần này. Tập hợp các giả định được gọi là mô hình tác vụ của bộ định thời.

Một số bộ định thời giả định rằng tất cả các tác vụ sẽ được thực thi đều được biết trước khi bắt đầu định thời và một số hỗ trợ các tác vụ đến (arrival task), có nghĩa là bộ định thời sẽ biết các tác vụ này khi các tác vụ khác đang được thực thi. Một số bộ định thời hỗ trợ các tình huống trong đó mỗi tác vụ $\tau \in T$ thực hiện lặp đi lặp lại, có thể mãi mãi và có thể định kỳ (*periodic*). Một tác vụ cũng có thể là rời rạc (sporadic task), có nghĩa là nó lặp lại và thời gian của nó không đều đặn, nhưng có một giới hạn thấp hơn về thời gian giữa các lần thực hiện tác vụ. Trong tình huống một tác vụ $\tau \in T$ thực hiện lặp đi lặp lại, chúng ta cần phải phân biệt giữa tác vụ τ và thực thi (execution/instance) $\tau_1, \tau_2, \dots$ của nó. Nếu mỗi tác vụ thực thi chính xác một lần thì không cần phân biệt như vậy.

Việc thực thi tác vụ có thể có các ràng buộc về mức độ ưu tiên (priority/precedence constraint), một yêu cầu mà một thực thi tác vụ trước một lần thực thi tác vụ khác. Nếu thực thi i phải diễn ra trước j , chúng ta có thể viết $i < j$. Ở đây, i và j có thể là các lần thực thi riêng biệt của cùng một tác vụ hoặc các lần thực thi các tác vụ khác nhau.

Một thực thi tác vụ i có thể có một số điều kiện tiên quyết để bắt đầu hoặc tiếp tục thực thi. Đây là những điều kiện phải được thỏa mãn trước khi tác vụ có thể thực thi. Khi các điều kiện tiên quyết được thỏa mãn, việc thực thi tác vụ được cho là đã được kích hoạt. Các điều kiện tiên quyết, ví dụ, chỉ định các điều kiện tiên quyết để bắt đầu thực thi tác vụ. Sự sẵn có của một khóa có thể là điều kiện tiên quyết để bắt đầu lại tác vụ.



Hình 7.1: Tóm tắt thời gian liên quan đến việc thực thi tác vụ

Tiếp theo, chúng ta xác định một số thuật ngữ được tóm tắt trong Hình 7.1.

Đối với một thực thi tác vụ i , chúng ta xác định thời gian xuất hiện/được tạo ra r_i (*release time*) là thời gian sớm nhất mà một tác vụ được kích hoạt. Chúng ta định nghĩa thời gian bắt đầu s_i là thời

gian mà quá trình thực thi thực sự bắt đầu. Rõ ràng, chúng ta có ràng buộc về thời gian như sau:

$$s_i \geq r_i \quad (7.1)$$

Chúng ta định nghĩa thời gian kết thúc f_i là thời gian mà tác vụ hoàn thành việc thực thi. Vì thế,

$$f_i \geq s_i \quad (7.2)$$

Thời gian phản hồi o_i được cho bởi công thức 11.3.

$$o_i = f_i - r_i \quad (7.3)$$

Do đó, thời gian phản hồi là thời gian trôi qua giữa thời điểm tác vụ được kích hoạt lần đầu tiên và khi nó hoàn thành việc thực thi.

Thời gian thực thi e_i của τ_i được định nghĩa là tổng thời gian tác vụ đang thực sự thực thi. Nó không bao gồm bất kỳ thời gian nào mà tác vụ đợi hoặc bị dừng. Nhiều chiến lược định thời giả định (thường là không thực tế) rằng thời gian thực thi của một tác vụ đã được biết trước và cố định. Nếu thời gian thực thi thay đổi, người ta thường cho rằng (thường là không thực tế) thời gian thực thi trong trường hợp xấu nhất (Worst Case Execution Time - WCET) đã được biết trước. Việc xác định thời gian thực thi của phần mềm có thể khá khó khăn.

Thời hạn thực thi (deadline) là thời gian mà một tác vụ phải được hoàn thành. Đôi khi, deadline là một ràng buộc thực tế do ứng

dụng áp đặt, trong đó việc trễ deadline được coi là một lỗi. Deadline như vậy được gọi là *hard deadline*. Định thời với hard deadline được gọi là định thời thời gian thực cứng (*hard real-time scheduling*).

Thông thường, deadline phản ánh một quyết định thiết kế không cần được thực thi nghiêm ngặt. Tốt hơn là nên đáp ứng hạn cuối, nhưng bỏ lỡ hạn cuối không phải là một lỗi. Nói chung, tốt hơn là đừng bỏ lỡ hạn cuối quá nhiều. Trường hợp này được gọi là định thời thời gian thực mềm (*soft real-time scheduling*).

Bộ định thời có thể sử dụng mức độ ưu tiên (priority) hơn là deadline. Bộ định thời dựa trên mức độ ưu tiên giả định mỗi tác vụ được gán một số gọi là mức độ ưu tiên và bộ định thời biểu sẽ luôn chọn thực hiện tác vụ có độ ưu tiên cao nhất (thường được biểu thị bằng số độ ưu tiên thấp nhất). Mức độ ưu tiên cố định (fixed priority) là mức độ ưu tiên không đổi trên tất cả các lần thực thi một tác vụ. Mức độ ưu tiên động (dynamic priority) được phép thay đổi trong quá trình thực thi.

Bộ định thời trung dụng dựa trên mức độ ưu tiên (preemptive priority-based) là bộ định thời hỗ trợ các tác vụ đến và luôn thực hiện tác vụ được kích hoạt với mức độ ưu tiên cao nhất. Bộ định thời không trung dụng dựa trên mức độ ưu tiên là bộ định thời sử dụng các mức độ ưu tiên để xác định tác vụ nào sẽ thực hiện tiếp theo sau khi việc thực thi tác vụ hiện tại hoàn thành, nhưng không

bao giờ làm ngắt tác vụ trong quá trình thực thi để lên lịch tác vụ khác.

7.1.3 So sánh các bộ định thời

Việc lựa chọn chiến lược định thời được điều chỉnh bởi các xem xét phụ thuộc vào mục tiêu của ứng dụng. Một mục tiêu khá đơn giản là tất cả các lần thực thi tác vụ đều đáp ứng thời hạn của chúng, $f_i \leq d_i$. Một kế hoạch định thời có thể thỏa mãn các điều kiện này được gọi là một định thời khả thi (feasible scheduling). Một bộ định thời mang lại một định thời khả thi cho bất kỳ bộ tác vụ nào (phù hợp với mô hình tác vụ của nó) mà có một định thời khả thi được cho là tối ưu về tính khả thi.

Một tiêu chí có thể được sử dụng để so sánh các thuật toán định thời là khả năng sử dụng bộ vi xử lý có thể thực hiện được. Hiệu suất sử dụng là phần trăm thời gian mà bộ vi xử lý dành để thực hiện các tác vụ (so với ở chế độ nhàn rỗi). Chỉ số này hữu ích nhất cho các tác vụ thực thi định kỳ. Thuật toán định thời cung cấp một định thời khả thi bất cứ khi nào mức sử dụng bộ vi xử lý nhỏ hơn hoặc bằng 100% rõ ràng là tối ưu về tính khả thi. Nó chỉ không đưa ra được một định thời khả thi trong những trường hợp mà tất cả các thuật toán định thời sẽ không đưa ra được một định thời khả thi.

Một tiêu chí khác có thể được sử dụng để so sánh các bộ định thời là độ trễ tối đa, được xác định cho một tập hợp các thực thi tác vụ T là:

$$L_{max} = \max_{i \in T} (f_i - d_i) \quad (7.4)$$

Đối với một định thời khả thi, con số này bằng 0 hoặc âm. Nhưng độ trễ tối đa cũng có thể được sử dụng để so sánh định thời không khả thi. Đối với các vấn đề thời gian thực mượt mà, có thể chấp nhận được con số này là số dương, miễn là nó không quá lớn.

Tiêu chí thứ ba có thể được sử dụng cho một tập hợp hữu hạn T thực hiện tác vụ là tổng thời gian hoặc khoảng chênh lệch, được xác định bởi:

$$M = \max_{i \in T} f_i - \min_{i \in T} r_i \quad (7.5)$$

Nếu mục tiêu của việc định thời là giảm thiểu khoảng chênh lệch, thì đây thực sự là một mục tiêu về mặt hiệu suất hơn là một yêu cầu thời gian thực.

7.1.4 Hiện thực bộ định thời

Bộ định thời có thể là một phần của trình biên dịch hoặc trình tạo mã (cho những quyết định định thời được đưa ra tại thời điểm thiết kế), một phần của hệ điều hành hoặc nhân vi mô (đối với các quyết định định thời được đưa ra tại thời điểm chạy) hoặc cả hai (nếu một số quyết định định thời được đưa ra tại thời điểm thiết kế và một số lúc chạy).

Một bộ định thời thời gian thực thì thường sẽ triển khai các tác vụ dưới dạng các luồng (hoặc dưới dạng các tiến trình, nhưng sự

khác biệt không quan trọng ở đây). Đôi khi, bộ định thời giả định những luồng này hoàn thành trong thời gian hữu hạn, và đôi khi nó không đưa ra giả định như vậy. Trong cả hai trường hợp, bộ định thời là một thủ tục được gọi vào những thời điểm nhất định. Đối với các bộ định thời rất đơn giản, không ưu tiên, thủ tục định thời có thể được gọi mỗi khi một tác vụ hoàn thành. Đối với bộ định thời có ưu tiên, thủ tục định thời được gọi khi bất kỳ điều nào sau đây xảy ra:

- Một ngắt bộ hẹn giờ xảy ra, ví dụ như ở một khoảng thời gian *jiffy*.
- Xảy ra ngắt I/O.
- Một dịch vụ hệ điều hành được gọi.
- Một tác vụ cố gắng có được một mutex.
- Một tác vụ kiểm tra một semaphore.

Đối với ngắt, quy trình định thời được gọi bởi ISR. Trong các trường hợp khác, thủ tục định thời được gọi bởi thủ tục hệ điều hành cung cấp dịch vụ. Trong cả hai trường hợp, stack chứa thông tin cần thiết để tiếp tục thực thi. Tuy nhiên, bộ định thời có thể chọn không chỉ tiếp tục thực thi. Tức là nó có thể chọn không quay lại ngay lập tức từ thủ tục ngắt hoặc dịch vụ. Thay vào đó, nó có thể chọn giành quyền ưu tiên trước bất kỳ tác vụ nào hiện đang chạy và bắt đầu hoặc tiếp tục tác vụ khác.

Để thực hiện ưu tiên này, bộ định thời cần ghi lại thực tế là tác vụ được ưu tiên (và, có lẽ, tại sao nó được ưu tiên), để sau đó nó có thể tiếp tục tác vụ này. Sau đó, nó có thể điều chỉnh con trỏ stack để tham chiếu đến trạng thái của tác vụ được bắt đầu hoặc tiếp tục. Tại thời điểm đó, một return được thực hiện, nhưng thay vì tiếp tục thực hiện với tác vụ đã được ưu tiên trước, việc thực thi sẽ tiếp tục cho một tác vụ khác.

Việc triển khai một công cụ định thời ưu tiên có thể khá khó khăn. Nó đòi hỏi sự kiểm soát đồng thời rất cẩn thận. Ví dụ: có thể cần phải vô hiệu hóa các ngắt đối với các phần quan trọng của quy trình để tránh kết thúc với một stack bị lỗi. Đây là lý do tại sao định thời là một trong những chức năng quan trọng nhất của nhân hệ điều hành hoặc nhân vi mô. Chất lượng của việc triển khai ảnh hưởng mạnh mẽ đến độ tin cậy và ổn định của hệ thống.

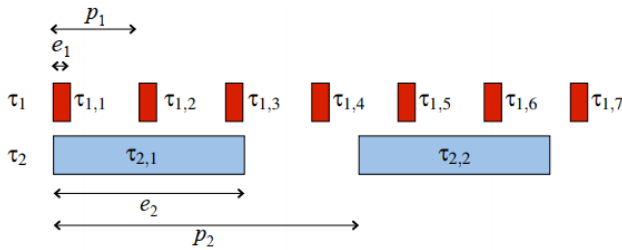
7.2 Bộ định thời Rate Monotonic

Hãy xem xét một kịch bản với $T = \{\tau_1, \tau_2, \dots, \tau_n\}$ của n tác vụ, trong đó các tác vụ phải thực hiện định kỳ. Cụ thể, chúng tôi giả định rằng mỗi tác vụ τ_i phải thực hiện để hoàn thành chính xác một lần trong mỗi khoảng thời gian p_i . Chúng tôi gọi p_i là chu kỳ của tác vụ. Do đó, thời hạn cho lần thực hiện thứ j của τ_i là $r_i, 1 + j p_i$, trong đó $r_{i,1}$ là thời gian bắt đầu lần thực thi đầu tiên.

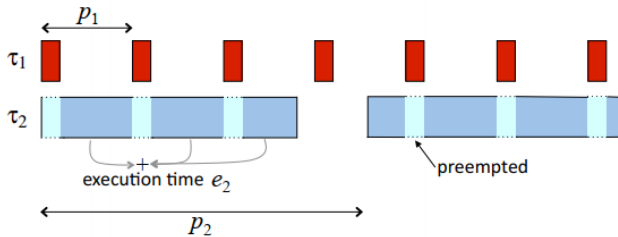
Liu và Layland đã giới thiệu một chiến lược lập kế hoạch ưu tiên đơn giản được gọi là Rate Monotonic (RM) là tối ưu về tính

khả thi giữa các bộ định thời đơn xử lý ưu tiên cố định cho mô hình tác vụ trên [11]. Chiến lược định thời này ưu tiên cao hơn cho tác vụ có chu kỳ ngắn hơn.

Dạng đơn giản nhất của bài toán chỉ có hai tác vụ, $T = \{\tau_1; \tau_2\}$ với thời gian thực hiện e_1 và e_2 và các chu kỳ p_1 và p_2 , như được mô tả trong Hình 7.2. Trong hình, thời gian thực hiện e_2 của tác vụ τ_2 dài hơn khoảng thời gian p_1 của nguyên công τ_1 . Do đó, nếu hai tác vụ này được thực thi trên cùng một bộ vi xử lý, thì rõ ràng là bộ định thời không trung dụng sẽ không mang lại một kết quả định thời khả thi. Nếu tác vụ τ_2 phải thực hiện để hoàn thành mà không bị ngắt, thì tác vụ τ_1 sẽ trễ *deadline* của nó.



Hình 7.2: Hai tác vụ chu kỳ p_1 và p_2

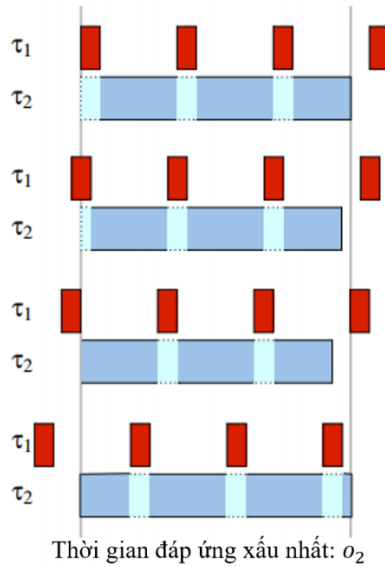


Hình 7.3: Hai tác vụ chu kỳ với định thời độ ưu tiên

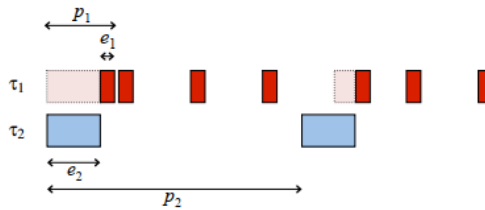
Một định thời trung dụng RM được thể hiện trong Hình 7.3. Trong hình đó, tác vụ τ_1 được ưu tiên cao hơn, vì chu kỳ của nó

nhỏ hơn. Vì vậy, τ_1 thực thi ở đầu mỗi khoảng thời gian, bất kể τ_2 có đang thực thi hay không. Nếu τ_2 đang thực thi, thì τ_1 sẽ chiếm quyền ưu tiên của nó. Hình này giả định rằng thời gian cần thiết để thực hiện quyền ưu tiên, được gọi là thời gian chuyển đổi ngữ cảnh (switching overhead), là không đáng kể. Định thời này là khả thi, trong khi nếu τ_2 được ưu tiên cao hơn, thì định thời sẽ không khả thi.

Đối với trường hợp hai tác vụ, dễ dàng chỉ ra rằng trong số tất cả các bộ định thời trung dụng có độ ưu tiên cố định, RM là tối ưu về tính khả thi, theo mô hình tác vụ giả định với thời gian chuyển đổi ngữ cảnh không đáng kể. Điều này rất dễ thấy được vì chỉ có hai định thời trung dụng cố định cho trường hợp đơn giản này, định thời RM, ưu tiên cao hơn cho tác vụ τ_1 và định thời không RM, ưu tiên cao hơn cho tác vụ τ_2 . Để thể hiện sự tối ưu, chúng ta chỉ cần chứng minh rằng nếu tồn tại định thời không phải RM là khả thi, thì định thời RM cũng có thể tạo ra một định thời khả thi.



Hình 7.4: Thời gian đáp ứng trường hợp xấu nhất



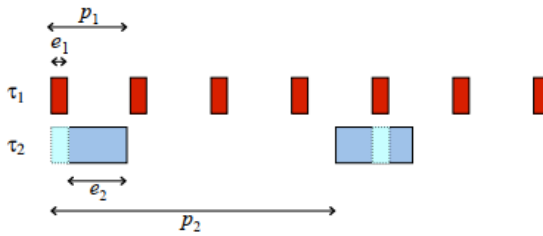
Hình 7.5: Định thời Non-RM ưu tiên cao hơn cho τ_2

Trước khi chúng ta có thể làm điều này, chúng ta cần xem xét sự liên quan có thể có của việc thực thi tác vụ có thể ảnh hưởng đến tính khả thi. Như trong Hình 7.4, thời gian đáp ứng của tác vụ có mức độ ưu tiên thấp hơn là xấu nhất khi giai đoạn bắt đầu của nó khớp với giai đoạn của các tác vụ có mức độ ưu tiên cao hơn. Nghĩa là, trường hợp xấu nhất xảy ra khi tất cả các tác vụ bắt đầu chu kỳ của chúng cùng một lúc. Do đó, chúng ta chỉ cần xem xét kịch bản này.

Trong trường hợp xấu nhất này, khi thời gian giải phóng phù hợp, định thời không phải RM là khả thi nếu và chỉ khi:

$$e_1 + e_2 \leq p_1 \quad (7.6)$$

Trường hợp này được minh họa trong Hình 7.5. Vì tác vụ τ_1 được ưu tiên bởi τ_2 , để τ_1 không trễ hạn của nó, chúng ta yêu cầu rằng $e_2 \leq p_1 - e_1$, để τ_2 có đủ thời gian để τ_1 thực hiện trước thời hạn của nó.



Hình 7.6: Định thời RM ưu tiên cao hơn cho τ_1

Xem xét Hình 7.6, rõ ràng là nếu Phương trình (7.6) được thỏa mãn, thì định thời RM là khả thi. Vì chỉ có hai định thời độ ưu tiên cố định, định thời RM là tối ưu về tính khả thi. Kỹ thuật chứng minh tương tự có thể được tổng quát hóa cho số lượng tác vụ tùy ý, tạo ra định lý sau [11]:

Định lý 7.1. Cho trước một bộ định thời ưu tiên và cố định, cùng một tập hữu hạn các tác vụ lặp lại $T = \{\tau_1, \tau_2, \dots, \tau_n\}$ với các giai đoạn liên quan $p_1, p_2, \dots, p_n$ và không có ràng buộc về mức độ ưu tiên, nếu bất kỳ sự phân công ưu tiên tỷ lệ đơn sẽ mang lại một định thời khả thi, thì sự phân công ưu tiên tỷ lệ đơn sẽ mang lại một định thời khả thi.

Định thời RM được thực hiện dễ dàng với bộ hẹn giờ ngắt với khoảng thời gian bằng ước số chung lớn nhất của các khoảng thời gian của tác vụ. Chúng cũng có thể được thực hiện với nhiều bộ hẹn giờ ngắt.

Nó chỉ ra rằng các định thời RM không phải lúc nào cũng đạt được hiệu suất 100%. Đặc biệt, những bộ định thời RM bị ràng buộc phải có mức độ ưu tiên cố định. Ràng buộc này dẫn đến các tình huống trong đó một bộ tác vụ mang lại định thời khả thi có hiệu suất sử dụng ít hơn 100% và vẫn không thể chấp nhận bất kỳ sự gia tăng thời gian thực hiện hoặc giảm đi trong các chu kỳ. Điều này có nghĩa là có những chu kỳ bộ vi xử lý nhàn rỗi không thể được sử dụng mà không gây ra thời hạn cuối.

May mắn thay, Trong nghiên cứu [11] chỉ ra rằng sự ảnh hưởng này là có giới hạn. Trước tiên, lưu ý rằng việc sử dụng n tác vụ độc lập với thời gian thực hiện e_i và chu kỳ p_i có thể được viết:

$$\mu = \sum_{i=1}^n \frac{e_i}{p_i} \quad (7.7)$$

Nếu $\mu = 1$, thì bộ vi xử lý bận trong 100% thời gian. Vì vậy, rõ ràng, nếu $\mu > 1$ cho bất kỳ bộ tác vụ nào, thì bộ tác vụ đó không có định thời khả thi. Liu và Layland chỉ ra rằng nếu μ nhỏ hơn hoặc bằng một mức sử dụng được giới hạn bởi:

$$\mu \leq n(2^{\frac{1}{n}} - 1) \quad (7.8)$$

thì định thời RM là khả thi.

Để hiểu kết quả này (khá đáng chú ý), hãy xem xét một vài trường hợp. Đầu tiên, nếu $n = 1$ (chỉ có một tác vụ), thì $n(2^{1/n} - 1) = 1$, vì vậy kết quả cho chúng ta biết rằng nếu việc sử dụng là 100% hoặc ít hơn, thì định thời RM là khả thi. Tất nhiên, điều này là hiển nhiên vì chỉ với một tác vụ, $\mu = e_1 = p_1$, và rõ ràng thời hạn chỉ có thể được đáp ứng nếu $e_1 \leq p_1$.

Nếu $n = 2$ thì $n(2^{1/n} - 1) \approx 0.828$. Do đó, nếu một bộ tác vụ có hai tác vụ không cố gắng sử dụng hơn 82,8% thời gian có sẵn của bộ vi xử lý, thì định thời RM sẽ đáp ứng tất cả các thời hạn cuối.

Khi n lớn hơn, giới hạn sử dụng tiến tới $\ln(2) \approx 0.693$. Đó là:

$$\lim_{n \rightarrow \infty} n \left(2^{\frac{1}{n}} - 1 \right) = \ln(2) = 0.693$$

Điều này có nghĩa là nếu một bộ tác vụ với bất kỳ số tác vụ nào không cố gắng sử dụng hơn 69,3% thời gian có sẵn của bộ vi xử lý, thì định thời RM sẽ đáp ứng tất cả các thời hạn cuối.

Trong phần tiếp theo, chúng tôi nói lỏng ràng buộc mức độ ưu tiên cố định và chỉ ra rằng các bộ định thời ưu tiên linh động có thể hoạt động tốt hơn các bộ định thời ưu tiên cố định, theo nghĩa là chúng có thể đạt được mức sử dụng cao hơn. Chi phí đôi khi là một sự phức tạp hơn.

7.3 Bộ định thời EDF

Cho một tập hợp hữu hạn các tác vụ không lặp lại với thời hạn cuối và không có ràng buộc về mức độ ưu tiên, một thuật toán định thời đơn giản là thời hạn sớm nhất (Earliest Due Date – EDD). Chiến lược EDD chỉ đơn giản là thực hiện các tác vụ theo thứ tự như thời hạn cuối của chúng, với tác vụ có thời hạn sớm nhất sẽ đi trước. Nếu hai tác vụ có cùng thời hạn, thì thứ tự tương đối của chúng không quan trọng.

EDD cũng tối ưu về tính khả thi, vì nó giảm thiểu độ trễ tối đa. Tuy nhiên, EDD không hỗ trợ việc xuất hiện tác vụ và do đó cũng không hỗ trợ định kỳ hay lặp lại của các tác vụ. May mắn thay, EDD dễ dàng được mở rộng để hỗ trợ những điều này, mang lại những gì được gọi là thời hạn sớm nhất thực thi đầu tiên (Earliest Deadline First - EDF) hoặc thuật toán Horn [12].

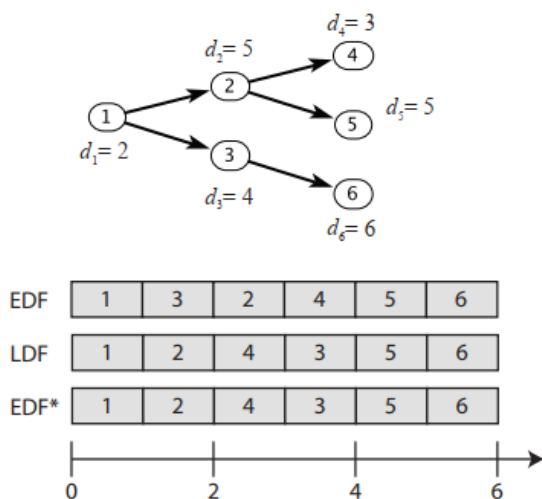
Định lý 7.2. *Cho tập n tác vụ độc lập $T = \{\tau_1, \tau_2, \dots, \tau_n\}$ với các thời hạn liên quan $d_1, d_2, \dots, d_n$ và thời gian đến tùy ý, bất kỳ thuật toán nào thực hiện tác vụ ngay lập tức với thời hạn cuối sớm nhất trong số tất cả các tác vụ đã đến là tối ưu để giảm thiểu độ trễ tối đa.*

Bằng chứng cho điều này sử dụng một đối số trao đổi tương tự. Hơn nữa, kết quả dễ dàng được mở rộng để hỗ trợ số lượng tác vụ đến không bị giới hạn. Chúng tôi để nó như một bài tập.

Lưu ý rằng EDF là một thuật toán định thời có độ ưu tiên thay đổi (*dynamic priority*). Nếu một tác vụ được thực thi nhiều lần, nó

có thể được chỉ định một mức độ ưu tiên khác nhau trên mỗi lần thực thi. Điều này có thể làm cho nó phức tạp hơn để thực hiện. Thông thường, đối với các tác vụ định kỳ, thời hạn cuối được sử dụng là thời hạn kết thúc của tác vụ, mặc dù chắc chắn có thể sử dụng các thời hạn khác cho các tác vụ.

Mặc dù EDF tốn kém hơn để triển khai so với RM, nhưng trên thực tế, hiệu suất của nó nói chung là vượt trội hơn. Thứ nhất, RM chỉ tối ưu về tính khả thi trong số các bộ định thời có độ ưu tiên cố định, trong khi EDF là tối ưu giữa các bộ định thời có độ ưu tiên thay đổi. Ngoài ra, EDF còn giảm thiểu độ trễ tối đa. Trên thực tế, EDF dẫn đến ít ưu tiên hơn, có nghĩa là ít chi phí hơn cho việc chuyển đổi ngữ cảnh. Điều này thường bù đắp cho sự phức tạp lớn hơn trong quá trình thực hiện. Ngoài ra, không giống như RM, bất kỳ định thời EDF nào với mức sử dụng ít hơn 100% đều có thể chịu được việc tăng thời gian thực hiện và/hoặc giảm thời gian mà vẫn khả thi.



Hình 7.7: Ví dụ định thời đa tác vụ với các độ ưu tiên khác nhau

7.3.1 EDF với các tiền lệ

EDF được chứng minh là tối ưu cho một bộ tác vụ không có tiền lệ. Nếu có tiền lệ (precedence) thì sao? Cho một tập hợp hữu hạn các tác vụ, mức độ ưu tiên giữa chúng có thể được biểu diễn bằng biểu đồ mức độ ưu tiên.

Ví dụ 7.1 cho thấy rằng EDF không phải là tối ưu nếu có các tiền lệ. Năm 1973, Lawler đã đưa ra một thuật toán đơn giản là tối ưu với các

Ví dụ 7.1: Xét sáu tác vụ $T = \{1, \dots, 6\}$, mỗi lần có thời gian thực hiện $e_i = 1$, với các mức tiền lệ như trong Hình 7.7. Sơ đồ có nghĩa là τ_1 phải thực hiện trước khi τ_2 hoặc τ_3 có thể thực hiện, tác vụ τ_2 phải thực hiện trước τ_4 hoặc τ_5 , và tác vụ τ_3 phải thực hiện trước τ_6 . Thời hạn cho mỗi tác vụ được thể hiện trong hình. Định thời có nhãn EDF là định thời EDF. Định thời này không khả thi. Tác vụ 4 bỏ lỡ thời hạn cuối của nó. Tuy nhiên, có một định thời khả thi. Định thời có nhãn LDF đáp ứng tất cả các thời hạn.

tiền lệ, theo nghĩa là nó giảm thiểu độ trễ tối đa [13]. Chiến lược rất đơn giản. Đưa ra một tập hợp hữu hạn và cố định gồm các tác vụ có thời hạn cuối, chiến lược của Lawler xây dựng định thời ngược lại, trước tiên chọn tác vụ cuối cùng để thực hiện. Tác vụ cuối cùng để thực hiện là tác vụ không có tác vụ nào khác phụ thuộc vào đó có thời hạn cuối muộn nhất. Thuật toán tiến hành xây dựng định thời ngược lại, mỗi lần chọn trong số các tác vụ mà những tác vụ phụ thuộc đã được lên định thời có thời hạn muộn nhất. Đối với ví dụ trước, định thời kết quả, có nhãn LDF trong Hình 7.7, là khả thi. Thuật toán của Lawler được gọi là thời hạn trễ nhất thực thi đầu tiên (Lasted Deadline First - LDF).

LDF tối ưu theo nghĩa là nó giảm thiểu tối đa độ trễ, và do đó nó cũng tối ưu về tính khả thi. Tuy nhiên, LDF không đáp ứng tốt cho các tác vụ mới xuất hiện. May mắn thay, có một phiên bản thay đổi đơn giản của EDF. EDF* (EDF có ưu tiên) hỗ trợ đáp ứng cho các tác vụ mới và giảm thiểu độ trễ tối đa. Với thay đổi này, chúng ta điều chỉnh thời hạn cuối của tất cả các tác vụ. Giả sử tập hợp tất cả các tác vụ là T . Đối với một tác vụ $i \in T$, đặt $D(i) \subset T$ là tập các thực thể của i trong biểu đồ ưu tiên. Đối với tất cả các lần thực thi tác vụ $i \in T$, chúng tôi xác định thời hạn cuối đã sửa đổi như sau:

$$d'_i = \min(d_i, \min_{j \in D(i)}(d'_j - e_j)) \quad (7.9)$$

EDF* sau đó cũng giống như EDF ngoại trừ việc nó sử dụng các thời hạn sửa đổi này.

EDF* có thể được coi là một kỹ thuật đề hợp lý hóa thời hạn cuối. Thay vì chấp nhận thời hạn cuối tùy ý như đã cho, thuật toán này đảm bảo rằng thời

Ví dụ 7.2: Trong Hình 7.7, chúng ta thấy rằng định thời EDF* giống với định thời LDF. Thời hạn sửa đổi như sau:

$$d'_1 = 1, d'_2 = 2, d'_3 = 4, d'_4 = 3, d'_5 = 5, d'_6 = 6$$

Điều quan trọng là thời hạn của *Tác vụ 2* đã thay đổi từ 5 thành 2, phản ánh thực tế là những người kế nhiệm của nó có thời hạn sớm. Điều này khiến EDF* lên lịch *Tác vụ 2* trước *Tác vụ 3*, dẫn đến một định thời khả thi.

hạn cuối có tính đến thời hạn cuối của tác vụ kế tiếp. Trong ví dụ, không có ý nghĩa gì khi *Tác vụ 2* có thời hạn cuối muộn hơn tác vụ kế tiếp, *Tác vụ 5*. Vì vậy, EDF* khắc phục sự bất thường này trước khi áp dụng EDF.

7.4 Định thời và loại trừ tương hỗ

Mặc dù các thuật toán được đưa ra cho đến nay là đơn giản về mặt khái niệm, nhưng hiệu quả của chúng trong thực tế khác xa so với các thiết kế hệ thống đơn giản và thường gây ngạc nhiên. Điều này đặc biệt đúng khi các tác vụ chia sẻ tài nguyên và sử dụng loại trừ lẫn nhau để bảo vệ quyền truy cập vào các tài nguyên đó.

7.4.1 Đảo ngược độ ưu tiên

Về nguyên tắc, bộ định thời ưu tiên dựa trên mức độ ưu tiên luôn thực thi tác vụ được kích hoạt cấp cao. Tuy nhiên, khi sử dụng loại trừ lẫn nhau, một tác vụ có thể bị chặn trong quá trình thực thi.

Nếu thuật toán định thời không tính đến khả năng này, các vấn đề nghiêm trọng có thể xảy ra.

Ví dụ 7.3: Máy dò đường Sao Hỏa, được hiển thị trong Hình 7.8, hạ cánh trên Sao Hỏa vào ngày 4 tháng 7 năm 1997. Một vài ngày sau khi thực hiện sứ mệnh, Máy dò đường bắt đầu bị bỏ lỡ thời hạn cuối thường xuyên, khiến toàn bộ hệ thống đặt lại, mỗi lần bị mất dữ liệu. Các kỹ sư trên mặt đất đã chẩn đoán vấn đề là đảo ngược ưu tiên, trong đó tác vụ khí tượng có mức độ ưu tiên thấp đang giữ khóa và chặn tác vụ có mức độ ưu tiên cao, trong khi các tác vụ có mức độ ưu tiên trung bình được thực thi.

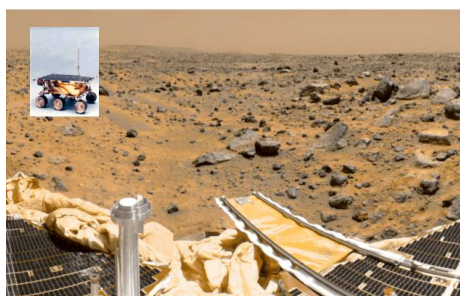
Đảo ngược mức độ ưu tiên là một sự bất thường về định thời trong đó tác vụ có mức độ ưu tiên cao bị chặn trong khi các tác vụ có mức độ ưu tiên thấp hơn không liên quan đang thực thi. Hiện tượng được minh họa trong Hình 7.9. Trong hình, *Tác vụ 3*, một tác vụ có mức độ ưu tiên thấp, nhận được một khóa tại thời điểm 1. Tại thời điểm 2, nó được ưu tiên bởi *Tác vụ 1*, một tác vụ có mức độ

ưu tiên cao, sau đó 3 khối đang cố gắng lấy cùng một khóa.

Tuy nhiên, trước khi *Tác vụ 3* đạt đến điểm mà nó giải phóng khóa, nó sẽ bị tấn công bởi một *Tác vụ 2* không liên quan, có mức độ ưu tiên trung bình. *Tác vụ 2* có thể chạy trong một khoảng thời gian không giới hạn và ngăn *Tác vụ 1* có mức độ ưu tiên cao hơn một cách hiệu quả. Điều này gần như chắc chắn là không được mong đợi.

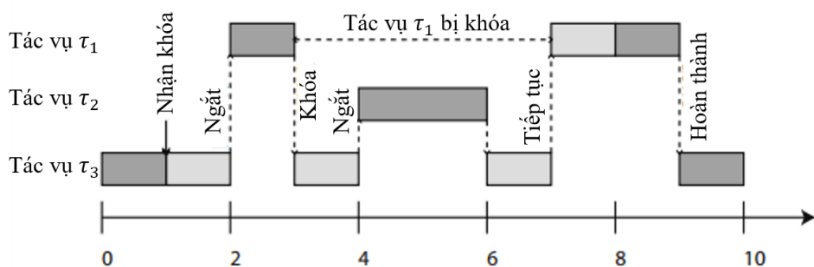
7.4.2 Giao thức kế thừa độ ưu tiên

Năm 1990, Sha và các cộng sự đã đưa ra một giải pháp cho vấn đề đảo ngược ưu tiên được gọi là thừa kế ưu tiên [14]. Trong giải pháp của họ, khi một khối tác vụ cố gắng lấy khóa, thì tác vụ giữ khóa sẽ kế thừa quyền ưu tiên của tác vụ bị chặn. Do đó, tác vụ giữ khóa không thể được thực hiện trước tác vụ có mức độ ưu tiên thấp hơn tác vụ cố gắng lấy khóa.



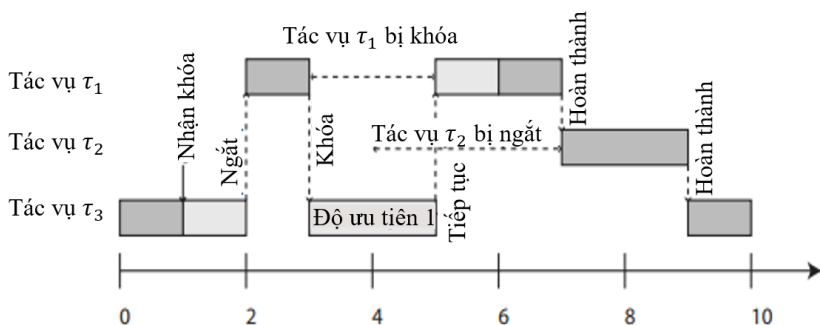
Hình 7.8: Máy dò đường sao Hỏa và hình ảnh bề mặt sao Hỏa từ camera của tàu đổ bộ

Ví dụ 7.4: Hình 7.10 minh họa sự kế thừa ưu tiên. Trong hình, khi khối *Tác vụ 1* cố gắng lấy khóa do *Tác vụ 3* nắm giữ, *Tác vụ 3* tiếp tục thực thi, nhưng bây giờ với mức độ ưu tiên cao hơn của *Tác vụ 1*. Do đó, khi *Tác vụ 2* được bật ở thời điểm 4, nó không ưu tiên trước *Tác vụ 3*. Thay vào đó, *Tác vụ 3* chạy cho đến khi nó giải phóng khóa vào thời điểm 5. Khi đó, *Tác vụ 3* hoàn nguyên về mức ưu tiên ban đầu (thấp) của nó và *Tác vụ 1* tiếp tục thực hiện. Chỉ khi *Tác vụ 1* hoàn thành thì *Tác vụ 2* mới có thể thực thi.



Hình 7.9: Minh họa nghịch đảo độ ưu tiên

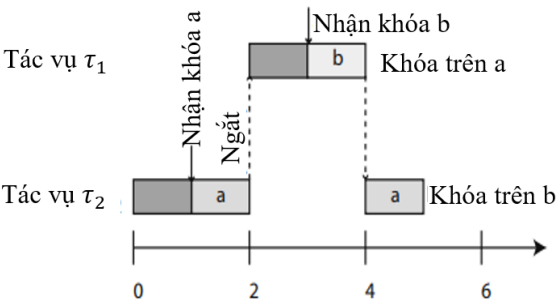
Tác vụ 1 có mức độ ưu tiên cao nhất, *Tác vụ 3* thấp nhất. *Tác vụ 3* có được một khóa trên một đối tượng được chia sẻ, đi vào một phần quan trọng. Nó được ưu tiên bởi *Tác vụ 1*, sau đó sẽ cố gắng lấy khóa và các khối. *Tác vụ 2* trước *Tác vụ 3* tại thời điểm 4, giữ cho *Tác vụ 1* có mức độ ưu tiên cao hơn bị chặn trong một khoảng thời gian không giới hạn. Trên thực tế, mức độ ưu tiên của *Tác vụ 1* và *Tác vụ 2* bị đảo ngược, vì *Tác vụ 2* có thể giữ cho *Tác vụ 1* đợi lâu tùy ý.



Hình 7.10: Minh họa giao thức kế thừa ưu tiên

Tác vụ 1 có mức độ ưu tiên cao nhất, *Tác vụ 3* thấp nhất. *Tác vụ 3* có được một khóa trên một đối tượng được chia sẻ, đi vào một phần quan trọng. Nó được ưu tiên bởi *Tác vụ 1*, sau đó sẽ cố gắng

lấy khóa và các khối. *Tác vụ 3* kế thừa mức độ ưu tiên của *Tác vụ 1*, ngăn chặn sự ưu tiên của *Tác vụ 2*.



Hình 7.11: Hình minh họa tắc nghẽn.

Tác vụ có mức độ ưu tiên thấp hơn bắt đầu trước và nhận được khóa a, sau đó được ưu tiên bởi tác vụ có mức độ ưu tiên cao hơn, có được khóa b và sau đó các khối cố gắng có được khóa a. Sau đó, Tác vụ có mức ưu tiên thấp hơn sẽ chặn việc cố gắng lấy khóa b và không thể tiến hành thêm.

7.4.3 Giao thức trần ưu tiên

Mức độ ưu tiên có thể tương tác với các khóa loại trừ lẫn nhau theo những cách thú vị hơn. Đặc biệt, vào năm 1990, ưu tiên có thể được sử dụng để ngăn chặn một số loại tắc nghẽn nhất định.

Ví dụ 7.5: Hình 7.11 minh họa một kịch bản trong đó hai tác vụ bị tắc nghẽn. Trong hình, *Tác vụ 1* có mức độ ưu tiên cao hơn. Tại thời điểm 1, *Tác vụ 2* có được khóa a. Tại thời điểm 2, *Tác vụ 1* trước *Tác vụ 2*, và tại thời điểm 3, có được khóa b. Trong khi giữ khóa b, nó cố gắng lấy khóa a. Vì a được giữ bởi *Tác vụ 2*, nó chặn. Tại thời điểm 4, *Tác vụ 2* tiếp tục thực hiện. Tại thời điểm 5, nó cố gắng lấy được khóa b, được giữ bởi *Tác vụ 1*. Tắc nghẽn!

Sự tắc nghẽn trong ví dụ trước có thể được ngăn chặn bằng một kỹ thuật thông minh được gọi là giao thức trần ưu tiên. Trong giao

Ví dụ 7.6: Giao thức trần ưu tiên ngăn chặn tắc nghẽn của Ví dụ 7.5, như trong Hình 7.12. Trong hình, khi *Tác vụ 1* cố gắng lấy khóa b tại thời điểm 3, nó bị ngăn cản. Khi đó, khóa a hiện đang được giữ bởi tác vụ khác (*Tác vụ 2*). Mức ưu tiên được gán cho khóa a bằng mức ưu tiên của *Tác vụ 1*, vì *Tác vụ 1* là tác vụ có mức ưu tiên cao nhất có thể nhận được khóa a. Vì mức độ ưu tiên của *Tác vụ 1* không cao hơn mức trần ưu tiên này, nên *Tác vụ 1* không được phép nhận được khóa b. Thay vào đó, *Tác vụ 1* bị chặn, cho phép *Tác vụ 2* chạy đến khi hoàn thành. Tại thời điểm 4, *Tác vụ 2* có được khóa b không bị cản trở, và tại thời điểm 5, nó giải phóng cả hai khóa. Khi nó đã giải phóng cả hai khóa, *Tác vụ 1*, có mức độ ưu tiên cao hơn, không còn bị chặn nữa, vì vậy nó tiếp tục thực thi, trước *Tác vụ 2*.

thức này, mọi khóa hoặc semaphore được gán mức ưu tiên bằng với mức ưu tiên của tác vụ có mức ưu tiên cao nhất có thể khóa nó. Một tác vụ τ chỉ có thể nhận được một khóa nếu mức độ ưu tiên của tác vụ

cao hơn mức ưu tiên của tất cả các khóa hiện đang được các tác vụ khác nắm giữ. Theo trực giác, nếu chúng tôi ngăn tác vụ τ nhận được khóa a, thì chúng tôi đảm bảo rằng tác vụ τ sẽ không giữ khóa sau đó khi cố gắng lấy các khóa khác do các tác vụ khác nắm giữ. Điều này ngăn chặn một số tắc nghẽn nhất định xảy ra.

Tất nhiên, việc triển khai giao thức trần ưu tiên đòi hỏi phải có khả năng xác định trước tác vụ nào nhận được khóa nào. Một chiến lược bảo thủ đơn giản là kiểm tra mã nguồn cho mỗi tác vụ và kiểm

kê các ổ khóa có được trong mã. Điều này là thận trọng vì một chương trình cụ thể có thể hoặc không thể thực thi bất kỳ dòng mã cụ thể nào, vì vậy chỉ vì một khóa được đề cập trong mã không nhất thiết có nghĩa là tác vụ sẽ cố gắng lấy khóa.

7.5 Định thời đa xử lý

Việc định thời các tác vụ trên một bộ vi xử lý đơn lẻ là khá khó. Định thời cho chúng trên nhiều bộ vi xử lý thậm chí còn khó hơn. Hãy xem xét vấn đề định thời cho một nhóm tác vụ hữu hạn cố định với các mức độ ưu tiên trên một số bộ vi xử lý hữu hạn với mục tiêu giảm thiểu khoảng chênh lệch. Vấn đề này được biết là NP-hard. Tuy nhiên, vẫn tồn tại các chiến lược định thời có năng suất và hiệu quả. Một trong những chiến lược đơn giản nhất được gọi là **thuật toán định thời mức Hu**. Nó chỉ định mức độ ưu tiên cho mỗi tác vụ τ dựa trên mức, là tổng thời gian thực hiện lớn nhất của các tác vụ trên một đường dẫn trong biểu đồ ưu tiên từ τ đến một tác vụ khác không có phần phụ thuộc. Các tác vụ có cấp độ lớn hơn có mức độ ưu tiên cao hơn các tác vụ có cấp độ nhỏ hơn.

Định thời mức Hu là một trong họ các phương pháp đường dẫn quan trọng vì nó nhấn mạnh đường dẫn qua biểu đồ ưu tiên với tổng thời gian thực hiện lớn nhất. Mặc dù nó không phải là tối ưu, nó được xem là gần với giải pháp tối ưu nhất cho hầu hết các đồ thị.

Sau khi các ưu tiên được chỉ định cho các tác vụ, danh sách bộ định thời sẽ sắp xếp các tác vụ theo mức độ ưu tiên và gán chúng

cho các bộ vi xử lý theo thứ tự của danh sách được sắp xếp khi các bộ vi xử lý khả dụng.

7.5.1 Bất thường trong định thời

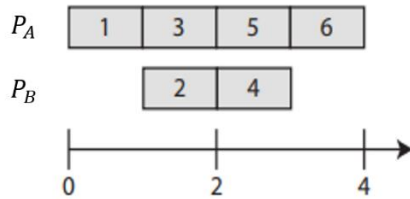
Trong số những bẫy lỗi nhất trong thiết kế hệ thống nhúng là định thời bất thường, trong đó các hành vi bất ngờ hoặc phản trực giác xuất hiện do những thay đổi nhỏ trong điều kiện hoạt động của hệ thống. Chúng tôi đã minh họa hai dị thường như vậy, đảo ngược ưu tiên và bế tắc. Có nhiều người khác. Mức độ có thể của các vấn đề có thể nảy sinh

Ví dụ 7.7: Đối với biểu đồ mức độ ưu tiên trong Hình 7.7, *Tác vụ 1* có mức 3, *Tác vụ 2* và *Tác vụ 3* có mức 2 và *Tác vụ 4*, *Tác vụ 5* và *Tác vụ 6* có mức 1. Do đó, bộ định thời mức Hu sẽ đưa ra mức ưu tiên cao nhất cho *Tác vụ 1*, *Tác vụ 2* và *Tác vụ 3* mức độ ưu tiên trung bình và các tác vụ *Tác vụ 4*, *Tác vụ 5* và *Tác vụ 6* có mức độ ưu tiên thấp nhất.

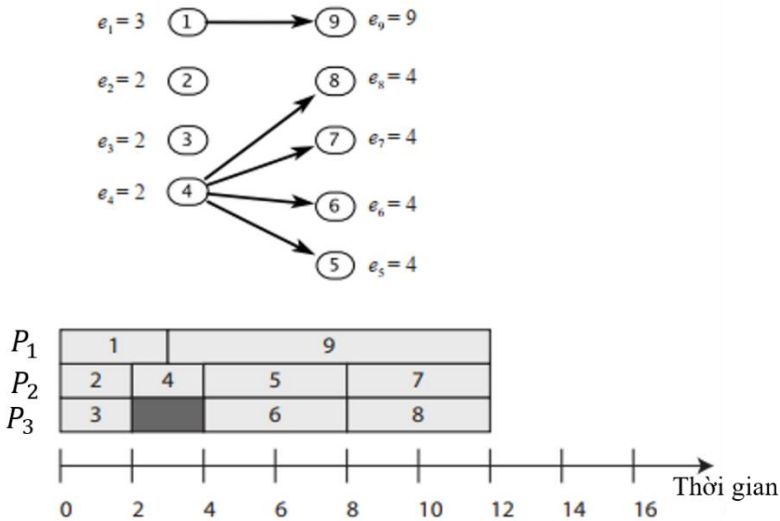
Ví dụ 7.8: Định thời hai bộ vi xử lý được xây dựng bằng thuật toán định thời mức Hu cho đồ thị mức độ ưu tiên trong Hình 7.7 được đưa ra trong Hình 7.13. Khoảng chênh lệch là 4.

được minh họa rõ ràng bởi cái gọi là sự bất thường của Richard [18]. Những điều này cho thấy rằng định thời đa xử lý là không cần thiết, có nghĩa là việc cải thiện hiệu suất ở cấp độ cục bộ có thể dẫn đến sự suy giảm hiệu suất ở cấp độ toàn cầu và dễ hỏng, có nghĩa là những thay đổi nhỏ có thể gây ra hậu quả lớn.

Các dị thường của Richard được tóm tắt trong định lý sau.



Hình 7.12: Định thời song song hai bộ vi xử lý cho các tác vụ.



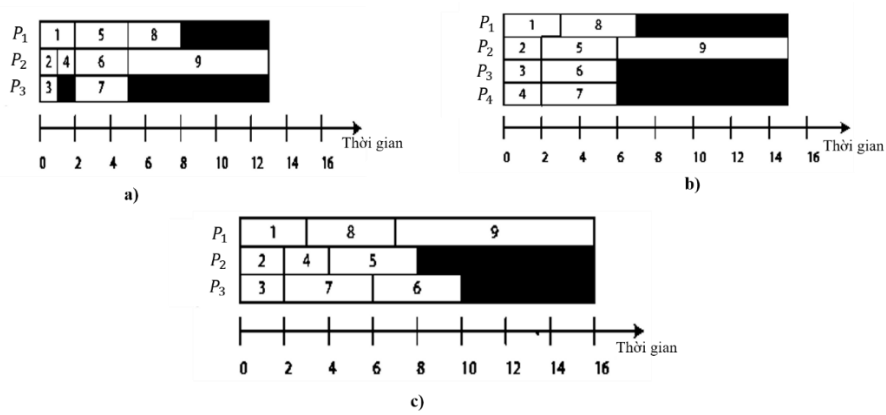
Hình 7.13: Biểu đồ mức độ ưu tiên với chín tác vụ

Định lý 7.4. Nếu một bộ tác vụ có mức ưu tiên cố định, thời gian thực thi và ràng buộc mức độ ưu tiên được định thời trên một số bộ vi xử lý cố định phù hợp với mức độ ưu tiên, thì việc tăng số lượng bộ vi xử lý, giảm thời gian thực thi hoặc làm suy yếu các ràng buộc ưu tiên có thể làm tăng độ dài định thời.

Chứng minh. Định lý này có thể được chứng minh bằng ví dụ trong Hình 7.15. Ví dụ có chín tác vụ với thời gian thực hiện như trong hình. Chúng tôi giả định các tác vụ được chỉ định mức độ ưu tiên để các tác vụ được đánh số thấp hơn có mức độ ưu tiên cao hơn

các tác vụ được đánh số cao hơn. Lưu ý rằng điều này không tương ứng với việc gán mức độ ưu tiên đường tới hạn, nhưng nó đủ để chứng minh định lý. Hình cho thấy định thời ba bộ vi xử lý phù hợp với các ưu tiên. Lưu ý rằng khoảng chênh lệch là 12.

Trước tiên, hãy xem xét điều gì sẽ xảy ra nếu tắt cả thời gian thực thi đều giảm đi một đơn vị thời gian. Hình 7.14(a) là một định thời tuân theo các mức độ ưu tiên và cấp thiết.



Hình 7.14: Các chế độ định thời với ràng buộc thời gian và độ ưu tiên

Lưu ý rằng chênh lệch đã tăng lên 13, mặc dù tổng số lượng tính toán đã giảm đáng kể. Vì thời gian tính toán hiếm khi được biết chính xác, dạng nguy cơ này đặc biệt đáng lo ngại.

Hãy xem xét điều gì sẽ xảy ra tiếp theo nếu chúng tôi thêm bộ vi xử lý thứ tư và giữ mọi thứ khác giống như trong vấn đề ban đầu. Kết quả định thời được hiển thị trên Hình 7.14(b). Một lần nữa,

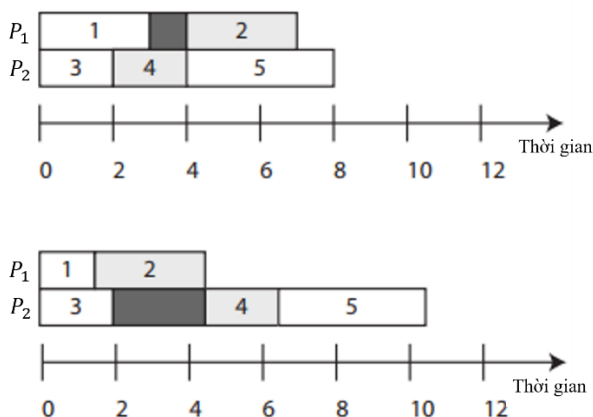
khoảng chênh lệch đã tăng lên (lần này là 15) mặc dù chúng tôi đã tăng thêm 33% độ mạnh xử lý so với ban đầu.

Cuối cùng hãy xem xét điều gì sẽ xảy ra nếu chúng ta làm suy yếu các ràng buộc về mức độ ưu tiên bằng cách loại bỏ các mức độ ưu tiên giữa *Tác vụ 4* và các *Tác vụ 7* và *Tác vụ 8*. Kết quả định thời được hiển thị như trên Hình 7.14(c).

Khoảng chênh lệch hiện đã tăng lên 16, mặc dù việc suy yếu các ràng buộc về mức độ ưu tiên làm tăng tính linh hoạt của định thời. Một sơ đồ định thời dựa trên mức độ ưu tiên đơn giản như thế này không tận dụng được các ràng buộc suy yếu.

Định lý này đặc biệt gây rắc rối khi chúng ta nhận ra rằng thời gian thực thi của phần mềm hiếm khi được biết chính xác. Chính sách định thời sẽ dựa trên giá trị gần đúng và hành vi tại thời điểm chạy có thể khá ngạc nhiên.

Một dạng dị thường khác phát sinh khi có các khóa loại trừ lẫn nhau. Một minh họa được đưa ra trong Hình 7.15. Trong ví dụ này, năm tác vụ được giao cho hai bộ vi xử lý bằng cách sử dụng bộ định thời phân công tĩnh. *Tác vụ 2* và *Tác vụ 4* tranh giành một mutex. Nếu thời gian thực hiện của *Tác vụ 1* bị giảm xuống, thì thứ tự thực hiện của *Tác vụ 2* và *Tác vụ 4* sẽ đảo ngược, dẫn đến tăng thời gian thực thi. Loại dị thường này khá phổ biến trong thực tế.



Hình 7.15: Sự bất thường do các khóa loại trừ tương hỗ

7.6 Tóm tắt

Phần mềm nhúng đặc biệt nhạy cảm với các hiệu ứng thời gian vì nó chắc chắn tương tác với các hệ thống vật lý bên ngoài. Do đó, một kỹ sư thiết kế cần phải chú ý đến việc định thời của các tác vụ. Chương này đã trình bày tổng quan về một số kỹ thuật cơ bản để định thời các tác vụ thời gian thực và định thời song song. Nó đã giải thích một số nguy cơ, chẳng hạn như đảo ngược ưu tiên và định thời bất thường. Một kỹ sư thiết kế nhận thức được những nguy cơ này sẽ được trang bị tốt hơn để đề phòng chúng.

7.7 Bài tập

Bài tập 1. Bài toán này nghiên cứu định thời độ ưu cố định. Hãy xem xét hai tác vụ được thực hiện định kỳ trên một bộ vi xử lý đơn, trong đó *Tác vụ 1* có chu kỳ $p_1 = 4$ và *Tác vụ 2* có chu kỳ $p_2 = 6$.

- (a) Cho thời gian thực hiện của *Tác vụ 1* là $e_1 = 1$. Tìm giá trị lớn nhất cho thời gian thực thi e_2 của *Tác vụ 2* sao cho định thời RM là khả thi.
- (b) Một lần nữa đặt thời gian thực hiện của *Tác vụ 1* là $e_1 = 1$. Gọi non-RM là định thời ưu tiên cố định không phải là định thời RM. Tìm giá trị lớn nhất cho thời gian thực hiện e_2 của *Tác vụ 2* sao cho non-RMS khả thi.
- (c) Đối với cả hai giải pháp của bạn cho (a) và (b) ở trên, hãy tìm cách sử dụng bộ vi xử lý. Cái nào tốt hơn?
- (d) Đối với định thời RM, có giá trị nào cho e_1 và e_2 mang lại hiệu suất sử dụng 100% không? Nếu vậy, hãy đưa ra một ví dụ.

Bài tập 2. Bài toán này nghiên cứu việc định thời ưu tiên thay đổi. Hãy xem xét hai tác vụ được thực thi định kỳ trên một bộ vi xử lý đơn lẻ, trong đó *Tác vụ 1* có chu kỳ $p_1 = 4$ và *Tác vụ 2* có chu kỳ $p_2 = 6$. Đặt thời hạn cuối cho mỗi lần gọi tác vụ là cuối chu kỳ. Nghĩa là, lệnh gọi đầu tiên của *Tác vụ 1* có thời hạn 4, lệnh gọi thứ hai của *Tác vụ 1* có thời hạn 8, v.v.

- (a) Cho thời gian thực hiện của *Tác vụ 1* là $e_1 = 1$. Tìm giá trị lớn nhất cho e_2 của *Tác vụ 2* sao cho EDF khả thi.
- (b) Đối với giá trị của e_2 mà bạn tìm thấy trong phần (a), hãy so sánh định thời EDF với định thời RM từ Bài tập 1 (a). Định thời nào có ít ưu tiên hơn? Định thời nào sử dụng tốt hơn?

Bài tập 3. Bài toán này so sánh định thời RM và EDF. Xét hai tác vụ có thời gian $p_1 = 2$ và $p_2 = 3$ và thời gian thực thi $e_1 = e_2 = 1$. Giả sử rằng thời hạn cuối cho mỗi lần thực hiện là cuối chu kỳ.

- (a) Áp dụng giải thuật RM cho bộ tác vụ này và tìm cách định thời bộ vi xử lý.
- (b) Chứng tỏ rằng bất kỳ sự gia tăng nào về e_1 hoặc e_2 đều làm cho định thời RM không khả thi. Nếu bạn giữ $e_1 = e_2 = 1$ và $p_2 = 3$ không đổi, liệu có thể giảm p_1 xuống dưới 2 mà vẫn có được một định thời khả thi không? Bằng bao nhiêu? Nếu bạn giữ $e_1 = e_2 = 1$ và $p_1 = 2$ không đổi, liệu có thể giảm p_2 xuống dưới 3 mà vẫn có được một định thời khả thi không? Bằng bao nhiêu?
- (c) Tăng thời gian thực hiện của *Tác vụ 2* lên $e_2 = 1.5$ và đưa ra định thời EDF. Nó có khả thi không? Việc sử dụng bộ vi xử lý thì sao?

Bài tập 4. Bài toán này, do Hokeun Kim đưa ra, cũng so sánh định thời của RM và EDF. Xem xét hai tác vụ được thực hiện định kỳ trên một bộ vi xử lý đơn lẻ, trong đó *Tác vụ 1* có chu kỳ $p_1 = 4$ và *Tác vụ 2* có chu kỳ $p_2 = 10$. Giả sử *Tác vụ 1* có thời gian thực hiện $e_1 = 1$ và *Tác vụ 2* có thời gian thực hiện $e_2 = 7$.

- (a) Phác thảo một định thời tỷ lệ đơn (cho 20 đơn vị thời gian, bội số chung nhỏ nhất của 4 và 10). Định thời có khả thi không?

- (b) Bây giờ, giả sử *Tác vụ 1* và *Tác vụ 2* tranh giành một khóa mutex, giả sử rằng khóa được nhận vào đầu mỗi lần thực thi và được giải phóng vào cuối mỗi lần thực thi. Ngoài ra, giả sử rằng việc giành hoặc giải phóng các khóa không mất thời gian và giao thức kế thừa ưu tiên được sử dụng. Định thời tỷ lệ đơn có khả thi không?
- (c) Vẫn giả sử rằng các *Tác vụ 1* và *Tác vụ 2* cạnh tranh cho khóa mutex, như trong phần (b). Giả sử rằng *Tác vụ 2* đang chạy một thuật toán anytime, đây là một thuật toán có thể kết thúc sớm và vẫn mang lại kết quả hữu ích. Ví dụ: nó có thể là một thuật toán xử lý hình ảnh sẽ cung cấp hình ảnh chất lượng thấp hơn khi kết thúc sớm. Tìm giá trị lớn nhất cho thời gian thực hiện e_2 của *Tác vụ 2* sao cho định thời tỷ lệ đơn khả thi. Xây dựng định thời kết quả, với thời gian thực hiện giảm cho *Tác vụ 2* và phác thảo định thời cho 20 đơn vị thời gian. Bạn có thể cho rằng thời gian thực hiện luôn là số nguyên dương.
- (d) Đối với bài toán ban đầu, trong đó $e_1 = 1$ và $e_2 = 7$, và không có khóa mutex, hãy phác thảo một định thời EDF cho 20 đơn vị thời gian. Để ràng buộc giữa các lần thực thi tác vụ có cùng thời hạn cuối, giả sử việc thực hiện *Tác vụ 1* có mức độ ưu tiên cao hơn việc thực hiện *Tác vụ 2*. Định thời có khả thi không?
- (e) Bây giờ hãy xem xét thêm tác vụ thứ ba, *Tác vụ 3*, có chu kỳ $p_3 = 5$ và thời gian thực thi $e_3 = 2$. Ngoài ra, giả sử như trong

phần (c) rằng chúng ta có thể điều chỉnh thời gian thực thi của *Tác vụ 2*.

Tìm giá trị lớn nhất cho thời gian thực hiện e_2 của *Tác vụ 2* sao cho định thời EDF khả thi và phác thảo định thời cho 20 đơn vị thời gian. Một lần nữa, bạn có thể giả định rằng thời gian thực thi luôn là số nguyên dương. Để có sự ràng buộc giữa các lần thực thi tác vụ có cùng thời hạn cuối, giả sử tác vụ i có mức ưu tiên cao hơn tác vụ j nếu $i < j$.)

Chương 8 THIẾT KẾ HỆ THỐNG DỰA TRÊN MÔ HÌNH

Trong các chương trước, chúng ta đã tìm hiểu qua các bước để thiết kế một hệ thống nhúng thực tế. Hướng tiếp cận này được đánh giá hiệu quả dựa trên kết quả chạy thực tế và các thay đổi được thực hiện dựa trên điều kiện thiết kế cụ thể. Tuy nhiên, một trong những hạn chế lớn nhất của cách tiếp cận này là khó có thể đánh giá tổng thể tính đúng đắn của thiết kế (thường được xác định bằng các yếu tố khoa học), khó có thể dự đoán được thay đổi và khó phân tích mở rộng hệ thống.

Có một cách tiếp cận thiết kế một cách hiện quả hơn về mặt khoa học, khắc phục được các hạn chế trên, gọi là thiết kế hệ thống dựa trên mô hình (model based design). Phương pháp thiết kế này về cơ bản dựa trên các cơ sở khoa học như toán học, vật lý, hóa học và sự phát triển của các công cụ mô hình hóa và mô phỏng hệ thống.

Trong chương này, chúng ta sẽ lần lượt tìm hiểu cơ bản về hướng tiếp cận mô hình này thông qua các dạng mô hình hóa thay đổi trạng thái của hệ thống gọi là **máy trạng thái** (state machine). Các dạng hệ thống thay đổi cơ bản được đề cập trong chương này gồm: hệ thống thay đổi liên tục theo thời gian, hệ thống thay đổi rời rạc và hệ thống lai (kết hợp của hai dạng trên).

Để thuận tiện cho việc tìm hiểu, chúng ta sử dụng nghiên cứu dựa trên một ví dụ về mạch điện áp đơn giản. Từ các kỹ thuật được

sử dụng để nghiên cứu sự thay đổi của của hệ thống đó, chúng ta sẽ mở rộng ra cho các thiết kế khác như các mạch, các quy trình hóa học, sinh học...

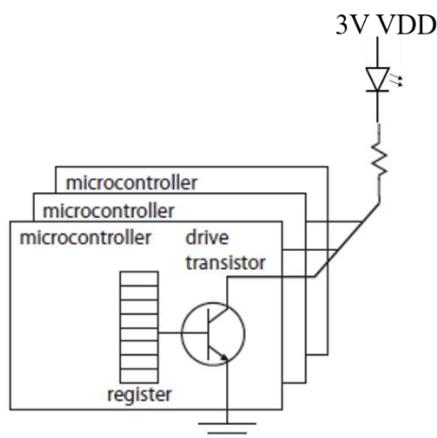
Nội dung của Chương 8 bao gồm:

- Mô hình hệ thống thay đổi liên tục theo thời gian
- Mô hình hệ thống thay đổi rời rạc
- Mô hình hệ thống lai
- Mô hình tổ hợp máy trạng thái

8.1 Mô hình hệ thống thay đổi liên tục

8.1.1 Ví dụ mở đầu

Giả sử chúng ta cần thiết kế một mạch điện để điều khiển một đèn diot đơn giản như trong Hình 8.1. Điện áp sử dụng là 3 V và transistor trong chân của vi điều khiển có khả năng lái một dòng 18 mA. Biết rằng điện áp rơi trên diot khi hoạt động là 2 V. Chúng ta cần thiết kế điện trở bao nhiêu ohm để mạch có thể hoạt động ổn định mà không ảnh hưởng đến vi điều khiển.



Hình 8.1: Thiết kế mạch điều khiển đèn diot

Để giải quyết vấn đề trên, chúng ta căn cứ vào mối quan hệ giữa điện áp, dòng điện và điện trở như thể hiện trong công thức (8.1).

$$V = R * I \quad (8.1)$$

Khi diot phát sáng, điện áp V trên điện trở sẽ bằng điện áp nguồn (3 V) trừ đi điện áp rơi trên diot (2 V); cụ thể, điện áp rơi trên điện trở là 1 V. Để duy trì một dòng điện 18 mA, điện trở cần có giá trị là: $R = V/I = 1/0.018 \approx 56$ (ohm).

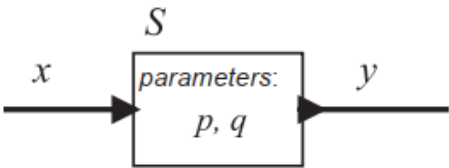
Trong việc thiết kế đơn giản trên, chúng ta sử dụng căn cứ thiết kế là mối quan hệ về toán học của các đại lượng theo công thức (8.1). Căn cứ vào công thức này, chúng ta dễ dàng xác định được sự thay đổi của giá trị điện trở tương ứng với những thay đổi ở đầu vào như thay điện áp nguồn, thay diot hay là thay vi điều khiển với khả năng lái dòng khác nhau.

Trong mô hình thiết kế trên, chúng ta xác định rõ yêu cầu và tín hiệu đầu vào, tín hiệu đầu ra và mối quan hệ giữa các đại lượng đầu vào và đầu ra. Tùy thuộc vào số lượng tín hiệu đầu vào và đầu ra, chúng ta có các mô hình thiết kế khác nhau. Các mô hình này gọi chung là **mô hình tác nhân** (Actor model).

8.1.2 Mô hình tác nhân – Actor model

8.1.2.1 Mô hình tác nhân một tín hiệu vào/ra

Trong ví dụ trên, một hệ thống vật lý được mô hình bởi một phương trình phản ánh mối liên hệ giữa tín hiệu đầu vào (điện áp) với tín hiệu đầu ra (dòng điện). Một hệ thống vật lý như vậy có thể được xem như một thành phần trong một hệ thống lớn hơn. Để mô hình hóa các hệ thống lớn, một hệ thống liên tục theo thời gian có thể được mô hình như một hộp đen với một ngõ vào và một ngõ ra như thể hiện trong Hình 8.2.



Hình 8.2: Mô hình hệ thống thay đổi liên tục một ngõ vào/ra
 Trong đó, tín hiệu đầu vào x và tín hiệu đầu ra y là các hàm có dạng:

$$x: R \rightarrow R, y: R \rightarrow R$$

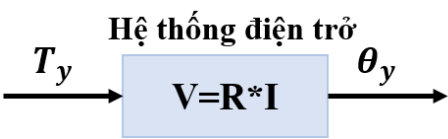
Ở đây miền đại diện cho thời gian và miền kết hợp đại diện cho giá trị của tín hiệu tại một thời điểm cụ thể. Miền R có thể được thay thế bằng R_+ (tập các số thực không âm) nếu chúng ta muốn

mô hình hóa rõ ràng một hệ thống đã tồn tại và bắt đầu hoạt động tại một thời điểm cụ thể. Mô hình của hệ thống là một hàm có dạng:

$$S: X \rightarrow Y \tag{8.2}$$

Trong đó, $X = Y = R^R$, tập hợp các hàm ánh xạ số thực đến số thực, như x và y ở trên. Hàm S có thể phụ thuộc vào các tham số của hệ thống, trong trường hợp này, các tham số có thể được hiển thị tùy ý trong hộp, và có thể được tùy chọn đưa vào ký hiệu hàm. Ví dụ, trong hình trên, nếu có tham số p và q , chúng ta có thể viết hàm hệ thống là $S_{p,q}$ hoặc thậm chí $S(p,q)$. Lưu ý rằng cả hai ký hiệu này đều đại diện cho các hàm có dạng trong Phương trình (8.2). Một mô hình trong đó đầu vào và đầu ra là các hàm được gọi là mô hình tác nhân.

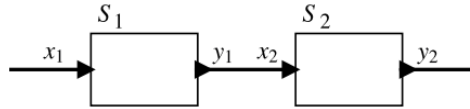
Xét mô hình tác nhân cho điện trở trong ví dụ trên. Mô hình tác nhân này có thể được mô tả như trong Hình 8.3.



Hình 8.3: Mô hình tác nhân của điện trở

Đầu vào và đầu ra của mô hình đều là các hàm thời gian liên tục (các hàm số của điện áp và dòng điện). Các thông số của tác nhân là điện áp đầu vào và dòng điện ngõ ra. Hàm tính các tác nhân được xác định bởi Phương trình (8.1).

Các mô hình tác nhân có thể kết hợp với nhau trong một hệ thống. Ví dụ, với hai tác nhân S_1 và S_2 , chúng ta có thể tạo thành một hệ thống nối tiếp như trong Hình 8.4.



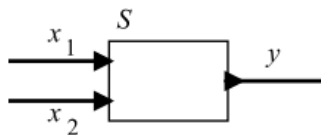
Hình 8.4: Hệ thống nối tiếp mô hình tác nhân

Trong mô hình hệ thống trên, đầu ra của S_1 chính là đầu vào S_2 ; nói cách khác, $y_1 = x_2$. Biểu diễn theo thời gian của sự kết hợp này như sau:

$$\forall t \in R, y_1(t) = x_2(t) \quad (8.3)$$

8.1.2.2 Mô hình tác nhân nhiều tín hiệu vào ra

Ngoài các mô hình tác nhân đơn giản một đầu vào, chúng ta có thể có các tác nhân có nhiều tín hiệu đầu vào và/hoặc nhiều tín hiệu đầu ra. Hình 8.5 trình bày ví dụ của một tác nhân với hai tín hiệu đầu vào và một tín hiệu đầu ra.

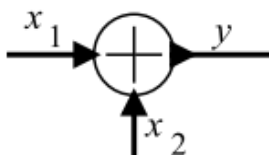


Hình 8.5: Mô hình tác nhân hai tín hiệu đầu vào

Biểu diễn toán học của mô hình hai tín hiệu đầu vào và một tín hiệu đầu ra như trên có thể là một bộ cộng tín hiệu như sau:

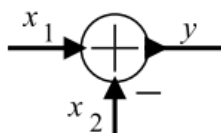
$$\forall t \in R, y(t) = x_1(t) + x_2(t) \quad (8.4)$$

Như đã đề cập ở trên, để dễ dàng hiểu được ý nghĩa của mô hình, biểu tượng được sử dụng để tượng trưng cho mô hình cộng thường được sử dụng như trong Hình 8.6.



Hình 8.6: Biểu tượng của mô hình bộ cộng

Tương tự, một mô hình bộ trừ có thể được minh họa bằng biểu tượng như trong Hình 8.7.



Hình 8.7: Biểu tượng của mô hình bộ trừ

Tác nhân này biểu thị một hàm $S: (R \rightarrow R)^2 \rightarrow (R \rightarrow R)$ được định nghĩa như sau:

$$\begin{aligned} \forall t \in R, \forall x_1, x_2 \in (R \rightarrow R), \\ (S(x_1, x_2))(t) = y(t) = x_1(t) - x_2(t) \end{aligned} \quad (8.5)$$

Chúng ta vừa tìm hiểu qua cơ bản về mô hình tác nhân của một hệ thống vật lý và biểu diễn mô hình thông qua biểu tượng tượng trưng. Trong phần còn lại của chương này, chúng ta sẽ không phân biệt giữa một hệ thống và mô hình tác nhân của nó, trừ khi sự phân biệt là cần thiết cho lập luận.

8.1.3 Thuộc tính của hệ thống

Trong phần này, chúng ta sẽ xem xét một số thuộc tính của các mô hình tác nhân và hệ thống mà nó mô tả, bao gồm tính nhân quả, tính không nhớ, tính tuyến tính, tính bất biến thời gian và tính ổn định.

8.1.3.1 Hệ thống nhân quả

Một cách trực quan, một hệ thống được gọi là hệ thống nhân quả nếu đầu ra của nó chỉ phụ thuộc vào các đầu vào hiện tại và quá khứ. Tuy nhiên, có nhiều khó khăn làm cho khái niệm này trở nên không chính xác.

Trước tiên, chúng ta xem qua định nghĩa cho khái niệm “đầu vào hiện tại và quá khứ”. Xét một tín hiệu liên tục theo thời gian $x: R \rightarrow A$, đối với một số tập A . Cho $x|_{t < \tau}$ là một hàm giới hạn theo thời gian với giá trị giới hạn là $t \leq \tau$ và trong giới hạn đó nó được xác định như sau: $x|_{t < \tau}(t) = x(t)$. Dựa vào định nghĩa này, nếu x là đầu vào của hệ thống thì $x|_{t < \tau}$ là “đầu vào hiện tại và quá khứ” tại thời điểm τ .

Tiếp theo, chúng ta xem xét một hệ thống liên tục theo thời gian $S: X \rightarrow Y$, trong đó $X = A^R$ và $Y = B^R$ đối với các tập A và B . Hệ thống này được gọi là hệ thống nhân quả nếu $\forall x_1, x_2 \in R$ và $\tau \in R$, ta có kết quả sau:

$$x_1|_{t \leq \tau} = x_2|_{t \leq \tau} \implies S(x_1)|_{t \leq \tau} = S(x_2)|_{t \leq \tau} \quad (8.6)$$

Từ công thức trên, chúng ta có thể hiểu một cách đơn giản hơn là một hệ thống gọi là hệ thống nhân quả nếu hệ thống có hai tín hiệu đầu vào x_1 và x_2 giống nhau cho đến một thời điểm τ nào đó (bao hàm cả τ) thì tín hiệu đầu ra cũng giống nhau cho đến thời điểm τ (bao hàm cả τ). Căn cứ vào định nghĩa này, tất cả các hệ thống chúng ta đã xem xét qua đều là các hệ thống nhân quả.

Thêm vào đó, chúng ta có khái niệm về hệ thống nhân quả chặt chẽ. Một hệ thống gọi là có quan hệ nhân quả chặt chẽ nếu $\forall x_1, x_2 \in R$ và $\tau \in R$, ta có kết quả sau:

$$x_1|_{t < \tau} = x_2|_{t < \tau} \Rightarrow S(x_1)|_{t \leq \tau} = S(x_2)|_{t \leq \tau} \quad (8.7)$$

Nói một cách dễ hiểu, hệ thống là nhân quả chặt chẽ nếu hệ thống có hai tín hiệu đầu vào x_1 và x_2 giống nhau cho đến một thời điểm τ nào đó (không bao hàm cả τ) thì tín hiệu đầu ra cũng giống nhau cho đến thời điểm τ (bao hàm cả τ). Đầu ra tại thời điểm t của một hệ thống nhân quả chặt chẽ sẽ không phụ thuộc vào đầu vào của nó tại thời điểm t . Nói cách khác, đầu ra chỉ phụ thuộc vào đầu vào trong quá khứ. Tất nhiên, một hệ thống nhân quả chặt chẽ cũng là một hệ thống nhân quả.

8.1.3.2 Hệ thống không nhớ

Một cách trực quan, một hệ thống có nhớ nếu đầu ra không chỉ phụ thuộc vào các tín hiệu đầu vào hiện tại mà còn phụ thuộc vào các tín hiệu đầu vào trong quá khứ (hoặc các tín hiệu đầu vào trong tương lai, nếu hệ thống không có quan hệ nhân quả). Chúng ta cùng

xem xét một hệ thống liên tục theo thời gian $S: X \rightarrow Y$, trong đó $X = A^R$ và $Y = B^R$ cho các tập A và B . Về mặt hình thức, hệ thống này là không có nhớ nếu tồn tại một hàm $f: A \rightarrow B$ sao cho $\forall x \in X$, ta có:

$$(S(x))(t) = f(x(t)), \forall t \in R \quad (8.8)$$

Tức là, đầu ra $(S(x))(t)$ tại thời điểm t chỉ phụ thuộc vào đầu vào $x(t)$ tại thời điểm t .

8.1.3.3 Tuyến tính và bất biến theo thời gian

Hệ thống tuyến tính và bất biến theo thời gian (Linear and Time Invariant - LTI) có các tính chất toán học đặc biệt chính xác. Phần lớn lý thuyết về hệ thống điều khiển phụ thuộc vào các đặc tính này. Các thuộc tính này tạo nên thành phần chính của các thí nghiệm về tín hiệu và hệ thống, và nằm ngoài phạm vi của giáo trình này. Tuy nhiên, trong quá trình nghiên cứu giáo trình, đôi khi chúng ta sẽ xem xét một số dạng đơn giản của các thuộc tính này. Do đó, chúng ta cũng cần thiết giới thiệu nhanh về hệ thống LTI.

Một hệ thống $S: X \rightarrow Y$, trong đó X và Y là các tập tín hiệu, là tuyến tính nếu nó thỏa mãn thuộc tính chồng chất sau đây:

$$\begin{aligned} \forall x_1, x_2 \in X, \forall a, b \in R, \\ (ax_1 + bx_2) = aS(x_1) + bS(x_2) \end{aligned} \quad (8.9)$$

Để xác định tính bất biến theo thời gian, trước tiên chúng ta định nghĩa một tác nhân liên tục theo thời gian đặc biệt được gọi là **độ**

trễ (delay). Chúng ta giả sử một hàm $D_\tau: X \rightarrow Y$, trong đó X và Y là tập hợp các tín hiệu thời gian liên tục, được định nghĩa như sau:

$$\forall x \in X, \forall y \in Y, (D_\tau(x))(t) = x(t - \tau) \quad (8.10)$$

Trong đó, τ là một tham số của tác nhân trễ. Một hệ thống $S: X \rightarrow Y$ được gọi là bất biến theo thời gian nếu:

$$\forall x \in X, \forall \tau \in R, S(D_\tau(x)) = D_\tau(S(x)) \quad (8.11)$$

Hệ thống LTI là một hệ thống vừa tuyến tính và vừa bất biến theo thời gian. Trong quá trình mô hình hóa sự thay đổi của các hệ thống vật lý, chúng ta luôn cố gắng để chọn mô hình LTI khi có thể. Nếu một kết quả hợp lý ước lượng gần đúng dẫn đến một mô hình LTI thì chúng ta nên thực hiện phép gần đúng này. Tuy nhiên, thực tế chúng ta không phải lúc nào cũng dễ dàng xác định xem giá trị gần đúng là hợp lý hoặc có thể tìm ra các mô hình mà giá trị gần đúng là hợp lý. Điều này dẫn đến việc chúng ta thường xây dựng các mô hình phức tạp hơn mức cần thiết.

8.1.3.4 Tính ổn định

Một hệ thống được gọi là ổn định (với đầu vào và đầu ra giới hạn – bounded input and bounded output - BIBO) nếu tín hiệu đầu ra được giới hạn cho tất cả các tín hiệu đầu vào được giới hạn. Xét một hệ thống liên tục theo thời gian với đầu vào w và đầu ra v . Đầu vào bị giới hạn nếu tồn tại một số thực $A < \infty$ sao cho $|w(t)| \leq A$ với mọi $t \in R$. Đầu ra bị giới hạn nếu tồn tại một số thực số $B < \infty$ sao cho $|v(t)| \leq B$ với mọi $t \in R$. Hệ thống là ổn định nếu với bất

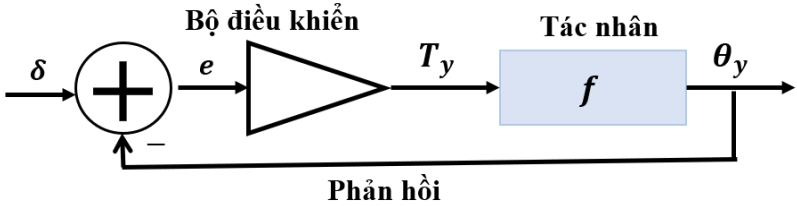
kỳ đầu vào nào bị giới hạn bởi A đều tồn tại một số B là giới hạn ở đầu ra.

8.1.4 Hệ thống kiểm soát có phản hồi

Một hệ thống **phản hồi trực tiếp** (direct feedback control) được định nghĩa là hệ thống có đầu ra của một tác nhân được phản hồi về và tác động đến đầu vào của tác nhân đó. Hình 8.8 trình bày ví dụ minh họa cho kiểm soát hồi đáp của hệ thống cân bằng độ ổn định của máy bay trực thăng. Hầu hết các hệ thống điều khiển sử dụng cơ chế phản hồi. Các hệ thống này kiểm soát một giá trị độ lỗi (e trong hình), là sự khác biệt giữa giá trị xử lý mong muốn và giá trị xử lý thực tế (θ_y trong hình), và sử dụng giá trị độ lỗi đó để chỉnh sửa cách xử lý. Sử dụng giá trị đo lỗi phản hồi và tín hiệu hiệu chỉnh tương ứng (T_y trong hình) sẽ giúp giảm lỗi trong tương lai. Thông thường, tín hiệu hiệu chỉnh chỉ có thể tác động đối với các lỗi trong tương lai. Do đó, hệ thống phản hồi thông thường phải bao gồm ít nhất một tác nhân quan hệ nhân quả chặt chẽ trong mỗi vòng lặp phản hồi.

Kiểm soát phản hồi là một chủ đề phức tạp, được đề cập nhiều trong các giáo trình và các khóa học. Ở đây, chúng ta chỉ đề cập đến vấn đề mang tính giới thiệu, vừa đủ để hiểu cơ bản về quan hệ tương tác giữa phần mềm và phần cứng vật lý. Hệ thống điều khiển phản hồi thường được thực hiện bằng cách sử dụng phần mềm

nhúng, và hệ thống tổng thể là một hệ thống kết hợp của phần mềm và cơ cấu cơ khí động học vật lý.



Hình 8.8: Hệ thống điều khiển có phản hồi

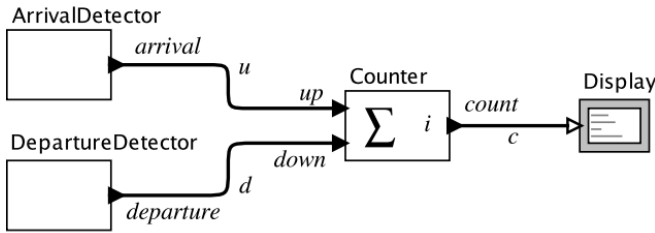
8.2 Mô hình hệ thống thay đổi rời rạc

Các mô hình hệ thống nhúng thường bao gồm cả thành phần liên tục và rời rạc. Nói một cách dễ hiểu, các thành phần liên tục thay đổi một cách liên tục theo thời gian trong khi các thành phần rời rạc thay đổi đột ngột tại các thời điểm rời rạc. Trong Mục 8.1 chúng ta đã xem xét mô hình các thành phần liên tục, và chỉ ra rằng sự thay đổi của các hệ thống thường có thể được mô hình hóa bằng các phương trình toán học, hoặc các mô hình tác nhân tương đương. Trái lại, các thành phần rời rạc không được mô hình hóa một cách thuận tiện thông qua các phương trình thông thường. Trong chương này, chúng ta sẽ tìm hiểu cách sử dụng máy trạng thái để mô hình hóa các hệ thống thay đổi rời rạc.

8.2.1 Các hệ thống thay đổi rời rạc

Một hệ thống rời rạc vận hành theo một chuỗi các bước rời rạc và được gọi là hệ thống thay đổi rời rạc (gọi tắt là hệ thống rời rạc). Một số hệ thống vốn một cách tự nhiên là rời rạc.

Ví dụ 8.1: Chúng ta xem xét ví dụ về một hệ thống đếm số lượng ô tô ra vào bãi đỗ để theo dõi xem có bao nhiêu ô tô ở trong bãi đỗ tại một thời điểm nào đó. Hệ thống đếm của bãi đỗ có thể được mô hình hóa như trong Hình 8.9. Bây giờ chúng ta tạm bỏ qua cách thiết kế các cảm biến phát hiện ra vào hoặc khởi hành của ô tô. Chúng ta chỉ đơn giản giả định rằng tác nhân *ArrivalDetector* tạo ra một sự kiện khi một chiếc ô tô đi vào và tác nhân *DepartureDetector* tạo ra một sự kiện khi một chiếc xe đi ra. Tác nhân *Counter* lưu giữ một số đếm đang chạy, bắt đầu từ giá trị ban đầu i . Mỗi khi số lượng thay đổi, nó tạo ra một sự kiện đầu ra cập nhật hiển thị trên màn hình *Display*.



Hình 8.9: Mô hình của một hệ thống theo dõi số lượng ô tô trong một nhà để xe.

Trong ví dụ trên, mỗi sự kiện ô tô đi vào hoặc ô tô đi ra được mô hình hóa như một sự kiện rời rạc. Một sự kiện rời rạc xảy ra ngay lập tức thay vì theo thời gian. Tác nhân *Counter* trong Hình 8.9 tương tự với khái niệm tác nhân trong phần trước. Tín hiệu u ở đầu vào của *Counter* là một hàm có dạng:

$$u: \mathbb{R} \rightarrow \{absent, present\}$$

Điều này có nghĩa là tại bất kỳ thời điểm $t \in \mathbb{R}$ nào, đầu vào là $u(t)$ hoặc không hiện diện; nó khác, *present* nghĩa là có xuất hiện sự kiện và *absent* là không có sự kiện nào tại thời điểm đó. Một tín hiệu dạng này được gọi là tín hiệu thuần túy (***pure signal***). Nó không mang giá trị gì, nhưng thay vào đó nó cung cấp tất cả thông tin của nó bằng cách hiện hữu hoặc vắng mặt tại bất kỳ thời điểm nào. Tín hiệu d trong Hình 8.9 cũng là một tín hiệu thuần túy.

Giả sử tác nhân *Counter* của chúng ta hoạt động như sau. Khi một sự kiện xuất hiện ở đầu vào ***up***, *Counter* sẽ tăng bộ đếm lên một đơn vị và tạo ra giá trị mới ở đầu ra. Khi một sự kiện xuất hiện ở đầu vào ***down***, *Counter* sẽ giảm một đơn vị và tạo ra giá trị mới ở đầu ra. Tại mọi thời điểm khác (khi cả hai đầu vào đều không có), nó không tạo ra đầu ra. Do đó, tín hiệu c trong Hình 8.9 có thể được mô hình hóa bởi một hàm có dạng như sau:

$$c: R \rightarrow \{absent\} \cup \mathbb{Z}$$

Tín hiệu này là không thuần túy, nhưng giống như u và d , nó có thể là không có hoặc có hiện diện. Không giống như u và d là khi nó hiện diện thì nó có một giá trị (một số nguyên).

Giả sử thêm rằng các đầu vào không có ở hầu hết các thời điểm, hoặc về mặt kỹ thuật, rằng các đầu vào là rời rạc. Sau đó, *Counter* thay đổi tương ứng với từng chuỗi sự kiện đầu vào. Điều này rất khác với thay đổi liên tục với một loạt đầu vào liên tục của bộ *Integrator*.

Đầu vào cho *Counter* là một cặp tín hiệu rời rạc mà tại một số thời điểm nhất định có một sự kiện (hiện diện), và vào các thời điểm khác không có sự kiện (không hiện diện). Đầu ra cũng là một tín hiệu rời rạc mà khi có đầu vào thì có giá trị là số tự nhiên và lúc khác thì không có hiện diện. Rõ ràng, tác nhân *Counter* chỉ hoạt động khi có đầu vào và không làm bất cứ điều gì khi đầu vào không hiện diện. Do đó, tác nhân *Counter* được gọi là hệ thống thay đổi rời rạc.

Chúng ta cũng cần lưu ý rằng, trên thực tế, các tín hiệu đầu vào rời rạc thì không nhất thiết tín hiệu đầu ra cũng là rời rạc.

Sự thay đổi của một hệ thống rời rạc có thể được mô tả như một chuỗi các bước mà chúng ta gọi là phản ứng (**reaction**) với mỗi bước được giả định là tức thời. Các phản ứng của một hệ thống rời rạc được kích hoạt bởi môi trường mà hệ thống rời rạc hoạt động. Trong trường hợp của ví dụ Hình 8.9, các phản ứng của tác nhân *Counter* được kích hoạt khi có một hoặc nhiều sự kiện đầu vào. Đó là, trong ví dụ này, các phản ứng được kích hoạt bởi sự kiện ô tô vào và ra. Khi cả hai đầu vào cho *Counter* đều không hiện diện thì không có phản ứng nào xảy ra.

Một phản ứng cụ thể sẽ theo dõi các giá trị của đầu vào tại một thời điểm cụ thể t và tính toán các giá trị đầu ra cho cùng thời điểm t . Giả sử một tác nhân có các cổng đầu vào $P = \{p_1; \dots; p_N\}$, trong đó p_i là tên của cổng đầu vào thứ i . Giả sử thêm rằng đối với mỗi

cổng đầu vào $p \in P$, một tập V_p biểu thị các giá trị có thể nhận được trên cổng p khi đầu vào có mặt. V_p được gọi là loại (**type**) của cổng p . Với một phản ứng, chúng ta coi mỗi $p \in P$ như một biến nhận giá trị $p \in V_p \cup \{absent\}$. Việc định giá các yếu tố đầu vào P là sự gán giá trị trong V_p cho mỗi biến $p \in P$ hoặc một khẳng định rằng p không hiện diện.

Nếu cổng p nhận một tín hiệu thuần túy, thì $V_p = \{present\}$. Giá trị duy nhất có thể khi không có sự vắng mặt là sự có mặt. Do đó, với một phản ứng, biến p sẽ có giá trị trong tập $\{present; absent\}$.

8.2.2 Khái niệm về trạng thái

Một cách trực quan, trạng thái của một hệ thống là tình trạng của nó tại một thời điểm cụ thể. Nói chung, trạng thái ảnh hưởng đến cách hệ thống phản ứng với đầu vào. Về mặt hình thức, chúng ta định nghĩa trạng thái là một mã hóa của những thứ xảy ra trong quá khứ có ảnh hưởng đến phản ứng của hệ thống đối với các đầu vào ở hiện tại hoặc tương lai. Trạng thái là tổng hợp sự kiện hay hoạt động của hệ thống trong quá khứ.

Chúng ta hãy xem xét tác nhân *Counter* trong Hình 8.9. Tác nhân này có trạng thái, trong trường hợp có xảy ra, có cùng giá trị với đầu ra tại bất kỳ thời điểm t nào. Trạng thái đó là sự tích lũy các giá trị đầu vào trong quá khứ và hoạt động một cách độc lập. Trạng thái $s(t)$ tại thời điểm t là một số nguyên, vì vậy $States \subset \mathbb{Z}$. Một

ga ra ô tô thực tế có số khoảng trống M không âm và hữu hạn. Vì vậy, không gian trạng thái cho tác nhân *Counter* được sử dụng theo cách này sẽ là $States = \{0, 1, 2, \dots, M\}$. Không gian trạng thái cho ga ra ô tô này là hữu hạn.

Các mô hình rời rạc với không gian trạng thái hữu hạn được gọi là máy trạng thái hữu hạn (Finite State Machine - FSM). Có những nhiều thuật phân tích hiệu quả đã được giới thiệu cho các mô hình máy trạng thái hữu hạn. Trong phần tiếp theo, chúng ta sẽ xem xét các kỹ thuật này.

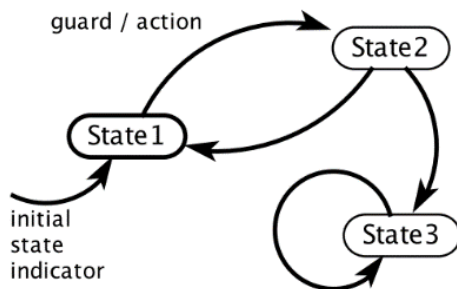
8.2.3 Máy trạng thái hữu hạn

Máy trạng thái là một mô hình của một hệ thống có sự thay đổi rời rạc mà phản ứng của hệ thống với tín hiệu đầu vào có thể phụ thuộc vào trạng thái hiện tại của nó. Một máy trạng thái hữu hạn (FSM) là máy trạng thái trong đó tập hợp các trạng thái có thể có là hữu hạn.

Nếu số lượng trạng thái là tương đối nhỏ thì FSM có thể được vẽ một cách thích hợp bằng cách sử dụng ký hiệu đồ họa như trong Hình 8.10. Ở đây, mỗi trạng thái được biểu diễn bằng một bong bóng, vì vậy đối với biểu đồ này, tập hợp các trạng thái được cho bởi:

$$States = \{State1, State2, State3\}$$

Ở đầu mỗi chuỗi phản ứng, có một trạng thái ban đầu, State 1, được biểu thị trong biểu đồ bằng một mũi tên chỉ vào nó.



Hình 8.10: Các ký hiệu liên quan máy trạng thái

8.2.3.1 Chuyển trạng thái

Sự chuyển tiếp (transformation) giữa các trạng thái quyết định sự thay đổi rời rạc của máy trạng thái và ánh xạ của giá trị đầu vào đến giá trị đầu ra. Chuyển trạng thái được biểu diễn dưới dạng một mũi tên cong, như trong Hình 8.10, đi từ trạng thái này sang trạng thái khác. Một quá trình chuyển trạng thái cũng có thể bắt đầu và kết thúc ở cùng một trạng thái, như được minh họa với State3 trong hình. Trong trường hợp này, quá trình chuyển trạng thái được gọi là giữ nguyên trạng thái (hay tự chuyển trạng thái).

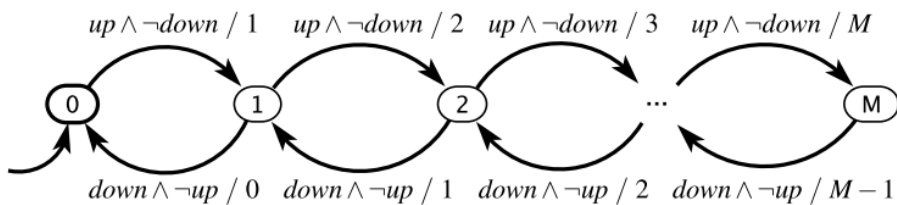
Trong Hình 8.10, sự chuyển trạng thái từ State1 sang State2 được gắn nhãn “guard/action”. *Guard* xác định điều kiện để sự chuyển trạng thái có thể được thực hiện theo phản ứng hay không. *Action* chỉ định đầu ra nào được tạo ra trên mỗi phản ứng.

Guard là một vị từ (một biểu thức có giá trị boolean) có giá trị là đúng (true) khi quá trình chuyển trạng thái sẽ được thực hiện, thay đổi trạng thái từ trạng thái đó đến một trạng thái khác. Khi *guard* là đúng, chúng ta nói rằng quá trình chuyển trạng thái đã

được kích hoạt. *Action* là một phép gán giá trị (hoặc không hiện diện) cho các cổng đầu ra. Bất kỳ cổng đầu ra nào không được đề cập trong quá trình chuyển trạng thái thì được hiểu là hoàn toàn không hiện diện. Nếu không có *Action* nào được đưa ra, thì tất cả các đầu ra hoàn toàn không hiện diện.

Nếu p_1 và p_2 là đầu vào thuần túy cho một hệ thống rời rạc thì sau đây là các ví dụ về các *guard* hợp lệ:

inputs: $up, down$: pure
output: $count$: $\{0, \dots, M\}$



Hình 8.11: Mô hình FSM cho ga ra ô tô của Hình 8.9

Các giá trị trong Hình 8.11 được làm rõ trong Bảng 8.1.

Bảng 8.1: Mô tả các thành phần của quá trình chuyển trạng thái

Giá trị đầu vào	Mô tả quá trình chuyển trạng thái
true	Chuyển trạng thái luôn được kích hoạt
p_1	Chuyển trạng thái được kích hoạt khi p_1 hiện diện
$\neg p_1$	Chuyển trạng thái được kích hoạt khi p_1 không hiện diện
$p_1 \wedge p_2$	Chuyển trạng thái được kích hoạt khi cả p_1 và p_2 cùng hiện diện
$p_1 \vee p_2$	Chuyển trạng thái được kích hoạt khi có một trong hai p_1 và p_2 hiện diện
$p_1 \wedge \neg p_2$	Chuyển trạng thái được kích hoạt khi có p_1 và p_2 không hiện diện

Các giá trị đầu vào là các toán tử luận lý tiêu chuẩn trong đó sự hiện diện của tín hiệu được coi là đồng nghĩa với giá trị đúng (true) và sự vắng mặt thì đồng nghĩa với sai (false). Kí hiệu $\neg$ thể hiện sự phủ định luận lý. Toán tử $\wedge$ là kết hợp luận lý (logic AND) và $\vee$ là toán tử “hoặc” luận lý (OR logic).

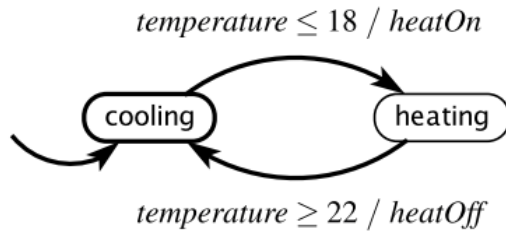
Giả sử rằng ngoài hệ thống rời rạc còn có một cổng đầu vào thứ ba p_3 với kiểu giá trị $V_{p_3} = N$. Khi đó, các giá trị trong bảng sau là các *guard* hợp lệ:

Bảng 8.2: Các giá trị guard hợp lệ

Giá trị đầu vào	Mô tả quá trình chuyển trạng thái
p_3	Chuyển trạng thái được kích hoạt khi p_3 hiện diện
$p_3 = 1$	Chuyển trạng thái được kích hoạt khi p_3 hiện diện và có giá trị là 1
$p_3 = 1 \wedge p_1$	Chuyển trạng thái được kích hoạt khi cả p_3 có giá trị 1 và p_1 không hiện diện
$p_3 > 5$	Chuyển trạng thái được kích hoạt khi p_3 có giá trị lớn hơn 5

Ví dụ 8.2: Việc sử dụng năng lượng trên toàn thế giới chủ yếu là trong các hệ thống sưởi, thông gió và điều hòa không khí (Heating, Ventilation, Air Conditioning - HVAC). Các mô hình chính xác về sự thay đổi nhiệt độ và hệ thống kiểm soát nhiệt độ có thể cải thiện đáng kể việc bảo tồn năng lượng. Mô hình như vậy bắt đầu với một bộ điều nhiệt vừa phải, điều chỉnh nhiệt độ để duy trì một điểm đặt hoặc nhiệt độ mong muốn. Cụm từ "máy điều nhiệt" bắt nguồn từ các từ Hy Lạp có nghĩa là "nóng" và "để đứng vững".

input: *temperature* : $\mathbb{R}$
outputs: *heatOn*, *heatOff* : pure



Hình 8.12: Mô hình máy điều nhiệt có độ trễ

Hãy xem xét một bộ điều nhiệt được mô hình hóa bởi FSM với $States = \{heating, cooling\}$ như trong Hình 8.12. Giả sử điểm đặt là $20^{\circ}C$. Nếu máy sưởi đang bật, thì bộ điều nhiệt sẽ cho phép nhiệt độ tăng qua điểm thiết lập cận trên là $22^{\circ}C$. Nếu máy sưởi tắt, thì nó cho phép nhiệt độ giảm xuống mức thiết lập cận dưới là $18^{\circ}C$. Chiến lược này được gọi là trễ. Nó tránh tiếng ồn ào mà lò sưởi sẽ bật và tắt nhanh chóng khi nhiệt độ gần với nhiệt độ điểm thiết lập.

Có một nhiệt độ đầu vào duy nhất với kiểu $\mathbb{R}$ và hai đầu ra thuần túy là *heatOn* và *heatOff*. Các đầu ra này sẽ chỉ hiện diện khi cần thay đổi trạng thái của lò sưởi (tức là khi lò sưởi đang bật và cần tắt, hoặc khi lò sưởi tắt và cần được bật).

FSM trong Hình 8.12 có thể được kích hoạt, giống như *Counter* trong ga ra. Trong trường hợp đó, hệ thống điều nhiệt sẽ phản ứng bất cứ khi nào đầu vào nhiệt độ được cung cấp. Ngoài ra, nó có thể được kích hoạt theo thời gian, nghĩa là nó phản ứng theo những khoảng thời gian đều đặn. Định nghĩa của FSM không thay đổi

trong hai trường hợp này. Nó phụ thuộc vào môi trường mà FSM có thể phản ứng khi nào nó cần.

Trong quá trình chuyển trạng thái, *Action* chỉ định giá trị kết quả trên các cổng đầu ra khi quá trình chuyển trạng thái được thực hiện. Nếu q_1 và q_2 là đầu ra thuần túy và q_3 có loại $\mathbb{N}$, thì sau đây là các ví dụ về *Action* hợp lệ:

Bảng 8.3: Các giá trị Action hợp lệ

Giá trị đầu ra	Mô tả
q_1	q_1 hiện diện và cả q_2 và q_3 không hiện diện
q_1, q_2	q_1 và q_2 hiện diện và q_3 không hiện diện
$q_3 := 1$	q_1 và q_2 không hiện diện và q_3 có giá trị bằng 1
$q_3 := 1, q_1$	q_1 hiện diện, q_2 không hiện diện và q_3 có giá trị bằng 1
Không đặt tả	q_1, q_2 và q_3 không hiện diện

Bất kỳ cổng đầu ra nào không được đề cập trong quá trình chuyển trạng thái được thực hiện là hoàn toàn vắng mặt. Khi gán giá trị cho một cổng đầu ra, chúng ta sử dụng ký hiệu *tên* := *giá trị* để phân biệt với cách gán giá trị cho một vị từ *tên* = *giá trị*. Như trong Hình 8.11, nếu chỉ có một đầu ra việc gán không cần đề cập đến tên cổng.

8.2.3.2 *Quá trình chuyển trạng thái*

Trong định nghĩa của máy trạng thái không thể hiện rang buộc khi nào phản ứng xảy ra. Chính môi trường xác định thời điểm phản ứng xảy ra. Khi môi trường xác định một trạng thái mà máy sẽ phản

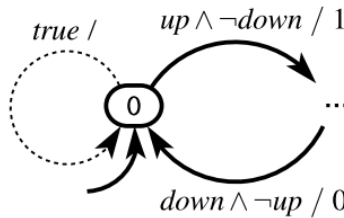
ứng, các đầu vào sẽ có một giá trị nhất định. Trạng thái của máy sẽ ấn định giá trị cho các cổng đầu ra và (có thể) chuyển đổi sang trạng thái mới. Nếu không có *guard* trên bất kỳ chuyển trạng thái nào ra khỏi trạng thái hiện tại được đánh giá là *true*, thì máy sẽ vẫn ở trạng thái cũ. (Trong trường hợp này, hệ thống không thay đổi trạng thái.)

Tất cả các yếu tố đầu vào có thể vắng mặt tại một phản ứng. Ngay cả trong trường hợp này, *guard* vẫn có thể được đánh giá là *true* và một quá trình chuyển trạng thái được thực hiện. Nếu đầu vào không hiện diện và không có *guard* trên bất kỳ quá trình chuyển trạng thái nào ra khỏi trạng thái hiện tại được đánh giá là *true*, thì máy sẽ bị chập chờn. Phản ứng chập chờn là phản ứng mà đầu vào và đầu ra đều không hiện diện và máy không thay đổi trạng thái. Không có tiến trình nào được thực hiện và không có gì thay đổi.

Ví dụ 8.3: Trong Hình 8.11, nếu trong bất kỳ phản ứng nào mà cả hai đầu vào đều không hiện diện thì máy sẽ bị chập chờn. Cụ thể, giả sử hệ thống đang ở trạng thái 0 và đầu vào *down* hiện diện thì chỉ *guard* trên chuyển trạng thái đi là có giá trị *false* và máy vẫn ở trạng thái cũ. Tuy nhiên, chúng ta không gọi đây là phản ứng chập chờn vì không phải tất cả các yếu tố đầu vào đều vắng mặt.

Mô tả không chính thức của chúng ta về ga ra ô tô trong Ví dụ 8.1 không nêu rõ điều gì sẽ xảy ra nếu *Counter* có giá trị 0 và một chiếc ô tô khởi hành. Một ưu điểm chủ yếu của các mô hình FSM là chúng xác định tất cả các khả năng xử lý có thể xảy ra trên hệ thống. Mô hình trong Hình 8.11 xác định những gì xảy ra trong

trường hợp này. *Counter* vẫn ở giá trị 0. Do đó, các mô hình FSM có thể phù hợp để đánh giá xác thực, xác định xem các xử lý được chỉ định có thực sự là các xử lý mong muốn hay không. Những đặt tả kỹ thuật không chính thức không thể phù hợp với các bài kiểm tra như vậy, hoặc ít nhất, là không hoàn toàn.



Hình 8.13: Một quá trình chuyển trạng thái mặc định

Mặc dù có vẻ như mô hình trong Hình 8.11 không xác định điều gì sẽ xảy ra nếu trạng thái là 0 và đầu vào *down* là có hiện diện, trạng thái không thay đổi và không có đầu ra nào được tạo ra. Phản ứng không được hiển thị rõ ràng trong sơ đồ. Đôi khi điều này rất hữu ích khi muốn nhấn mạnh những phản ứng cần được thể hiện một cách rõ ràng. Một cách thuận tiện để làm điều này là sử dụng một chuyển trạng thái mặc định (default state), được hiển thị trong Hình 8.13. Trong hình minh họa, chuyển trạng thái mặc định được biểu thị bằng các đường đứt nét và được gắn nhãn “true/”. Chuyển trạng thái mặc định xảy ra nếu không có chuyển trạng thái không mặc định nào được kích hoạt và nếu *guard* của nó được xác định là *true*. Do đó, trong Hình 8.13, quá trình chuyển trạng thái mặc định được kích hoạt nếu $up \wedge \neg down$ có giá trị là *false*, và khi chuyển trạng thái mặc định được thực hiện, đầu ra không có.

Các chuyển trạng thái mặc định cung cấp một ký hiệu thuận tiện, nhưng chúng không thực sự cần thiết. Bất kỳ quá trình chuyển trạng thái mặc định nào cũng có thể được thay thế bằng một quá trình chuyển trạng thái thông thường với một *guard* được chọn lựa thích hợp. Ví dụ, trong Hình 8.13, chúng ta có thể sử dụng một chuyển đổi thông thường với *guard* có dạng $\neg(up \wedge \neg down)$.

Việc sử dụng cả chuyển trạng thái thông thường và chuyển trạng thái mặc định trong sơ đồ có thể được coi là một cách để chỉ định mức độ ưu tiên cho các chuyển trạng thái. Quá trình chuyển trạng thái thông thường có mức độ ưu tiên cao hơn so với quá trình chuyển trạng thái mặc định. Khi cả hai đều có *guard* được đánh giá là *true*, quá trình chuyển trạng thái thông thường sẽ ưu tiên được thực hiện. Một số hình thức cho máy trạng thái hỗ trợ nhiều hơn hai mức độ ưu tiên.

8.2.3.3 Hàm cập nhật

Ký hiệu hình vẽ minh họa cho các FSM xác định một mô hình toán học cụ thể về sự thay đổi của một máy trạng thái. Một ký hiệu toán học có cùng ý nghĩa với ký hiệu hình vẽ đôi khi thuận tiện hơn, đặc biệt là đối với các máy trạng thái lớn sẽ làm cho sơ đồ hình vẽ các ký hiệu trở nên phức tạp. Khi sử dụng ký hiệu toán học, một FSM bao gồm một bộ năm yếu tố sau:

(States; Inputs; Outputs; update; initialState)

Trong đó:

- *States* là một tập hợp hữu hạn các trạng thái;
- *Inputs* là một tập hợp các giá trị đầu vào;
- *Outputs* là một tập hợp các giá trị đầu ra;
- *update*: $States \times Inputs \rightarrow States \times Outputs$ là một hàm cập nhật, ánh xạ một trạng thái và giá trị đầu vào đến trạng thái tiếp theo và giá trị đầu ra;
- *initialState* là trạng thái ban đầu.

FSM phản ứng theo một chuỗi các phản ứng. Tại mỗi phản ứng, FSM có một trạng thái hiện tại và phản ứng có thể chuyển sang trạng thái tiếp theo và đó sẽ là trạng thái hiện tại của phản ứng tiếp theo. Chúng ta có thể đánh số các trạng thái này bắt đầu bằng 0 cho trạng thái ban đầu. Cụ thể, giả sử $s: \mathbb{N} \rightarrow States$ là một hàm cho các trạng thái của một FSM tại phản ứng $n \in \mathbb{N}$. Ban đầu, $s(0) = initialState$.

Cho $x: \mathbb{N} \rightarrow Inputs$ và $y: \mathbb{N} \rightarrow Outputs$ biểu thị giá trị đầu vào và đầu ra ở mỗi phản ứng. Do đó, $x(0) \in Inputs$ là giá trị đầu vào đầu tiên và $y(0) \in Outputs$ là giá trị đầu ra đầu tiên. Sự thay đổi của máy trạng thái được cho bởi phương trình sau:

$$(s(n+1), y(n)) = update(s(n), x(n))$$

Phương trình này cung cấp trạng thái và đầu ra tiếp theo của trạng thái hiện tại và đầu vào. Hàm cập nhật mã hóa tất cả các thông số kỹ thuật chuyển trạng thái, *guard* và đầu ra trong FSM. Thuật ngữ hàm chuyển trạng thái thường được sử dụng thay cho hàm cập

nhật. Giá trị đầu vào và đầu ra cũng có dạng toán học. Giả sử một FSM có các cổng đầu vào $P = \{p_1; p_2; \dots; p_N\}$, trong đó mỗi $p \in P$ có một kiểu V_p tương ứng. Khi đó, *Inputs* là một tập hợp các hàm có công thức như sau:

$$i: P \rightarrow V_{p_1} \cup \dots \cup V_{p_N} \cup \{absent\}$$

trong đó với mỗi $p \in P$, $i(p) \in V_p \cup \{absent\}$ cho giá trị của cổng đầu vào p . Do đó, một hàm $i \in Inputs$ là giá trị của các cổng đầu vào.

Ví dụ 8.4: FSM trong Hình 8.11 có thể được biểu diễn toán học như sau:

$$\begin{aligned} States &= \{0, 1, \dots, M\} \\ Inputs &= (\{up, down\} \rightarrow \{present, absent\}) \\ Outputs &= (\{count\} \rightarrow \{0, 1, \dots, M, absent\}) \\ initialState &= 0 \end{aligned}$$

Hàm cập nhật được cho bởi công thức sau:

$$update(s, i) = \begin{cases} (s + 1, s + 1) & \text{if } C_1 \\ (s - 1, s - 1) & \text{if } C_2 \\ (s, absent) & \text{otherwise} \end{cases} \quad (8.13)$$

Trong đó,

$$C_1 = s < M \wedge i(up) = present \wedge i(down) = absent$$

$$C_2 = s > 0 \wedge i(up) = absent \wedge i(down) = present$$

Cho mọi $s \in States$ và $i \in Inputs$. Lưu ý rằng giá trị đầu ra $o \in Outputs$ là một hàm có dạng $o: \{count\} \rightarrow$

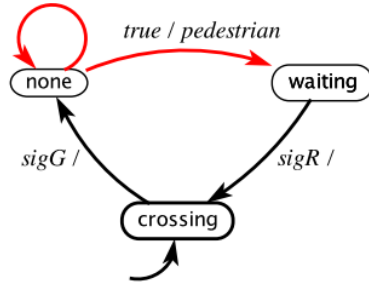
$\{0, 1, \dots, M, absent\}$. Trong công thức (8.13, trường hợp đầu tiên đưa ra giá trị đầu ra là $o = s + 1$, mà chúng ta hiểu rằng hàm hằng số với mọi $q \in Q = \{count\}$ tạo ra $o(q) = s + 1$. Khi có nhiều hơn một cổng đầu ra, chúng ta sẽ cần phải rõ ràng hơn về giá trị đầu ra nào được gán cho cổng đầu ra nào. Trong những trường hợp như vậy, chúng ta có thể sử dụng cùng một ký hiệu mà chúng ta sử dụng cho các *action* trong sơ đồ.

8.2.4 Thuyết không xác định

Hầu hết các máy trạng thái phản ứng với đầu vào và tạo ra đầu ra. Những đầu vào này phải đến từ một nơi nào đó, và đầu ra phải đến một nơi nào đó được gọi chung là môi trường của máy trạng thái.

Câu hỏi đặt ra là môi trường được mô hình hóa như thế nào. Trong ví dụ về đèn giao thông, chúng ta có thể xây dựng mô hình dòng người đi bộ trong thành phố để phục vụ mục đích này. Tuy nhiên, đây có thể sẽ là một mô hình rất phức tạp và có thể sẽ chi tiết hơn nhiều so với mức cần thiết. Chúng ta muốn bỏ qua những chi tiết không cần thiết và tập trung vào thiết kế của đèn giao thông. Chúng ta có thể làm điều này bằng cách sử dụng máy trạng thái không xác định (nondeterministic state machine).

inputs: *sigR*, *sigG*, *sigY* : pure
outputs: *pedestrian* : pure
true /



Hình 8.14: Mô hình máy trạng thái không xác định cho bộ điều khiển đèn cho người đi bộ

Ví dụ 8.5: Mô hình FSM trong Hình 8.14 mô hình hóa sự xuất hiện của người đi bộ tại vị trí dành cho người đi bộ qua đường có bộ điều khiển đèn giao thông. FSM này có ba đầu vào. Đầu ra duy nhất của nó là người đi bộ.

Trạng thái ban đầu là băng qua đường. Khi nhận được tín hiệu *sigG*, FSM chuyển sang trạng thái *none*. Cả hai cạnh chuyển trạng thái từ trạng thái này đều có *guard* là *true*, cho biết rằng chúng luôn được kích hoạt. Vì cả hai đều được kích hoạt nên máy trạng thái này không có tính xác định. FSM có thể giữ nguyên trạng thái và không tạo ra đầu ra, hoặc nó có thể chuyển sang trạng thái chờ và tạo ra đầu ra thuần túy là có người đi bộ.

Nếu đối với bất kỳ trạng thái nào của máy trạng thái, có hai quá trình chuyển trạng thái riêng biệt với các *guard* có giá trị là *true* trong cùng một phản ứng, thì máy trạng thái là không xác định. Trong sơ đồ cho một máy trạng thái như vậy, các chuyển trạng thái

làm cho máy trạng thái không xác định có thể có màu đỏ. Trong ví dụ của Hình 8.14, các chuyển trạng thái thoát khỏi trạng thái 0 là những chuyển trạng thái làm cho máy trạng thái không xác định.

Cũng có thể định nghĩa các máy trạng thái mà có nhiều hơn một trạng thái ban đầu. Một máy trạng thái như vậy cũng không có tính xác định. Trong cả hai trường hợp, FSM không xác định chỉ định một tập các phản ứng có thể xảy ra hơn là một phản ứng đơn lẻ. Về mặt vận hành, mọi phản ứng trong tập đều có thể xảy ra. FSM không xác định không đưa ra lệnh nào về khả năng xảy ra các phản ứng khác nhau. Ví dụ, việc luôn sử dụng vòng lặp chuyển trạng thái trở về trạng thái none trong Hình 8.14 là hoàn toàn đúng. Một mô hình xác định khả năng xảy ra (dưới dạng xác suất) là một mô hình ngẫu nhiên, khá khác biệt với một mô hình không xác định.

8.2.4.1 Mô hình hình thức

Về mặt hình thức, FSM không xác định được biểu diễn dưới dạng bộ năm yếu tố, tương tự như FSM xác định,

(States; Inputs; Outputs; possibleUpdates; initialStates)

Ba phần tử đầu tiên giống với FSM xác định, nhưng hai phần tử cuối cùng không giống:

- *States* là một tập hợp hữu hạn các trạng thái;
- *Inputs* là một tập hợp các giá trị đầu vào;
- *Outputs* là một tập hợp các giá trị đầu ra;

- *possibleUpdates*: $States \times Inputs \rightarrow 2^{States*Outputs}$ gọi là quan hệ cập nhật (*update relation*). *Update relation* ánh xạ trạng thái giá trị đầu vào và một tập hợp các cặp có thể có (trạng thái tiếp theo, giá trị đầu ra);
- *initialStates* là một tập hợp các trạng thái ban đầu.

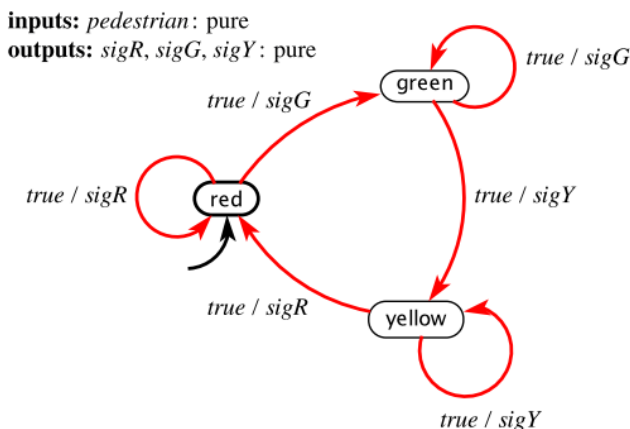
Hình thức của hàm *possibleUpdates* cho biết có thể có nhiều hơn một trạng thái và/hoặc giá trị đầu ra tiếp theo với trạng thái hiện tại và giá trị đầu vào. Miền giá trị của hàm là $2^{States*Outputs}$. Chúng ta đề cập đến chức năng *possibleUpdates* như một quan hệ cập nhật, để nhấn mạnh sự khác biệt này. Thuật ngữ quan hệ chuyển trạng thái (*transition relation*) cũng thường được sử dụng thay cho quan hệ cập nhật.

Để củng cố thực tế rằng có thể có nhiều hơn một trạng thái ban đầu cho một FSM không xác định, *initialStates* là một tập hợp chứ không phải là một phần tử của các trạng thái.

8.2.4.2 *Sử dụng thuyết không xác định*

Bản thân thuyết không xác định là một khái niệm toán học thú vị. Thuyết không xác định có hai cách sử dụng chính trong việc mô hình hóa các hệ thống nhúng:

Mô hình hóa môi trường: Thường hữu ích khi ẩn các chi tiết không liên quan về cách môi trường hoạt động, dẫn đến mô hình FSM không xác định. Chúng ta đã thấy một ví dụ về mô hình môi trường như vậy trong Hình 8.14.



Hình 8.15: FSM không xác định cho tín hiệu đèn giao thông

Thông số kỹ thuật: Thông số kỹ thuật hệ thống áp dụng các yêu cầu đối với một số tính năng của hệ thống, trong khi vẫn để các tính năng khác không bị giới hạn. Thuyết không xác định cũng là một kỹ thuật lập mô hình hữu ích trong các thiết kế như vậy. Ví dụ, hãy xem xét một thông số kỹ thuật mà đèn giao thông tuần tự chuyển sang đỏ, xanh, vàng, và không tính đến thời gian giữa các đầu ra. FSM không xác định trong Hình 8.15 mô hình hóa đặc điểm kỹ thuật này. Việc *guard* có giá trị *true* trên mỗi cạnh chuyển trạng thái chỉ ra rằng quá trình chuyển trạng thái có thể được thực hiện ở bất kỳ bước nào. Về mặt kỹ thuật, điều đó có nghĩa là mỗi quá trình chuyển trạng thái đều được kích hoạt cho bất kỳ giá trị đầu vào nào trong tập *Inputs*.

8.3 Hệ thống lai

Các hệ thống nhúng thường có sự kết hợp giữa đặc tính thay đổi liên tục và thay đổi rời rạc. Đối với hệ thống thay đổi liên tục, chúng

ta sử dụng các phương trình toán học và các mô hình tác nhân tương ứng của chúng. Đối với hệ thống thay đổi rời rạc, chúng tôi sử dụng máy trạng thái. Các hệ thống như thế gọi là **Hệ thống lai** (hybrid system). Khái niệm hệ thống lai là một công cụ hữu hiệu để hiểu các hệ thống trong thực tế.

8.3.1 Mô hình phương thức (modal model)

Trong phần này, chúng ta sẽ giới thiệu về các mô hình máy trạng thái có thể được tạo ra để chấp nhận các đầu vào và đầu ra liên tục và kết hợp các thay đổi liên tục và rời rạc.

8.3.1.1 Mô hình tác nhân cho máy trạng thái

Trong Phần 8.2.3.1, chúng ta đã tìm hiểu máy trạng thái có đầu vào được xác định bởi tập *Inputs* mà có thể là tín hiệu thuần túy hoặc có thể mang một giá trị. Trong cả hai trường hợp, máy trạng thái có một số cổng đầu vào, trong trường hợp tín hiệu thuần túy là *present* hoặc *absent*, và trong trường hợp các tín hiệu được gán một giá trị cụ thể tại mỗi phản ứng của máy trạng thái.

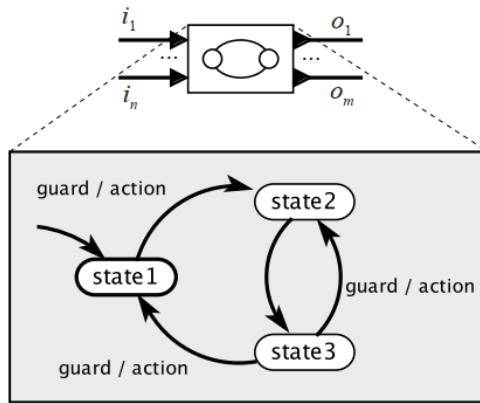
Chúng ta cũng tìm hiểu trong Phần 8.2.3.1 rằng các *action* của quá trình chuyển trạng thái thiết lập các giá trị của đầu ra. Các đầu ra cũng có thể được biểu diễn bằng các cổng, và một lần nữa các cổng có thể mang tín hiệu thuần túy hoặc tín hiệu được gán giá trị. Trong trường hợp tín hiệu thuần túy, quá trình chuyển trạng thái được thực hiện cho thấy đầu ra là *present* hay *absent* và trong trường hợp tín hiệu có giá trị, nó sẽ gán một giá trị hoặc khẳng định

rằng tín hiệu là *absent*. Đầu ra được cho là *absent* giữa các quá trình chuyển trạng thái.

Với cách xem xét đầu vào hoặc đầu ra này của các máy trạng thái, dễ dàng coi máy trạng thái như một tác nhân như minh họa trong Hình 8.16. Trong hình này, chúng ta giả sử một số n cổng đầu vào có tên là $i_1 \dots i_n$. Tại mỗi phản ứng, các cổng này có một giá trị *present* hoặc *absent* (nếu cổng mang tín hiệu thuần túy) hoặc là một phần tử của tập các giá trị (nếu cổng mang tín hiệu được gán giá trị). Các kết quả đầu ra cũng tương tự như thế. *Guard* của quá trình chuyển trạng thái xác định các tập con của các giá trị có thể có trên các cổng đầu vào và các *action* gán giá trị cho các cổng đầu ra. Với một mô hình tác nhân như vậy, thật đơn giản để tạo ra các FSM chấp nhận các tín hiệu thời gian liên tục làm đầu vào.

8.3.1.2 Đầu vào liên tục

Cho đến hiện tại, chúng ta xem xét các cỗ máy trạng thái hoạt động theo một chuỗi các phản ứng rời rạc. Chúng ta đã giả định rằng đầu vào và đầu ra đều vắng mặt giữa các phản ứng. Bây giờ, chúng ta sẽ tổng quát hóa điều này để cho phép đầu vào và đầu ra là các tín hiệu liên tục theo thời gian.

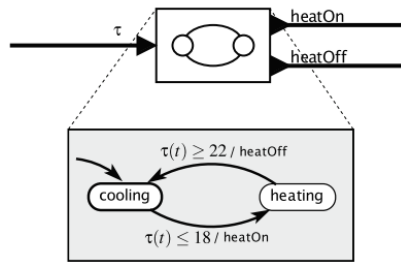


Hình 8.16: FSM được biểu thị như một tác nhân.

Để các mô hình máy trạng thái cùng tồn tại với các mô hình dựa trên thời gian, chúng ta cần diễn giải các chuyển trạng thái xảy ra trên cùng một dòng thời gian được sử dụng cho phần dựa trên thời gian của hệ thống. Khái niệm về phản ứng rời rạc được mô tả trong Phần 8.2.1 là đủ cho mục đích này. Tuy nhiên, để phù hợp với tín hiệu liên tục, chúng ta sẽ tìm hiểu thiết kế hệ thống để đầu vào và đầu ra có thể không hiện diện giữa các phản ứng nữa. Cụ thể, chúng ta sẽ định nghĩa một chuyển trạng thái xảy ra khi *guard* trên của quá trình chuyển trạng thái từ trạng thái hiện tại được kích hoạt. Như đã đề cập trong phần trước, trong khoảng thời gian giữa các phản ứng, một máy trạng thái được xem là không chuyển trạng thái giữa các chế độ. Nhưng đầu vào và đầu ra không còn bắt buộc phải có giá trị là *absent* trong thời gian đó.

Ví dụ 8.6: Xem xét một máy điều nhiệt được mô phỏng như một máy trạng thái với các trạng thái $\Sigma = \{heating, cooling\}$, được thể hiện trong Hình 8.17. Đây là một biến thể của mô hình của Ví dụ

8.5 trong đó thay vì một đầu vào rời rạc được cung cấp một nhiệt độ tại mỗi phản ứng, ở đây đầu vào là tín liên tục theo thời gian $\tau: \mathbb{R} \rightarrow \mathbb{R}$ trong đó $\tau(t)$ đại diện cho nhiệt độ tại thời điểm t . Trạng thái ban đầu là *cooling* và quá trình chuyển trạng thái ra khỏi trạng thái này được kích hoạt sớm nhất tại thời điểm t sau thời điểm bắt đầu là khi $\tau(t) \leq 18$. Ở ví dụ này, chúng ta giả sử đầu ra là các tín hiệu thuần túy *heatOn* và *heatOff*.



Hình 8.17: Bộ điều nhiệt được mô hình hóa như một FSM với tín hiệu đầu vào liên tục

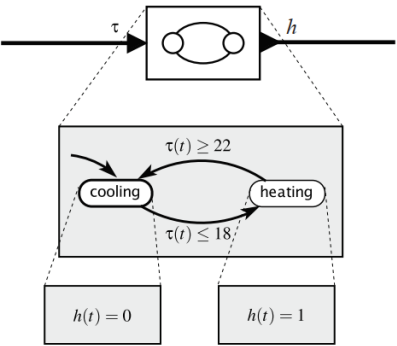
Trong ví dụ trên, các đầu ra chỉ hiện diện tại thời điểm các quá trình chuyển trạng thái được thực hiện. Chúng ta cũng có thể khái quát hóa các FSM để hỗ trợ các đầu ra thời gian liên tục theo thời gian. Tuy nhiên, để làm được điều này, chúng ta cần có khái niệm về các tình chỉnh trạng thái.

8.3.2 Tình chỉnh trạng thái (state refinement)

Một hệ thống lai kết hợp mỗi trạng thái của FSM với một hành vi động (hành vi thay đổi theo thời gian – dynamic behavior). Trong ví dụ đầu tiên dưới đây, chúng ta chỉ sử dụng khả năng này để tạo ra một đầu ra liên tục theo thời gian.

Ví dụ 8.7: Giả sử rằng thay vì các đầu ra rời rạc như trong Ví dụ 8.6, chúng ta muốn tạo ra một tín hiệu điều khiển có giá trị là 1 khi lò sưởi đang bật và 0 khi tắt. Một tín hiệu điều khiển như vậy có thể trực tiếp điều khiển lò sưởi. Bộ điều nhiệt trong Hình 8.18 thực hiện điều này. Trong hình đó, mỗi trạng thái có một sự tinh chỉnh cho giá trị của đầu ra h khi hệ thống đang ở trạng thái đó.

Trong một hệ thống lai, trạng thái hiện tại của máy trạng thái có một sự tinh chỉnh trạng thái (state refinement) dẫn đến hành vi thay đổi ở đầu ra (có thể xem) như một chức năng của đầu vào. Trong ví dụ đơn giản trên, đầu ra là không đổi ở mỗi trạng thái, đó là khái niệm khá phổ biến về sự thay đổi. Hệ thống lai có thể phức tạp hơn nhiều.

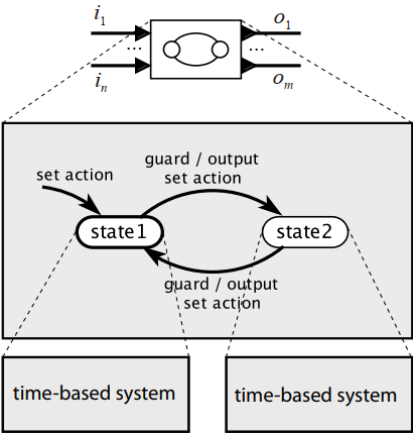


Hình 8.18: Một bộ điều nhiệt có đầu ra thời gian liên tục

Cấu trúc chung của mô hình hệ thống lai được thể hiện trên Hình 8.19. Trong đó, một máy trạng thái gồm có hai trạng thái được minh họa. Mỗi trạng thái được liên kết với một bộ tinh chỉnh trạng thái gọi là “time-based system” (TBS), một dạng hệ thống điều khiển

dựa trên thời gian. Việc tinh chỉnh trạng thái xác định các hành vi thay đổi của các đầu ra và (có thể) các biến trạng thái liên tục được thêm vào. Ngoài ra, mỗi quá trình chuyển trạng thái có thể tùy chọn chỉ định các tập hành động gán giá trị của các biến trạng thái thêm vào như vậy khi quá trình chuyển trạng thái được thực hiện. Ví dụ của Hình 8.18 khá đơn giản ở chỗ nó không có biến trạng thái liên tục, không có hành động đầu ra và không có tập hợp các hành động.

Một hệ thống lai đôi khi được gọi là mô hình phương thức (modal model) vì nó có một số chế độ hữu hạn, một chế độ cho mỗi trạng thái của FSM và khi ở trong một chế độ đó, sự thay đổi của nó được xác định bởi quá trình tinh chỉnh trạng thái. Các trạng thái của FSM có thể được gọi là các chế độ hơn là các trạng thái, như chúng ta sẽ thấy, giúp tránh nhầm lẫn với các trạng thái của các quá trình tinh chỉnh.



Hình 8.19: Kí hiệu cho hệ thống lai

Trong phần tiếp theo, chúng ta sẽ cùng thảo luận về các mô hình máy tự động thay đổi theo thời gian.

8.4 Tổ hợp máy trạng thái

Máy trạng thái là một phương thức thuận tiện để mô hình hóa các hành vi của hệ thống. Một nhược điểm mà chúng có là đối với hầu hết các hệ thống được xem xét, số lượng trạng thái rất lớn, thậm chí là vô hạn. Các công cụ tự động có thể xử lý không gian trạng thái lớn, tuy nhiên con người gặp nhiều khó khăn với các hệ thống có không gian trạng thái lớn.

Một nguyên tắc lâu đời trong kỹ thuật là các hệ thống phức tạp phải được mô tả thành các thành phần của các hệ thống đơn giản hơn. Trong phần này, chúng ta tìm hiểu một số cách để thực hiện việc này với các máy trạng thái. Tuy nhiên, người đọc cần lưu ý rằng có nhiều cách thiết lập máy trạng thái. Các thành phần trông giống nhau trên bề mặt có thể có nghĩa là những điều khác nhau với những người khác nhau. Các quy tắc ký hiệu của một mô hình được gọi là cú pháp (syntax) của nó, và ý nghĩa của ký hiệu được gọi là ngữ nghĩa (semantics).

Kỹ thuật tổ hợp đầu tiên mà chúng ta xem xét là tổ hợp đồng thời (concurrent composition). Hai hoặc nhiều máy trạng thái phản ứng đồng thời hoặc độc lập. Nếu các phản ứng xảy ra đồng thời, chúng tôi gọi nó là **tổ hợp đồng bộ** (synchronous composition).

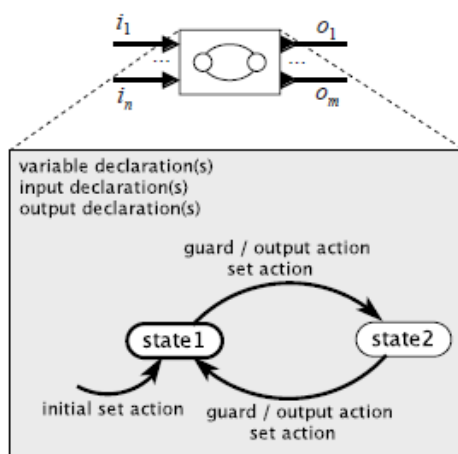
Nếu chúng phản ứng độc lập thì chúng ta gọi đó là **tổ hợp bất đồng bộ** (asynchronous composition).

Kỹ thuật tổ hợp thứ hai mà chúng ta sẽ xem xét là tổ hợp phân cấp (hierarchical composition). Máy trạng thái phân cấp cũng có thể cho phép các hệ thống phức tạp được mô tả thành của các hệ thống đơn giản hơn.

8.4.1 Tổ hợp đồng thời

Để nghiên cứu tổ hợp đồng thời của máy trạng thái, chúng ta sẽ xem xét qua một chuỗi các mô hình mẫu của tổ hợp. Các mô hình mẫu này có thể được kết hợp để xây dựng các hệ thống phức tạp tùy ý.

Chúng ta bắt đầu với trường hợp đơn giản nhất, tổ hợp song song (side-by-side composition), trong đó các máy trạng thái không có giao tiếp với nhau. Sau đó, chúng ta sẽ tìm hiểu về việc cho phép giao tiếp thông qua các biến chia sẻ. Chúng ta sẽ thấy giao tiếp sử dụng biến chia sẻ khiến các mô hình trở nên phức tạp như thế nào. Sau đó, chúng tôi xem xét giao tiếp thông qua các cổng với tổ hợp nối tiếp và sau đó mở rộng sang các kết nối tùy ý. Chúng ta xem xét cả dạng đồng bộ và bất đồng bộ cho mỗi loại tổ hợp.



Hình 8.20: Tóm tắt ký hiệu các máy trạng thái được sử dụng trong chương này

Hình 8.20 mô tả tóm tắt các ký hiệu được sử dụng trong các mô hình máy trạng thái được sử dụng trong chương này. Trong đó, *Variable declaration(s)* là các khai báo biến, *Input declaration(s)* là các khai báo đầu vào; *Output declaration(s)* là các khai báo đầu ra; *Guard/output action, set action* là các chỉ dẫn chuyển trạng thái, hành động ở ngõ ra và hành động thiết lập hệ thống của quá trình chuyển trạng thái; *Initial action* là hành động khởi tạo; *state1* và *state2* là các trạng thái của hệ thống.

8.4.1.1 Khái niệm đồng bộ

Thuật ngữ đồng bộ (synchrony) có nghĩa là (1) xảy ra hoặc tồn tại cùng một lúc hoặc (2) vận hành cùng một tốc độ. Trong kỹ thuật và khoa học máy tính, thuật ngữ này có nhiều ý nghĩa diễn đạt khác nhau nhưng vẫn thống nhất với định nghĩa trên.

Khi đề cập đến các phần mềm đồng thời được xây dựng bằng cách sử dụng các luồng (thread) hoặc tiến trình (process), giao tiếp đồng bộ đề cập đến một dạng giao tiếp điểm hẹn (rendezvous communication), trong đó người gửi thông điệp phải đợi người nhận sẵn sàng nhận và người nhận phải đợi người gửi. Về mặt khái niệm, các *thread* xem việc truyền thông xảy ra cùng một lúc, phù hợp với định nghĩa (1). Trong Java, từ khóa đồng bộ hóa định nghĩa các khối mã lệnh không được phép thực thi đồng thời. Thật kỳ lạ, hai khối mã được đồng bộ hóa không thể “xảy ra” (thực thi) cùng một lúc, điều này không phù hợp với cả hai định nghĩa.

Trong thế giới phần mềm, vẫn có nghĩa thứ ba của từ đồng bộ. Nghĩa thứ ba này được sử dụng trong giáo trình. Ý nghĩa thứ ba này làm nền tảng cho các ngôn ngữ đồng bộ. Hai ý tưởng chính chi phối các ngôn ngữ này. Đầu tiên, đầu ra của các thành phần trong một chương trình (về mặt khái niệm) đồng thời với đầu vào của chúng (điều này được gọi là giả thuyết đồng bộ). Thứ hai, các thành phần trong một chương trình thực thi (về mặt khái niệm) đồng thời và tức thời. Các tiến trình thực thi thực sự không xảy ra đồng thời cũng như ngay lập tức, và các đầu ra không thực sự đồng thời với các đầu vào, nhưng việc thực thi đúng phải hoạt động như thể chúng vốn có. Việc sử dụng từ đồng bộ này phù hợp với cả hai định nghĩa trên; việc thực thi của các thành phần diễn ra cùng lúc và hoạt động cùng tốc độ như nhau.

Trong thiết kế mạch, từ đồng bộ đề cập đến một phong cách thiết kế trong đó đồng hồ được phân phối khắp các mạch chốt (latch) ghi lại đầu vào của chúng trên các cạnh của xung clock. Thời gian giữa các cạnh trên xung clock cần đủ để mạch giữa các chốt ổn định. Về mặt khái niệm, mô hình này tương tự như mô hình trong các ngôn ngữ đồng bộ. Giả sử rằng các mạch giữa các chốt có độ trễ bằng 0 tương đương với giả thuyết đồng bộ và phân phối xung clock toàn cục cho phép thực thi đồng thời và tức thời.

Trong kỹ thuật hệ thống điện, đồng bộ có nghĩa là các dạng sóng điện có cùng tần số và cùng pha. Trong xử lý tín hiệu,

đồng bộ có nghĩa là các tín hiệu có cùng tốc độ lấy mẫu hoặc tốc độ mẫu của chúng là bội số cố định của nhau. Thuật ngữ luồng dữ liệu đồng bộ dựa trên nghĩa thứ hai của từ đồng bộ.

Ví dụ 8.8: Xem xét hai FSM A và B trong Hình 8.22. A có một đầu ra duy nhất *a*, và B có một đầu ra duy nhất *b*. Tổ hợp C có hai đầu ra, *a* và *b*. Nếu tổ hợp là đồng bộ thì ở phản ứng đầu tiên *a* sẽ hiện diện và *b* sẽ không hiện diện. Ở phản ứng thứ hai, nó sẽ là ngược lại. Trong các phản ứng tiếp theo, *a* và *b* sẽ tiếp tục hiện diện xen kẽ.

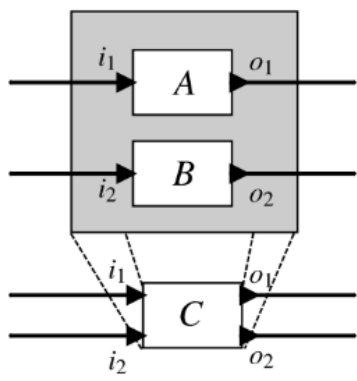
8.4.1.2 Tổ hợp đồng bộ song song

Mô hình mẫu tổ hợp đầu tiên mà chúng ta xem xét là bố cục song song (side-by-side) được minh họa cho hai tác nhân trong Hình 8.21. Trong mô hình này, chúng ta giả định rằng các đầu vào và đầu ra của hai tác nhân rời rạc, tức là các máy trạng thái không

giao tiếp. Cụ thể, tác nhân A có đầu vào i_1 và đầu ra o_1 , và tác nhân B có đầu vào i_2 và đầu ra o_2 . Tổ hợp của hai tác nhân chính là tác nhân C với đầu vào i_1 và i_2 và đầu ra o_1 và o_2 .

Trong trường hợp đơn giản nhất, nếu hai tác nhân là máy trạng thái mở rộng với các biến thì các biến đó cũng rời rạc. Trong mục 8.4.1.4 chúng ta sẽ tìm hiểu điều gì sẽ xảy ra khi hai máy trạng thái chia sẻ các biến. Ở dạng tổ hợp đồng bộ, phản ứng của C là phản ứng đồng thời của A và B.

Tác nhân tổ hợp C có thể đổi tên các cổng đầu vào và đầu ra này, nhưng ở đây chúng ta giả định rằng nó sử dụng cùng tên các tác nhân thành phần.

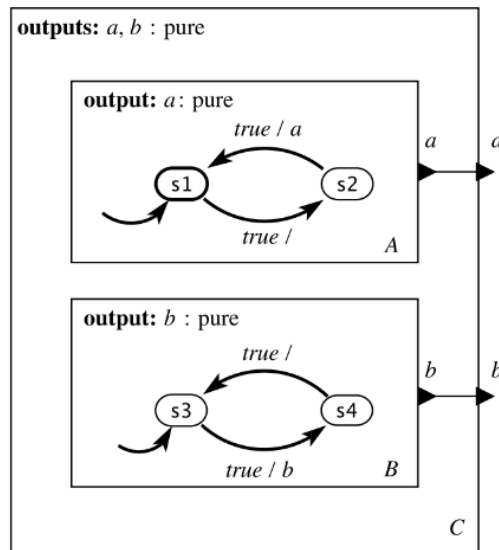


Hình 8.21: Tổ hợp song song của hai tác nhân.

Tổ hợp đồng bộ song song rất đơn giản vì một số lý do. Đầu tiên, hãy nhớ lại từ Phần 8.2.3.2 rằng môi trường xác định thời điểm phản ứng của máy trạng thái. Trong tổ hợp đồng bộ song song, môi trường không cần biết rằng C là tổ hợp của hai máy trạng thái. Các

tổ hợp như vậy có tính mô-đun theo nghĩa là bản thân thành phần trở thành một thành phần có thể được tạo ra thêm như thể chính nó là một thành phần nguyên tử.

Hơn nữa, nếu hai máy trạng thái A và B là xác định, thì tổ hợp đồng bộ song song cũng có tính chất xác định. Chúng ta nói rằng một thuộc tính là tổ hợp nếu thuộc tính do các tổ hợp tạo ra cũng là một thuộc tính của tổ hợp đó. Đối với tổ hợp đồng bộ song song, thuyết xác định là một thuộc tính tổng hợp.



Hình 8.22: Sơ đồ máy trạng thái của tổ hợp hai tác nhân

Ngoài ra, tổ hợp đồng bộ song song của các FSM tự nó là một FSM. Một cách chặt chẽ để đưa ra ngữ nghĩa của tổ hợp là xác định một máy trạng thái duy nhất cho tổ hợp. Giả sử rằng, như trong Mục 8.2.3.3, máy trạng thái A và B được cho bởi bộ năm giá trị:

$$A = (States_A; Inputs_A; Outputs_A; update_A; initialState_A)$$

$$B = (States_B; Inputs_B; Outputs_B; update_B; initialState_B)$$

Khi đó, tổ hợp đồng bộ song song C được tạo bởi:

$$States_C = States_A \times States_B \quad (8.14)$$

$$Inputs_C = Inputs_A \times Inputs_B \quad (8.15)$$

$$Outputs_C = Outputs_A \times Outputs_B \quad (8.16)$$

$$initialState_C = (initialState_A; initialState_B) \quad (8.17)$$

Và hàm cập nhật được xác định bởi:

$$Update_C((s_A; s_B); (i_A; i_B)) = ((s'_A; s'_B); (o_A; o_B))$$

Tại $(s'_A; o_A) = update_A(s_A; i_A)$ và $(s'_B; o_B) = update_B(s_B; i_B)$, $\forall s_A \in States_A, s_B \in States_B, i_A \in Inputs_A, \text{ và } i_B \in Inputs_B$

$Inputs_A$ và $Inputs_B$ là tập hợp các giá trị. Mỗi định giá trong tập hợp là một giá trị gán cho các cổng. Nghĩa là:

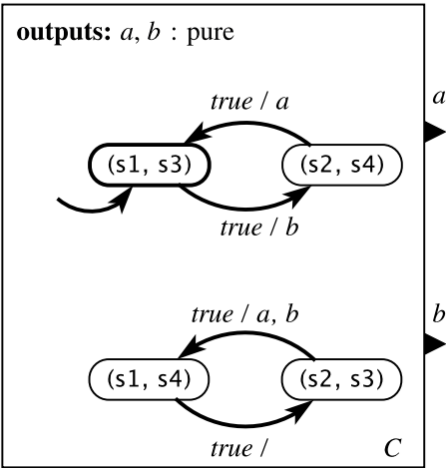
$$Inputs_C = Inputs_A \times Inputs_B$$

Nói cách khác, giá trị đầu vào của C phải bao gồm giá trị đầu vào của A và giá trị đầu vào của B.

Như thường lệ, FSM C đơn có thể

Ví dụ 8.9: Tổ hợp đồng bộ song song C trong Hình 5.3 là được đưa ra dưới dạng một FSM đơn trong Hình 8.23. Lưu ý rằng máy trạng thái này hoạt động chính xác như được mô tả trong Ví dụ 8.7. Các đầu ra a và b hiện diện xen kẽ. Lưu ý hơn nữa là $(s_1; s_4)$ và $(s_2; s_3)$ không phải là trạng thái có thể truy cập được.

được cho bằng hình ảnh thay vì biểu tượng như minh họa trong ví dụ tiếp theo.



Hình 8.23: Máy trạng thái đơn đưa ra ngữ nghĩa của thành phần đồng bộ song song

8.4.1.3 Tổ hợp không đồng bộ song song

Trong tổ hợp không đồng bộ của máy trạng thái, các máy thành phần phản ứng độc lập. Khẳng định này khá mơ hồ, và trên thực tế, nó có nhiều cách hiểu khác nhau. Mỗi cách diễn giải đưa ra một ngữ nghĩa cho tổ hợp. Chìa khóa cho từng ngữ nghĩa là cách xác định phản ứng của tổ hợp C trong Hình 8.24. Có hai khả năng:

- *Ngữ nghĩa 1:* Phản ứng của C là phản ứng của một trong A hoặc B, trong đó sự lựa chọn là không xác định.
- *Ngữ nghĩa 2:* Phản ứng của C là phản ứng của A, B hoặc cả A và B, trong đó sự lựa chọn là không xác định. Một biến có thể không cho phép phản ứng.

Dấu hiệu ² hay không ² đều có thể theo ² hoặc ² 1, định nghĩa ² là:

$$update_C((s_A; s_B); (i_A; i_B)) = ((s'_A; s'_B); (o'_A; o'_B))$$

Trong đó

$$(s'_A; o'_A) = update_A(s_A; i_A)$$

$$s'_B = s_B$$

$$o'_B = absent$$

hoặc

$$(s'_B; o'_B) = update_B(s_B; i_B)$$

$$A = s_A$$

$$o'_A = absent$$

Với mọi $s_A \in States_A$, $s_B \in States_B$, $i_A \in Inputs_A$, và $i_B \in Inputs_B$. Nghĩa là là $O'_B = absent$ là tất cả *output* của B không có. Ngữ nghĩa 2 có thể được định nghĩa một cách tương tự (xem thêm Bài tập 2 của chương này).

8.4.1.4 Biến chia sẻ

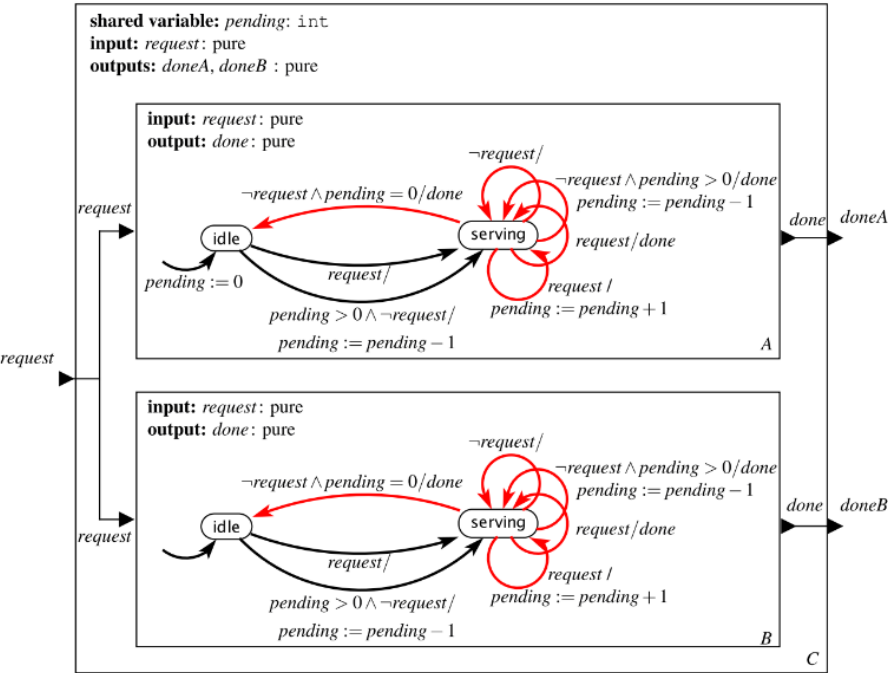
Máy trạng thái mở rộng có các biến cục bộ có thể được đọc và ghi như một phần của việc chuyển trạng thái. Đôi khi, nó hữu ích khi thiết lập máy trạng thái để cho phép các biến được chia sẻ giữa một nhóm máy. Đặc biệt, các biến được chia sẻ như vậy có thể hữu ích cho việc mô hình các ngắt (interrupt) và luồng xử lý (thread). Tuy nhiên, cần phải cẩn thận để đảm bảo rằng ngữ nghĩa của mô hình phù hợp với chương trình có chứa *interrupt* hoặc *thread*. Nhiều vấn đề phức tạp nảy sinh, bao gồm cả sự nhất quán của bộ nhớ (memory consistency) và khái niệm về các thao tác nguyên tử (atomic operation).

Ví dụ 8.10: Xem xét hai máy chủ có thể nhận yêu cầu từ một mạng. Mỗi yêu cầu cần một khoảng thời gian, không xác định để được phục vụ, vì vậy các máy chủ chia sẻ một loạt các yêu cầu. Nếu một máy chủ bận, máy chủ khác có thể phản hồi một yêu cầu, ngay cả khi yêu cầu đến máy chủ đầu tiên.

Quy trình này phù hợp với một mô hình tương tự như trong Hình 5.2, trong đó A và B là máy chủ. Chúng ta có thể mô hình hóa các máy chủ dưới dạng máy trạng thái như trong Hình 8.25. Trong mô hình này, một biến được chia sẻ đang chờ xử lý sẽ đếm số lượng công việc yêu cầu. Khi một yêu cầu đến máy tổ hợp C, một trong hai máy chủ là không xác định được chọn để phản ứng, giả sử tổ hợp là không đồng bộ trong ngữ nghĩa 1. Nếu máy chủ đó đang ở chế độ nghỉ (idle) thì nó sẽ thực thi yêu cầu. Nếu máy chủ đang thực thi một yêu cầu khác thì một trong hai điều có thể xảy ra: nó có thể ngẫu nhiên kết thúc việc thực thi yêu cầu hiện tại mà nó hiện đang làm, đưa ra kết quả và tiếp tục thực thi yêu cầu mới, hoặc nó có thể tăng số lượng tác vụ chờ xử lý và tiếp tục thực thi yêu cầu hiện tại. Sự lựa chọn giữa chúng là không xác định, để mô hình hóa thực tế rằng thời gian cần thiết để thực thi một yêu cầu là không xác định.

Nếu C phản ứng khi không có yêu cầu thì một lần nữa máy chủ A hoặc B sẽ được chọn một cách ngẫu nhiên để phản ứng. Nếu máy chủ phản ứng đang ở chế độ nghỉ và có một hoặc nhiều yêu cầu

đang chờ xử lý, sau đó máy chủ chuyển trạng thái sang phục vụ yêu cầu và giảm biến ghi nhận số lượng yêu cầu đang chờ xử lý. Nếu máy chủ phản ứng đang bận thì một trong ba điều có thể xảy ra. Nó có thể tiếp tục phục vụ yêu cầu hiện tại, trong trường hợp đó nó chỉ đơn giản là chuyển trạng thái của nó về trạng thái hiện tại là đang phục vụ yêu cầu. Hoặc nó có thể kết thúc việc thực thi yêu cầu hiện tại, trong trường hợp đó nó sẽ chuyển sang trạng thái nghỉ nếu không có yêu cầu nào đang chờ xử lý hoặc chuyển đổi trở lại trạng thái phục vụ và giảm dần nếu có yêu cầu đang chờ xử lý.



Hình 8.25: Mô hình của hai máy chủ với hàng đợi tác vụ được chia sẻ

Mô hình trong ví dụ 8.10 cho thấy nhiều điểm cần quan tâm của các hệ thống đồng thời. Đầu tiên, vì ngữ nghĩa đan xen, quyền truy cập vào biến chia sẻ là các thao tác nguyên tử, một điều khá khó khăn để đảm bảo trong thực tế. Thứ hai, lựa chọn ngữ nghĩa 1 là hợp lý trong trường hợp này vì đầu vào đi đến cả hai máy thành phần vì vậy bất kể máy thành phần nào phản ứng, sẽ không có sự kiện đầu vào nào bị bỏ lỡ. Tuy nhiên, ngữ nghĩa này sẽ không hoạt động nếu hai máy có đầu vào độc lập, vì khi đó các yêu cầu có thể bị bỏ sót. Ngữ nghĩa 2 có thể giúp ngăn chặn điều đó, nhưng môi trường nên sử dụng chiến lược nào để xác định máy nào phản ứng? Điều gì xảy ra nếu cả hai đầu vào độc lập đều có yêu cầu hiện tại cùng phản ứng của C? Nếu chúng ta chọn ngữ nghĩa 4 để cho phép cả hai máy phản ứng đồng thời, khi cả hai máy đều cập nhật biến được chia sẻ? Các bản cập nhật không còn là nguyên tử nữa vì chúng có sự đan xen ngữ nghĩa.

Lưu ý thêm rằng việc chọn tổ hợp không đồng bộ theo ngữ nghĩa 1 cho phép các hành vi không phù hợp sử dụng các máy ở chế độ nghỉ. Giả sử rằng máy A là đang phục vụ, máy B ở chế độ nghỉ và một yêu cầu đến. Nếu lựa chọn không xác định dẫn đến máy A phản ứng, sau đó nó sẽ chỉ đơn giản là tăng thêm việc đang chờ xử lý. Khi lựa chọn không xác định sẽ dẫn đến kết quả là phản ứng B đưa máy ở chế độ nghỉ vào sử dụng. Trên thực tế, ngữ nghĩa 1 cho phép các chế độ không bao giờ sử dụng một trong các máy móc.

Các biến chia sẻ cũng có thể được sử dụng trong các thành phần đồng bộ nhưng xuất hiện một số điểm phức tạp. Đặc biệt, điều gì sẽ xảy ra nếu trong cùng một phản ứng, một máy đọc một biến chia sẻ để xác định *guard* và một máy khác ghi vào biến chia sẻ? Chúng ta có yêu cầu viết trước khi đọc không? Điều gì sẽ xảy ra nếu quá trình chuyển đổi đang ghi đến biến chia sẻ cũng đọc cùng một biến xuất hiện trong *guard* của nó? Một khả năng là chọn **ngữ nghĩa xen kẽ đồng bộ**, nơi các máy thành phần phản ứng theo thứ tự tùy ý, được chọn không xác định. Chiến lược này có nhược điểm rằng một thành phần của hai máy xác định có thể là không xác định. Một phiên bản thay thế của ngữ nghĩa xen kẽ đồng bộ có các máy thành phần phản ứng theo một trật tự cố định được xác định bởi môi trường hoặc bởi một số cơ chế bổ sung, chẳng hạn như sự ưu tiên.

Những khó khăn của các biến được chia sẻ, đặc biệt với tổ hợp không đồng bộ, phản ánh sự phức tạp vốn có của các mô hình đồng thời với các biến dùng chung. Giải pháp tối ưu yêu cầu một ngữ nghĩa phức tạp hơn. Cho đến hiện tại, chúng ta đã xem xét thành phần của các máy không trực tiếp giao tiếp. Tiếp theo, chúng ta sẽ xem xét điều gì sẽ xảy ra khi đầu ra của một máy này là đầu vào của máy khác.

8.4.2 Tổ hợp xếp tầng

Xét hai máy trạng thái A và B được cấu tạo như Hình 8.26. Các đầu ra của máy A cung cấp đầu vào cho B. Kiểu tổ hợp này được gọi là tổ hợp xếp tầng (cadecase composition).

Trong hình, cổng đầu ra o_1 từ A cung cấp các sự kiện đến cổng đầu vào i_2 của B. Giả sử dữ liệu kiểu của o_1 là V_1 (có nghĩa là o_1 có thể nhận giá trị từ V_1 hoặc không có) và kiểu dữ liệu của i_2 là V_2 . Sau đó, một yêu cầu cho thành phần này hợp lệ là:

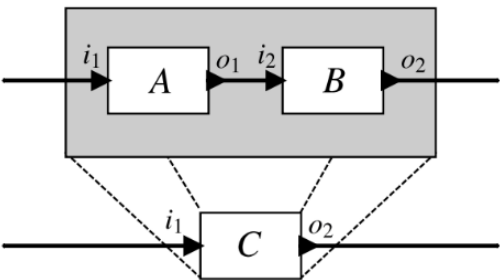
$$V_1 \subseteq V$$

Điều này khẳng định rằng bất kỳ đầu ra nào do A tạo ra trên cổng o_1 là đầu vào được chấp nhận cho B trên cổng i_2 .

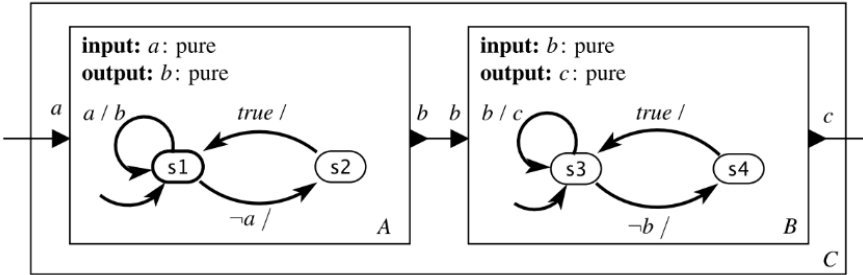
Đối với tổ hợp xếp tầng, nếu chúng ta muốn tổ hợp bất đồng bộ, thì chúng ta cần chỉ dẫn một số máy móc để đệm dữ liệu được gửi từ A đến B. Chúng ta sẽ thảo luận về tổ hợp bất đồng bộ đến Chương 6, trong đó luồng dữ liệu và quá trình các mô hình mạng tính toán sẽ đưa ra các tổ hợp bất đồng bộ. Trong chương này chúng ta sẽ chỉ xem xét tổ hợp đồng bộ cho hệ thống thác.

Trong tổ hợp đồng bộ của cấu trúc tầng trong Hình 8.26, một phản ứng của C bao gồm phản ứng của cả A và B, trong đó A phản ứng trước, tạo ra sản phẩm (nếu có), và sau đó B phản ứng. Về mặt luân lý, chúng ta xem điều này xảy ra trong thời gian là 0, vì vậy hai phản ứng có nghĩa **đồng thời và tức thời**. Nhưng chúng có quan

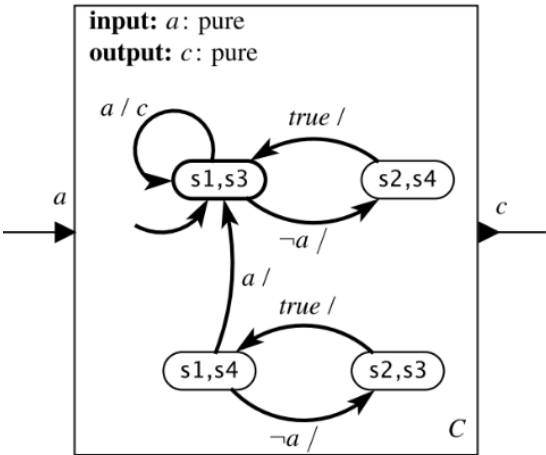
hệ nhân quả ở chỗ các kết quả đầu ra của A có thể ảnh hưởng đến hoạt động của B.



Hình 8.26: Tổ hợp xếp chồng hai tác nhân



Hình 8.27: Tổ hợp xếp chồng hai FSM



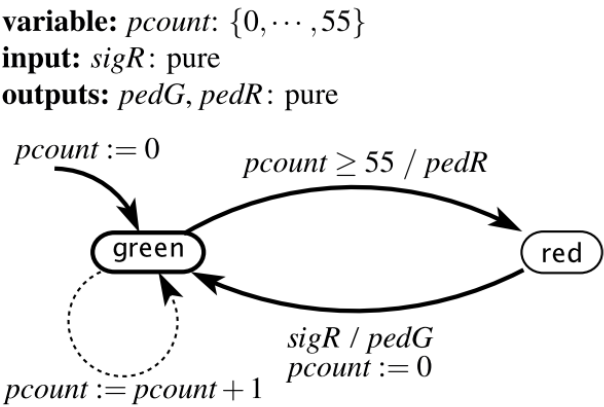
Hình 8.28: Ngữ nghĩa của tổ hợp xếp chồng hai FSM

Ví dụ 8.11: Xem xét thành phần tầng của hai FSM trong Hình 8.27. Giả sử ngữ nghĩa đồng bộ, ý nghĩa của phản ứng C được đưa ra trong Hình 8.28. Con số đó nói rõ rằng phản ứng của hai máy là đồng thời và tức thời. Khi chuyển từ trạng thái ban đầu (s_1, s_3) sang (s_2, s_4) (xảy ra khi không có đầu vào a), máy thành phần C không đi qua (s_2, s_3)! Trên thực tế, (s_2, s_3) không phải là một trạng thái có thể truy cập được! Trong cách này, một phản ứng đơn của C bao gồm một phản ứng của cả A và B.

Để xây dựng máy tổ hợp như trong Hình 8.28, trước tiên hãy tạo không gian trạng thái dưới dạng tích chéo của không gian trạng thái của các máy thành phần, rồi xác định quá trình chuyển đổi được thực hiện trong những điều kiện nào. Điều quan trọng cần nhớ là quá trình chuyển đổi diễn ra đồng thời, ngay cả khi quá trình này dẫn đến quá trình kia một cách hợp lý.

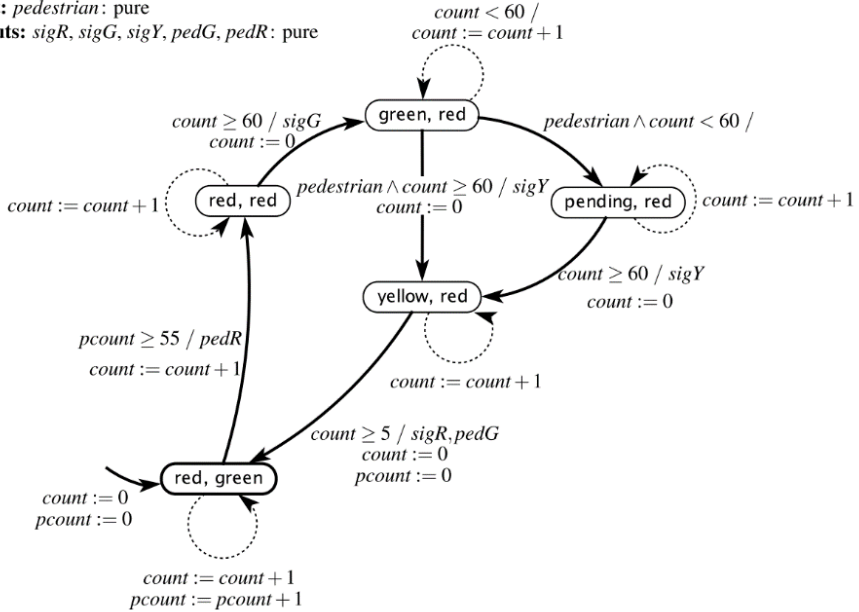
Ví dụ 8.12: Giả sử rằng chúng ta muốn kết hợp với mô hình đèn tín hiệu băng qua đường dành cho người đi bộ, như thể hiện trong Hình 8.29. Đầu ra sig_R của đèn giao thông có thể cung cấp đầu vào sig_R của đèn dành cho người đi bộ. Theo tổ hợp xếp tầng đồng bộ, ý nghĩa của hỗn hợp được cho trong Hình 8.30. Lưu ý rằng các trạng thái không an toàn, chẳng hạn như (xanh, xanh), là trạng thái khi cả ô tô và người đi bộ đều có đèn xanh, không phải trạng thái có thể truy cập và do đó không được hiển thị.

Ở dạng đơn giản nhất, tổ hợp xếp tầng ngụ ý thứ tự các phản ứng của các thành phần. Vì thứ tự này được xác định rõ ràng, chúng ta không gặp nhiều khó khăn với các biến được chia sẻ như chúng ta đã làm với tổ hợp song song. Tuy nhiên, chúng ta sẽ thấy rằng trong các thành phần chung, việc đưa ra tín hiệu điều khiển không đơn giản như vậy.



Hình 8.29: Mô hình đèn đi bộ trong tổ hợp xếp chồng đồng bộ

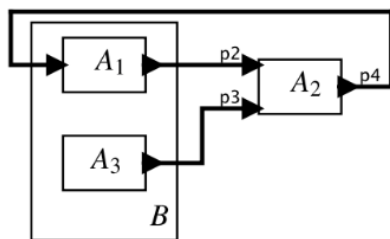
variables: $count: \{0, \dots, 60\}, pcount: \{0, \dots, 55\}$
input: $pedestrian: pure$
outputs: $sigR, sigG, sigY, pedG, pedR: pure$



Hình 8.30: Sơ đồ ngữ nghĩa của tổ hợp xếp tần đồng bộ của đèn đi bộ

8.4.3 Tổ hợp tổng quát

Tổ hợp song song và xếp tầng cung cấp các khối xây dựng cơ bản để xây dựng thêm cấu tạo phức tạp của máy móc. Hãy xem xét ví dụ về tổ hợp trong Hình 8.31. A_1 và A_3 là các tác nhân song song tạo nên tổ hợp máy B . B và A_2 là một tổ hợp xếp tầng, với các sự kiện B cấp cho A_2 . Tuy nhiên, B và A_2 cũng là tổ hợp xếp tầng theo thứ tự ngược lại, với các dữ liệu cấp A_2 cho B . Các chu kỳ như vậy được gọi là phản hồi (feedback) và chúng đưa ra một câu hỏi để suy ngẫm: máy nào sẽ phản ứng đầu tiên, B hoặc A_2 ? Câu hỏi này sẽ được giải quyết trong chương tiếp theo khi chúng ta xem xét mô hình tính toán phản ứng đồng bộ.



Hình 8.31: Sự kết hợp của tổ hợp song song và xếp tầng trong kết nối của máy trạng thái

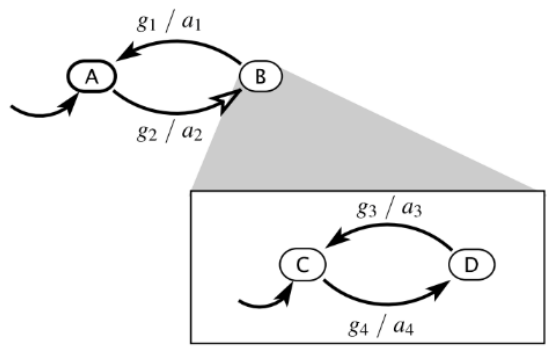
8.4.4 Máy trạng thái phân cấp

Trong phần này, chúng ta xem xét các FSM phân cấp (Hierarchical FSM - HFSM), có từ thời Statecharts [15]. Có nhiều biến thể của Statecharts, thường có sự khác biệt nhỏ về ngữ nghĩa giữa chúng [16]. Ở đây, chúng ta sẽ tập trung vào một số khía cạnh đơn giản hơn và chúng ta sẽ chọn một biến thể ngữ nghĩa cụ thể.

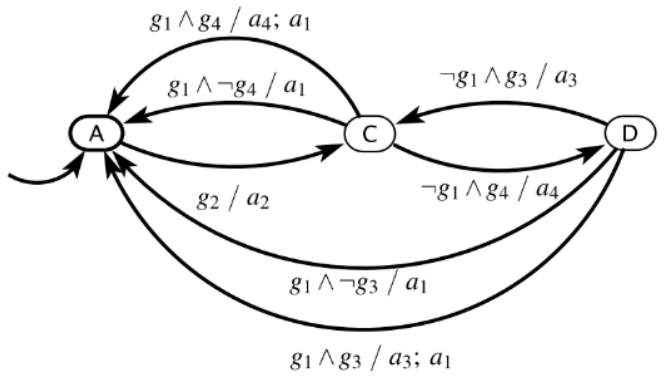
Ý tưởng chính trong các HFSM là sự tinh chỉnh trạng thái. Trong Hình 8.32, trạng thái B có tinh chỉnh là một FSM khác có hai trạng thái, C và D. Ý nghĩa của nó đối với máy trạng thái B là nó ở một trong các trạng thái C hoặc D.

Ý nghĩa của hệ thống phân cấp trong Hình 8.32 có thể được hiểu bằng cách so sánh nó với FSM tương đương trong Hình 8.33. Máy khởi đầu ở trạng thái A. Khi *guard* g_2 có giá trị là *true*, máy chuyển sang trạng thái B, cũng có nghĩa là chuyển sang trạng thái C, trạng thái ban đầu của quá trình tinh chỉnh. Khi thực hiện chuyển đổi này sang C, máy thực hiện hành động a_2 có thể tạo ra một sự kiện đầu ra hoặc thiết lập một biến nếu đây là máy trạng thái mở rộng.

Sau đó, có hai cách để thoát khỏi trạng thái C. Cách thứ nhất là *guard* g_1 có giá trị là *true* và máy thoát trạng thái B và quay trở lại trạng thái A. Cách thứ hai là *guard* g_4 có giá trị là *true* và máy chuyển tiếp thành trạng thái D. Một câu hỏi thú vị là điều gì sẽ xảy ra nếu cả hai *guard* g_1 và g_4 đều có giá trị là *true*.



Hình 8.32: Tinh chỉnh trạng thái của máy trạng thái phân cấp.

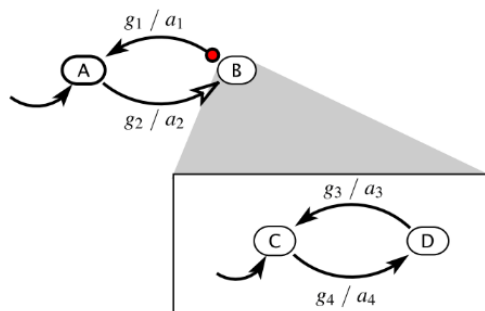


Hình 8.33: Sơ đồ ngữ nghĩa của HFSM

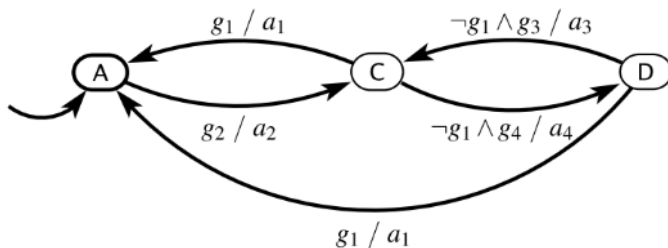
Các biến thể khác nhau của Statecharts có thể đưa ra các lựa chọn khác nhau tại thời điểm này. Rõ ràng là máy sẽ kết thúc ở trạng thái A, nhưng hành động nào sẽ được thực hiện, a_4 , a_1 , hay cả

hai? Những câu hỏi như vậy giúp giải thích sự gia tăng của các biến thể khác nhau của Statecharts.

Chúng ta chọn một ngữ nghĩa cụ thể có đặc tính mô đun hấp dẫn [10]. Trong ngữ nghĩa này, một phản ứng của HFSM được định nghĩa theo độ sâu phân cấp (depth-first fashion). Sự tinh chỉnh sâu nhất của trạng thái hiện tại sẽ phản ứng đầu tiên, sau đó là các tinh chỉnh cao hơn. Trong Hình 8.32, điều này có nghĩa là nếu máy ở trạng thái B (có nghĩa là nó ở C hoặc D), sau đó máy tinh chỉnh phản ứng trước. Nếu nó là C, và *guard* g_4 là *true*, quá trình chuyển trạng thái được thực hiện đến D và hành động a_4 được thực hiện. Sau đó, FSM cấp cao nhất sẽ phản ứng. Nếu *guard* g_1 cũng là *true* thì máy chuyển sang trạng thái A. Điều quan trọng là về mặt luận lý, hai quá trình chuyển đổi này là đồng thời và tức thời, do đó máy không thực sự chuyển sang trạng thái D. Tuy nhiên, hành động a_4 được thực hiện và hành động a_1 cũng vậy. Sự kết hợp này tương ứng với chuyển tiếp trên cùng của Hình 8.33.

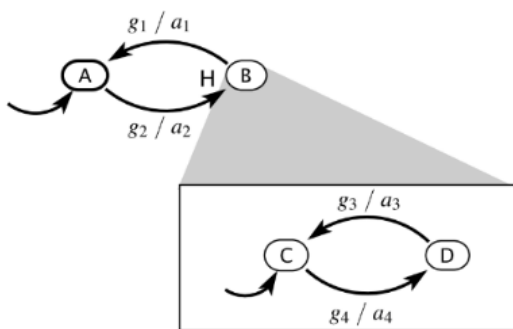


Hình 8.34: Máy trạng thái phân cấp với chuyển trạng thái có độ ưu tiên.



Hình 8.35: Sơ đồ ngữ nghĩa HFSM với chuyển trạng thái có độ ưu tiên.

Một sự khác biệt là nếu hai hành động (không vắng mặt) được thực hiện trong cùng một phản ứng, chúng có thể xung đột. Ví dụ: hai hành động có thể ghi các giá trị khác nhau vào cùng một cổng đầu ra. Hoặc chúng có thể thiết lập cùng một biến thành các giá trị khác nhau. Sự lựa chọn của chúng ta là các hành động được thực hiện theo trình tự, như được gợi ý bởi dấu chấm phẩy trong hành động a_4 ; a_1 . Như trong một ngôn ngữ mệnh lệnh như C, dấu chấm phẩy biểu thị một chuỗi. Nếu hai hành động xung đột, cái sau chiếm ưu thế hơn.



Hình 8.36: Dạng khác của HFSM với lịch sử chuyển tiếp

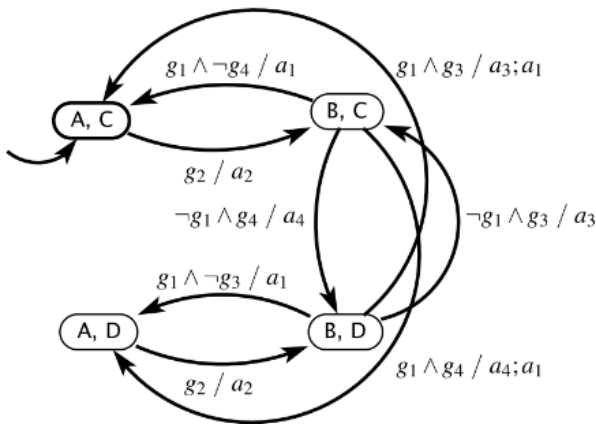
Bằng cách sử dụng một quá trình chuyển trạng thái trước có thể tránh được những vấn đề này như được thể hiện trong Hình 8.34 với ngữ nghĩa được mô tả trong Hình 8.35. *Guard* của quá trình chuyển trạng thái trước được đánh giá trước khi phản ứng tinh chỉnh và nếu bất kỳ *guard* có giá trị là true, thì bộ tinh chỉnh không phản ứng. Do đó, nếu máy ở trạng thái B và g_1 có giá trị là true, thì không có hành động nào của a_3 hoặc a_4 được thực hiện. Chuyển trạng thái trước được hiển thị bằng một vòng tròn màu đỏ ở trạng thái bắt đầu của quá trình chuyển trạng thái.

Lưu ý trong Hình 8.32 và Hình 8.33 rằng bất cứ khi nào máy vào trạng thái B, nó luôn vào trạng thái C và không bao giờ là D ngay cả khi nó trước đó ở D khi rời B. Quá trình chuyển trạng thái từ A sang B được gọi là **chuyển trạng thái tái thiết lập** (reset transition) vì quá trình tinh chỉnh đích được thiết lập lại về trạng thái ban đầu, bất kể nó đã ở trạng thái nào trước đó. Quá trình chuyển trạng thái tái thiết lập được chỉ ra bằng ký hiệu với đầu mũi tên rỗng ở điểm cuối của quá trình chuyển đổi.

Trong Hình 8.36, quá trình chuyển trạng thái từ A sang B là **quá trình chuyển đổi lịch sử**, một dạng khác của quá trình chuyển trạng thái tái thiết lập. Trong ký hiệu của chúng ta, một đầu mũi tên đậm biểu thị một quá trình chuyển trạng thái lịch sử. Nó cũng có thể được đánh dấu bằng chữ “H” để nhấn mạnh. Khi quá trình chuyển trạng thái lịch sử được thực hiện, điểm đến tinh chỉnh tiếp tục ở bất

kỳ trạng thái nào mà nó ở lần cuối cùng (hoặc trạng thái ban đầu của nó ở điểm nhập đầu tiên).

Ngữ nghĩa của quá trình chuyển trạng thái lịch sử được thể hiện trong Hình 8.37. Trạng thái ban đầu được gán ký hiệu (A, C) để chỉ ra rằng máy đang ở trạng thái A, và khi và chỉ khi nó vào B tiếp theo, nó sẽ đi đến C. Lần đầu tiên nó đi đến B, nó sẽ ở trạng thái được ký hiệu (B, C) để chỉ ra rằng đang ở trạng thái B và cụ thể hơn là C. Nếu sau đó nó chuyển sang (B, D), rồi quay lại A, sẽ kết thúc ở trạng thái có ký hiệu (A, D), có nghĩa là ở trạng thái A, nhưng nếu khi nó đi vào B nó sẽ đến D. Nghĩa là, nhớ lịch sử của quá trình chuyển trạng thái là ghi nhớ xem hệ thống đã ở trạng thái nào khi rời khỏi B.



Hình 8.37: Sơ đồ ngữ nghĩa của HFSM với lịch sử chuyển trạng thái.

Như với tổ hợp đồng thời, máy trạng thái phân cấp chấp nhận nhiều ý nghĩa có thể có. Sự khác biệt có thể rất nhỏ. Cần có sự kiểm

tra cẩn thận để đảm bảo rằng các mô hình rõ ràng và ngữ nghĩa của chúng phù hợp với những gì đang được mô hình hóa.

8.5 Tóm tắt

Trong chương này, chúng ta đã tìm hiểu về hướng tiếp cận mô hình hóa của hệ thống. Về cơ bản, một hệ thống nhúng có thể thuộc các dạng sau: thay đổi liên tục theo thời gian, thay đổi rời rạc và hệ thống kết hợp của hai dạng trên. Để mô tả hệ thống thay đổi liên tục, chúng ta sử dụng các mô hình toán học hoặc tác nhân để mô tả hệ thống. Đối với các hệ thống thay đổi rời rạc hoặc kết hợp của hai dạng, chúng ta sử dụng mô hình máy trạng thái để biểu diễn.

Có nhiều dạng máy trạng thái khác nhau giúp mô tả các hệ thống với các yêu cầu khác nhau. Một mô hình máy trạng thái về cơ bản gồm các trạng thái có thể xảy ra của hệ thống, giá trị và trạng thái khởi tạo, các chuyển tiếp trạng thái, giá trị điều khiển chuyển trạng thái và kết quả đầu ra cho từng phép chuyển trạng thái đó.

Hướng tiếp cận hệ thống từ mô hình giúp thuận tiện trong việc kiểm soát sự thay đổi của hệ thống, cũng như cho phép phân tích và phát triển hệ thống một cách tổng quát hơn, chính xác hơn. Hướng tiếp cận này phù hợp cho các thiết kế có độ phức tạp cao, cần nhiều phân tích và đánh giá.

8.6 Bài tập

Bài tập 1. Một âm thoa, Hình 8.38, bao gồm một ngón tay kim loại (gọi là âm thoa) được dịch chuyển bằng cách dùng búa đập vào nó.

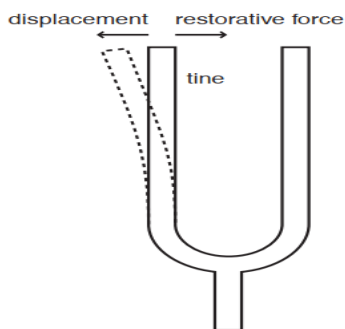
Sau khi được dịch chuyển, nó rung động. Nếu kim không có ma sát, nó sẽ rung mãi mãi. Chúng ta có thể biểu thị độ dịch chuyển của âm thoa sau khi được đánh ở thời điểm 0 dưới dạng một hàm $y: R_+ \rightarrow R$. Nếu chúng ta giả sử rằng độ dịch chuyển ban đầu do búa tác động vào là một đơn vị thì bằng kiến thức vật lý chúng ta có thể xác định rằng với mọi $t \in R_+$, độ dịch chuyển thỏa mãn phương trình vi phân sau:

$$\ddot{y}(t) = -\omega_0^2 y(t)$$

Trong đó, ω_0^2 là hằng số phụ thuộc vào khối lượng và độ cứng của âm thoa và $\ddot{y}(t)$ biểu thị đạo hàm cấp hai theo thời gian của y . Chúng ta dễ dàng chứng minh rằng y được cho bởi phương trình

$$\forall t \in R_+, y(t) = \cos(\omega_0 t)$$

là một nghiệm của phương trình vi phân (chỉ cần lấy đạo hàm cấp hai của nó). Như vậy, độ dịch chuyển của âm thoa là hình sin. Nếu ta chọn vật liệu làm âm thoa sao cho $\omega_0 = 2\pi * 440 \text{ radians/s}$ thì âm thoa sẽ tạo ra âm A-440 trên thang âm.



Hình 8.38: Mô hình âm thoa

- (a) $y(t) = \cos(\omega_0 t)$ có phải là nghiệm duy nhất không? Nếu không, hãy chỉ ra một số nghiệm khác.
- (b) Giả sử nghiệm là $y(t) = \cos(\omega_0 t)$, độ dịch chuyển ban đầu là bao nhiêu?
- (c) Xây dựng mô hình của âm thoa trên với đầu ra là y , sử dụng các mô hình tác nhân cơ bản như Integrator, Adder, Scaler... Giả thuyết xem như độ dịch chuyển ban đầu là một tham số. Sơ đồ cần có ký hiệu rõ ràng.

Bài tập 2. Chứng minh rằng nếu một hệ thống $S: A^R \rightarrow B^R$ là hệ thống nhân quả chặt chẽ và không có nhớ thì đầu ra của nó là không đổi. Không đổi có nghĩa là đầu ra $(S(x))(t)$ tại thời điểm t không phụ thuộc vào t .

Bài tập 3. Bài tập này nghiên cứu tính tuyến tính.

- (a) Chứng minh rằng mô hình máy bay trực thăng giới thiệu ở trên là tuyến tính khi và chỉ khi vận tốc góc ban đầu $\theta_y(0) = 0$.
- (b) Chứng minh rằng nối tiếp của hai tác nhân tuyến tính bất kỳ là tuyến tính.

(c) Mở rộng định nghĩa về độ tuyến tính để áp dụng cho các tác nhân có hai tín hiệu đầu vào và một tín hiệu đầu ra. Chứng minh rằng tác nhân bộ cộng là tuyến tính.

Bài tập 4. Cho mô hình tác nhân một bộ đếm sự kiện đơn giản trong Hình 8.39.



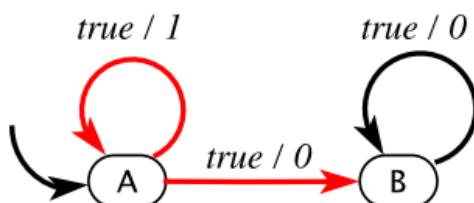
Hình 8.39: Mô hình bộ đếm sự kiện

Tác nhân này bắt đầu với trạng thái i và khi xuất hiện một sự kiện ở đầu vào, tăng trạng thái và gửi giá trị mới đến đầu ra. Trong mô hình này, e là một tín hiệu thuần túy, và c có dạng $c: \mathbb{N} \rightarrow \{absent\} \cup \mathbb{N}$, $\forall i \in \mathbb{N}$. Giả sử chúng ta sử dụng bộ đếm sự kiện như vậy trong một trạm thời tiết để đếm số lần nhiệt độ tăng trên ngưỡng nào đó. Yêu cầu của bài tập này là tạo ra một tín hiệu đầu vào hợp lý e cho bộ đếm sự kiện. Hãy thiết kế một máy trạng thái có đầu vào là tín hiệu $\tau: R \rightarrow \{absent\} \cup \mathbb{Z}$ cung cấp nhiệt độ hiện tại (tính bằng độ C), giá trị cập nhật mỗi giờ một lần. Đầu ra $e: R \rightarrow \{absent, present\}$ sẽ là một tín hiệu thuần túy đến bộ đếm sự kiện.

- (a) Thiết kế máy trạng thái chỉ cần tạo ra một đầu ra hiện tại bất cứ khi nào đầu vào có mặt và lớn hơn 38° . Nếu không, đầu ra sẽ vắng mặt.
- (b) Thiết kế lại máy trạng thái ở câu (a) với độ trễ. Cụ thể, nó sẽ tạo ra đầu ra ở lần đầu tiên đầu vào lớn hơn 38° và sau đó, nó sẽ tạo ra đầu ra bất cứ lúc nào đầu vào lớn hơn 38° nhưng đã giảm xuống dưới 36 độ kể từ lần cuối cùng đầu ra được tạo ra.
- (c) Thiết kế phiên bản thứ ba máy trạng thái cùng độ trễ như trong phần (b) nhưng cũng tạo ra đầu ra nhiều nhất một lần mỗi ngày.

Bài tập 5. Cho mô hình trạng thái máy sau:

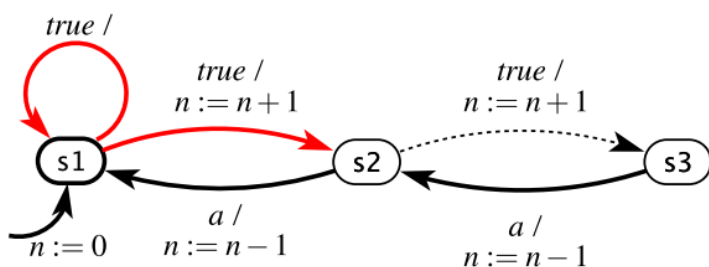
output: $y: \{0, 1\}$



Khẳng định sau là đúng hay sai và đưa ra một lập luận bổ sung:
 “Đầu ra cuối cùng sẽ là một hằng số 0, hoặc cuối cùng sẽ là một hằng số 1. Nghĩa là, đối với một số $n \in \mathbb{N}$, sau phản ứng thứ n , đầu ra sẽ là 0 hoặc 1 trong mọi phản ứng tiếp theo.”

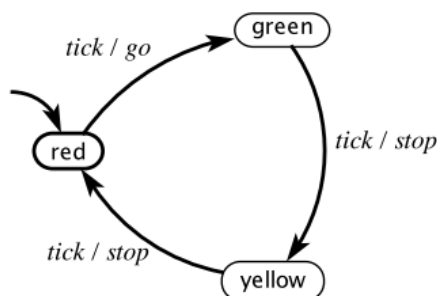
Bài tập 6. Máy trạng thái sau có bao nhiêu trạng thái có thể đạt được?

input: a : pure
variable: $n \in \mathbb{Z}$



Bài tập 7. Cho sơ đồ máy trạng thái hữu hạn xác định mô hình hóa một đèn giao thông đơn giản như sau

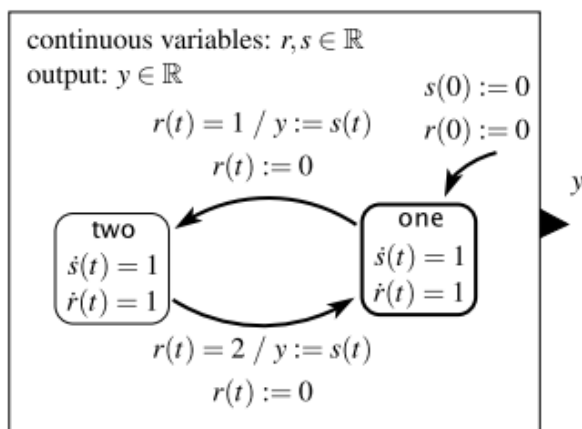
input: *tick*: pure
output: *go, stop*: pure



- (a) Viết rõ mô tả về FSM này dưới dạng một bộ 5 thành phần:
 (*States, Inputs, Outputs, update, initialState*)
- (b) Giả sử *tick* có mặt tại mỗi phản ứng, hãy xác định *execution trace* của FSM có độ dài là 4.
- (c) Xem xét việc hợp nhất các trạng thái đỏ và vàng thành một trạng thái dừng duy nhất. Các chuyển trạng thái được hướng vào hoặc ra khỏi các trạng thái đó bây giờ được chuyển hướng vào hoặc ra khỏi một trạng thái dừng mới. Các chuyển trạng thái khác và các đầu vào và đầu ra vẫn giữ nguyên. Trạng thái dừng mới là trạng thái ban đầu mới. Máy trạng thái mới này có tính xác định hay không? Tại sao hoặc tại sao không? Nếu nó là xác định, hãy vẽ ra *observable trace* có độ dài 4. Nếu nó là không xác định, hãy vẽ cây tính toán đến mức 4.

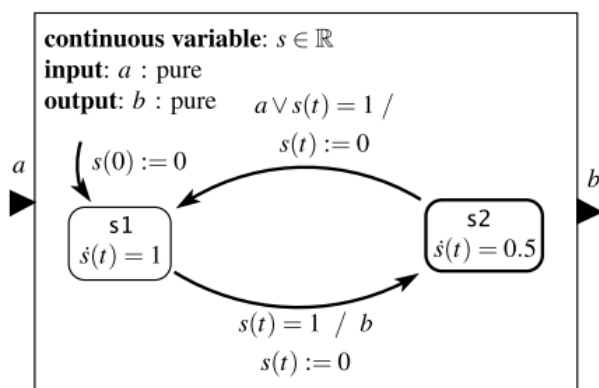
Bài tập 8. Mục tiêu của bài tập này là hiểu một AT và sau đó thay đổi nó như được chỉ định.

- (a) Đối với AT được hiển thị bên dưới, mô tả đầu ra *y*. Tránh ký hiệu không chính xác hoặc cầu thả.



(b) Giả sử có một thiết lập lại đầu vào thuần túy mới và khi đầu vào này xuất hiện, hệ thống lai sẽ bắt đầu lại, hoạt động như thể nó đang bắt đầu lại ở thời điểm 0. Thay đổi hệ thống lai từ phần (a) để thực hiện việc này.

Bài tập 9. Hãy xem xét sơ đồ một AT sau đây:



Giả sử rằng các tín hiệu đầu vào a và b là các tín hiệu rời rạc theo thời gian liên tục, nghĩa là mỗi tín hiệu có thể được cho dưới dạng một hàm dạng $a: R \rightarrow \{present, absent\}$, trong đó hầu như

tại mọi thời điểm $t \in R$, $a(t) = absent$. Giả sử rằng máy trạng thái có thể thực hiện nhiều nhất một quá trình chuyển đổi tại mỗi thời điểm khác nhau t và máy đó bắt đầu thực hiện tại thời điểm $t = 0$.

(a) Vẽ sơ đồ đầu ra b nếu đầu vào a chỉ hiện diện tại các thời điểm sau:

$$t = 0.75, 1.5, 2.25, 3, 3.75, 4.5, \dots$$

Bao gồm ít nhất các thời điểm từ $t = 0$ đến $t = 5$.

(b) Vẽ sơ đồ đầu ra b nếu đầu vào a chỉ hiện diện tại các thời điểm $t = 0, 1, 2, 3, \dots$

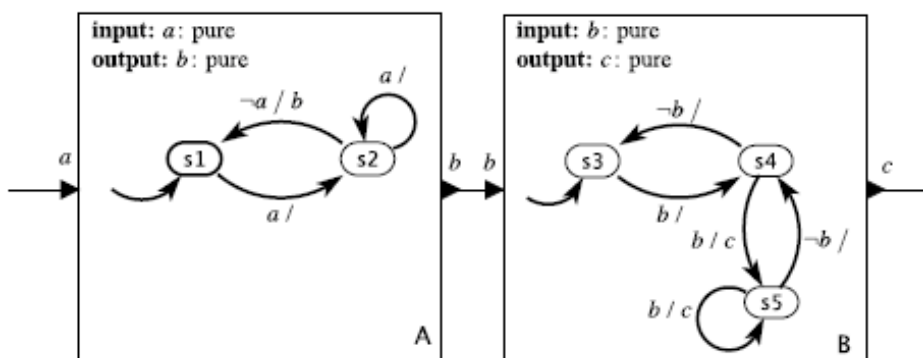
(c) Giả sử rằng đầu vào a có thể là bất kỳ tín hiệu rời rạc nào, hãy tìm giới hạn dưới về lượng thời gian giữa các sự kiện b . Tín hiệu đầu vào a nào (nếu có) đạt được giới hạn dưới này?

Bài tập 10. Bạn có một nguồn tín hiệu tương tự tạo ra âm thanh thuần túy. Bạn có thể bật hoặc tắt nguồn bằng sự kiện đầu vào *On* hoặc *Off*. Xây dựng một AT cung cấp tín hiệu *On* và *Off* dưới dạng đầu ra, được kết nối với đầu vào của bộ tạo âm thanh. Hệ thống sẽ hoạt động như sau. Khi nhận được một chuông sự kiện đầu vào, nó sẽ tạo ra âm thanh có độ dài 80 ms bao gồm ba đợt, mỗi đợt kéo dài 20 ms của âm thuần cách nhau hai khoảng im lặng 10 ms. Hệ thống sẽ xử lý ra sao nếu nó nhận được hai sự kiện đồ chuông cách nhau 50 ms?

Bài tập 11. Ô tô ngày nay có các tính năng được liệt kê dưới đây. Triển khai từng tính năng như một AT.

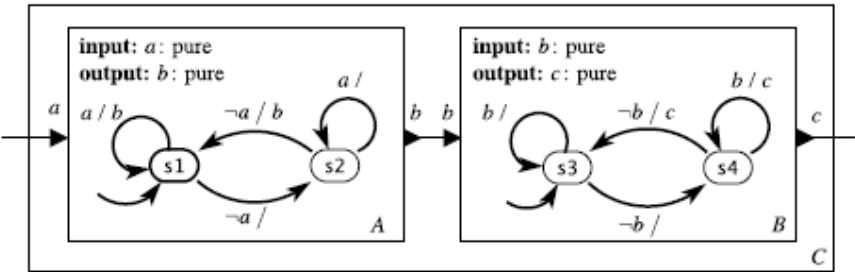
- (a) Đèn vòm được bật ngay khi bất kỳ cánh cửa nào được mở ra. Nó vẫn bật trong 30 giây sau khi tất cả các cửa đóng lại. Những cảm biến nào là cần thiết?
- (b) Sau khi khởi động động cơ, một tiếng bíp vang lên và đèn cảnh báo đỏ sẽ được sáng nếu có hành khách chưa thắt dây an toàn. Bíp ngừng phát ra sau 30 giây hoặc ngay khi thắt dây an toàn, tùy điều kiện nào đến sớm hơn. Đèn cảnh báo bật sáng khi thắt dây an toàn. Gợi ý: Giả sử các cảm biến cung cấp một sự kiện cảnh báo khi hệ thống đánh lửa được bật và có ghế ngồi có hành khách không thắt dây an toàn hoặc nếu hệ thống đánh lửa đã bật và hành khách ngồi trên ghế mà không thắt dây an toàn. Giả sử thêm rằng các cảm biến cung cấp sự kiện noWarn khi hành khách rời khỏi ghế ngồi, hoặc khi khóa an toàn bị thắt hoặc khi tắt máy.

Bài tập 12. Xét tổ hợp đồng bộ sau đây của hai máy trạng thái A và B:



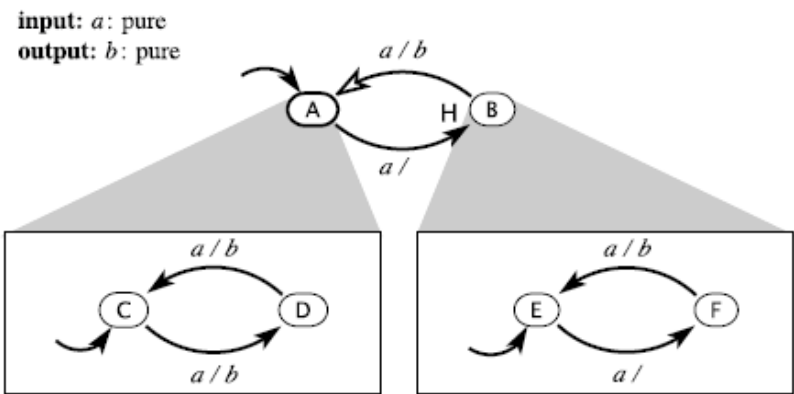
Xây dựng một máy trạng thái duy nhất C đại diện cho tổ hợp. Trạng thái nào của tổ hợp không thể truy cập được?

Bài tập 13. Xét tổ hợp đồng bộ sau đây của hai máy trạng thái A và B:



Xây dựng một máy trạng thái duy nhất C đại diện cho tổ hợp. Trạng thái nào của tổ hợp không thể truy cập được?

Bài tập 14. Hãy xem xét máy trạng thái phân cấp sau:



Xây dựng một máy FSM tương đương để đưa ra ngữ nghĩa của hệ thống phân cấp. Mô tả theo cách khác là hoạt động đầu vào/đầu ra của máy này. Có máy nào đơn giản hơn thể hiện cùng một hoạt động không?

TÀI LIỆU THAM KHẢO

- [1] Marcel J.M. Pelgrom, “*Analog-to-Digital Conversion*,” 1st Edition, Springer Netherland, 2010, ISBN: 978-90-481-8887-1, DOI 10.1007/978-90-481-8888-8.
- [2] Marcel J.M. Pelgrom, “*Analog-to-Digital Conversion*,” 2nd Edition, Springer Netherland, 2010, ISBN: 978-1-4614-1370-7, DOI 10.1007/978-1-4614-1371-4.
- [3] Marcel J.M. Pelgrom, “*Analog-to-Digital Conversion*,” 3rd Edition, Springer Netherland, 2010, ISBN: 978-3-319-44970-8, DOI 10.1007/978-3-319-44971-5.
- [4] Kamen, E. W., “*Industrial Controls and Manufacturing*,” Academic Press., 1999.
- [5] Eden, M. and M. Kagan, “*The PentiumR processor with MMXTM technology*,” In IEEE International Conference (COMPCON), CA, USA, 1997. pp. 260–262.
- [6] Tremblay, M., J. M. O’Connor, V. Narayannan, and H. Liang, “*VIS speeds new media processing*” IEEE Micro, 16(4), 1996, pp.10–20.
- [7] Wolfe, V., S. Davidson, and I. Lee, “*RTC: Language support for real-time concurrency*,” Real-Time Systems, 5(1), 1993, pp.63–87.
- [8] Texas Instruments, “*Stellaris® LM3S8962 Evaluation Board*,” User’s Manual, February 9, 2010.

- [9] Gamma, E., R. Helm, R. Johnson, and J. Vlissides, “*Design Patterns: Elements of Reusable Object-Oriented Software*,” Addison Wesley, 1994.
- [10] Coffman, E. G., Jr., M. J. Elphick, and A. Shoshani, “*System deadlocks*,” Computing Surveys, Vol. 3, Iss. 2, 1971, pp. 67–78.
- [11] Liu, C. L. and J. W. Layland, “*Scheduling algorithms for multiprogramming in a hard real time environment*,” Journal of the ACM, Vol. 20, Iss. 1, 1973, pp. 46–61.
- [12] Horn, W., “*Some simple scheduling algorithms*,” Naval Research Logistics Quarterly, Vol. 21, Iss. 1, 1974, pp.177 – 185.
- [13] Lawler, E. L., “*Optimal scheduling of a single machine subject to precedence constraints*,” Management Science, Vol. 19, Iss. 5, 1973, pp.544–546.
- [14] Sha, L., R. Rajkumar, and J. P. Hehoczky, “*Priority inheritance protocols: An approach to real-time synchronization*,” IEEE Transactions on Computers, Vol. 39, Iss. 9, 1990, pp.1175–1185.
- [15] Harel, D., “*Statecharts: A visual formalism for complex systems*,” Science of Computer Programming, Vol. 8, 1987, pp.231–274.
- [16] von der Beeck, M., “*A comparison of Statecharts variants*,” In Third International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems, Springer-Verlag, Lubeck,

Germany, vol. 863 " of Lecture Notes in Computer Science, 1994, pp. 128–148

- [17] Graham, R. L., "*Bounds on multiprocessing timing anomalies,*" SIAM Journal on Applied Mathematics, Vol. 17, Iss. 2, 1969, pp.416–429.
- [18] Lee, Edward A., and Sanjit A. Seshia, "*Introduction to embedded systems: A cyber-physical systems approach,*" The MIT Press, Second Edition, Version 2.2, 2017.